

PsyConn: Conectividad funcional bajo psicodélicos clásicos (psilocibina y LSD)

Conectividad, teoría de grafos y estadística inferencial sobre ds006072 y ds003059

Juan

2026-06-03

Table of contents

1	1 Carga y validación de conectomas en R	2
1.1	1.1 Importación y validación	3
1.2	1.2 Comprobación de integridad	7
2	2 Análisis de redes funcionales (teoría de grafos)	8
2.1	2.1 Métricas globales y nodales	8
2.1.1	2.1.1 Marco teórico: del grafo de Euler al conectoma humano	8
2.1.2	2.1.2 Construcción del grafo cerebral	8
2.1.3	2.1.3 Métricas calculadas	9
2.2	2.2 Justificación de las elecciones metodológicas	11
2.3	2.3 Referencias clave del chunk	12
3	3 Métricas avanzadas	25
3.1	3.1 Entropía dinámica de la sincronía intra-red — métrica entrópica #1 de Carhart-Harris (2014)	25
3.2	3.2 Shannon entropy de la distribución de grados (Viol et al. 2017)	27
4	4 Control de pipelines de denoising — sensibilidad de las métricas topológicas	39
5	5 Visualización exploratoria de conectividad	47
5.1	5.1 Conectomas por red funcional (nilearn)	47
5.2	5.2 Heatmaps, conectomas circulares y métricas nodales (R)	50
6	6 Análisis estadístico: PSIL vs MTP	60
6.1	6.1 Tests pareados globales y nodales — núcleo confirmatorio del TFM	60
6.1.1	6.1.1 FDR within-family: justificación y aplicación	72

6.1.2	6.1.2 Visualización de resultados: boxplots, rainclouds, forest-style dot plot	77
6.1.3	6.1.3 Análisis de influencia Leave-One-Out (LOO) — métricas globales y nodales	83
6.2	6.2 Convergencia inter-pipeline del contraste — operacionalización del criterio S0	86
6.3	6.3 Conectividad positiva, negativa y anticorrelaciones (POS/NEG)	94
6.3.1	6.3.1 Visualizaciones POS/NEG	107
6.4	6.4 Conectividad inter-red 8×8 (between-network) — Yeo-7 + Subcortex Tian	113
6.4.1	6.4.1 Visualizaciones inter-red 8×8 (con barras de error y BayesFactor)	125
6.5	6.5 Conectividad basada en semillas (seed-based) — subcortex Tian para REBUS-1	128
6.6	6.5.1 Convergencia inter-pipeline del seed-based (criterio S0)	141
6.7	6.6 Correlación cerebro-experiencia (MEQ30, EBI)	147
7	7 Análisis de sensibilidad y robustez	163
7.1	7.1 Detectabilidad del diseño (sensitivity analysis, no post-hoc power)	165
7.2	7.2 Tests de permutación pareados (sign-flip exacto)	171
7.3	7.3 Verificación de supuestos — QQ-plots y Shapiro-Wilk heredado	177
7.4	7.4 Bootstrap del intervalo de confianza del tamaño de efecto	179

1 1 Carga y validación de conectomas en R

Importa, valida y limpia las matrices de conectividad por sujeto y condición generadas en la fase de extracción (PsiloPrep §4.5 para ds006072; LSDPrep §5 para ds003059) para el **pipeline primario** de cada dataset, y construye los objetos R que consumen las secciones de análisis posteriores (§2–§6): matrices de conectividad por sujeto, diferencia intra-sujeto del contraste primario (psicodélico — comparador), promedios grupales y la lista de etiquetas del atlas combinado Schaefer 200 + Tian S2 (232 ROIs corteza+subcortex; Schaefer et al., 2018; Tian et al., 2020). El bloque es dataset-agnóstico: itera sobre `ACTIVE_DATASETS` y deja todo en una lista `CONNECTOMES` indexada por dataset (`ds006072` = TEDANA+aCompCor, conditions `psil/mtp`; `ds003059` = AsIs, conditions `lsd/plcb`). El primario de ds006072 (TEDANA+aCompCor) se fijó en PsiloPrep §5.2 tras el QC inter-pipeline y se documenta en PsiloPrep §3.13. Para correr el resto del pipeline sobre un pipeline de denoising secundario (TEDANA, TEDANA+24P, OC+Siegel, OC+24P, OC+aCompCor, OC+36P+GSR; AsIs+GSR, AsIs+aCompCor) basta con cambiar `pipeline/conn_dir` en la entrada de `DATASETS` correspondiente.

```
# Para iterar ambos datasets, descomentar la línea de abajo:
# ACTIVE_DATASETS <- c("ds006072", "ds003059")
ACTIVE_DATASETS <- "ds006072"
```

1.1 1.1 Importación y validación

```
#=====
# FASE 1 - Carga y validación de conectomas en R - §1.1
# Importación y validación - BLOQUE DATASET-AGNÓSTICO (itera sobre ACTIVE_DATASETS)
#=====
# Carga las matrices de conectividad por sujeto y condición generadas en la
# fase de extracción para el pipeline PRIMARIO de cada dataset:
#   ds006072 → TEDANA+aCompCor (NORDIC + fMRIPrep + TEDANA ICA + 24P motion +
#                               10 PC aCompCor; Kundu 2013; Moeller 2021;
#                               Behzadi 2007; DuPre 2021; primario fijado
#                               en PsiloPrep §5.2, cf. PsiloPrep §3.13)
#                               - generadas en PsiloPrep §4.5; atlas en §4.2.
#   ds003059 → AsIs (replicación fiel de Carhart-Harris 2016;
#                   cf. LSDPrep @sec-ds003059-triangulacion)
#                   - generadas en LSDPrep §5.
# El atlas (Schaefer 200 + Tian S2 = 232 ROIs; Schaefer 2018, Tian 2020) es
# el MISMO para ambos; sólo difiere el nombre del CSV de etiquetas copiado
# a cada results/<ds>/stats/.
#
# Salida: lista CONNECTOMES, una entrada por dataset activo, con campos de
# nombre uniforme:
#   $id $pipeline $conn_dir $n_rois $conditions (c1=psicodélico, c2=comparador)
#   $cond_pretty $atlas_labels (232)
#   $mean_subject : list(<c1> = list(sub-XX = 232x232, ...), <c2> = ...)
#   $diff_subject : list(sub-XX = mean_<c1> - mean_<c2>, ...) ← contraste primario
#   $group_mean   : list(<c1> = 232x232, <c2> = 232x232) ← desde sujetos comunes
#   $group_diff   : 232x232 = group_mean[[c1]] - group_mean[[c2]]
#   $common_subjects : char (intersección c1 c2 - diseño crossover intra-sujeto)
#
# Para ejecutar UN solo dataset, restringir ACTIVE_DATASETS.
#=====

library(tidyverse)
library(igraph)

PROJECT_ROOT <- "D:/ProfessionalProyectos/PsyConn"
N_ROIS <- 232L # Schaefer 200 + Tian S2 (PsiloPrep §4.2)

# ---- Configuración por dataset (pipeline PRIMARIO de cada uno) -----
# conditions: c1 = psicodélico (psil/lsd), c2 = comparador (mtp/plcb).
```

```

# El contraste primario es siempre c1 - c2.
DATASETS <- list(
  ds006072 = list(
    id = "ds006072",
    pipeline = "TEDANA+aCompCor",
    conn_dir = file.path(PROJECT_ROOT, "derivatives/ds006072/conn/TEDANA_aCompCor"),
    conditions = c("psil", "mtp"),
    cond_pretty = c(psil = "PSIL", mtp = "MTP"),
    n_rois = N_ROIS,
    atlas_labels_csv = file.path(PROJECT_ROOT,
                                  "results/ds006072/stats/atlas_labels_ds006072.csv")
  ),
  ds003059 = list(
    id = "ds003059",
    pipeline = "AsIs",
    conn_dir = file.path(PROJECT_ROOT, "derivatives/ds003059/conn/AsIs"),
    conditions = c("lsd", "plcb"),
    cond_pretty = c(lsd = "LSD", plcb = "PLCB"),
    n_rois = N_ROIS,
    atlas_labels_csv = file.path(PROJECT_ROOT,
                                  "results/ds003059/stats/atlas_labels_ds003059.csv")
  )
)

# ---- Helpers comunes (idénticos para ambos datasets) -----
# Leer matriz desde CSV de Python (pandas to_csv con índice de fila/columna).
# Contrato: matriz numérica, cuadrada, simétrica, sin NaN/Inf. PsiloPrep §4.5
# garantiza estas tres propiedades (`np.corrcoef` produce simétrica; `to_zcorr`
# aplica nan_to_num + clip + fill_diagonal(0)). La verificación defensiva
# aquí declara el contrato y aborta si alguna corre futura del productor
# lo viola.
read_matrix <- function(file_path) {
  mat <- as.matrix(read.csv(file_path, row.names = 1, check.names = FALSE))
  storage.mode(mat) <- "numeric"
  if (!isSymmetric(mat, tol = 1e-8))
    stop("[read_matrix] matriz no simétrica: ", file_path)
  if (any(is.na(mat)) || !all(is.finite(mat)))
    stop("[read_matrix] matriz con NaN o Inf: ", file_path)
  mat
}

# Cargar todas las matrices de sujeto de una condición.

```

```

# En disco, derivatives/<ds>/conn/<pipeline>/ contiene {group/, <c1>/, <c2>/}
# como directorios hermanos: dentro de <cond>/ solo hay sub-XX/, no group/.
load_subject_matrices <- function(condition_dir, pattern) {
  subject_dirs <- list.dirs(condition_dir, recursive = FALSE, full.names = TRUE)
  mats <- list()
  for (d in subject_dirs) {
    f <- list.files(d, pattern = pattern, full.names = TRUE)
    if (length(f) == 0) next
    mats[[basename(d)]] <- read_matrix(f[1])
  }
  mats
}

# ---- Cargador de conectomas por dataset -----
load_connectomes <- function(cfg) {
  cat(sprintf("\n##### $1.1 Conectomas - %s (pipeline %s) #####\n",
    cfg$id, cfg$pipeline))
  c1 <- cfg$conditions[1]; c2 <- cfg$conditions[2]

  # Atlas: Background + 232 ROIs → eliminar Background → 232 etiquetas.
  if (!file.exists(cfg$atlas_labels_csv))
    stop("[S1.1] ", cfg$id, ": no existe ", cfg$atlas_labels_csv,
      " - ejecuta la fase de extracción (PsiloPrep $4.5 / LSDPrep $5) que copia el atlas.")
  atlas_labels <- read.csv(cfg$atlas_labels_csv, stringsAsFactors = FALSE)$label
  atlas_labels <- atlas_labels[-1]
  if (length(atlas_labels) != cfg$n_rois)
    stop("[S1.1] ", cfg$id, ": el atlas tiene ", length(atlas_labels),
      " etiquetas, se esperaban ", cfg$n_rois, ".")
  cat(sprintf(" Atlas: %d etiquetas (Schaefer 200 + Tian 32)\n", length(atlas_labels)))

  # Conectomas por sujeto - patrón {subj}_mean_<cond>.csv en conn_dir/<cond>/<subj>/
  # (la fase de extracción escribe estos nombres aun con 1 run/condición:
  # skip blando del promediado).
  mean_subject <- list()
  for (cd in c(c1, c2)) {
    mean_subject[[cd]] <- load_subject_matrices(
      file.path(cfg$conn_dir, cd), sprintf("_mean_%s\\.csv$", cd))
  }
  cat(sprintf(" Sujetos %s: %d | %s: %d\n",
    cfg$cond_pretty[[c1]], length(mean_subject[[c1]]),
    cfg$cond_pretty[[c2]], length(mean_subject[[c2]])))
}

```

```

# Validación de dimensiones
for (cd in c(c1, c2)) for (s in names(mean_subject[[cd]])) {
  m <- mean_subject[[cd]][[s]]
  if (nrow(m) != cfg$n_rois || ncol(m) != cfg$n_rois)
    stop("[S1.1] ", cfg$id, " - dimensión incorrecta ", cfg$cond_pretty[[cd]],
         " ", s, ": ", nrow(m), "x", ncol(m))
}

# Sujetos comunes (crossover intra-sujeto c1 vs c2): el diseño pareado
# necesita N >= 2 por construcción (un único sujeto pareado degenera el
# contraste a sd=0). Si una pasada futura sobre un pipeline alternativo
# pierde sujetos en una de las dos condiciones, el guard fuerza un fallo
# explícito en lugar de propagar resultados sin sentido aguas abajo.
common <- intersect(names(mean_subject[[c1]]), names(mean_subject[[c2]]))
cat(sprintf(" Sujetos comunes (%s %s): %d\n",
           cfg$cond_pretty[[c1]], cfg$cond_pretty[[c2]], length(common)))
if (length(common) < 2)
  stop("[S1.1] ", cfg$id, ": diseño pareado requiere N >= 2 sujetos comunes; encontrados "
       length(common), ".")

diff_subject <- setNames(
  lapply(common, function(s) mean_subject[[c1]][[s]] - mean_subject[[c2]][[s]]),
  common)

# group_mean / group_diff se RECOMPUTAN aquí sobre el conjunto `common`,
# en vez de leer los `group/group_mean<cond>.npz` precomputados por
# PsiloPrep §4.6. Esto garantiza que el promedio coincide con el N
# efectivo del test pareado de §6 - relevante si en un pipeline
# alternativo se pierde un sujeto en una sola condición.
group_mean <- list()
group_mean[[c1]] <- Reduce("+", mean_subject[[c1]][common]) / length(common)
group_mean[[c2]] <- Reduce("+", mean_subject[[c2]][common]) / length(common)
group_diff <- group_mean[[c1]] - group_mean[[c2]]
stopifnot(
  "group_mean debe ser matriz" = is.matrix(group_mean[[c1]]),
  "group_mean debe ser cuadrada" = nrow(group_mean[[c1]]) == ncol(group_mean[[c1]]),
  "group_mean debe ser 232x232" = nrow(group_mean[[c1]]) == cfg$n_rois)

cat(sprintf(" OK conectomas cargados, validados y limpios (%s / %s)\n",
           cfg$id, cfg$pipeline))
list(id = cfg$id, pipeline = cfg$pipeline, conn_dir = cfg$conn_dir,
     n_rois = cfg$n_rois, conditions = cfg$conditions, cond_pretty = cfg$cond_pretty,

```

```

    atlas_labels = atlas_labels, mean_subject = mean_subject,
    diff_subject = diff_subject, group_mean = group_mean,
    group_diff = group_diff, common_subjects = common)
}

CONNECTOMES <- setNames(lapply(ACTIVE_DATASETS, function(d) load_connectomes(DATASETS[[d]]))
                        ACTIVE_DATASETS)

```

1.2 1.2 Comprobación de integridad

```

#=====
# FASE 1 - Carga y validación de conectomas en R - §1.2
# Comprobación de integridad - recorre TODAS las matrices de TODOS los
# sujetos y condiciones de cada dataset cargado en §1.1.
#
# read_matrix() ya garantiza simetría + ausencia de NaN/Inf en la carga;
# este chunk añade dim == (n_rois, n_rois), diag == 0 y rango plausible
# para Pearson-z, y reporta el rango agregado por dataset × condición.
#=====
for (.id in names(CONNECTOMES)) {
  cd <- CONNECTOMES[[.id]]
  for (cond in cd$conditions) {
    mats <- cd$mean_subject[[cond]]
    if (length(mats) == 0) next
    rng_min <- Inf; rng_max <- -Inf
    for (s in names(mats)) {
      m <- mats[[s]]
      stopifnot(
        "dim incorrecta" = identical(dim(m), c(cd$n_rois, cd$n_rois)),
        "diag != 0"      = all(diag(m) == 0))
      rng_min <- min(rng_min, min(m))
      rng_max <- max(rng_max, max(m))
    }
    message(sprintf("[%s | %s] N=%d dim=%dx%d rango=[%.3f, %.3f] (pipeline %s)",
                    cd$id, cd$cond_pretty[[cond]], length(mats),
                    cd$n_rois, cd$n_rois, rng_min, rng_max, cd$pipeline))
  }
}

```

2 2 Análisis de redes funcionales (teoría de grafos)

2.1 2.1 Métricas globales y nodales

2.1.1 2.1.1 Marco teórico: del grafo de Euler al conectoma humano

La teoría de grafos nace en 1736, cuando Euler demostró la imposibilidad de recorrer los siete puentes de Königsberg pasando por cada uno una sola vez, abstrayendo el problema en un conjunto de **vértices** (las orillas) unidos por **aristas** (los puentes) [Euler, 1736; Barabási, 2016, Cap. 2]. Esta abstracción —un grafo $G = (V, E)$, donde V es el conjunto de n nodos y E el conjunto de l enlaces— es hoy la herramienta estándar para describir cualquier sistema complejo cuyos elementos interactúan: redes sociales, metabólicas, tecnológicas o neuronales [Newman, 2003; Boccaletti et al., 2006; Barabási, 2016].

Aplicada al cerebro, la teoría de grafos formaliza la intuición de que el sistema nervioso es una red de regiones especializadas (segregación funcional) que cooperan dinámicamente (integración funcional) [Tononi et al., 1994; Friston, 1994]. Bullmore & Sporns (2009, *Nat. Rev. Neurosci.*) consolidaron este marco en neuroimagen, y Rubinov & Sporns (2010, *NeuroImage*) lo sistematizaron en la **Brain Connectivity Toolbox** (BCT), de la que aquí adoptamos las definiciones canónicas.

2.1.2 2.1.2 Construcción del grafo cerebral

A partir de la matriz de conectividad funcional $W = (w_{ij})$ obtenida en PsiloPrep §4 (rama psilocibina) o en LSDPrep §5 (rama LSD) — correlación de Pearson sobre series temporales BOLD del atlas combinado Schaefer 200 + Tian S2, $n = 232$ ROIs:

1. **Nodos** = parcelas atlas (regiones con perfiles funcionales homogéneos [Wang et al., 2009; Sporns & Betzel, 2016]).
2. **Aristas** = $|w_{ij}|$ (magnitud de la asociación estadística).
3. **Umbralización proporcional**: se conserva el 20% superior de las correlaciones positivas más fuertes (umbralización por densidad fija, Rubinov & Sporns 2010, 2011; cf. Achard & Bullmore 2007). Las anticorrelaciones se descartan porque (i) su interpretación topológica es controvertida [Rubinov & Sporns, 2011], (ii) son sensibles al GSR [Murphy & Fox, 2017] y (iii) violan la positividad asumida por la mayoría de métricas. La sensibilidad al umbral se evalúa replicando el análisis a densidades del 10 % y 30 % (Tabla suplementaria S1).

Para métricas válidas en grafos densos (mean strength, global efficiency, eigenvector centrality) se conserva la matriz **completa** sin umbralizar, siguiendo Rubinov & Sporns (2011) sobre redes “weight-conserving”.

2.1.3 2.1.3 Métricas calculadas

Las métricas se agrupan en cuatro familias [Rubinov & Sporns, 2010]: **integración, segregación, centralidad y modularidad**. Notación: $a_{ij} \in \{0, 1\}$ enlace binario, $w_{ij} \in [0, 1]$ peso, k_i grado, d_{ij} longitud del camino más corto entre i y j , N conjunto de nodos.

2.1.3.1 Métricas globales

Densidad — fracción de aristas presentes sobre las posibles:

$$\kappa = \frac{2l}{n(n-1)}$$

Estimador básico del coste de cableado [Bullmore & Sporns, 2012].

Grado y fuerza — $k_i = \sum_{j \in N} a_{ij}$; análogo ponderado $s_i = \sum_{j \in N} w_{ij}$. La **fuerza media** $\bar{s} = \frac{1}{n} \sum_i s_i$ resume el peso total de conexiones por nodo [Barrat et al., 2004].

Eficiencia global [Latora & Marchiori, 2001, *Phys. Rev. Lett.*] — inversa del camino característico, robusta a grafos desconectados; mide la capacidad de comunicación paralela:

$$E_{\text{glob}} = \frac{1}{n(n-1)} \sum_{i \neq j \in N} \frac{1}{d_{ij}}$$

con $d_{ij} = \sum_{w_{uv} \in g_{ij}} f(w_{uv})$, $f(w) = 1/w$ (mapa peso $\rightarrow$ longitud de la BCT).

Camino característico (CPL) [Watts & Strogatz, 1998] — promedio de las distancias geodésicas:

$$L = \frac{1}{n} \sum_{i \in N} \frac{\sum_{j \neq i} d_{ij}}{n-1}$$

Marcador clásico de integración global.

Coefficiente de clustering ponderado [Barrat et al., 2004, *PNAS*] — generalización ponderada del clustering local de Watts–Strogatz que pondera cada triángulo por la *intensidad* media de las dos aristas que entran al nodo, normalizando por la fuerza del nodo:

$$C_i^w = \frac{1}{s_i(k_i-1)} \sum_{j,h \in N} \frac{w_{ij} + w_{ih}}{2} a_{ij} a_{ih} a_{jh}, \quad C^w = \frac{1}{n} \sum_{i \in N} C_i^w$$

donde $s_i = \sum_j w_{ij}$ es la fuerza del nodo y a_{ij} la matriz de adyacencia binaria del grafo umbralizado. Es el estimador que implementa `igraph::transitivity(type = "weighted")` y por tanto el que se reporta en los CSV de §2 (columnas `clustering global` y `local_clustering nodal`). Mide segregación local: clustering alto indica unidades funcionales densamente interconectadas.

Modularidad Q [Newman, 2004, 2006] — bondad de la partición de la red en módulos no solapantes M :

$$Q = \frac{1}{2l} \sum_{i,j \in N} \left(w_{ij} - \frac{s_i s_j}{2l} \right) \delta(m_i, m_j)$$

donde $\delta(m_i, m_j) = 1$ si i y j están en el mismo módulo. La optimización de Q es NP-difícil; usamos el algoritmo de Louvain [Blondel et al., 2008, *J. Stat. Mech.*]. Por la degeneración de Good et al. (2010, *Phys. Rev. E*) — el paisaje de Q tiene multitud de máximos cercanos al óptimo, de modo que dos ejecuciones independientes pueden devolver particiones distintas con Q casi idéntica — aplicamos el **procedimiento de consenso de Lancichinetti & Fortunato (2012, *Sci. Rep.* 2:336)**: se generan $N = 100$ particiones Louvain base, se construye la matriz de co-clasificación $D_{ij} = (1/N) \sum_{p=1}^N \mathbb{1}[p_i = p_j]$, se umbraliza a $\tau = 0.5$ y se re-aplica Louvain sobre D iterativamente hasta que dos iteraciones devuelven la misma partición (o hasta `max_it` = 10). La modularidad Q reportada se evalúa sobre el **grafo original** con la partición consensuada (no sobre la matriz D , cuyo Q no sería interpretable). Diagnósticos de convergencia (`Q_sd`, `Q_range` de las 100 particiones base, `cons_iterations`, `cons_converged`) acompañan cada fila del CSV.

Small-worldness σ [Watts & Strogatz, 1998; Humphries & Gurney, 2008]

$$\sigma = \frac{C/C_{\text{rand}}}{L/L_{\text{rand}}}$$

con C y L calculadas sobre el **mismo grafo umbralizado** observado, y $C_{\text{rand}}, L_{\text{rand}}$ promediados sobre $k = 10$ grafos nulos Erdős–Rényi con la misma n y la misma densidad. Si el grafo umbralizado de distancias resulta desconectado, L se sustituye por $1/E_{\text{glob}}$ sobre el mismo grafo umbralizado (Latora & Marchiori, 2001). $\sigma > 1$ indica topología small-world (alta segregación + alta integración), distintiva del cerebro sano [Bassett & Bullmore, 2006] y cuya alteración bajo psilocibina/LSD ha sido reportada [Petri et al., 2014; Carhart-Harris et al., 2014].

Asortatividad r [Newman, 2002] — coeficiente de Pearson entre los grados de nodos en los extremos de cada arista. $r > 0$: hubs conectados entre sí (núcleo resiliente, *rich-club*); $r < 0$: hubs dispersos y vulnerables [Bullmore & Sporns, 2009].

2.1.3.2 Métricas nodales

Centralidad de intermediación (betweenness) [Freeman, 1978; Brandes, 2001]:

$$b_i = \frac{1}{(n-1)(n-2)} \sum_{\substack{h,j \in N \\ h \neq j, h \neq i, j \neq i}} \frac{\rho_{hj}(i)}{\rho_{hj}}$$

fracción de caminos más cortos que pasan por i . Identifica cuellos de botella en el flujo de información.

Centralidad de cercanía (closeness) [Freeman, 1978]:

$$L_i^{-1} = \frac{n-1}{\sum_{j \neq i} d_{ij}}$$

inverso de la distancia media a todos los demás nodos.

Centralidad de eigenvector [Bonacich, 1972] — i -ésima componente del autovector principal de W ; nodo central si está conectado a otros nodos centrales. Calculada sobre el grafo completo (Rubinov & Sporns 2011).

Within-module degree z-score z_i [Guimerà & Amaral, 2005, *Nature*]:

$$z_i = \frac{k_i(m_i) - \overline{k(m_i)}}{\sigma_{k(m_i)}}$$

donde $k_i(m_i) = \sum_{j \in m_i} w_{ij}$ es la fuerza de i **dentro** de su módulo m_i , y $\overline{k(m_i)}$, $\sigma_{k(m_i)}$ son la media y desviación estándar de las fuerzas intra-módulo de los nodos en m_i .

Coefficiente de participación P_i [Guimerà & Amaral, 2005]:

$$P_i = 1 - \sum_{m \in M} \left(\frac{k_i(m)}{k_i} \right)^2$$

diversidad de conexiones inter-modulares. $P_i \rightarrow 0$: todas las conexiones intra-módulo; $P_i \rightarrow 1$: distribuidas uniformemente entre módulos.

Clasificación de roles z-P [Guimerà & Amaral, 2005] — hubs ($z \geq 2.5$) se subdividen en provinciales ($P \leq 0.30$), conectores ($0.30 < P \leq 0.75$) y kinless ($P > 0.75$); no-hubs en ultra-periféricos, periféricos, conectores no-hub y kinless no-hub. Particularmente relevante en psicodélicos, donde se observa redistribución del balance provincial/conector hub [Petri et al., 2014; Lord et al., 2019].

2.2 2.2 Justificación de las elecciones metodológicas

Decisión	Justificación
Atlas Schaefer 200 + Tian S2	Cobertura cortical + subcortical; parcelas funcionalmente homogéneas [Schaefer et al., 2018; Tian et al., 2020].
Densidad 20 % (rango 10–30 %)	Estándar en BCT para grafos cerebrales; análisis de sensibilidad evita conclusiones umbral-dependientes [Rubinov & Sporns, 2010; Garrison et al., 2015].

Decisión	Justificación
Descartar pesos negativos	Interpretación topológica controvertida; sensible a GSR; mayoría de métricas asumen positividad [Rubinov & Sporns, 2011; Murphy & Fox, 2017].
Métricas densas vs. umbralizadas	Strength, eficiencia global, eigenvector son válidas en grafos densos; clustering, modularidad, betweenness requieren grafo esparcido [Rubinov & Sporns, 2010, 2011].
Consenso Louvain (Lancichinetti & Fortunato 2012)	Mitiga la degeneración de particiones [Good et al., 2010] de forma más robusta que un simple max- Q sobre N ejecuciones independientes: agrega las 100 particiones base vía matriz de co-clasificación y re-aplica Louvain iterativamente hasta convergencia.
Convención $L_{ij} = 1/ w_{ij} $	Mapa peso→longitud estándar BCT [Rubinov & Sporns, 2010].

Sobre el clustering ponderado conviene anotar una decisión metodológica específica: la literatura ofrece al menos dos formulaciones convivientes. La de Barrat et al. (2004) que adoptamos aquí pondera cada triángulo por la **media aritmética** de los pesos de las dos aristas adyacentes al nodo y normaliza por la fuerza del nodo; la de Onnela et al. (2005, *Phys. Rev. E*) usa la **media geométrica** de los tres pesos del triángulo, normalizados por el peso máximo de la red, $t_i^{w, \text{Onnela}} = \frac{1}{2} \sum_{j,h} (\hat{w}_{ij} \hat{w}_{ih} \hat{w}_{jh})^{1/3}$. Las dos métricas correlan fuertemente en grafos densos pero difieren en magnitud absoluta: Barrat es sensible a la fuerza del nodo, Onnela enfatiza la coherencia relativa de los pesos del triángulo. Elegimos Barrat porque es el estimador canónico que implementa **igraph** (la herramienta estándar en el TFM), lo que garantiza reproducibilidad sin reimplementación local, y porque para los fines del análisis exploratorio basta con un índice consistente entre sujetos y pipelines: la triangulación inter-pipeline de §4 controla la robustez a la elección, no a la fórmula concreta.

2.3 2.3 Referencias clave del chunk

Achard & Bullmore (2007) *PLoS Comput. Biol.* 3:e17 — eficiencia y coste. Barabási (2016) *Network Science*, Cambridge UP — texto de referencia. Barrat et al. (2004) *PNAS* 101:3747 — strength y clustering ponderado en redes con peso (estimador adoptado por **igraph::transitivity**). Bassett & Bullmore (2006) *Neuroscientist* 12:512 — small-world cerebral. Blondel et al. (2008) *J. Stat. Mech.* P10008 — algoritmo de Louvain. Brandes (2001) *J. Math. Sociol.* 25:163 — algoritmo de betweenness. Bullmore & Sporns (2009) *Nat. Rev. Neurosci.* 10:186 — revisión seminal. Bullmore & Sporns (2012) *Nat. Rev. Neurosci.* 13:336 — economía de redes. Carhart-Harris et al. (2014) *Front. Hum. Neurosci.* 8:20 — psicodélicos. Freeman (1978) *Soc. Netw.* 1:215 — centralidad clásica. Garrison et al. (2015)

NeuroImage 118:651 — efecto del umbral. Good et al. (2010) *Phys. Rev. E* 81:046106 — degeneración de modularidad. Guimerà & Amaral (2005) *Nature* 433:895 — cartografía z-P. Humphries & Gurney (2008) *PLoS ONE* 3:e0002051 — small-worldness σ . Lancichinetti & Fortunato (2012) *Sci. Rep.* 2:336 — consenso Louvain. Latora & Marchiori (2001) *Phys. Rev. Lett.* 87:198701 — eficiencia global. Lord et al. (2019) *NeuroImage* 199:127 — psilocibina y modularidad. Murphy & Fox (2017) *NeuroImage* 154:169 — GSR. Newman (2002) *Phys. Rev. Lett.* 89:208701 — asortatividad. Newman (2004, 2006) *Phys. Rev. E* — modularidad. Onnela et al. (2005) *Phys. Rev. E* 71:065103 — clustering ponderado por media geométrica (alternativa a Barrat discutida en §2.2). Petri et al. (2014) *J. R. Soc. Interface* 11:20140873 — psilocibina + topología. Rubinov & Sporns (2010) *NeuroImage* 52:1059 — referencia canónica BCT. Rubinov & Sporns (2011) *NeuroImage* 56:2068 — redes con peso negativo. Schaefer et al. (2018) *Cereb. Cortex* 28:3095 — atlas Schaefer. Sporns & Betzel (2016) *Annu. Rev. Psychol.* 67:613 — módulos cerebrales. Tian et al. (2020) *Nat. Neurosci.* 23:1421 — atlas Tian subcortical. Tononi et al. (1994) *PNAS* 91:5033 — integración/segregación. Watts & Strogatz (1998) *Nature* 393:440 — small-world.

```
#=====
# FASE 2 - Métricas de teoría de grafos (igraph) - §2.1
# Métricas globales y nodales - BLOQUE DATASET-AGNÓSTICO (itera sobre CONNECTOMES)
#=====
# Calcula las métricas de teoría de grafos por sujeto, condición y grupo sobre
# el atlas combinado Schaefer 200 + Tian S2 = 232 ROIs, para el pipeline
# PRIMARIO de cada dataset (ds006072 → TEDANA+aCompCor; ds003059 → AsIs). Los
# conectomas y todo lo necesario vienen de la lista CONNECTOMES construida en §1.1
# (campos: $mean_subject[<cond>][<subj>], $group_mean[<cond>], $common_subjects,
# $atlas_labels, $n_rois, $conditions = c(c1=psicodélico, c2=comparador), ...).
#
# Las funciones de cómputo (to_distance, proportional_threshold,
# consensus_louvain, mat_to_igraph, mat_to_igraph_full, calc_global_metrics,
# calc_nodal_metrics) son IDÉNTICAS para ambos datasets - no dependen de la
# config. Sólo varía, vía CONNECTOMES, qué matrices se procesan y a qué
# directorio de salida van (derivatives/<ds>/graphs/<pipeline>/).
#
# Notas metodológicas que sostienen el cómputo (cf. §2.2 para la justificación
# completa):
# - clustering ponderado = estimador de Barrat et al. (2004, PNAS 101:3747),
#   implementado por igraph::transitivity(type="weighted"). Onnela et al.
#   (2005, PRE 71:065103) ofrece una alternativa de media geométrica que
#   se discute pero no se reporta.
# - modularidad = consenso de Lancichinetti & Fortunato (2012, Sci. Rep.
#   2:336): N=100 particiones Louvain base → matriz de co-clasificación →
#   reaplica Louvain iterativamente hasta convergencia (no es best-of-N).
#
```

```

# Salidas por dataset (en derivatives/<ds>/graphs/<pipeline>/):
#   global/<cond>/<subj>_global_metrics.csv   nodal/<cond>/<subj>_nodal_metrics.csv
#   group/<cond>/group_global_<cond>.csv      group/<cond>/group_nodal_<cond>.csv
#   sensitivity_thresholds.csv (densidades 0.10 / 0.20 / 0.30 - Tabla S1 del TFM)
# Y en memoria: lista GRAPHS, una entrada por dataset, con $graph_dir,
# $subjects, $louvain_memberships (estos últimos quedan en memoria sin consumidor
# directo tras la retirada de $3.2 modularity_signed en la ronda 1 de revisión;
# se conservan para inspección/uso futuro).
#=====

library(tidyverse)
library(igraph)

PROJECT_ROOT      <- "D:/ProfessionalProyectos/PsyConn"
PRIMARY_THRESHOLD <- 0.20 # umbral proporcional principal a reportar (Rubinov & Sporns 2010)

# Seed maestra para las rutinas estocásticas del pipeline de grafos:
# (a) `consensus_louvain` (N=100 restarts de Louvain, Lancichinetti & Fortunato 2012);
# (b) nulos Erdős-Rényi de `small_worldness` (N_NULLS=10, Watts & Strogatz 1998 /
#     Humphries & Gurney 2008). Se fija dentro de `calc_global_metrics` para que
#     cada llamada arranque desde un estado RNG conocido - los valores numéricos
#     dependen entonces sólo del contenido de la matriz de entrada, no del orden
#     de ejecución. Cascada: $2.1 (CSVs reproducibles), $3 (sensibilidad
#     multi-threshold heredada), $6.1 (pool sin drift entre reruns), $6.2
#     (recálculo por pipeline de control).
GLOBAL_SEED <- 2025L

stopifnot("$2.1 requiere CONNECTOMES (ejecutar $1.1 antes)" = exists("CONNECTOMES"))

#=====
# Funciones utilitarias (dataset-agnósticas)
#=====

to_distance <- function(mat, eps = 1e-6) {
  mat_abs <- abs(mat)
  mat_abs[mat_abs < eps] <- 0
  dist_mat <- ifelse(mat_abs > 0, 1 / mat_abs, 0)
  diag(dist_mat) <- 0
  dist_mat
}

#=====

```

```

# UMBRALIZACIÓN PROPORCIONAL - Rubinov & Sporns (2010, NeuroImage)
# Mantiene el top `prop` de conexiones positivas más fuertes. Las conexiones
# negativas se eliminan: su interpretación como aristas en grafos con propósito
# topológico es controvertida (Rubinov & Sporns, 2011) y son sensibles al GSR
# (Murphy & Fox, 2017). Primario: prop=0.20. Sensibilidad: 0.10 y 0.30.
# El efecto del pipeline de denoising sobre estas métricas se cuantifica en §4.
#=====
proportional_threshold <- function(mat, prop = PRIMARY_THRESHOLD) {
  mat_pos <- mat
  mat_pos[mat_pos < 0] <- 0
  vals_upper <- mat_pos[upper.tri(mat_pos)]
  vals_pos <- vals_upper[vals_upper > 0]
  if (length(vals_pos) == 0) return(mat_pos) # matriz toda ceros: sin cambio
  thresh <- quantile(vals_pos, 1 - prop)
  mat_pos[mat_pos < thresh] <- 0
  diag(mat_pos) <- 0
  mat_pos
}

#=====
# Consensus Louvain (Lancichinetti & Fortunato, 2012, Sci. Rep. 2:336)
# Mitiga la degeneración de modularidad (Good et al., 2010): construye una
# matriz de co-clasificación D_ij = fracción de las N ejecuciones en que i y j
# caen en el mismo módulo, la umbraliza a tau, y reparticiona iterativamente
# hasta convergencia. Q se calcula sobre el GRAFO ORIGINAL con la partición
# consensuada. Devuelve list(membership, Q, n_iterations, converged,
# Q_base_sd, Q_base_range).
#=====
consensus_louvain <- function(G_w, N = 100, tau = 0.5, max_it = 10) {
  n <- igraph::vcount(G_w)
  base_runs <- lapply(seq_len(N), function(i) {
    cm <- cluster_louvain(G_w)
    list(membership = as.integer(membership(cm)), Q = modularity(cm))
  })
  Q_base <- sapply(base_runs, function(x) x$Q)
  partitions <- lapply(base_runs, function(x) x$membership)

  converged <- FALSE
  it_count <- 0
  for (it in seq_len(max_it)) {
    it_count <- it
    p1 <- partitions[[1]]

```

```

same_all <- all(vapply(partitions[-1], function(p)
  identical(p, p1), logical(1)))
if (same_all) { converged <- TRUE; break }

D <- matrix(0, n, n)
for (p in partitions) D <- D + outer(p, p, "==")
D <- D / length(partitions)
diag(D) <- 0
D[D < tau] <- 0
if (sum(D) == 0) break

G_cons <- igraph::graph_from_adjacency_matrix(
  D, mode = "undirected", weighted = TRUE, diag = FALSE)
partitions <- lapply(seq_len(N), function(i)
  as.integer(membership(cluster_louvain(G_cons))))
}

final_memb <- partitions[[1]]
Q_final <- igraph::modularity(G_w, final_memb, weights = E(G_w)$weight)
list(membership = final_memb, Q = Q_final, n_iterations = it_count,
     converged = converged, Q_base_sd = sd(Q_base),
     Q_base_range = max(Q_base) - min(Q_base))
}

mat_to_igraph <- function(mat, node_names, prop = PRIMARY_THRESHOLD) {
  if (nrow(mat) != ncol(mat)) stop("Matriz no cuadrada: ", nrow(mat), "x", ncol(mat))
  mat_t <- proportional_threshold(mat, prop)
  mat_t <- (mat_t + t(mat_t)) / 2 # simetría numérica
  diag(mat_t) <- 0
  g <- igraph::graph_from_adjacency_matrix(mat_t, mode = "undirected",
                                           weighted = TRUE, diag = FALSE)

  V(g)$name <- node_names
  g
}

mat_to_igraph_full <- function(mat, node_names) {
  # Grafo completo sin umbral, pesos absolutos - para métricas válidas en
  # grafos densos: mean_strength, global_efficiency, eigenvector centrality.
  mat_abs <- abs(mat)
  mat_abs <- (mat_abs + t(mat_abs)) / 2
  diag(mat_abs) <- 0
  g <- igraph::graph_from_adjacency_matrix(mat_abs, mode = "undirected",

```



```

weighted = TRUE, diag = FALSE)

V(g)$name <- node_names
g
}

=====
# Métricas globales
=====

calc_global_metrics <- function(G_full, G_thr, mat) {

  # Seed fijada por llamada (cf. GLOBAL_SEED, §2.1 header): garantiza que
  # `consensus_louvain` y los nulos Erdős-Rényi de `small_worldness` producen
  # el mismo número desde el mismo input, independientemente del orden de
  # ejecución dentro del bucle de sujetos/pipelines.
  set.seed(GLOBAL_SEED)

  # --- Grafo COMPLETO (válidas sin umbral) ---
  w_full <- abs(E(G_full)$weight)
  G_full_abs <- igraph::set_edge_attr(G_full, "weight", value = w_full)
  mean_str <- mean(strength(G_full_abs))

  # Grafo de distancias desde la matriz bruta. to_distance() invierte los
  # pesos:  $L_{ij} = 1/|w_{ij}|$  (convención BCT, Rubinov & Sporns 2010). distances()
  # los usa como costes de Dijkstra. E_glob = mean(1/d_ij) - Latora & Marchiori
  # (2001). NO aplicar 1/weight aquí (doble inversión).
  dist_mat_full <- to_distance(abs(mat))
  Gd_full <- mat_to_igraph_full(dist_mat_full, V(G_full)$name)
  eff_full <- tryCatch({
    d <- igraph::distances(Gd_full, weights = E(Gd_full)$weight)
    diag(d) <- NA
    mean(1 / d, na.rm = TRUE)
  }, error = \(e) NA_real_)
  cpl_full <- tryCatch(mean_distance(Gd_full, weights = E(Gd_full)$weight,
    directed = FALSE), error = \(e) NA_real_)

  # --- Grafo UMBRALIZADO (requieren grafo esparcido) ---
  w_thr <- abs(E(G_thr)$weight)
  G_thr_abs <- igraph::set_edge_attr(G_thr, "weight", value = w_thr)

  cons <- consensus_louvain(G_thr_abs, N = 100, tau = 0.5, max_it = 10)

  assort <- tryCatch(igraph::assortativity_degree(G_thr, directed = FALSE),

```

```

      error = \(e) NA_real_)

# Small-worldness sigma (Watts & Strogatz 1998; Humphries & Gurney 2008).
# sigma = (C/C_rand) / (L/L_rand); sigma > 1 = topología small-world.
# Coherencia conceptual: C_obs y L_obs se calculan ambas sobre el MISMO grafo
# umbralizado, y se comparan con la media de N_NULLS=10 nulos Erdős-Rényi de
# misma n y misma densidad (mejor que el sigma de un único sample). Si el
# grafo umbralizado de distancias resulta desconectado para algún sujeto, se
# usa 1/E_glob_thr como sustituto de L_obs (Latora & Marchiori 2001).
C_obs      <- mean(transitivity(G_thr_abs, type = "weighted"), na.rm = TRUE)
n_nodes    <- vcount(G_thr_abs)
p_density  <- edge_density(G_thr_abs, loops = FALSE)

dist_mat_thr <- to_distance(
  as.matrix(igraph::as_adjacency_matrix(G_thr_abs, attr = "weight", sparse = FALSE)))
Gd_thr <- igraph::graph_from_adjacency_matrix(
  dist_mat_thr, mode = "undirected", weighted = TRUE, diag = FALSE)
L_obs <- tryCatch({
  d_thr <- igraph::distances(Gd_thr, weights = E(Gd_thr)$weight)
  diag(d_thr) <- NA
  if (any(is.infinite(d_thr), na.rm = TRUE)) {
    # Grafo desconectado → inverso de la eficiencia global (Latora-Marchiori).
    inv_d <- 1 / d_thr
    inv_d[is.infinite(d_thr)] <- 0
    e_glob_thr <- mean(inv_d, na.rm = TRUE)
    if (is.na(e_glob_thr) || e_glob_thr == 0) NA_real_ else 1 / e_glob_thr
  } else {
    mean(d_thr, na.rm = TRUE)
  }
}, error = \(e) NA_real_)

N_NULLS <- 10L
sigma_sw <- tryCatch({
  rand_metrics <- vapply(seq_len(N_NULLS), function(i) {
    G_rand <- igraph::sample_gnp(n_nodes, p_density, directed = FALSE)
    if (igraph::ecount(G_rand) < 2) return(c(NA_real_, NA_real_))
    C_r <- transitivity(G_rand, type = "global")
    L_r <- mean_distance(G_rand, directed = FALSE, unconnected = TRUE)
    c(C_r, L_r)
  }, numeric(2))
  C_rand <- mean(rand_metrics[1, ], na.rm = TRUE)
  L_rand <- mean(rand_metrics[2, ], na.rm = TRUE)

```

```

    if (is.na(C_rand) || C_rand == 0 || is.na(L_rand) || L_rand == 0 ||
        is.na(L_obs) || L_obs == 0) NA_real_
    else (C_obs / C_rand) / (L_obs / L_rand)
  }, error = \"(e) NA_real_)

# Diagnóstico de la partición consensuada: tamaños de módulo. Comunidades
# de un solo nodo (singletons) hacen que within_module_z se colapse a 0 en
# ese nodo y distorsionan la cartografía z-P; reportarlas sin filtrarlas
# (la partición se preserva tal cual la devuelve el consenso).
.module_sizes <- as.integer(table(cons$membership))
.n_singletons <- sum(.module_sizes == 1L)

metrics <- tibble(
  mean_strength      = mean_str,
  global_efficiency  = eff_full,
  char_path_length   = cpl_full,
  density            = edge_density(G_thr, loops = FALSE),
  mean_degree        = mean(degree(G_thr)),
  clustering          = C_obs,
  modularity         = cons$Q,
  n_communities      = length(.module_sizes),
  min_module_size    = min(.module_sizes),
  median_module_size = stats::median(.module_sizes),
  n_singletons       = .n_singletons,
  assortativity      = assort,
  small_worldness    = sigma_sw,
  threshold_used     = PRIMARY_THRESHOLD,
  Q_sd               = cons$Q_base_sd,
  Q_range            = cons$Q_base_range,
  cons_iterations    = cons$n_iterations,
  cons_converged     = cons$converged
)
attr(metrics, "louvain_membership") <- cons$membership
metrics
}

#=====
# Métricas nodales
#=====
calc_nodal_metrics <- function(G_full, G_thr, louvain_memb = NULL) {

  w_full      <- abs(E(G_full)$weight)

```

```

G_full_abs <- igraph::set_edge_attr(G_full, "weight", value = w_full)
strength_i <- strength(G_full_abs)
eig_cent    <- eigen_centrality(G_full_abs, weights = w_full)$vector

w_thr      <- abs(E(G_thr)$weight)
G_thr_abs <- igraph::set_edge_attr(G_thr, "weight", value = w_thr)

if (!is.null(louvain_memb)) {
  memb <- louvain_memb
} else {
  memb <- consensus_louvain(G_thr_abs, N = 100, tau = 0.5, max_it = 10)$membership
}
n      <- vcount(G_thr)
A_abs <- as_adjacency_matrix(G_thr_abs, attr = "weight", sparse = FALSE)

# within_module_z: z-score intra-módulo (Guimerà & Amaral 2005, eq. 3;
# Rubinov & Sporns 2010, Table A1). k_i(m_i) = fuerza de i SÓLO hacia otros
# nodos de su módulo, comparada con la distribución intra-módulo de m_i.
within_z <- vapply(seq_len(n), function(i) {
  idx <- which(memb == memb[i])
  if (length(idx) < 2) return(0)
  k_within <- rowSums(A_abs[idx, idx, drop = FALSE])
  k_i      <- k_within[match(i, idx)]
  if (sd(k_within) == 0) return(0)
  (k_i - mean(k_within)) / sd(k_within)
}, numeric(1))

participation <- vapply(seq_len(n), function(i) {
  ki <- sum(A_abs[i, ])
  if (ki == 0) return(0)
  1 - sum(vapply(unique(memb), function(m)
    (sum(A_abs[i, memb == m]) / ki)^2, numeric(1)))
}, numeric(1))

# Clasificación de roles z-P (Guimerà & Amaral 2005). Hubs: z >= 2.5.
classify_role <- function(z, p) {
  if (z >= 2.5) {
    if (p <= 0.30) "R5_provincial_hub"
    else if (p <= 0.75) "R6_connector_hub"
    else "R7_kinless_hub"
  } else {
    if (p <= 0.05) "R1_ultra_peripheral"

```

```

    else if (p <= 0.62) "R2_peripheral"
    else if (p <= 0.80) "R3_nonhub_connector"
    else
        "R4_nonhub_kinless"
  }
}
roles <- mapapply(classify_role, within_z, participation)

w_thr_safe <- ifelse(w_thr > 0, w_thr, 1e-6)

tibble(
  node          = V(G_thr)$name,
  strength      = strength_i,
  eigenvector   = eig_cent,
  degree        = degree(G_thr),
  betweenness   = betweenness(G_thr_abs, weights = 1 / w_thr_safe),
  closeness     = closeness(G_thr_abs, weights = 1 / w_thr_safe),
  local_clustering = transitivity(G_thr_abs, type = "weighted", isolates = "zero"),
  within_module_z = within_z,
  participation = participation,
  role          = roles,
  module        = as.integer(memb)
)
}

#=====
# Pipeline de cómputo por dataset
#=====
run_graph_metrics <- function(cd) {
  cat(sprintf("\n##### $2.1 Grafos - %s (pipeline %s) #####\n",
            cd$id, cd$pipeline))
  c1 <- cd$conditions[1]; c2 <- cd$conditions[2]
  GRAPH_DIR <- file.path(PROJECT_ROOT, "derivatives", cd$id, "graphs", cd$pipeline)
  group_out_dir <- file.path(GRAPH_DIR, "group")
  for (d in c(file.path(GRAPH_DIR, "global", c1), file.path(GRAPH_DIR, "global", c2),
                file.path(GRAPH_DIR, "nodal", c1), file.path(GRAPH_DIR, "nodal", c2),
                file.path(group_out_dir, c1), file.path(group_out_dir, c2)))
    dir.create(d, showWarnings = FALSE, recursive = TRUE)

  # --- Diagnóstico del impacto de la umbralización (primer sujeto común, c1) ---
  .diag_subj <- cd$common_subjects[1]
  .mat_raw <- cd$mean_subject[[c1]][[.diag_subj]]
  .mat_thr <- proportional_threshold(.mat_raw, PRIMARY_THRESHOLD)

```

```

.n_edges_full <- cd$n_rois * (cd$n_rois - 1) / 2
cat(sprintf("  Umbralización proporcional %.0f%% - sujeto diagnóstico %s:\n",
            PRIMARY_THRESHOLD * 100, .diag_subj))
cat(sprintf("    conexiones activas: %d → %d (0.1f%% retenidas) | mean_strength: %.3f → %",
            sum(.mat_raw[upper.tri(.mat_raw)] != 0),
            sum(.mat_thr[upper.tri(.mat_thr)] != 0),
            100 * sum(.mat_thr[upper.tri(.mat_thr)] != 0) / sum(.mat_raw[upper.tri(.mat_raw)] != 0),
            mean(rowSums(abs(.mat_raw))), mean(rowSums(abs(.mat_thr))),
            sum(.mat_raw[upper.tri(.mat_raw)] != 0) / .n_edges_full,
            sum(.mat_thr[upper.tri(.mat_thr)] != 0) / .n_edges_full))
rm(.diag_subj, .mat_raw, .mat_thr, .n_edges_full)

# --- Métricas por sujeto -----
subjects <- cd$common_subjects
louvain_memberships <- list()
non_converged <- character(0)
for (subj in subjects) {
  message(" ", cd$id, " | ", subj)
  for (cond in c(c1, c2)) {
    mat <- cd$mean_subject[[cond]][[subj]]
    G_full <- mat_to_igraph_full(mat, cd$atlas_labels)
    G_thr <- mat_to_igraph(mat, cd$atlas_labels)
    global_m <- calc_global_metrics(G_full, G_thr, mat)
    memb_cond <- attr(global_m, "louvain_membership")
    louvain_memberships[[paste0(subj, "_", cond)]] <- memb_cond
    if (isFALSE(global_m$cons_converged))
      non_converged <- c(non_converged, paste0(subj, "_", cond))
    write.csv(global_m,
              file.path(GRAPH_DIR, "global", cond, paste0(subj, "_global_metrics.csv")),
              row.names = FALSE)
    write.csv(calc_nodal_metrics(G_full, G_thr, louvain_memb = memb_cond),
              file.path(GRAPH_DIR, "nodal", cond, paste0(subj, "_nodal_metrics.csv")),
              row.names = FALSE)
  }
}
if (length(non_converged) > 0)
  warning(sprintf(
    "$2.1 - %s: el consenso Louvain NO convergió en max_it=10 para %d sujeto×cond: %s",
    cd$id, length(non_converged), paste(non_converged, collapse = ", ")))

# --- Métricas grupales -----
for (cond in c(c1, c2)) {

```

```

mat      <- cd$group_mean[[cond]]
G_full   <- mat_to_igraph_full(mat, cd$atlas_labels)
G_thr    <- mat_to_igraph(mat, cd$atlas_labels)
out_cond <- file.path(group_out_dir, cond)
group_m  <- calc_global_metrics(G_full, G_thr, mat)
write.csv(group_m,
           file.path(out_cond, paste0("group_global_", cond, ".csv")),
           row.names = FALSE)
write.csv(calc_nodal_metrics(G_full, G_thr,
                             louvain_memb = attr(group_m, "louvain_membership")),
           file.path(out_cond, paste0("group_nodal_", cond, ".csv")),
           row.names = FALSE)
}

# --- Análisis de sensibilidad: densidades 10% / 20% / 30% -----
# (Tabla S1 del TFM.) Aquí la eficiencia se calcula sobre el grafo
# UMBRALIZADO a cada densidad - intencional, para mostrar cómo varían las
# métricas topológicas con el umbral. La columna se llama efficiency_thresholded
# para distinguirla de la global_efficiency del CSV principal, que se computa
# sobre el grafo COMPLETO sin umbralizar. Ref: Rubinov & Sporns (2010).
sensitivity_results <- map_dfr(c(0.10, 0.20, 0.30), function(prop) {
  map_dfr(cd$common_subjects, function(subj) {
    map_dfr(c(c1, c2), function(cond) {
      mat <- cd$mean_subject[[cond]][[subj]]
      G_full_s <- mat_to_igraph_full(mat, cd$atlas_labels)
      G_thr_s <- mat_to_igraph(mat, cd$atlas_labels, prop = prop)
      w_thr_s <- abs(E(G_thr_s)$weight)
      G_thr_abs_s <- igraph::set_edge_attr(G_thr_s, "weight", value = w_thr_s)
      set.seed(GLOBAL_SEED) # cf. H2-D: reproducibilidad de consensus_louvain.
      cons_s <- consensus_louvain(G_thr_abs_s, N = 100, tau = 0.5, max_it = 10)
      best_Q_s <- cons_s$Q
      w_full_s <- abs(E(G_full_s)$weight)
      G_full_abs_s <- igraph::set_edge_attr(G_full_s, "weight", value = w_full_s)
      dist_s <- to_distance(as.matrix(
        igraph::as_adjacency_matrix(G_thr_abs_s, attr = "weight", sparse = FALSE)))
      Gd_s <- igraph::graph_from_adjacency_matrix(
        dist_s, mode = "undirected", weighted = TRUE, diag = FALSE)
      eff_s <- tryCatch({
        d_s <- igraph::distances(Gd_s, weights = E(Gd_s)$weight)
        diag(d_s) <- NA
        mean(1 / d_s, na.rm = TRUE)
      }, error = \(e) NA_real_)
    })
  })
})

```

```

        tibble(
          subject      = subj,
          condition    = cond,
          density      = prop,
          clustering    = mean(transitivity(G_thr_abs_s, type = "weighted"), na.rm = TRUE),
          modularity    = best_Q_s,
          efficiency_thresholded = eff_s,
          mean_strength = mean(strength(G_full_abs_s))
        )
      })
    })
  })
  dir.create(GRAPH_DIR, showWarnings = FALSE, recursive = TRUE)
  write.csv(sensitivity_results,
            file.path(GRAPH_DIR, "sensitivity_thresholds.csv"), row.names = FALSE)

  cat(sprintf(" OK §2.1 - %s - métricas de red + sensibilidad guardadas en %s\n",
              cd$id, GRAPH_DIR))
  invisible(list(id = cd$id, pipeline = cd$pipeline, graph_dir = GRAPH_DIR,
                 subjects = subjects, louvain_memberships = louvain_memberships))
}

GRAPHS <- setNames(lapply(names(CONNECTOMES), function(ds) run_graph_metrics(CONNECTOMES[[ds]],
                                     names(CONNECTOMES))), names(CONNECTOMES))
message("OK §2.1 - métricas de teoría de grafos calculadas para: ",
        paste(names(GRAPHS), collapse = ", "))

# Comprobación final de integridad - por dataset cargado
for (.id in names(CONNECTOMES)) {
  cd <- CONNECTOMES[[.id]]
  ex <- cd$mean_subject[[ cd$conditions[1] ]][[1]]
  stopifnot(length(cd$atlas_labels) == nrow(ex),
            all(diag(ex) == 0))
  message(sprintf("[%s] atlas_labels == nrow(mat): %s | diag = 0: %s",
                  cd$id, length(cd$atlas_labels) == nrow(ex), all(diag(ex) == 0)))
}

```


3 3 Métricas avanzadas

3.1 3.1 Entropía dinámica de la sincronía intra-red — métrica entrópica #1 de Carhart-Harris (2014)

Esta sub-sección implementa la **primera de las dos métricas que Carhart-Harris et al. (2014, *Front Hum Neurosci* 8:20) etiquetan como “entropía”** en la formulación de la *entropic brain hypothesis* (la segunda — entropía del repertorio de motifs entre 4 nodos límbicos, Tagliazucchi et al. 2014, *Hum Brain Mapp* — queda pendiente por ser más nicho y requerir dFC sobre un sub-grafo de 4 nodos concretos). A diferencia de las métricas topológicas de §2 y de la degree entropy estática de §3.2 (ambas operan sobre la matriz de FC promediada), esta es **dinámica sobre la sincronía intra-red** y es la operacionalización *literal* que da nombre a la hipótesis.

i Segunda entropía EBH (motifs límbicos, Tagliazucchi 2014) — extensión futura

La segunda métrica entrópica de la EBH literal — entropía del repertorio de motifs de conectividad entre 4 nodos límbicos (HIP × ACC bilateral, 64 motifs posibles) a lo largo del tiempo (Tagliazucchi et al. 2014, *Hum Brain Mapp*) — no se implementa en este pipeline porque requiere construir dFC con sliding window sobre ese sub-grafo de 4 nodos concretos y enumerar los 64 motifs binarios posibles en cada ventana. Pertenecería al dominio dinámico junto a **synch_entropy_assoc** y cerraría el plano de entropía dinámica de R9 con una operacionalización alternativa; queda anotada como extensión natural cuando se amplíe el N (cf. R9 «Limitaciones y caveats»).

Definición (Carhart-Harris et al. 2014, §“Increased network entropy in the psychedelic state” y Fig. 6). Para cada red canónica de reposo:

1. *Sincronía intra-red instantánea.* En cada instante t se mide cuánto se desvían las señales de las ROIs de la red respecto a su media: $V_N(t) = \text{var}_{i \in N}(z(x_i)(t))$, con $z(x_i) =$ serie temporal de la ROI i estandarizada (media 0, sd 1). $V_N(t)$ bajo red muy sincronizada (todas las ROIs cerca de la media); $V_N(t)$ alto red desincronizada — “if the signal in all of the voxels within a given network deviates little from the network’s mean signal, then variance is low... if the signal in voxels fluctuate erratically, then variance is high”.
2. *Distribución temporal.* Se discretiza $\{V_N(t)\}_{t=1}^T$ en N_BINS_SYNC bins de igual anchura sobre el rango observado → distribución $p_b =$ fracción de instantes en el bin b — “we discretized the time course of intra-network synchrony over time into equal sized bins... we built probability distributions of the variance of the intra-network synchrony across time”.
3. *Entropía de Shannon.* $H_N = -\sum_b p_b \log_2 p_b$ (bits). H_N es máxima cuando la red visita un repertorio amplio de estados de sincronía (metaestabilidad alta) y mínima cuando se mantiene en un estado fijo — “prediction of internal network synchrony... is more difficult [post-psilocybin]” distribución más plana H_N mayor.

Predicción a priori (entropic brain hypothesis). Carhart-Harris et al. (2014) reportan $\uparrow$ entropía bajo psilocibina **específicamente en redes de asociación de alto nivel** (DMN, ejecutiva/control, salience), **no** en sensoriales/motoras. Por tanto:

- Métrica primaria = **synch_entropy_assoc** = media de H_N sobre {Default, Cont, SalVentAttn} (los equivalentes Yeo-7 de las redes de asociación). Test **one-sided PSIL > MTP** (hipótesis direccional pre-especificada), con two-sided + Wilcoxon como sensibilidad.
- **synch_entropy_sensory** = media sobre {Vis, SomMot} $\rightarrow$ **control negativo** (predicción nulo).
- Tests por red individual (7 redes Yeo corticales) con BH-FDR conjunta (réplica de la estructura del paper, que testó 9 redes Smith con Bonferroni 0.05/9). Subcortex (Tian S2) se reporta aparte (régimen de SNR distinto).

i Decisiones de implementación (Carhart-Harris 2014 no las especifica todas)

- **Parcelación.** El paper usó las 9 redes ICA de Smith et al. (2009). Aquí se usan las **7 redes Yeo-7** parseadas de los nombres de ROI Schaefer (**7Networks_***), más “Subcortex” (32 ROIs Tian S2) aparte — Tian et al. (2020, *Nat Neurosci* 23:1421, **tian_2020**) documentan que la parcelación subcortical tiene un régimen de SNR distinto al cortical, por lo que conviene un pool de tests propio para la entropía intra-red en ese dominio. El mapeo Smith-2009 $\rightarrow$ Yeo-7 no es 1:1 pero las redes de asociación de alto nivel (DMN, FPN/control, salience) están bien representadas en ambas taxonomías.
- **Estandarización por ROI.** Se z-score cada serie de ROI antes de la varianza espacial, para que las ROIs de mayor amplitud no dominen $V_N(t)$ (el paper no lo menciona; es la práctica estándar bajo Schaefer-200, atlas heterogéneo en tamaño de parcelas). Consecuencia cuantitativa: los valores absolutos de H_N no son comparables 1:1 con la Fig. 6 del paper; el contraste pareado within-subject se preserva. Como sensibilidad se reporta también **synch_entropy_raw** sin z-score (sobre BOLD crudo) en el mismo CSV — el ranking pareado debería ser estable entre ambas variantes; si no lo es, el resultado primario debe entenderse como dependiente de la normalización.
- **Nº de bins.** Primario **N_BINS_SYNCH** = 20 (con $T \approx 217$ vol ~ 11 instantes/bin). El paper dice “equal sized bins” sin especificar. Para verificar la robustez al parámetro se ejecuta un **sweep de sensibilidad** sobre $N_{\text{bins}} \in \{15, 20, 30, 40\}$ (resultados en **synch_entropy_sensitivity_nbins.csv**); el agregado **synch_entropy_assoc** y su d_z se reportan por cada valor de N_{bins} .
- **Bordes de bin.** Comunes a las dos condiciones de cada sujeto y red (rango combinado PSIL+MTP de ese sujeto-red) para que $H_N(\text{PSIL})$ y $H_N(\text{MTP})$ sean comparables sobre la misma “regla” intra-sujeto. Consecuencia operativa: los grids difieren entre sujetos y entre redes, por lo que el agregado **synch_entropy_assoc** (media sobre Default, Cont, SalVentAttn) mezcla bits computados sobre anchos de

bin distintos — el agregado es bien-definido como índice pareado within-subject pero su magnitud absoluta no es intercambiable entre sujetos.

- **Censura/scrubbing.** La métrica usa la *distribución* de $V_N(t)$, no su autocorrelación, así que es robusta a volúmenes descartados (cf. PsiloPrep §4.5): importa qué valores toma la sincronía, no el orden.
- **Orden de columnas del timeseries.** Se asume que las columnas siguen el orden de `atlas_labels` (Schaefer 1-200, luego Tian S2 1-32). El productor (PsiloPrep §4.5) ya garantiza este orden.
- **Corrección de comparaciones múltiples.** Carhart-Harris (2014, Fig. 6) usó **Bonferroni 0.05/9** sobre 9 redes con $N=15$ pre/post. Aquí se aplica **BH-FDR conjunta** sobre las 7 redes corticales Yeo (con Wilcoxon en paralelo; Lumley et al. 2002), coherente con el marco S0 (PFM exploratorio + triangulación inter-pipeline, no significancia estricta) — la sustitución es defendible bajo $N=5-6$. El CSV `synch_entropy_tests.csv` reporta ambas correcciones (`p_FDR` primaria + `p_bonf` columna paralela) para permitir triangulación con el paper original.
- **Pool FDR.** Independiente del de §6.1 (dominio dinámico vs topológico/señal estática). La métrica primaria agregada `synch_entropy_assoc` es direccional a priori $\rightarrow$ no entra en el pool de las 7 redes (Rothman, 1990, *Epidemiology* 1:43).
- **$N = 5-6$.** Test pareado con 4-5 g.l. $\rightarrow$ potencia muy limitada; resultados **exploratorios** (cf. callouts de §6.1).

3.2 3.2 Shannon entropy de la distribución de grados (Viol et al. 2017)

! FC entropy del histograma de pesos: sustituida por la entropía de la distribución de grados (Viol 2017)

La versión previa del chunk calculaba “FC entropy” como Shannon entropy del histograma de los valores z-Fisher de la matriz de conectividad estática. Esa métrica concreta **no se corresponde** con ninguna de las dos que Carhart-Harris et al. (2014, *Front Hum Neurosci*) etiquetan como “entropía” en la hipótesis entrópica — ambas **dinámicas** sobre series temporales:

1. *Shannon entropy de la varianza temporal de la sincronía intra-red* (metastability sobre 9 redes de Smith 2009; su Fig. 6) — implementada en §3.1.
2. *Entropy del repertorio/secuencia de motifs de conectividad* entre 4 nodos límbicos (HIP $\times$ ACC bilateral, 64 motifs posibles) a lo largo del tiempo — queda pendiente (cf. §3.1, Tagliazucchi et al. 2014).

Shannon sobre el histograma de pesos cuantifica **dispersión de fuerzas de conexión** — plausible, pero no es una entropía reconocida en la literatura de la EBH. **Se elimina esa versión** y, en su lugar, se calcula la métrica estática que **sí** tiene respaldo directo:

la **Shannon entropy de la distribución de grados del grafo umbralizado** ($S = -\sum_k P(k) \log_2 P(k)$, en bits; Viol et al., 2017, *Sci Rep* 7:7388 — artículo de la carpeta 5. **entropic_brain**), que Viol et al. presentan como su resultado principal y como medición *directa* de la entropía de la conectividad funcional consistente con la EBH (red regular $\rightarrow S = 0$; distribución de grados ancha $\rightarrow S$ alta; predicción a priori $\uparrow$ bajo el psicodélico). Esta métrica entra en el **pool BH-FDR de topología de §6.1** junto a las globales y nodales de §2.

Unidades. Se reporta en **bits** ($\log_2$) por convención estándar de teoría de la información. Viol et al. (2017, eq. 5) usan log natural y reportan en *nats*; los valores cuantitativos no son 1:1 con su Fig. 3/4 ($S_{\text{bits}} = S_{\text{nats}} / \ln 2 \approx 1.443 S_{\text{nats}}$). La inferencia (test pareado, d_z) es invariante a la base del logaritmo.

Threshold. Viol et al. (2017, Fig. 3) reportan S como función de 16 mean degrees ($\langle k \rangle \in \{24, \dots, 39\}$) y comparan cada original contra **iso-entropic randomized networks** (Maslov–Sneppen) para aislar el efecto distribucional respecto al estructural. Aquí se reporta S en un **único threshold primario** (densidad 0.20, mean degree 46) que entra al pool §6.1; como sensibilidad complementaria se añaden también las densidades 0.10 y 0.30 (mismas que §2.1 Tabla S1), reportadas en **degree_entropy_sensitivity.csv**. Bajo el marco S0 (PFM exploratorio + triangulación inter-pipeline) la cobertura parcial se considera suficiente; el barrido iso-entrópico de Viol no se implementa.

```
#=====
# §3 - Métricas avanzadas (entropía)
#   A) Entropía dinámica de la sincronía intra-red (Carhart-Harris 2014)
#   B) Entropía de la distribución de grados (Viol et al. 2017, Sci Rep)
# Itera sobre ACTIVE_DATASETS; la métrica A (synch entropy intra-red) se
# omite silenciosamente si no hay timeseries en disco para un dataset.
# Productores aguas abajo: §6.1 (degree_entropy entra al pool FDR de
# topología junto a las globales+nodales de §2); §6.6 (synch_entropy_assoc
# como variable cerebral en el análisis cerebro × experiencia).
# La modularidad con signo Q* (Rubinov & Sporns 2011) que esta sección
# implementó previamente se retiró en la ronda 1 de revisión (H2): la
# implementación reutilizaba la partición Louvain de §2.1 (sobre |w|
# umbralizado al 20 %), por lo que el Q* reportado era restringido a esa
# partición y no el Q* maximizado sobre signed que el paper introduce -
# Juan optó por dejar §3 centrada en las dos métricas con anclaje EBH directo.
#=====

library(tidyverse)
library(gghalves)

#   Funciones puras (dataset-agnósticas)
```

```

shannon_entropy_degree <- function(deg) {
  deg <- deg[is.finite(deg)]
  n <- length(deg)
  if (n == 0L) return(NA_real_)
  if (length(unique(deg)) == n && n > 3L)
    warning("`degree` parece continuo (todos los valores únicos) - ¿se está usando `strength`")
  deg <- round(deg)
  p <- as.numeric(table(deg)) / n
  p <- p[p > 0]
  -sum(p * log2(p))
}

# Varianza espacial intra-red por instante temporal. Si `zscore = TRUE`
# (primario, práctica estándar bajo Schaefer-200), z-score por ROI antes de
# la varianza espacial; si FALSE (sensibilidad), opera sobre BOLD tal cual
# lo proporciona el productor (las ROIs de alta amplitud dominan V_N(t)).
# El paper Carhart-Harris 2014 no especifica z-norm explícita; los dos
# variantes se reportan en paralelo (H4 ronda 1).
intra_network_variance <- function(ts_mat, roi_idx, zscore = TRUE) {
  if (length(roi_idx) < 2L) return(numeric(0))
  sub <- ts_mat[, roi_idx, drop = FALSE]
  if (zscore) {
    sub <- scale(sub)
    bad <- apply(sub, 2, function(col) !all(is.finite(col)))
    if (any(bad)) sub[, bad] <- NA
  }
  apply(sub, 1, function(row) {
    row <- row[is.finite(row)]
    if (length(row) > 1L) var(row) else NA_real_
  })
}

shannon_from_values <- function(v, breaks) {
  v <- v[is.finite(v)]
  if (length(v) == 0L) return(NA_real_)
  counts <- hist(v, breaks = breaks, plot = FALSE, include.lowest = TRUE)$counts
  if (sum(counts) == 0L) return(NA_real_)
  p <- counts / sum(counts); p <- p[p > 0]
  -sum(p * log2(p))
}

# Función principal por dataset

```

```

run_advanced_metrics <- function(cd, gd) {
  c1_key <- cd$conditions[1]; c2_key <- cd$conditions[2]
  C1     <- toupper(c1_key); C2     <- toupper(c2_key)

  RESULTS_DS <- file.path(PROJECT_ROOT, "results", cd$id)
  out_adv    <- file.path(RESULTS_DS, "stats/statistical_results/advanced")
  plot_dir_adv <- file.path(RESULTS_DS, "figures/stats/advanced")
  dir.create(out_adv,          showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_dir_adv,    showWarnings = FALSE, recursive = TRUE)

  common_subj <- cd$common_subjects
  atlas_labels <- cd$atlas_labels
  n_rois      <- cd$n_rois
  # Paleta de condiciones: magenta #C2185B (PSIL) + teal oscuro #00695C
  # (MTP). Se evita el par rojo/azul porque está reservado para
  # conectividad POS/NEG en S5 y S6.3; el morado #8E44AD también está
  # tomado por RATIO en el dotplot POSNEG, así que se descarta para
  # condición.
  cond_colors <- setNames(c("#C2185B", "#00695C"), c(C1, C2))

  get_network <- function(label)
    if (startsWith(label, "7Networks_")) strsplit(label, "_")[[1]][3] else "Subcortex"
  node_network <- vapply(atlas_labels, get_network, character(1))

  .load_csv_dir <- function(dir, cond_label) {
    purrr::map_dfr(list.files(dir, pattern = "\\\\.csv$", full.names = TRUE), function(f) {
      df <- read.csv(f, stringsAsFactors = FALSE)
      df$subject <- sub("_.*", "", basename(f))
      df$condition <- cond_label
      df
    })
  }

  global_subj <- bind_rows(
    .load_csv_dir(file.path(gd$graph_dir, "global", c1_key), C1),
    .load_csv_dir(file.path(gd$graph_dir, "global", c2_key), C2)) %>%
    group_by(subject, condition) %>%
    summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE)), .groups = "drop")
  nodal_data <- bind_rows(
    .load_csv_dir(file.path(gd$graph_dir, "nodal", c1_key), C1),
    .load_csv_dir(file.path(gd$graph_dir, "nodal", c2_key), C2))

```

```

# A) Shannon entropy de la distribución de grados (Viol 2017)
# Primario: degree del grafo umbralizado al 20 % (heredado de §2.1 nodal CSV).
# Sensibilidad: re-umbralizar las matrices a 10 % y 30 % y recomputar S.
degree_entropy_results <- NULL
if ("degree" %in% names(nodal_data)) {
  degree_entropy_results <- nodal_data %>%
    group_by(subject, condition) %>%
    summarise(degree_entropy = shannon_entropy_degree(degree), .groups = "drop")
write.csv(degree_entropy_results,
          file.path(out_adv, "degree_entropy.csv"), row.names = FALSE)
de_summary <- degree_entropy_results %>% group_by(condition) %>%
  summarise(mean_S = mean(degree_entropy, na.rm = TRUE), .groups = "drop")
cat(sprintf("\n=== Shannon entropy distribución de grados (Viol 2017) - %s ===\n", cd$id))
cat(sprintf(" Media S (bits): %s=%.4f %s=%.4f Δ(%s-%s)=%.4f\n",
            C1, de_summary$mean_S[de_summary$condition == C1],
            C2, de_summary$mean_S[de_summary$condition == C2],
            C1, C2,
            de_summary$mean_S[de_summary$condition == C1] -
            de_summary$mean_S[de_summary$condition == C2]))
de_wide <- degree_entropy_results %>%
  pivot_wider(names_from = condition, values_from = degree_entropy) %>%
  filter(!is.na(.data[[C1]]), !is.na(.data[[C2]]))
if (nrow(de_wide) >= 3L) {
  t_de_1s <- t.test(de_wide[[C1]], de_wide[[C2]], paired = TRUE,
                    alternative = "greater")
  cat(sprintf(" t pareado one-sided (%s>%s): t(%.0f)=%.3f, p=%.4f",
            C1, C2, t_de_1s$parameter, t_de_1s$statistic, t_de_1s$p.value))
  cat(" [informativo; el test del pool §6.1 es two-sided]\n")
}
cat(sprintf(" Predicción a priori (Viol 2017): %s > %s (distribución de grados más anchas)\n",
            C1, C2))

# Sensibilidad multi-threshold (H6 ronda 1): re-umbralizar las matrices
# signed originales a 10/20/30 % de densidad (mismas tres del barrido de
# §2.1) y recomputar S para cada (sujeto, condición, densidad). El valor
# a 0.20 debe coincidir (módulo redondeo) con el primario calculado desde
# el CSV nodal - sirve también como check de coherencia. Cobertura parcial
# del barrido 16-threshold de Viol et al. (2017, Fig. 3) bajo marco S0.
# `mat_to_igraph` (definida en §2.1) aplica `proportional_threshold`
# internamente, así que se llama con la matriz original y `prop = p`.
sensitivity_dens <- c(0.10, 0.20, 0.30)
degree_entropy_sensitivity <- map_dfr(sensitivity_dens, function(p) {

```



```

    map_dfr(common_subj, function(sub) {
      map_dfr(c(c1_key, c2_key), function(cond_key) {
        M      <- cd$mean_subject[[cond_key]][[sub]]
        Gthr <- mat_to_igraph(M, atlas_labels, prop = p)
        deg  <- igraph::degree(Gthr)
        tibble(subject = sub, condition = toupper(cond_key),
                density = p, degree_entropy = shannon_entropy_degree(deg))
      })
    })
  })
write.csv(degree_entropy_sensitivity,
          file.path(out_adv, "degree_entropy_sensitivity.csv"),
          row.names = FALSE)
de_sens_summary <- degree_entropy_sensitivity %>%
  group_by(density, condition) %>%
  summarise(mean_S = mean(degree_entropy, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = condition, values_from = mean_S) %>%
  mutate(delta = .data[[C1]] - .data[[C2]])
cat(sprintf("  Sensibilidad multi-threshold (H6) - mean S por densidad:\n"))
for (i in seq_len(nrow(de_sens_summary))) {
  r <- de_sens_summary[i, ]
  cat(sprintf("    d=%.2f  %s=%.4f  %s=%.4f  Δ=%.4f\n",
              r$density, C1, r[[C1]], C2, r[[C2]], r$delta))
}
} else {
  warning(sprintf("%s: columna `degree` ausente en nodal_data - se omite degree_entropy.",
                  cd$id))
}

#  B) Entropía dinámica de la sincronía intra-red (Carhart-Harris 2014)
N_BINS_SYNC      <- 20L
N_BINS_SWEEP     <- c(15L, 20L, 30L, 40L) # H5 ronda 1
YEO_CORTICAL     <- c("Vis", "SomMot", "DorsAttn", "SalVentAttn", "Limbic", "Cont", "De")
ASSOC_NETS       <- c("Default", "Cont", "SalVentAttn")
SENSORY_NETS     <- c("Vis", "SomMot")

load_ts_blocks <- function(sub, cond_dir) {
  sub_dir <- file.path(cd$conn_dir, cond_dir, sub)
  ts_files <- list.files(sub_dir, pattern = "_timeseries\\.csv$", full.names = TRUE)
  if (length(ts_files) == 0L) return(list())
  lapply(ts_files, function(f) {
    m <- as.matrix(read.csv(f, header = FALSE))
  })
}

```



```

    if (ncol(m) != n_rois && nrow(m) == n_rois) m <- t(m)
    if (ncol(m) != n_rois)
      stop(sprintf("%s/%s: %s dim %d×%d - ninguna == %d ROIs",
                    sub, cond_dir, basename(f), nrow(m), ncol(m), n_rois))
    m
  })
}

paired_test_tbl <- function(df, value_col, unit_label) {
  w <- df %>% transmute(subject, condition, val = .data[[value_col]]) %>%
    pivot_wider(names_from = condition, values_from = val) %>%
    filter(is.finite(.data[[C1]]), is.finite(.data[[C2]])) %>% arrange(subject)
  base <- tibble(unit = unit_label, n_pairs = nrow(w),
                 mean_C1 = if (nrow(w)) mean(w[[C1]]) else NA_real_,
                 mean_C2 = if (nrow(w)) mean(w[[C2]]) else NA_real_,
                 diff = if (nrow(w)) mean(w[[C1]] - w[[C2]]) else NA_real_,
                 cohen_dz = NA_real_, p_onesided = NA_real_,
                 p_twosided = NA_real_, p_wilcox = NA_real_)
  if (nrow(w) < 3L || length(unique(w[[C1]] - w[[C2]])) < 2L) return(base)
  d <- w[[C1]] - w[[C2]]
  base$cohen_dz <- mean(d) / sd(d)
  base$p_onesided <- t.test(w[[C1]], w[[C2]], paired = TRUE, alternative = "greater")$p.value
  base$p_twosided <- t.test(w[[C1]], w[[C2]], paired = TRUE)$p.value
  base$p_wilcox <- suppressWarnings(
    wilcox.test(w[[C1]], w[[C2]], paired = TRUE, alternative = "greater",
                exact = FALSE)$p.value)
  base
}

message(sprintf("Calculando entropía de la sincronía intra-red - %s...", cd$id))
nets_present <- intersect(c(YEO_CORTICAL, "Subcortex"), unique(node_network))

vN_by_sc <- setNames(map(common_subj, function(s) {
  setNames(map(c(c1_key, c2_key), function(cdir) {
    runs <- load_ts_blocks(s, cdir)
    if (length(runs) == 0L) { warning(s, "/", cdir, ": sin timeseries"); return(NULL) }
    setNames(map(nets_present, function(net)
      unlist(map(runs, ~intra_network_variance(.x, which(node_network == net), zscore = TRUE)
      nets_present)
    )), c(C1, C2))
  })), common_subj)

```

```

# H4 ronda 1: cálculo paralelo SIN z-score (sensibilidad sobre BOLD crudo).
vN_by_sc_raw <- setNames(map(common_subj, function(s) {
  setNames(map(c(c1_key, c2_key), function(cdir) {
    runs <- load_ts_blocks(s, cdir)
    if (length(runs) == 0L) return(NULL)
    setNames(map(nets_present, function(net)
      unlist(map(runs, ~intra_network_variance(.x, which(node_network == net), zscore = FALSE)
    nets_present)
  )), c(C1, C2))
}), common_subj)

synch_long <- map_dfr(common_subj, function(s) {
  sc      <- vN_by_sc[[s]]
  sc_raw  <- vN_by_sc_raw[[s]]
  if (is.null(sc[[C1]]) || is.null(sc[[C2]])) return(NULL)
  map_dfr(nets_present, function(net) {
    n_roi  <- length(which(node_network == net))
    # z-score primario: bordes pooled sobre PSIL+MTP del mismo sujeto-red
    pooled <- c(sc[[C1]][[net]], sc[[C2]][[net]])
    pooled <- pooled[is.finite(pooled)]
    if (length(pooled) < N_BINS_SYNCH * 2L || diff(range(pooled)) == 0)
      return(tibble(subject = s, network = net, condition = c(C1, C2),
        n_rois = n_roi, metastability = NA_real_,
        synch_entropy = NA_real_, synch_entropy_raw = NA_real_))
    breaks <- seq(min(pooled), max(pooled), length.out = N_BINS_SYNCH + 1L)
    # raw - bordes pooled sobre PSIL+MTP del mismo sujeto-red, sin z-score
    pooled_raw <- c(sc_raw[[C1]][[net]], sc_raw[[C2]][[net]])
    pooled_raw <- pooled_raw[is.finite(pooled_raw)]
    breaks_raw <- if (length(pooled_raw) >= N_BINS_SYNCH * 2L &&
      diff(range(pooled_raw)) > 0)
      seq(min(pooled_raw), max(pooled_raw), length.out = N_BINS_SYNCH + 1L) else NULL
    map_dfr(c(C1, C2), function(cond) {
      v      <- sc[[cond]][[net]];      v      <- v[is.finite(v)]
      v_raw  <- sc_raw[[cond]][[net]];  v_raw  <- v_raw[is.finite(v_raw)]
      tibble(subject = s, network = net, condition = cond, n_rois = n_roi,
        metastability = if (length(v) > 1L) sd(v) else NA_real_,
        synch_entropy = shannon_from_values(v, breaks),
        synch_entropy_raw = if (!is.null(breaks_raw))
          shannon_from_values(v_raw, breaks_raw)
        else NA_real_)
    })
  })
})

```

```

})
write.csv(synch_long, file.path(out_adv, "synch_entropy_per_network.csv"), row.names = FALSE)

synch_tests <- tibble()
synch_summary <- NULL

if (nrow(synch_long) > 0) {
  synch_summary <- synch_long %>% group_by(subject, condition) %>%
    summarise(
      synch_entropy_assoc = mean(synch_entropy[network %in% ASSOC_NETS],
                                na.rm = TRUE),
      synch_entropy_sensory = mean(synch_entropy[network %in% SENSORY_NETS],
                                   na.rm = TRUE),
      synch_entropy_cortical_mean = mean(synch_entropy[network %in% YEO_CORTICAL],
                                          na.rm = TRUE),
      synch_entropy_subcortex = mean(synch_entropy[network == "Subcortex"],
                                     na.rm = TRUE),
      # H4 ronda 1: agregados sobre la versión raw (sensibilidad)
      synch_entropy_assoc_raw = mean(synch_entropy_raw[network %in% ASSOC_NETS],
                                     na.rm = TRUE),
      synch_entropy_sensory_raw = mean(synch_entropy_raw[network %in% SENSORY_NETS],
                                       na.rm = TRUE),
      .groups = "drop")
  write.csv(synch_summary, file.path(out_adv, "synch_entropy_summary.csv"),
            row.names = FALSE)

  # H3 ronda 1: BH-FDR primaria + Bonferroni en columna paralela.
  tests_per_net <- map_dfr(YEO_CORTICAL, function(net)
    paired_test_tbl(synch_long %>% filter(network == net), "synch_entropy", net)) %>%
    mutate(p_FDR = p.adjust(p_twosided, method = "fdr"),
           p_bonf = p.adjust(p_twosided, method = "bonferroni"),
           significant_FDR = is.finite(p_FDR) & p_FDR < 0.05,
           pool = "Yeo-7 cortical (BH-FDR primaria, Bonferroni paralela)")
  test_subcx <- paired_test_tbl(synch_long %>% filter(network == "Subcortex"),
                                "synch_entropy", "Subcortex") %>%
    mutate(p_FDR = NA_real_, p_bonf = NA_real_,
           significant_FDR = NA, pool = "independiente (subcortical)")
  tests_agg <- bind_rows(
    paired_test_tbl(synch_summary, "synch_entropy_assoc",
                   "ASSOC (Default+Cont+SalVentAttn) [PRIMARIO 1-sided]"),
    paired_test_tbl(synch_summary, "synch_entropy_sensory",
                   "SENSORY (Vis+SomMot) [control negativo]"),

```

```

paired_test_tbl(synch_summary, "synch_entropy_cortical_mean",
                "Cortical-mean (7 Yeo)"),
paired_test_tbl(synch_summary, "synch_entropy_subcortex", "Subcortex"),
# H4 ronda 1: control raw del agregado primario y sensory
paired_test_tbl(synch_summary, "synch_entropy_assoc_raw",
                "ASSOC raw (sin z-score, H4 sensibilidad)"),
paired_test_tbl(synch_summary, "synch_entropy_sensory_raw",
                "SENSORY raw (sin z-score, H4 sensibilidad)")
) %>% mutate(p_FDR = NA_real_, p_bonf = NA_real_, significant_FDR = NA,
            pool = "agregados - a priori direccional o sensibilidad, fuera del pool de ")
synch_tests <- bind_rows(tests_per_net, test_subcx, tests_agg)
write.csv(synch_tests, file.path(out_adv, "synch_entropy_tests.csv"), row.names = FALSE)

# --- Reporte en consola -----
cat("\n=====\\n")
cat(sprintf(" §3.1 - Entropía dinámica sincronía intra-red - %s (Carhart-Harris 2014)\\n",
            cd$id))
cat(sprintf(" N_BINS_SYNCH=%d | redes corticales=%d | sujetos pareados=%d\\n",
            N_BINS_SYNCH, length(YEO_CORTICAL),
            nrow(distinct(synch_summary, subject))))
cat("-----\\n")
prim <- tests_agg %>% filter(grepl("PRIMARIO", unit))
cat(sprintf(" PRIMARIA synch_entropy_assoc: %s=%.4f %s=%.4f Δ=%.4f dz=%.3f\\n",
            C1, prim$mean_C1, C2, prim$mean_C2, prim$diff, prim$cohen_dz))
cat(sprintf(" t one-sided (%s>%s) p=%.4f | two-sided p=%.4f | Wilcoxon p=%.4f\\n",
            C1, C2, prim$p_onesided, prim$p_twosided, prim$p_wilcox))
ctrl <- tests_agg %>% filter(grepl("control", unit))
cat(sprintf(" CONTROL synch_entropy_sensory: Δ=%.4f two-sided p=%.4f (predicción: n",
            ctrl$diff, ctrl$p_twosided))
cat("-----\\n")
cat(sprintf(" Por red Yeo-7 (Δ = %s-%s; p_FDR = BH conjunta, two-sided):\\n", C1, C2))
for (i in seq_len(nrow(tests_per_net))) {
  r <- tests_per_net[i, ]
  cat(sprintf(" %-12s Δ=%.4f dz=%.2f p=%.4f p_FDR=%.4f %s\\n",
              r$unit, r$diff, r$cohen_dz, r$p_twosided, r$p_FDR,
              ifelse(isTRUE(r$significant_FDR), "*", "")))
}
cat(sprintf(" %-12s Δ=%.4f dz=%.2f p=%.4f (subcortical, fuera del pool)\\n",
            test_subcx$unit, test_subcx$diff, test_subcx$cohen_dz, test_subcx$p_twosided))
cat(" NOTA: pool FDR independiente del de §6.1 (dominio dinámico).\\n")
cat("=====\\n")

```

```

# --- H5 ronda 1: sweep de sensibilidad sobre N_BINS -----
# Reutiliza vN_by_sc (z-score primario); recalcula bordes y entropía por
# (sujeto, condición, red ASSOC) para cada N_BINS, agrega sobre ASSOC_NETS
# y reporta d_z + p one-sided pareado.
synch_entropy_sweep_nbins <- map_dfr(N_BINS_SWEEP, function(nb) {
  sl_nb <- map_dfr(common_subj, function(s) {
    sc <- vN_by_sc[[s]]
    if (is.null(sc[[C1]]) || is.null(sc[[C2]])) return(NULL)
    map_dfr(intersect(nets_present, ASSOC_NETS), function(net) {
      pooled <- c(sc[[C1]][[net]], sc[[C2]][[net]])
      pooled <- pooled[is.finite(pooled)]
      if (length(pooled) < nb * 2L || diff(range(pooled)) == 0) return(NULL)
      breaks <- seq(min(pooled), max(pooled), length.out = nb + 1L)
      map_dfr(c(C1, C2), function(cond) {
        v <- sc[[cond]][[net]]; v <- v[is.finite(v)]
        tibble(subject = s, network = net, condition = cond,
                synch_entropy = shannon_from_values(v, breaks))
      })
    })
  })
  if (nrow(sl_nb) == 0) return(NULL)
  assoc_agg <- sl_nb %>% group_by(subject, condition) %>%
    summarise(synch_entropy_assoc = mean(synch_entropy, na.rm = TRUE),
              .groups = "drop")
  test <- paired_test_tbl(assoc_agg, "synch_entropy_assoc",
                          sprintf("N_BINS=%d", nb))
  test %>% transmute(n_bins = nb, n_pairs, cohen_dz,
                    p_onesided, p_twosided, p_wilcox,
                    mean_C1, mean_C2, diff)
})
write.csv(synch_entropy_sweep_nbins,
          file.path(out_adv, "synch_entropy_sensitivity_nbins.csv"),
          row.names = FALSE)
cat(sprintf("\n Sensibilidad N_BINS (H5) - agregado ASSOC pareado one-sided:\n"))
for (i in seq_len(nrow(synch_entropy_sweep_nbins))) {
  r <- synch_entropy_sweep_nbins[i, ]
  cat(sprintf("    N_BINS=%2d  dz=%+.3f  p_1-sided=%.4f  p_2-sided=%.4f\n",
              r$n_bins, r$cohen_dz, r$p_onesided, r$p_twosided))
}
cat("\n")

# --- Figuras -----

```

```

df_prim <- synch_summary %>%
  transmute(subject, condition, value = synch_entropy_assoc) %>% filter(is.finite(value))
p_synch_assoc <- ggplot(df_prim, aes(condition, value, fill = condition)) +
  ggthemes::geom_half_violin(side = "l", alpha = 0.45, trim = FALSE) +
  geom_boxplot(width = 0.14, outlier.shape = NA, alpha = 0.85) +
  geom_jitter(width = 0.07, size = 2.5, alpha = 0.85) +
  geom_line(aes(group = subject), color = "grey55", linewidth = 0.4, alpha = 0.6) +
  scale_fill_manual(values = cond_colors) +
  theme_minimal(base_size = 12) + theme(legend.position = "none") +
  labs(title = "[GRUPO · Dinámico] Entropía sincronía intra-red (redes asociación)",
        subtitle = "Carhart-Harris et al. 2014 (Front Hum Neurosci)",
        x = "Condición", y = "H de la sincronía intra-red (bits)")
ggsave(file.path(plot_dir_adv, "adv_raincloud_synch_entropy_assoc.png"),
        p_synch_assoc, width = 7, height = 5, dpi = 300)

df_net <- synch_long %>% filter(is.finite(synch_entropy)) %>%
  mutate(network = factor(network, levels = c(YEO_CORTICAL, "Subcortex")))
p_synch_net <- ggplot(df_net, aes(condition, synch_entropy, fill = condition)) +
  geom_boxplot(width = 0.5, outlier.shape = NA, alpha = 0.85) +
  geom_line(aes(group = subject), color = "grey60", linewidth = 0.3, alpha = 0.5) +
  geom_jitter(width = 0.1, size = 1.6, alpha = 0.7) +
  facet_wrap(~ network, nrow = 2, scales = "free_y") +
  scale_fill_manual(values = cond_colors) +
  theme_minimal(base_size = 11) + theme(legend.position = "none") +
  labs(title = sprintf("[GRUPO · Dinámico] Entropía sincronía intra-red por red | %s",
                        C1, C2),
        subtitle = sprintf("Carhart-Harris 2014: ↑ en Default/Cont/SalVentAttn, nulo en %s",
                            C1, C2),
        x = "Condición", y = "H de la sincronía intra-red (bits)")
ggsave(file.path(plot_dir_adv, "adv_facet_synch_entropy_by_network.png"),
        p_synch_net, width = 11, height = 6, dpi = 300)

message(sprintf("OK §3 - entropía dinámica completada - %s", cd$id))
} else {
  message(sprintf("§3 - sin timeseries para entropía dinámica - %s (omitiendo §3.1 synch en %s)",
                  cd$id, cd$network))
}

list(
  degree_entropy_results = degree_entropy_results,
  synch_tests = synch_tests,
  synch_summary = synch_summary
)

```

```

}

# Iterar sobre datasets activos
ADVANCED_METRICS <- setNames(
  lapply(ACTIVE_DATASETS, function(ds)
    run_advanced_metrics(CONNECTOMES[[ds]], GRAPHS[[ds]])),
  ACTIVE_DATASETS)

```

4 4 Control de pipelines de denoising — sensibilidad de las métricas topológicas

§4 es un **control técnico** de la cadena: cuantifica en qué medida los pipelines de denoising producen valores similares de cuatro métricas topológicas (`mean_strength`, `global_efficiency`, `clustering`, `modularity`) sobre los mismos sujetos $\times$ condiciones. La pregunta es metodológica —¿coinciden los pipelines en el *valor* del topograma?—, no inferencial; la convergencia del **contraste** PSIL vs MTP entre pipelines (criterio de credibilidad declarado en el callout-important S0 de §6.1) se operacionaliza en §6.2 sobre el pool topología completo. Las dos sub-secciones leen la misma lista de pipelines pero responden a preguntas distintas: §4 mide acuerdo entre lecturas, §6.2 mide reproducibilidad del efecto. Por eso §4 se reporta sin corrección por comparaciones múltiples (rol **descriptivo** / **control técnico**); el peso inferencial está en §6.1 (pool topología, BH-FDR within-family) y §6.2 (convergencia 6/7 del contraste).

La sección compara el pipeline **primario** de cada dataset —ya calculado en §2.1— contra sus pipelines paralelos. Para **ds006072** (primario TEDANA+aCompCor, fijado en PsiloPrep §5.2 / PsiloPrep §3.13), los seis pipelines de control cubren los tres ejes de triangulación pre-registrados: ME-ICA (TEDANA puro, TEDANA+24P), single-echo no-GSR (OC+Siegel-like, OC+24P, OC+aCompCor) y single-echo con GSR (OC+36P+GSR). Para **ds003059**, AsIs vs AsIs+aCompCor (no-GSR) vs AsIs+GSR (GSR); al tratarse de un dataset derivado (single-echo) el eje ME-ICA no aplica y la triangulación se reduce a los ejes (b) y (c). Si las correlaciones inter-pipeline son altas tanto dentro de la familia no-GSR como frente a GSR, las lecturas del topograma son técnicamente reproducibles y la decisión de descartar correlaciones negativas en `proportional_threshold()` es válida para todos los pipelines (Murphy y Fox, 2017; Ciric et al., 2017; Behzadi et al., 2007; Rubinov y Sporns, 2011; Sinclair et al., 2015); las cotas de clasificación —«robusta» $r > 0.90$, «sensible» $r < 0.70$ — siguen la rúbrica operativa de Cicchetti (1994). La comparación canónica TEDANA+aCompCor (primario) OC+aCompCor (regresores idénticos, input distinto) cuantifica el aporte neto de ME-ICA. Como en §1–§2, el chunk itera sobre los datasets activos (`GRAPHS` / `CONNECTOMES`) y escribe en `derivatives/<ds>/graphs/<pipeline>/` y `results/<ds>/figures/stats/robust/`. La traducción de estos acuerdos técnicos al criterio S0 de convergencia del contraste se hace en §6.2.


```

#=====
# FASE 4 - Control de pipelines de denoising - BLOQUE DATASET-AGNÓSTICO
#=====
# Sensibilidad de las métricas topológicas (mean_strength, global_efficiency,
# clustering, modularity) a la estrategia de denoising. Para cada dataset
# activo compara el pipeline PRIMARIO (CONNECTOMES[[ds]]$pipeline, ya calculado
# en §2.1) contra los pipelines de control:
#   ds006072 → TEDANA+aCompCor vs TEDANA (ME-ICA puro), TEDANA+24P,
#               OC+Siegel, OC+24P, OC+aCompCor (no-GSR) y OC+36P+GSR (GSR)
#   ds003059 → AsIs                vs AsIs+aCompCor (no-GSR) y AsIs+GSR (GSR)
#
# Triangulación (cf. PsiloPrep §4.3 / LSDPrep @sec-ds003059-triangulacion):
#   (a) primario vs alternativas ME-ICA (TEDANA, TEDANA+24P) → cuanto
#       captura ME-ICA puro vs su combinación con nuisance externo.
#   (b) primario vs alternativas single-echo no-GSR → robustez frente al
#       método de regresión de confounds. La comparación canónica
#       TEDANA+aCompCor OC+aCompCor (regresores idénticos, input
#       distinto) aísla el aporte neto de ME-ICA.
#   (c) primario vs alternativa GSR → robustez frente a GSR específicamente.
# §4 = control TÉCNICO (¿los pipelines coinciden en el VALOR del topograma?).
# Si r = 1 en todas las comparaciones, las lecturas son reproducibles y el
# descarte de correlaciones negativas en proportional_threshold() es coherente
# entre pipelines. La convergencia del CONTRASTE PSIL vs MTP entre pipelines
# (criterio S0 = 6/7) se evalúa en §6.2, no aquí.
# Refs: Murphy & Fox (2017); Ciric et al. (2017); Behzadi et al. (2007);
# Rubinov & Sporns (2011); Sinclair et al. (2015); Cicchetti (1994) para
# las cotas operativas de status_str().
#
# Reutiliza funciones definidas en §1.1/§2.1 (load_subject_matrices,
# mat_to_igraph, mat_to_igraph_full, calc_global_metrics, proportional_threshold,
# to_distance) y las listas CONNECTOMES (§1.1) y GRAPHS (§2.1). No produce
# estado consumido aguas abajo (sólo CSVs + figura por dataset) → sin shim.
#=====

library(tidyverse)
library(ggplot2)
library(patchwork)

PROJECT_ROOT <- "D:/ProfessionalProyectos/PsyConn"
stopifnot("$4 requiere CONNECTOMES y GRAPHS (ejecutar §1.1 y §2.1 antes)" =
  exists("CONNECTOMES") && exists("GRAPHS"))

```



```

metrics_list <- c("mean_strength", "global_efficiency", "clustering", "modularity")

# Pipelines de CONTROL por dataset (el primario sale de CONNECTOMES[[ds]]$pipeline)
CONTROL_PIPELINES_BY_DS <- list(
  ds006072 = tibble::tribble(
    ~name,      ~conn_dir,
    "TEDANA",    file.path(PROJECT_ROOT, "derivatives/ds006072/conn/TEDANA"),
    "TEDANA+24P", file.path(PROJECT_ROOT, "derivatives/ds006072/conn/TEDANA_24P"),
    "OC+Siegel", file.path(PROJECT_ROOT, "derivatives/ds006072/conn/OC_siegel"),
    "OC+24P",    file.path(PROJECT_ROOT, "derivatives/ds006072/conn/OC_24P"),
    "OC+aCompCor", file.path(PROJECT_ROOT, "derivatives/ds006072/conn/OC_aCompCor"),
    "OC+36P+GSR", file.path(PROJECT_ROOT, "derivatives/ds006072/conn/OC_36P_GSR"),
  ),
  ds003059 = tibble::tribble(
    ~name,      ~conn_dir,
    "AsIs+aCompCor", file.path(PROJECT_ROOT, "derivatives/ds003059/conn/AsIs_aCompCor"),
    "AsIs+GSR",    file.path(PROJECT_ROOT, "derivatives/ds003059/conn/AsIs_GSR"),
  )
)

# Rúbrica de acuerdo entre pipelines (Cicchetti, 1994: "excellent" >0.90,
# "good" 0.75-0.90, "fair" 0.60-0.74, "poor" <0.60). Conservamos tres
# bandas operativas que comprimen good+fair → "moderada"; el corte
# 0.70 se queda con la cota inferior de "good" para ser conservadores.
status_str <- function(r) {
  if (is.na(r))      "NA"
  else if (r > 0.90) "robusta"
  else if (r < 0.70) "sensible"
  else               "moderada"
}

#=====
# Control de pipelines para un dataset
#=====
run_pipeline_control <- function(ds) {
  cd <- CONNECTOMES[[ds]]
  gd <- GRAPHS[[ds]]
  ctrl <- CONTROL_PIPELINES_BY_DS[[ds]]
  if (is.null(cd) || is.null(gd) || is.null(ctrl)) {
    warning("$4: falta config/datos para ", ds, " - omitido."); return(invisible(NULL))
  }
  primary <- cd$pipeline

```

```

c1 <- cd$conditions[1]; c2 <- cd$conditions[2]
GRAPH_DIR <- gd$graph_dir
plot_dir <- file.path(PROJECT_ROOT, "results", ds, "figures/stats/robust")
dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(GRAPH_DIR, showWarnings = FALSE, recursive = TRUE)
# Paleta de condiciones: magenta #C2185B (PSIL) + teal oscuro #00695C
# (MTP) - rojo/azul están
# reservados para POS/NEG en §5 y §6.3.
cond_colors <- setNames(c("#C2185B", "#00695C"), c(c1, c2))
cond_labels <- cd$cond_pretty

cat(sprintf("\n##### §4 Control de pipelines - %s (primario: %s) #####\n",
           ds, primary))

# --- 1. Métricas de grafo para cada pipeline de control -----
compute_metrics_for_pipeline <- function(name, conn_dir) {
  d1 <- file.path(conn_dir, c1); d2 <- file.path(conn_dir, c2)
  if (!dir.exists(d1) || !dir.exists(d2)) {
    warning(sprintf(" %s | pipeline %s: directorios %s / %s no encontrados - saltando",
                    ds, name, basename(d1), basename(d2)))
    return(NULL)
  }
  s1 <- load_subject_matrices(d1, sprintf("_mean_%s\\.csv$", c1))
  s2 <- load_subject_matrices(d2, sprintf("_mean_%s\\.csv$", c2))
  subjs <- intersect(intersect(names(s1), names(s2)), cd$common_subjects)
  if (length(subjs) == 0) {
    warning(sprintf(" %s | pipeline %s: sin sujetos comunes con %s", ds, name, primary))
    return(NULL)
  }
  message(" ", name, ": calculando sobre ", length(subjs), " sujetos comunes con ", primary)
  map_dfr(subjs, function(subj) {
    map_dfr(c(c1, c2), function(cond) {
      mat <- if (cond == c1) s1[[subj]] else s2[[subj]]
      G_full <- mat_to_igraph_full(mat, cd$atlas_labels)
      G_thr <- mat_to_igraph(mat, cd$atlas_labels)
      m <- calc_global_metrics(G_full, G_thr, mat)
      tibble(pipeline = name, subject = subj, condition = cond,
              mean_strength = m$mean_strength, global_efficiency = m$global_efficiency,
              clustering = m$clustering, modularity = m$modularity)
    })
  })
}

```

```

message("  Calculando métricas para ", nrow(ctrl), " pipelines de control (puede tardar)..")
metrics_secondary <- map2_dfr(ctrl$name, ctrl$conn_dir, compute_metrics_for_pipeline)
if (nrow(metrics_secondary) == 0) {
  warning(" ", ds, ": ningún pipeline de control produjo métricas - sección omitida para e
  return(invisible(NULL))
}
message("    métricas secundarias: ", nrow(metrics_secondary), " filas, ",
        n_distinct(metrics_secondary$pipeline), " pipelines")

# --- 2. Métricas del primario desde los CSVs de §2.1 -----
# Fail-soft: si los CSVs no están en disco (p. ej. §2.1 no se ha
# re-ejecutado tras un cambio de primario o de dataset), avisamos sin
# detener la ejecución global de PsyConn; el `pivot_wider` siguiente
# generará NA y cor.test/wilcox.test devolverán NA por par.
primary_dirs <- file.path(GRAPH_DIR, "global", c(c1, c2))
primary_csvs <- unlist(lapply(primary_dirs,
                             \ (d) list.files(d, pattern = "_global_metrics\\.csv$",
                             full.names = TRUE)))

if (length(primary_csvs) == 0) {
  warning(sprintf(" §4 (%s): no se encontraron CSVs de métricas del primario %s en %s - ej
                  ds, primary, file.path(GRAPH_DIR, "global")))
}
subjs_sec <- unique(metrics_secondary$subject)
metrics_primary <- map_dfr(subjs_sec, function(subj) {
  map_dfr(c(c1, c2), function(cond) {
    f <- file.path(GRAPH_DIR, "global", cond, paste0(subj, "_global_metrics.csv"))
    if (!file.exists(f)) return(NULL)
    m <- read.csv(f)
    tibble(pipeline = primary, subject = subj, condition = cond,
           mean_strength = m$mean_strength[1], global_efficiency = m$global_efficiency[1],
           clustering = m$clustering[1], modularity = m$modularity[1])
  })
})

# --- 3. Tabla combinada wide -----
combined <- bind_rows(metrics_primary, metrics_secondary) %>%
  pivot_wider(id_cols = c(subject, condition), names_from = pipeline,
              values_from = all_of(metrics_list), names_glue = "{.value}__{pipeline}")
write.csv(combined, file.path(GRAPH_DIR, "graph_metrics_pipeline_comparison.csv"),
          row.names = FALSE)

# --- 4. Comparación formal: primario vs cada control -----

```

```

# r y CI95 = acuerdo entre valores de la métrica (Pearson sobre subj×cond);
# p_bias = Wilcoxon pareado de (xT - xP) 0, i.e. bias sistemático del
# control respecto al primario sobre el mismo (sujeto, condición)
# - NO es el test del contraste PSIL vs MTP (eso vive en §6 y
# se generaliza a todos los pipelines en §6.2);
# med_diff% = mediana de la diferencia relativa (más robusta que la media
# cuando |xT| está cerca de 0 en alguna métrica).
cat(sprintf("\n == Control de pipelines (%s): %s vs cada alternativa ==\n", ds, primary))
cat(" r = acuerdo entre valores | CI95 = IC de r (Fisher z) | p_bias = Wilcoxon (sesgo en")
cat(sprintf(" %-15s %-22s %6s %17s %7s %8s %9s\n",
            "Pipeline", "Métrica", "r", "CI95(r)", "p_cor", "p_bias", "med_diff%"))
cat(" ", strrep("-", 92), "\n", sep = "")
corr_summary <- map_dfr(ctrl$name, function(pipe) {
  map_dfr(metrics_list, function(mm) {
    col_T <- paste0(mm, "__", primary); col_P <- paste0(mm, "__", pipe)
    if (!(col_T %in% names(combined)) || !(col_P %in% names(combined))) return(NULL)
    xT <- combined[[col_T]]; xP <- combined[[col_P]]
    ct <- tryCatch(cor.test(xT, xP, method = "pearson"), error = \ (e) NULL)
    wt <- tryCatch(wilcox.test(xT, xP, paired = TRUE, exact = FALSE), error = \ (e) NULL)
    r_val <- if (!is.null(ct)) ct$estimate else NA_real_
    r_lo <- if (!is.null(ct) && !is.null(ct$conf.int)) ct$conf.int[1] else NA_real_
    r_hi <- if (!is.null(ct) && !is.null(ct$conf.int)) ct$conf.int[2] else NA_real_
    p_cor <- if (!is.null(ct)) ct$p.value else NA_real_
    p_bias <- if (!is.null(wt)) wt$p.value else NA_real_
    # mediana de la diferencia relativa (más estable que la media con
    # métricas que pueden tomar valores 0 en algún sujeto, p. ej. modularity).
    med_diff_pct <- median((xP - xT) / (abs(xT) + 1e-9) * 100, na.rm = TRUE)
    ci_str <- if (is.na(r_lo)) " - " else sprintf("[%5.2f, %5.2f]", r_lo, r_hi)
    cat(sprintf(" %-15s %-22s %6.3f %17s %7.4f %8.4f %8.2f%%\n",
                pipe, mm, r_val, ci_str, p_cor, p_bias, med_diff_pct))
    tibble(dataset = ds, pipeline = pipe, has_gsr = ctrl$has_gsr[ctrl$name == pipe],
            metric = mm, r = r_val, r_ci_low = r_lo, r_ci_high = r_hi,
            p_cor = p_cor, p_bias = p_bias, median_diff_pct = med_diff_pct)
  })
})
write.csv(corr_summary, file.path(GRAPH_DIR, "pipeline_comparison_summary.csv"),
          row.names = FALSE)

# --- 5. Figura: scatter primario vs cada control (filas × 4 columnas) ---
make_scatter <- function(pipe, metric) {
  col_T <- paste0(metric, "__", primary); col_P <- paste0(metric, "__", pipe)
  if (!(col_T %in% names(combined)) || !(col_P %in% names(combined))) return(NULL)

```

```

ct <- tryCatch(cor.test(combined[[col_T]], combined[[col_P]], method = "pearson"),
               error = \(e) NULL)
r_label <- if (!is.null(ct)) sprintf("r = %.3f, p = %.4f", ct$estimate, ct$p.value) else
has_gsr <- ctrl$has_gsr[ctrl$name == pipe]
pipe_tag <- if (has_gsr) paste0(pipe, " [GSR]") else paste0(pipe, " [no-GSR]")
ggplot(combined, aes(x = .data[[col_T]], y = .data[[col_P]], color = condition)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey60", linewidth = 1) +
  geom_point(size = 2.5, alpha = 0.8) +
  scale_color_manual(values = cond_colors, labels = cond_labels) +
  annotate("text", x = -Inf, y = Inf, label = r_label, hjust = -0.05, vjust = 1.5,
           size = 3.0, color = "grey30") +
  labs(title = paste0(metric, " - ", pipe_tag),
       x = paste0(metric, " (", primary, ")"), y = paste0(metric, " (", pipe, ")"),
       color = NULL) +
  theme_minimal(base_size = 9) +
  theme(legend.position = "bottom", plot.title = element_text(size = 9, face = "bold"))
}

plots <- list()
for (pipe in ctrl$name) for (mm in metrics_list)
  plots[[length(plots) + 1L]] <- make_scatter(pipe, mm)
plots <- plots[!vapply(plots, is.null, logical(1))]
if (length(plots) > 0) {
  fig <- wrap_plots(plots, ncol = length(metrics_list)) +
    plot_annotation(
      title = sprintf("Control de pipelines (%s): %s vs alternativas", ds, primary),
      subtitle = paste(
        sprintf("Filas = %s.", paste(sprintf("%s (%s)", ctrl$name,
        ifelse(ctrl$has_gsr, "GSR", "no-GSR")), collapse = ", ")),
        "Columnas = mean_strength, global_efficiency, clustering, modularity.",
        "Línea discontinua = identidad. Punto = sujeto × condición.", sep = "\n"),
      theme = theme(plot.title = element_text(size = 13, face = "bold"),
                    plot.subtitle = element_text(size = 9))
    ) & theme(legend.position = "bottom")
  fig_h <- max(5, 3 + 2.7 * nrow(ctrl))
  # `+` es válido como nombre de fichero pero se reinterpreta como espacio
  # en URLs y rompe regex/glob sin escapar - sanitizamos sólo el filename;
  # los títulos y labels siguen mostrando el nombre con `+`.
  safe_primary <- gsub("\\+", "_", primary)
  ggsave(file.path(plot_dir, sprintf("sensitivity_pipelines_%s_vs_others.png", safe_primary)),
          fig, width = 14, height = fig_h, dpi = 300)
  message(" Guardado: sensitivity_pipelines_", safe_primary, "_vs_others.png")
}

```

```

# --- 6. Interpretación automática -----
cat("\n === Interpretación automática ===\n")
for (i in seq_len(nrow(corr_summary))) {
  cat(sprintf(" %-15s %-9s %-22s → %-12s (r=%.3f, med_diff=%.2f%%)\n",
    corr_summary$pipeline[i],
    if (corr_summary$has_gsr[i]) "[GSR]" else "[no-GSR]",
    corr_summary$metric[i], status_str(corr_summary$r[i]),
    corr_summary$r[i], corr_summary$median_diff_pct[i]))
}
cat(sprintf("\n --- Sensibilidad a GSR (%s vs alternativa con GSR) ---\n", primary))
gsr_row <- corr_summary %>% filter(has_gsr)
if (nrow(gsr_row) > 0) {
  sens <- gsr_row$metric[!is.na(gsr_row$r) & gsr_row$r < 0.70]
  if (length(sens) > 0)
    cat(" Métricas alteradas por GSR: ", paste(sens, collapse = ", "),
      " - reportar en la sección de Limitaciones del TFM.\n", sep = "")
  else if (all(!is.na(gsr_row$r) & gsr_row$r > 0.90))
    cat(" → Métricas robustas a GSR: las conclusiones topológicas son válidas con y sin GSR.\n")
  else
    cat(" → Sensibilidad moderada a GSR - examinar caso por caso.\n")
} else cat(" (Sin pipeline GSR disponible - comparación omitida.)\n")

noGSR_names <- ctrl$name[!ctrl$has_gsr]
cat(sprintf("\n --- Robustez frente a alternativas no-GSR (%s vs %s) ---\n",
  primary, paste(noGSR_names, collapse = " / ")))
noGSR_row <- corr_summary %>% filter(!has_gsr)
if (nrow(noGSR_row) > 0) {
  sens <- noGSR_row %>% filter(!is.na(r) & r < 0.70) %>%
    mutate(tag = paste0(pipeline, ":", metric)) %>% pull(tag)
  if (length(sens) > 0)
    cat(" Discrepancias ", primary, " vs alternativas no-GSR: ",
      paste(sens, collapse = ", "),
      " - indica dependencia del método de regresión de confounds.\n", sep = "")
  else if (all(!is.na(noGSR_row$r) & noGSR_row$r > 0.90))
    cat(" → Métricas topológicas independientes del método no-GSR (los pipelines convergen).\n")
  else
    cat(" → Convergencia parcial entre pipelines no-GSR.\n")
}
invisible(corr_summary)
}

PIPELINE_CONTROL <- setNames(lapply(names(GRAPHS), run_pipeline_control), names(GRAPHS))

```

```
message("OK §4 - control técnico inter-pipeline completado para: ",
       paste(names(GRAPHs), collapse = ", "))
```

5 5 Visualización exploratoria de conectividad

Genera, para cada dataset activo y su pipeline primario, las figuras exploratorias de conectividad: glass-brain por red funcional (nilearn, §5.1) y heatmaps + conectomas circulares + heatmaps nodales por sujeto/red (R, §5.2). Atlas combinado Schaefer 200 (Yeo-7) + Tian S2 = 232 ROIs; redes mostradas: Default, Vis, Subcortex. Como en §1-§4, ambos chunks iteran sobre `ACTIVE_DATASETS`; las figuras van a `results/<ds>/figures/...`. No producen estado que consuman secciones posteriores → sin shim. (Los nombres de los `.npy` de conectomas grupales/por sujeto difieren entre datasets — herencia de las fases de extracción `PsiloPrep §4.5` (`ds006072`) y `LSDPrep §5` (`ds003059`) — y van resueltos en la config de cada chunk.)

5.1 5.1 Conectomas por red funcional (nilearn)

```
#####
# FASE 5 - Visualización exploratoria de conectividad - §5.1
# Conectomas por red funcional (nilearn) - BLOQUE DATASET-AGNÓSTICO
#####
# Para cada dataset activo, plot_connectome (glass brain) por red funcional
# Yeo-7 + pseudo-red "Subcortex" (Tian S2), por sujeto y a nivel de grupo,
# sobre el pipeline primario. Lee los .npy de la fase de extracción
# (PsiloPrep §4.5 ds006072 / LSDPrep §5 ds003059); los nombres de fichero difieren entre
# datasets → van en la config DATASETS. Figuras → results/<ds>/figures/networks/.
# c1 = condición psicodélica (psil/lsd); c2 = comparador (mtp/plcb).
#####

from Nilearn import plotting
import numpy as np
import os
import pandas as pd

PROJECT_ROOT    = "D:/ProfessionalProjects/PsyConn"
ATLAS_DIR       = os.path.join(PROJECT_ROOT, "derivatives", "atlas")
COORDS_CSV      = os.path.join(ATLAS_DIR, "Schaefer200_Tian32_coords.csv")
NETWORK_NAMES   = ["Default", "SalVentAttn", "Cont", "Subcortex", "Vis"] # eje REBUS + senso
# ACTIVE_DATASETS = ["ds006072", "ds003059"] # para uno solo: ["ds006072"]
ACTIVE_DATASETS = ["ds006072"]
```



```

DATASETS = {
    "ds006072": dict(
        pipeline="TEDANA_aCompCor", c1="psil", c2="mtp", c1_label="PSIL", c2_label="MTP",
        atlas_labels_csv=f"{PROJECT_ROOT}/results/ds006072/stats/atlas_labels_ds006072.csv",
        subj_c1_npy="{subj}_mean_connectome.npy", subj_c2_npy="{subj}_mean_connectome.npy",
        subj_diff_npy="{subj}_diff_connectome.npy",
        group_c1_npy="group_mean_psil.npy", group_c2_npy="group_mean_mtp.npy",
        group_diff_npy="group_diff.npy",
    ),
    "ds003059": dict(
        pipeline="AsIs", c1="lsd", c2="plcb", c1_label="LSD", c2_label="Placebo",
        atlas_labels_csv=f"{PROJECT_ROOT}/results/ds003059/stats/atlas_labels.csv",
        subj_c1_npy="{subj}_mean_connectome.npy", subj_c2_npy="{subj}_plcb_connectome.npy",
        subj_diff_npy="{subj}_diff_connectome.npy",
        group_c1_npy="group_mean_connectome.npy", group_c2_npy="group_plcb_connectome.npy",
        group_diff_npy="group_diff_connectome.npy",
    ),
}

SCHAEFER_RANGE = range(0, 200)
TIAN_RANGE      = range(200, 232)
coords_all      = pd.read_csv(COORDS_CSV)[["x", "y", "z"]].to_numpy()

def plot_net(mat, coords_net, title, out_file):
    plotting.plot_connectome(mat, coords_net, title=title, edge_threshold="90%",
                             node_size=20, display_mode="ortho", colorbar=True,
                             output_file=out_file)

def run_network_viz(ds):
    cfg = DATASETS[ds]
    print(f"\n{'#'*60}\n# $5.1 conectomas por red - {ds} (pipeline {cfg['pipeline']})\n{'#'*60}")
    conn_dir = f"{PROJECT_ROOT}/derivatives/{ds}/conn/{cfg['pipeline']}"
    c1_dir   = os.path.join(conn_dir, cfg["c1"])
    c2_dir   = os.path.join(conn_dir, cfg["c2"])
    fig_dir  = f"{PROJECT_ROOT}/results/{ds}/figures/networks"

    labels    = pd.read_csv(cfg["atlas_labels_csv"])["label"].tolist()
    labels_rois = [l for l in labels if l.lower() != "background"]
    assert len(labels_rois) == coords_all.shape[0] == 232, (
        f"{ds}: atlas inconsistente ({len(labels_rois)} labels vs {coords_all.shape[0]} coord")

    def network_indices(net_name):

```



```

    if net_name == "Subcortex":
        return list(TIAN_RANGE)
    return [i for i in SCHAEFER_RANGE if net_name in labels_rois[i]]

if not os.path.isdir(c1_dir):
    print(f"    {ds}: no existe {c1_dir} - dataset omitido.")
    return
subjects = sorted(d for d in os.listdir(c1_dir)
                  if os.path.isdir(os.path.join(c1_dir, d)) and d != "group")

for net in NETWORK_NAMES:
    print(f"\n   == Red: {net} ==")
    nodes = network_indices(net)
    print(f"    nodos detectados: {len(nodes)}")
    if len(nodes) == 0:
        print(f"    sin nodos para '{net}' - saltando.")
        continue
    cn = coords_all[nodes]

    # --- por sujeto ---
    for subj in subjects:
        f1 = os.path.join(c1_dir, subj, cfg["subj_c1_npy"].format(subj=subj))
        f2 = os.path.join(c2_dir, subj, cfg["subj_c2_npy"].format(subj=subj))
        fd = os.path.join(c1_dir, subj, cfg["subj_diff_npy"].format(subj=subj))
        if not all(os.path.exists(f) for f in (f1, f2, fd)):
            print(f"    omitido {subj}: archivos faltantes")
            continue
        sd = os.path.join(fig_dir, net, "subjects", subj)
        os.makedirs(sd, exist_ok=True)
        m1, m2, md = np.load(f1), np.load(f2), np.load(fd)
        plot_net(m1[np.ix_(nodes, nodes)], cn, f"{subj} {cfg['c1_label']} RSFC ({net})",
                os.path.join(sd, f"{subj}_{cfg['c1_label']}_{net}.png"))
        plot_net(m2[np.ix_(nodes, nodes)], cn, f"{subj} {cfg['c2_label']} RSFC ({net})",
                os.path.join(sd, f"{subj}_{cfg['c2_label']}_{net}.png"))
        plot_net(md[np.ix_(nodes, nodes)], cn,
                f"{subj} Δ {cfg['c1_label']}-{cfg['c2_label']} ({net})",
                os.path.join(sd, f"{subj}_Diff_{net}.png"))
        print(f"    {subj}: figuras {net} guardadas")

    # --- grupo ---
    g1 = os.path.join(conn_dir, "group", cfg["group_c1_npy"])
    g2 = os.path.join(conn_dir, "group", cfg["group_c2_npy"])

```

```

    gd = os.path.join(conn_dir, "group", cfg["group_diff_npy"])
    if not all(os.path.exists(f) for f in (g1, g2, gd)):
        print(f"    archivos grupales faltantes para {net} - ejecutar primero la fase de
        continue
    gfd = os.path.join(fig_dir, net, "group")
    os.makedirs(gfd, exist_ok=True)
    gm1, gm2, gmd = np.load(g1), np.load(g2), np.load(gd)
    plot_net(gm1[np.ix_(nodes, nodes)], cn, f"Group {cfg['c1_label']} RSFC ({net})",
            os.path.join(gfd, f"group_{cfg['c1_label']}_{net}.png"))
    plot_net(gm2[np.ix_(nodes, nodes)], cn, f"Group {cfg['c2_label']} RSFC ({net})",
            os.path.join(gfd, f"group_{cfg['c2_label']}_{net}.png"))
    plot_net(gmd[np.ix_(nodes, nodes)], cn,
            f"Group  $\Delta$  {cfg['c1_label']}-{cfg['c2_label']} ({net})",
            os.path.join(gfd, f"group_Diff_{net}.png"))
    print(f"    grupo: figuras {net} guardadas")

for _ds in ACTIVE_DATASETS:
    run_network_viz(_ds)
print("\n §5.1 - visualización multi-red completada para todos los datasets activos.")

```

5.2 5.2 Heatmaps, conectomas circulares y métricas nodales (R)

```

#=====
# FASE 5 - Visualización exploratoria de conectividad - §5.2
# Heatmaps, conectomas circulares y métricas nodales (R) - DATASET-AGNÓSTICO
#=====
# Para cada dataset activo: heatmaps grupales (global y por red, signed/POS/NEG),
# conectomas circulares (ggraph) y heatmaps nodales por sujeto. Lee las matrices
# grupales CSV (group/group_mean_<cond>.csv, group/group_diff.csv) y los CSV
# nodales por sujeto generados en §2.1 (derivatives/<ds>/graphs/<pipeline>/nodal/).
# c1 = psicodélica (psil/lsd); c2 = comparador (mtp/plcb). Figuras →
# results/<ds>/figures/. No deja estado consumido aguas abajo → sin shim.
#=====

library(igraph)
library(tidyverse)
library(pheatmap)
library(ggraph)
library(ggplot2)
library(patchwork)

```

```

library(cowplot)
library(png)          # para los composites resumen de §5.2 (3-panel + 6-panel)

PROJECT_ROOT    <- "D:/ProfessionalProyectos/PsyConn"
# ACTIVE_DATASETS <- c("ds006072", "ds003059")  # para uno solo: <- "ds006072"

# Config por dataset (pipeline primario). c1/c2 = nombres de carpeta de condición;
# c1_label/c2_label = etiquetas legibles (también claves en COND_COLOR y cond_label).
DATASETS_VIZ <- list(
  ds006072 = list(id = "ds006072", pipeline = "TEDANA_aCompCor",
    c1 = "psil", c2 = "mtp", c1_label = "PSIL", c2_label = "MTP",
    atlas_labels_csv = file.path(PROJECT_ROOT,
      "results/ds006072/stats/atlas_labels_ds006072.csv")),
  ds003059 = list(id = "ds003059", pipeline = "AsIs",
    c1 = "lsd", c2 = "plcb", c1_label = "LSD", c2_label = "PLCB",
    atlas_labels_csv = file.path(PROJECT_ROOT,
      "results/ds003059/stats/atlas_labels.csv"))
)

NETWORKS_OF_INTEREST <- c("Default", "SalVentAttn", "Cont", "Subcortex", "Vis")
TOPN_NET              <- 12

# ---- Utilidades comunes -----
save_plot_to <- function(plot_obj, path, width = 8, height = 6, dpi = 300)
  ggsave(filename = path, plot = plot_obj, width = width, height = height, dpi = dpi)

read_nodal_csv <- function(file) read.csv(file, stringsAsFactors = FALSE)

mat_to_igraph_safe <- function(mat, node_names) {
  diag(mat) <- 0
  g <- graph_from_adjacency_matrix(mat, mode = "undirected", weighted = TRUE)
  V(g)$name <- node_names
  g
}

# Wrapper robusto para pheatmap: pheatmap muere ("breaks" are not unique)
# con matrices de varianza nula (todo ceros, frecuente en NEG, o scale="row"
# sobre filas constantes). Salta esos casos y envuelve en tryCatch.
pheatmap_safe <- function(mat, ..., .label = "") {
  if (!is.matrix(mat) || nrow(mat) < 2 || ncol(mat) < 2) {
    message("  skip pheatmap (", .label, ", dim=", nrow(mat), "x", ncol(mat), ")")
  }

```

```

    return(invisible(NULL))
}
m_sd <- suppressWarnings(sd(as.numeric(mat), na.rm = TRUE))
if (is.na(m_sd) || m_sd == 0) {
  message("    skip pheatmap (", .label, ", matriz constante"); return(invisible(NULL))
}
tryCatch(pheatmap(mat, ...),
  error = function(e) { message("    pheatmap fallo (", .label, "): ",
    conditionMessage(e)); invisible(NULL) })
}

# Wrapper que escribe a PNG sólo si la matriz es plotable. El patrón antiguo
# (png(path); pheatmap_safe(mat, ...); dev.off()) abría el dispositivo ANTES
# del guard de varianza, por lo que cuando la matriz era constante pheatmap
# saltaba pero dev.off() cerraba igualmente un PNG de 581 bytes (vacío) en
# disco. Aquí el guard se aplica antes de abrir el dispositivo y no queda
# fichero huérfano si no hay nada que dibujar.
pheatmap_to_png <- function(.path, mat, ..., .label = "",
  .width = 800, .height = 600, .res = 150) {
  if (!is.matrix(mat) || nrow(mat) < 2 || ncol(mat) < 2) {
    message("    skip pheatmap (", .label, ", dim=", nrow(mat), "x", ncol(mat), ")")
    return(invisible(NULL))
  }
  m_sd <- suppressWarnings(sd(as.numeric(mat), na.rm = TRUE))
  if (is.na(m_sd) || m_sd == 0) {
    message("    skip pheatmap (", .label, ", matriz constante")
    return(invisible(NULL))
  }
  png(.path, width = .width, height = .height, res = .res)
  on.exit(try(dev.off(), silent = TRUE), add = TRUE)
  tryCatch(pheatmap(mat, ...),
    error = function(e) message("    pheatmap fallo (", .label, "): ",
      conditionMessage(e)))

  invisible(NULL)
}

# Atlas combinado: Schaefer (prefijo "7Networks_") → red Yeo-7; Tian S2 → "Subcortex".
get_network <- function(label)
  if (startsWith(label, "7Networks_")) strsplit(label, "_")[[1]][3] else "Subcortex"

make_circular_connectome_to <- function(mat, node_labels, out_path,
  top_pct = 0.02, title = "", show_labels = TRUE) {

```

```

vals <- sort(abs(mat[upper.tri(mat)]), decreasing = TRUE)
n_keep <- max(1L, ceiling(length(vals) * top_pct))
if (length(vals) == 0 || n_keep > length(vals)) return(invisible(NULL))
thr <- vals[n_keep]
mat_thr <- mat; mat_thr[abs(mat_thr) < thr] <- 0
g <- mat_to_igraph_safe(mat_thr, node_labels)
png(out_path, width = 1200, height = 1200, res = 150)
p <- ggraph(g, layout = "circle") +
  geom_edge_link(aes(width = abs(weight), alpha = abs(weight), color = weight)) +
  geom_node_point(size = 3) +
  {if (show_labels) geom_node_text(aes(label = name), repel = TRUE, size = 3)} +
  scale_edge_colour_gradient2(low = "blue", mid = "grey90", high = "red", midpoint = 0) +
  theme_void() + labs(title = title)
print(p); dev.off()
}

#=====
# Visualizaciones para un dataset
#=====
run_viz_R <- function(cd) {
  cat(sprintf("\n##### $5.2 Visualizaciones (R) - %s (pipeline %s) #####\n",
    cd$id, cd$pipeline))
  c1 <- cd$c1; c2 <- cd$c2; L1 <- cd$c1_label; L2 <- cd$c2_label
  CONN_DIR <- file.path(PROJECT_ROOT, "derivatives", cd$id, "conn", cd$pipeline)
  GRAPH_DIR <- file.path(PROJECT_ROOT, "derivatives", cd$id, "graphs", cd$pipeline)
  out_dir <- file.path(PROJECT_ROOT, "results", cd$id, "figures")
  # Paleta de condiciones: magenta #C2185B (psicodélica) + teal oscuro
  # #00695C (comparador). Se evita rojo/azul porque está reservado para
  # POS/NEG en $5 y $6.3 y su reutilización para condición es
  # contraproducente; el morado #8E44AD también está tomado (RATIO en
  # dotplot POSNEG).
  COND_COLOR <- setNames(c("#C2185B", "#00695C"), c(L1, L2))
  cond_label <- setNames(c(L1, L2, sprintf("Delta (%s - %s)", L1, L2)),
    c(L1, L2, "Delta"))
  delta_tag <- sprintf("Delta (%s - %s)", L1, L2)

  # --- 1) Atlas (Schaefer 200 + Tian S2 = 232 ROIs) -----
  atlas_labels <- read.csv(cd$atlas_labels_csv)$label
  atlas_labels <- atlas_labels[-1] # quita Background
  n_nodes <- length(atlas_labels)
  atlas_labels_clean <- gsub("^7Networks_", "", atlas_labels)
  node_network <- sapply(atlas_labels, get_network)

```

```

stopifnot("Atlas debe tener 232 ROIs" = n_nodes == 232L)

# --- 2) Matrices grupales (CSV): <c1> / <c2> / Delta = <c1> - <c2> -----
rd <- function(f) as.matrix(read.csv(file.path(CONN_DIR, f), row.names = 1))
group_mats <- setNames(
  list(rd(sprintf("group/group_mean_%s.csv", c1)),
        rd(sprintf("group/group_mean_%s.csv", c2)),
        rd("group/group_diff.csv")),
  c(L1, L2, "Delta"))

# --- 3) Heatmaps y circulares - global/group -----
global_group_dir <- file.path(out_dir, "global", "group")
dir.create(global_group_dir, showWarnings = FALSE, recursive = TRUE)
for (cond_name in c(L1, L2, "Delta")) {
  group_mat <- group_mats[[cond_name]]
  pheatmap_to_png(
    file.path(global_group_dir, paste0("heatmap_global_", cond_name, ".png")),
    group_mat, cluster_rows = FALSE, cluster_cols = FALSE,
    show_rownames = FALSE, show_colnames = FALSE,
    color = colorRampPalette(c("blue", "white", "red"))(100), border_color = NA,
    main = paste0("[GRUPO · Global] Conectividad funcional ± | ", cond_label[[cond_name]]),
    .label = paste0("global ± ", cond_name),
    .width = 1000, .height = 1000)
  mat_pos <- group_mat; mat_pos[mat_pos <= 0] <- 0
  pheatmap_to_png(
    file.path(global_group_dir, paste0("heatmap_global_POS_", cond_name, ".png")),
    mat_pos, cluster_rows = FALSE, cluster_cols = FALSE,
    show_rownames = FALSE, show_colnames = FALSE,
    color = colorRampPalette(c("white", "red"))(100), border_color = NA,
    main = paste0("[GRUPO · Global] Conectividad funcional (+) | ", cond_label[[cond_name]]),
    .label = paste0("global POS ", cond_name),
    .width = 1000, .height = 1000)
  mat_neg <- group_mat; mat_neg[mat_neg >= 0] <- 0
  pheatmap_to_png(
    file.path(global_group_dir, paste0("heatmap_global_NEG_", cond_name, ".png")),
    mat_neg, cluster_rows = FALSE, cluster_cols = FALSE,
    show_rownames = FALSE, show_colnames = FALSE,
    color = colorRampPalette(c("blue", "white"))(100), border_color = NA,
    main = paste0("[GRUPO · Global] Conectividad funcional (-) | ", cond_label[[cond_name]]),
    .label = paste0("global NEG ", cond_name),
    .width = 1000, .height = 1000)
}

```

```

# --- 3b) Composites resumen global -----
# Componen los 3 heatmaps globales signed (1×3 = MTP|PSIL|Δ) y los 6 mapas
# POS/NEG (3×2 = filas MTP/PSIL/Δ × cols POS/NEG) en una sola figura cada
# uno. Pensados para incrustar como única figura en R2 (sustituyen 3+6
# incrustaciones individuales). Lee los PNGs que se acaban de guardar en
# disco y los pega con grid::rasterGrob + cowplot::plot_grid; los títulos
# legibles ya vienen embebidos por pheatmap_safe (`main=`).
panel_png <- function(path) {
  img <- png::readPNG(path, native = FALSE)
  cowplot::ggdraw() + cowplot::draw_grob(grid::rasterGrob(img, interpolate = TRUE))
}
composite_3 <- cowplot::plot_grid(
  panel_png(file.path(global_group_dir, paste0("heatmap_global_", L2, ".png"))),
  panel_png(file.path(global_group_dir, paste0("heatmap_global_", L1, ".png"))),
  panel_png(file.path(global_group_dir, "heatmap_global_Delta.png")),
  nrow = 1)
save_plot_to(composite_3,
  file.path(global_group_dir, "summary_global_3panel.png"),
  width = 18, height = 6, dpi = 150)
composite_6 <- cowplot::plot_grid(
  panel_png(file.path(global_group_dir, paste0("heatmap_global_POS_", L2, ".png"))),
  panel_png(file.path(global_group_dir, paste0("heatmap_global_NEG_", L2, ".png"))),
  panel_png(file.path(global_group_dir, paste0("heatmap_global_POS_", L1, ".png"))),
  panel_png(file.path(global_group_dir, paste0("heatmap_global_NEG_", L1, ".png"))),
  panel_png(file.path(global_group_dir, "heatmap_global_POS_Delta.png")),
  panel_png(file.path(global_group_dir, "heatmap_global_NEG_Delta.png")),
  ncol = 2)
save_plot_to(composite_6,
  file.path(global_group_dir, "summary_pos_neg_6panel.png"),
  width = 14, height = 18, dpi = 150)

# --- 4) Por red funcional -----
for (net in NETWORKS_OF_INTEREST) {
  net_group_dir <- file.path(out_dir, "networks", net, "group")
  dir.create(net_group_dir, showWarnings = FALSE, recursive = TRUE)
  idx_net <- which(node_network == net)
  if (length(idx_net) < 4) next
  labels_net <- atlas_labels_clean[idx_net]

  # 4a. Heatmaps grupales por red (<c1> / <c2>)
  for (cond_name in c(L1, L2)) {
    group_mat <- group_mats[[cond_name]]

```

```

mat_net <- group_mat[idx_net, idx_net]
top_idx <- idx_net[order(rowSums(abs(mat_net))), decreasing = TRUE][1:min(TOPN_NET, length(top_idx))]
mat_top <- group_mat[top_idx, top_idx]
lbs <- atlas_labels_clean[top_idx]
pheatmap_to_png(
  file.path(net_group_dir, paste0("heatmap_", cond_name, ".png")),
  mat_top, cluster_rows = TRUE, cluster_cols = TRUE,
  labels_row = lbs, labels_col = lbs, fontsize_row = 8, fontsize_col = 8,
  main = paste0("[GRUPO · ", net, "] Top hubs ± | ", cond_label[[cond_name]]),
  .label = paste0(net, " ± ", cond_name))
mat_pos <- mat_net; mat_pos[mat_pos <= 0] <- 0
top_idx_pos <- idx_net[order(rowSums(mat_pos), decreasing = TRUE)[1:min(TOPN_NET, length(top_idx_pos))]]
mat_top_pos <- group_mat[top_idx_pos, top_idx_pos]; mat_top_pos[mat_top_pos <= 0] <- 0
pheatmap_to_png(
  file.path(net_group_dir, paste0("heatmap_POS_", cond_name, ".png")),
  mat_top_pos, cluster_rows = TRUE, cluster_cols = TRUE,
  labels_row = atlas_labels_clean[top_idx_pos], labels_col = atlas_labels_clean[top_idx_pos],
  fontsize_row = 8, fontsize_col = 8, color = colorRampPalette(c("white", "red"))(100),
  main = paste0("[GRUPO · ", net, "] Top hubs (+) | ", cond_label[[cond_name]]),
  .label = paste0(net, " POS ", cond_name))
mat_neg <- mat_net; mat_neg[mat_neg >= 0] <- 0
top_idx_neg <- idx_net[order(rowSums(abs(mat_neg)), decreasing = TRUE)[1:min(TOPN_NET, length(top_idx_neg))]]
mat_top_neg <- group_mat[top_idx_neg, top_idx_neg]; mat_top_neg[mat_top_neg >= 0] <- 0
pheatmap_to_png(
  file.path(net_group_dir, paste0("heatmap_NEG_", cond_name, ".png")),
  mat_top_neg, cluster_rows = TRUE, cluster_cols = TRUE,
  labels_row = atlas_labels_clean[top_idx_neg], labels_col = atlas_labels_clean[top_idx_neg],
  fontsize_row = 8, fontsize_col = 8, color = colorRampPalette(c("blue", "white"))(100),
  main = paste0("[GRUPO · ", net, "] Top hubs (-) | ", cond_label[[cond_name]]),
  .label = paste0(net, " NEG ", cond_name))
}

# 4b. Heatmaps Delta por red
group_mat <- group_mats[["Delta"]]
mat_net <- group_mat[idx_net, idx_net]
mat_pos <- mat_net; mat_pos[mat_pos <= 0] <- 0
top_idx_pos <- idx_net[order(rowSums(mat_pos), decreasing = TRUE)[1:min(TOPN_NET, length(top_idx_pos))]]
mat_top_pos <- group_mat[top_idx_pos, top_idx_pos]; mat_top_pos[mat_top_pos <= 0] <- 0
pheatmap_to_png(
  file.path(net_group_dir, "heatmap_POS_Delta.png"),
  mat_top_pos, cluster_rows = TRUE, cluster_cols = TRUE,
  labels_row = atlas_labels_clean[top_idx_pos], labels_col = atlas_labels_clean[top_idx_pos],

```



```

    fontsize_row = 8, fontsize_col = 8, color = colorRampPalette(c("white", "red"))(100),
    main = paste0("[GRUPO · ", net, "] Top hubs (+) | ", delta_tag),
    .label = paste0(net, " POS Delta"))
mat_neg <- mat_net; mat_neg[mat_neg >= 0] <- 0
top_idx_neg <- idx_net[order(rowSums(abs(mat_neg))), decreasing = TRUE][1:min(TOPN_NET, 1)]
mat_top_neg <- group_mat[top_idx_neg, top_idx_neg]; mat_top_neg[mat_top_neg >= 0] <- 0
pheatmap_to_png(
  file.path(net_group_dir, "heatmap_NEG_Delta.png"),
  mat_top_neg, cluster_rows = TRUE, cluster_cols = TRUE,
  labels_row = atlas_labels_clean[top_idx_neg], labels_col = atlas_labels_clean[top_idx_neg],
  fontsize_row = 8, fontsize_col = 8, color = colorRampPalette(c("blue", "white"))(100),
  main = paste0("[GRUPO · ", net, "] Top hubs (-) | ", delta_tag),
  .label = paste0(net, " NEG Delta"))

# 4c. Conectomas circulares grupales por red
for (cond_name in c(L1, L2, "Delta")) {
  mat_net <- group_mats[[cond_name]][idx_net, idx_net]
  make_circular_connectome_to(mat_net, labels_net,
    file.path(net_group_dir, paste0("circular_", cond_name, ".png")), 0.2,
    paste0("[GRUPO · ", net, "] Conectoma circular ± | ", cond_label[[cond_name]]), "
    show_labels = TRUE)
  mat_pos <- mat_net; mat_pos[mat_pos <= 0] <- 0
  if (any(mat_pos != 0))
    make_circular_connectome_to(mat_pos, labels_net,
      file.path(net_group_dir, paste0("circular_POS_", cond_name, ".png")), 0.2,
      paste0("[GRUPO · ", net, "] Conectoma circular (+) | ", cond_label[[cond_name]]), "
      show_labels = TRUE)
  mat_neg <- mat_net; mat_neg[mat_neg >= 0] <- 0
  if (any(mat_neg != 0))
    make_circular_connectome_to(mat_neg, labels_net,
      file.path(net_group_dir, paste0("circular_NEG_", cond_name, ".png")), 0.2,
      paste0("[GRUPO · ", net, "] Conectoma circular (-) | ", cond_label[[cond_name]]), "
      show_labels = TRUE)
}

# 4e. Composites resumen por red para incrustación en R2 (Fase 2). Lee
# los PNGs ya guardados - heatmaps de §5.2 y glass-brain de §5.1 - y los
# compone con cowplot::plot_grid. Si falta algún input (matriz constante
# en pheatmap_to_png o §5.1 no ejecutado para esta red), salta el
# composite afectado y avisa por consola.

# Composite (1) - top-12 hubs: MTP | PSIL en una sola fila.

```

```

top12_paths <- c(file.path(net_group_dir, paste0("heatmap_", L2, ".png")),
                file.path(net_group_dir, paste0("heatmap_", L1, ".png")))
if (all(file.exists(top12_paths))) {
  composite_top12 <- cowplot::plot_grid(
    panel_png(top12_paths[1]),
    panel_png(top12_paths[2]),
    nrow = 1)
  save_plot_to(composite_top12,
               file.path(net_group_dir, "summary_top12_MTP_PSIL.png"),
               width = 14, height = 5, dpi = 150)
} else {
  message("    composite top12 ", net, " omitido (falta heatmap MTP o PSIL)")
}

# Composite (2) - Δ (PSIL - MTP) apilado vertical: conectoma circular ±
# arriba a anchura completa y glass-brain ortho debajo a anchura completa.
# Layout focal del contraste; sustituye al composite MTP|PSIL|Δ previo
# que se renderizaba demasiado pequeño en la memoria. rel_heights = c(2.4, 1)
# respeta los aspect ratios naturales (circular 1:1, glass-brain 2.4:1)
# para que ambos paneles compartan la misma anchura efectiva sin recortes.
cc_path_delta <- file.path(net_group_dir, "circular_Delta.png")
gb_path_delta <- file.path(net_group_dir, paste0("group_Diff_", net, ".png"))
if (file.exists(cc_path_delta) && file.exists(gb_path_delta)) {
  composite_delta <- cowplot::plot_grid(
    panel_png(cc_path_delta),
    panel_png(gb_path_delta),
    nrow = 2, ncol = 1, rel_heights = c(2.4, 1))
  save_plot_to(composite_delta,
               file.path(net_group_dir, "summary_circular_glass_Delta.png"),
               width = 10, height = 14, dpi = 150)
} else {
  missing <- c(cc_path_delta, gb_path_delta)[
    !file.exists(c(cc_path_delta, gb_path_delta))]
  message("    composite circular+glass Δ ", net, " omitido (faltan: ",
          paste(basename(missing), collapse = ", "), ")")
}

# 4d. Heatmaps nodales por sujeto - CSVs nodales de $2.1 (graphs/<pipe>/nodal/<cond>/)
nodal_dir_1 <- file.path(GRAPH_DIR, "nodal", c1)
nodal_dir_2 <- file.path(GRAPH_DIR, "nodal", c2)
subjects <- intersect(sub("_.*", "", list.files(nodal_dir_1, pattern = "\\..csv$")),
                     sub("_.*", "", list.files(nodal_dir_2, pattern = "\\..csv$")))

```

```

for (subj in subjects) {
  subj_net_dir <- file.path(out_dir, "networks", net, "subjects", subj)
  dir.create(subj_net_dir, showWarnings = FALSE, recursive = TRUE)
  cond_dirs <- setNames(list(nodal_dir_1, nodal_dir_2), c(L1, L2))
  for (cond in names(cond_dirs)) {
    nodal_file <- list.files(cond_dirs[[cond]], pattern = subj, full.names = TRUE)
    if (length(nodal_file) == 0) next
    nodal_mat <- read_nodal_csv(nodal_file[1])
    if (nrow(nodal_mat) > length(atlas_labels)) nodal_mat <- nodal_mat[1:length(atlas_labels), ]
    nodal_mat$node_label <- atlas_labels
    nodal_mat$network <- sapply(nodal_mat$node_label, get_network)
    nodal_net <- nodal_mat %>% filter(network == net)
    if (nrow(nodal_net) < 4) next
    top_net <- nodal_net %>% arrange(desc(abs(strength))) %>% slice(1:min(TOPN_NET, nrow(nodal_net)))
    base_mat <- top_net %>%
      select(degree, strength, betweenness, closeness, eigenvector,
             local_clustering, within_module_z, participation) %>% as.matrix()
    rownames(base_mat) <- gsub("^7Networks_", "", top_net$node_label)
    mat_pos <- base_mat
    mat_pos[, "strength"] <- ifelse(mat_pos[, "strength"] > 0, mat_pos[, "strength"], 0)
    heatmap_to_png(
      file.path(subj_net_dir, paste0("heatmap_POS_", cond, ".png")),
      mat_pos, scale = "row", cluster_rows = TRUE, cluster_cols = FALSE, fontsize_row = 8,
      main = paste0("[", subj, " · ", net, "] Top hubs (+) | ", cond_label[[cond]]),
      .label = paste0(subj, " ", net, " POS ", cond),
      .width = 900)
    mat_neg <- base_mat
    mat_neg[, "strength"] <- ifelse(mat_neg[, "strength"] < 0, mat_neg[, "strength"], 0)
    heatmap_to_png(
      file.path(subj_net_dir, paste0("heatmap_NEG_", cond, ".png")),
      mat_neg, scale = "row", cluster_rows = TRUE, cluster_cols = FALSE, fontsize_row = 8,
      main = paste0("[", subj, " · ", net, "] Top hubs (-) | ", cond_label[[cond]]),
      .label = paste0(subj, " ", net, " NEG ", cond),
      .width = 900)
    top_hubs <- nodal_mat %>% arrange(desc(abs(strength))) %>% slice(1:50)
    g_hubs <- ggplot(top_hubs, aes(x = reorder(node_label, strength), y = strength)) +
      geom_col(fill = COND_COLOR[[cond]]) + coord_flip() + theme_minimal() +
      scale_x_discrete(labels = function(x) gsub("^7Networks_", "", x)) +
      labs(title = paste0("[", subj, " · ", net, "] Node strength ± | ", cond_label[[cond]]),
           x = "Node", y = "Strength")
    save_plot_to(g_hubs, file.path(subj_net_dir, paste0("top_hubs_", cond, ".png")))
  }
}

```

```

    }
    message(" OK ", cd$id, " | red ", net, " completada.")
  }
  cat(sprintf(" OK §5.2 - %s - figuras en %s\n", cd$id, out_dir))
  invisible(NULL)
}

for (.ds in ACTIVE_DATASETS) run_viz_R(DATASETS_VIZ[.ds])
message("OK §5.2 - visualizaciones (R) generadas para: ",
        paste(ACTIVE_DATASETS, collapse = ", "))

```

6 6 Análisis estadístico: PSIL vs MTP

6.1 6.1 Tests pareados globales y nodales — núcleo confirmatorio del TFM

Esta sección operacionaliza la hipótesis principal del TFM: la psilocibina (PSIL), comparada con metilfenidato como placebo activo (MTP), altera la **organización topológica** de la red funcional cerebral en una dirección coherente con la **hipótesis del cerebro entrópico** (Carhart-Harris et al., 2014, *Front Hum Neurosci*; Carhart-Harris & Friston, 2019, *Pharmacol Rev*). Se aplican tests pareados within-subject (t-test pareado primario + Wilcoxon como sensibilidad) sobre un pool de 13 métricas topológicas, con corrección BH-FDR conjunta within-family.

Pool topología (13 tests). Ocho métricas globales del grafo (`mean_strength`, `global_efficiency`, `char_path_length`, `density`, `clustering`, `modularity`, `assortativity`, `small_worldness`) + cuatro métricas nodales agregadas por sujeto×condición (`eigenvector`, `betweenness`, `closeness`, `participation`) + una métrica avanzada: la **Shannon entropy de la distribución de grados** del grafo umbralizado (Viol et al., 2017, *Sci Rep* 7:7388, “Shannon entropy of brain functional complex networks under the influence of the psychedelic Ayahuasca” — la única entropía *topológica* estática con respaldo directo en la literatura de la entropic brain hypothesis: Viol et al. la presentan como su resultado principal y como medición *directa* de la entropía de la conectividad funcional; predicción a priori ↑ **bajo PSIL** al ensancharse la distribución de grados, cf. su Fig. 3-4). Tres nodales (`strength`, `degree`, `local_clustering`) se retiraron del pool en la ronda 1 de revisión por ser redundantes con sus homólogas globales (`mean_strength`; `density × (n-1)`; `clustering`). Se predice (Carhart-Harris et al., 2016, *PNAS*; Müller et al., 2018, *NeuroImage*; Lord et al., 2019, *NeuroImage*; Luppi et al., 2023, *Nat Commun*) ↑**integración** / ↓**segregación** bajo PSIL: ↓`modularity`, ↓`clustering`, ↑`global_efficiency`, ↑`degree_entropy`.

i Variable de orden — disponible en ds006072

Contexto: ds006072 es un diseño crossover, randomized, single-blind con washout intra-sujeto > 4 semanas (Siegel et al., 2024, *Nature*; Raut et al., 2025). Cada sujeto recibe PSIL (25 mg oral) y MTP (40 mg oral) en sesiones separadas.

Disponibilidad del orden: A diferencia de ds003059, ds006072 **sí publica participants.tsv** con los campos `drug1`, `drug2`, `ses_drug1`, `ses_drug2`, lo que permite reconstruir la secuencia (PSIL→MTP o MTP→PSIL) y el periodo (1 = primer fármaco, 2 = segundo) por sujeto.

Balance de orden en el subconjunto preprocesado actual (N=6):

Sujeto	drug1	drug2	secuencia
sub-P1	MTP	PSIL	MTP→PSIL
sub-P3	MTP	PSIL	MTP→PSIL
sub-P4	MTP	PSIL	MTP→PSIL
sub-P5	PSIL	MTP	PSIL→MTP
sub-P6	MTP	PSIL	MTP→PSIL
sub-P7	PSIL	MTP	PSIL→MTP

El balance está sesgado (4 MTP-first vs 2 PSIL-first). El chunk reporta este balance como **descriptor de transparencia** — la inferencia de §6.1 se basa en t-test pareado y Wilcoxon, no en un modelo lineal mixto. Si en el futuro se quisiera formalizar el efecto de orden con `valor ~ condition * sequence + (1|subject)`, el diseño es rank-deficient con N=6 e imbalance 4:2 (`sequence` queda cuasi-colineal con la diferencia intra-sujeto) y la covariable no podría estimarse; por eso aquí se reporta sin entrar al modelo.

! Marco estadístico simplificado (decisión S0, 2026-05-23)

La corrección por atenuación basada en ICC test-retest se ha **retirado** del pipeline. Con N=5–6 sujetos y k=2 runs intra-sesión, el ICC no es estimable de forma estadísticamente útil ni mide la fiabilidad inter-sesión que el contraste PSIL vs MTP requeriría (Bonett 2002; Walter et al. 1998; Noble et al. 2019, discusión sobre TRT inter-sesión). En su lugar, la **robustez estadística** se cierra aguas abajo en §7 mediante (i) LOO por sujeto, (ii) bootstrap del IC del efecto, (iii) permutación intra-sujeto (sign-flip exacto) y (iv) convergencia inter-pipeline (de PsiloPrep §5.2). Cada métrica se reporta con d_z + CI bootstrap + p permutación + p FDR(N_familia); un efecto se considera **creíble** si cumple los tres criterios: CI bootstrap no cruza 0, p permutación < 0.05, y la dirección converge en la mayoría de los pipelines de denoising disponibles según el dataset.

Definición canónica del criterio S0 (referencia para §6.2 y consumidores aguas abajo): sea n_{dir} el número de pipelines (primario incluido) cuyo signo de d_z coincide con el signo de d_z del primario para una métrica dada. El criterio direccional `converge_S0`

es `n_dir >= SO_THRESHOLD_BY_DS`, donde la constante `SO_THRESHOLD_BY_DS` toma los valores de la tabla siguiente. Las cifras se eligen proporcionales («mayoría sustancial»: 85 % para ds006072; unanimidad para ds003059 donde la triangulación es más estrecha).

Dataset	Pipelines totales	SO_THRESHOLD_BY_DS
ds006072 (PSIL/MTP, rama primaria)	7 (1 primario + 6 controles)	6 (6/7 con misma dirección de signo)
ds003059 (LSD/PLCB)	3 (1 primario + 2 controles)	3 (3/3 con misma dirección de signo)

La **operacionalización cuantitativa** (cálculo del flag por métrica) se ejecuta en §6.2 sobre el pool topología (13 métricas) iterando los pipelines de `CONTROL_PIPELINES_BY_DS` (§4) más el primario. Decisión completa en `reviews/PsiloPrep_S7-qc_ronda3_estadistico.md` § S0; cf. §6.2 para el detalle por métrica.

Tests pareados (t-test pareado primario + Wilcoxon como sensibilidad), corrección FDR within-family, d de Cohen reportado en dos variantes en paralelo: $d_z = \overline{\Delta}/\text{sd}(\Delta)$ como effect size primario (test-coupled con `pwr.t.test` para la sensibilidad §7) y $d_{av} = \overline{\Delta}/\sqrt{(s_1^2 + s_2^2)/2}$ para comparabilidad con la literatura (Lakens 2013). Diseño within-subject.

```
#=====
# § 6 - Análisis estadístico: funciones auxiliares y datos por dataset
# 6.1 - Tests pareados globales y nodales
#=====
# loo_influence_analysis() y analizar_metrica() son dataset-agnósticas:
# reciben C1/C2 como parámetros en lugar de hardcodear "PSIL"/"MTP".
# make_stats_13_1(cd, gd, am) construye el estado completo por dataset;
# los resultados se almacenan en STATS_13_1 (lista nombrada por dataset id).
#=====

library(tidyverse)
library(igraph)
library(rstatix)

# H12 ronda 1: path al participants.tsv promovido a constante para que el
# único punto de configuración esté arriba (análogo a PROJECT_ROOT). Si el
# disco F: cambia o se trabaja desde otra máquina, basta editar aquí. La
# constante se usa también en §6.6 (cerebro-experiencia) - mantener
# sincronizadas ambas referencias si esta cambia.
PARTICIPANTS_TSV_DS006072 <- "F:/JIVillL/openneuro/ds006072/participants.tsv"
```

```

loo_influence_analysis <- function(df_long, metric_col, metric_name, C1, C2,
                                  condition_col = "condition",
                                  subject_col   = "subject") {
  # LOO influence para tests pareados.
  # Criterio: |z-score(diff_i)| > 2 (regla |z|>2 - Tukey 1977; equivalente
  # a cobertura ~95 % bajo normalidad). H2-A r1 §6.3 (ronda 3 quirúrgica
  # de §6.1, 2026-05-24): el threshold previo `4/n` derivado de Cook 1977
  # asume la distancia estandarizada  $D_i$  [0,1]; aplicarlo al cambio
  # absoluto de la media  $|\Delta_{full} - \Delta_{-i}|$  (con las unidades de la métrica)
  # es dimensionalmente incorrecto y produce flags espurios - para
  # métricas con valores en [0,1] casi nunca se cruza, para métricas con
  # rango amplio (p. ej. RATIO 2-17) sobre-flagea. El z-score sobre `diff`
  # es escala-invariante y conserva la interpretación de outlier.
  # Devuelve lista: $robust (3 tests), $influence (por sujeto),
  # $n_influential, $agreement.

  df_wide <- df_long %>%
    select(all_of(c(subject_col, condition_col, metric_col))) %>%
    pivot_wider(names_from = all_of(condition_col),
                values_from = all_of(metric_col)) %>%
    filter(!is.na(.data[[C1]]), !is.na(.data[[C2]])) %>%
    arrange(across(all_of(subject_col))) %>%
    mutate(diff = .data[[C1]] - .data[[C2]])

  n <- nrow(df_wide)
  if (n < 3) return(NULL)

  mean_diff_full <- mean(df_wide$diff)
  Z_THRESH      <- 2

  df_wide <- df_wide %>%
    mutate(
      loo_mean_diff = map_dbl(seq_len(n), ~mean(diff[-.x])),
      loo_influence = abs(mean_diff_full - loo_mean_diff),
      diff_zscore   = as.numeric(scale(diff)),
      influential   = !is.na(diff_zscore) & abs(diff_zscore) > Z_THRESH
    )

  test_full  <- t.test(df_wide[[C1]], df_wide[[C2]], paired = TRUE)
  test_wilcox <- wilcox.test(df_wide[[C1]], df_wide[[C2]],
                             paired = TRUE, exact = FALSE)
  df_no_infl <- df_wide %>% filter(!influential)

```

```

test_noinfl <- if (nrow(df_no_infl) >= 3)
  t.test(df_no_infl[[C1]], df_no_infl[[C2]], paired = TRUE)
else NULL

robust_tbl <- tibble(
  metric      = metric_name,
  test        = c("t pareado (todos)",
                  "Wilcoxon (todos)",
                  "t pareado (sin influyentes)"),
  n           = c(n, n, nrow(df_no_infl)),
  statistic    = c(test_full$statistic,
                  test_wilcox$statistic,
                  if (!is.null(test_noinfl)) test_noinfl$statistic
                  else NA_real_),
  p_value     = c(test_full$p.value,
                  test_wilcox$p.value,
                  if (!is.null(test_noinfl)) test_noinfl$p.value
                  else NA_real_)
) %>%
  mutate(
    significant = p_value < 0.05,
    zscore_thresh = Z_THRESH
  )

sigs      <- robust_tbl$significant
agreement <- (sum(sigs, na.rm = TRUE) == 3 ||
             sum(sigs, na.rm = TRUE) == 0)

infl_tbl <- df_wide %>%
  select(all_of(c(subject_col, C1, C2, "diff",
                  "loo_influence", "influential", "diff_zscore"))) %>%
  mutate(metric = metric_name)

list(
  robust      = robust_tbl,
  influence   = infl_tbl,
  n_influential = sum(df_wide$influential),
  agreement   = agreement
)
}

library(effsize)

```



```

library(ggrepel)
library(gghalves)
library(patchwork)

# Diagnóstico - qué variante de Cohen's d devuelve effsize::cohen.d en
# diseño pareado, y por qué §6.1 no la usa directamente.
#
# Hay TRES convenciones razonables para el effect size en un crossover
# (Lakens 2013, "Calculating and reporting effect sizes...", §3.4):
#
#   d_z = mean( $\Delta$ ) / sd( $\Delta$ )                (test-coupled con pwr.t.test
#                                                    type="paired"; el que
#                                                    §7.1 consume)
#   d_av = mean( $\Delta$ ) / sqrt((sd(g1)^2 + sd(g2)^2)/2)  (variante "average SD";
#                                                         comparabilidad con
#                                                         literatura que reporta
#                                                         entre grupos)
#   d_rm = d_z · sqrt(2·(1 - r))                (repeated measures Cohen's
#                                                         d en escala "between-
#                                                         equivalente"; descuenta la
#                                                         correlación intra-sujeto.
#                                                         Borenstein 2009; Lakens
#                                                         2013, eq. 8)
#
# `effsize::cohen.d(g1, g2, paired = TRUE)$estimate` devuelve d_rm, NO d_z
# ni d_av. Con datos muy correlacionados (típico de un crossover) el
# factor sqrt(2·(1-r)) puede ser 1 y d_rm queda muy lejos de d_z. Esto
# motiva que `analizar_metrica` calcule d_z y d_av a mano (cf. l. 2417-2420)
# en lugar de delegar en `effsize::cohen.d`.
{
  g1_test <- c(1, 2, 3, 4, 5)
  g2_test <- c(0.5, 1.5, 2, 3, 4)
  d_effsize <- cohen.d(g1_test, g2_test, paired = TRUE)$estimate
  dz_manual <- mean(g1_test - g2_test) / sd(g1_test - g2_test)
  dav_manual <- mean(g1_test - g2_test) / sqrt((sd(g1_test)^2 + sd(g2_test)^2) / 2)
  r_pair <- cor(g1_test, g2_test)
  drm_pred <- dz_manual * sqrt(2 * (1 - r_pair))
  effsize_es_drm <- abs(d_effsize - drm_pred) < 0.01

  cat(sprintf("effsize::cohen.d (paired)           = %.4f\n", d_effsize))
  cat(sprintf("d_z manual [mean/sd(diff)]         = %.4f\n", dz_manual))
}

```

```

cat(sprintf("d_av manual [mean/SDpool]          = %.4f\n", dav_manual))
cat(sprintf("d_rm = d_z · sqrt(2·(1-r))          = %.4f   (r = %.4f)\n",
            drm_pred, r_pair))
cat(sprintf("effsize coincide con d_rm: %s\n",
            ifelse(effsize_es_drm,
                  "SI - confirmado (effsize devuelve d_rm)",
                  "NO - variante de effsize inesperada; revisar versión del paquete")))
cat("Decisión: §6.1 calcula d_z y d_av a mano; effsize::cohen.d no se usa.\n")
}

# -- Función de análisis estadístico PAREADO (dataset-agnóstica) -----
# C1/C2: etiquetas uppercase de condiciones (ej. "PSIL"/"MTP" o "LSD"/"PLCB").
# Siempre ambos tests - reporte paralelo, no selección condicional.
# Ref: Lumley et al. (2002, Annual Review of Public Health).
#
# Effect sizes (H2-C ronda 1): se reportan EN PARALELO las dos variantes
# pareadas que Lakens (2013) recomienda para diseño within-subject:
#   d_z = mean(diffs) / sd(diffs)                (test-coupled; usado
#                                                  por pwr.t.test para
#                                                  sensibilidad §7.1)
#   d_av = mean(diffs) / sqrt((sd(g1)^2 + sd(g2)^2)/2) (comparabilidad con
#                                                  literatura que reporta
#                                                  d entre grupos)
# Nota: `effsize::cohen.d(g1, g2, paired = TRUE)` devuelve d_rm =
# d_z·sqrt(2·(1-r)) (verificado empíricamente en el bloque diagnóstico
# previo) - NI d_z NI d_av. Por eso d_z y d_av se calculan a mano para
# evitar la ambigüedad del paquete.
analizar_metrica <- function(data, metric, C1, C2) {
  subs <- intersect(
    data %>% filter(condition == C1) %>% pull(subject),
    data %>% filter(condition == C2) %>% pull(subject)
  )
  if (length(subs) < 3)
    return(tibble(
      metric = metric,
      !!paste0("mean_", C1) := NA_real_,
      !!paste0("mean_", C2) := NA_real_,
      diff = NA_real_, p_value = NA_real_, p_wilcox = NA_real_,
      shapiro_p = NA_real_, d_z = NA_real_, d_av = NA_real_,
      method = "insufficient_n"))
  g1 <- (data %>% filter(condition == C1, subject %in% subs) %>%

```

```

      arrange(subject))[[metric]]
g2 <- (data %>% filter(condition == C2, subject %in% subs) %>%
      arrange(subject))[[metric]]
g1 <- as.numeric(g1); g2 <- as.numeric(g2)

diffs <- g1 - g2

if (length(unique(diffs[!is.na(diffs)])) < 2)
  return(tibble(
    metric          = metric,
    !!paste0("mean_", C1) := mean(g1, na.rm = TRUE),
    !!paste0("mean_", C2) := mean(g2, na.rm = TRUE),
    diff = mean(diffs, na.rm = TRUE), p_value = NA_real_, p_wilcox = NA_real_,
    shapiro_p = NA_real_, d_z = NA_real_, d_av = NA_real_,
    method = "constant"))

# H8-A r1 §6.3 (ronda 3 quirúrgica de §6.1, 2026-05-24): `na.rm = TRUE`
# en mean(diffs), sd(diffs), sd(g1), sd(g2) y los means del tibble final.
# Sin él, cualquier NA pareado en `diffs` propaga a `d_z`/`d_av` mientras
# que `t.test(paired=TRUE)` elimina pares con NA internamente y sí
# produce un p-value sobre el subconjunto restante - desacople entre
# test y effect size. Afloraba en §6.3 (b) NEG-por-red (cells con
# p_value pero d_z = NA).
test_t    <- tryCatch(t.test(g1, g2, paired = TRUE),          error = \(e) NULL),
test_w    <- tryCatch(wilcox.test(g1, g2, paired = TRUE, exact = FALSE), error = \(e) NULL),
p_shapiro <- tryCatch(shapiro.test(diffs[!is.na(diffs)])$p.value, error = \(e) NA_real_),
d_z       <- tryCatch(mean(diffs, na.rm = TRUE) / sd(diffs, na.rm = TRUE),
sd_pool   <- tryCatch(sqrt((sd(g1, na.rm = TRUE)^2 + sd(g2, na.rm = TRUE)^2) / 2),
d_av      <- tryCatch(mean(diffs, na.rm = TRUE) / sd_pool,

tibble(
  metric          = metric,
  !!paste0("mean_", C1) := mean(g1, na.rm = TRUE),
  !!paste0("mean_", C2) := mean(g2, na.rm = TRUE),
  diff            = mean(diffs, na.rm = TRUE),
  p_value         = if (!is.null(test_t)) test_t$p.value else NA_real_,
  p_wilcox        = if (!is.null(test_w)) test_w$p.value else NA_real_,
  d_z             = d_z,
  d_av           = d_av,
  shapiro_p       = p_shapiro,
  method          = "parallel (t + Wilcoxon)"
)

```

```

}

# -- Función principal por dataset: construye el estado completo de §6.1 ----
make_stats_13_1 <- function(cd, gd, am) {
  c1_key <- cd$conditions[1]; c2_key <- cd$conditions[2]
  C1 <- toupper(c1_key);      C2 <- toupper(c2_key)

  RESULTS_DS <- file.path(PROJECT_ROOT, "results", cd$id)
  out_dir <- file.path(RESULTS_DS, "stats/statistical_results")
  plot_dir <- file.path(RESULTS_DS, "figures/stats")
  out_adv <- file.path(RESULTS_DS, "stats/statistical_results/advanced")
  dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)
  dir.create(out_adv, showWarnings = FALSE, recursive = TRUE)

  # Paleta de condiciones: magenta #C2185B (PSIL) + teal oscuro #00695C
  # (MTP) - rojo/azul están
  # reservados para POS/NEG en §5 y §6.3.
  cond_colors <- setNames(c("#C2185B", "#00695C"), c(C1, C2))

  # participants.tsv: ds006072 únicamente (único dataset con drug1/drug2 publicado)
  order_tbl <- NULL
  if (cd$id == "ds006072") {
    if (file.exists(PARTICIPANTS_TSV_DS006072)) {
      participants <- read.delim(PARTICIPANTS_TSV_DS006072,
                                stringsAsFactors = FALSE)
      order_tbl <- participants %>%
        transmute(
          subject = participant_id,
          sequence = ifelse(drug1 == "PSIL", "PSIL_first", "MTP_first")
        ) %>%
        tidyr::crossing(condition = c("PSIL", "MTP")) %>%
        left_join(participants %>%
                  transmute(subject = participant_id, drug1, drug2),
                  by = "subject") %>%
        mutate(period = ifelse(condition == drug1, 1L, 2L)) %>%
        select(subject, condition, sequence, period)
      cat(sprintf("[%s] Tabla de orden - balance:\n", cd$id))
      print(table(order_tbl %>% distinct(subject, sequence) %>% pull(sequence)))
    } else {
      warning(sprintf("participants.tsv no encontrado - orden omitido para %s.", cd$id))
    }
  }
}

```

```

}

# Leer métricas globales/nodales desde gd$graph_dir
read_metrics_dir <- function(directory, condition_label) {
  files <- list.files(directory, pattern = "\\\\.csv$", full.names = TRUE)
  map_dfr(files, function(f) {
    df <- read.csv(f, stringsAsFactors = FALSE)
    df$subject <- sub("_.*", "", basename(f))
    df$condition <- condition_label
    df
  })
}

global_data <- bind_rows(
  read_metrics_dir(file.path(gd$graph_dir, "global", c1_key), C1),
  read_metrics_dir(file.path(gd$graph_dir, "global", c2_key), C2)
)

nodal_data <- bind_rows(
  read_metrics_dir(file.path(gd$graph_dir, "nodal", c1_key), C1),
  read_metrics_dir(file.path(gd$graph_dir, "nodal", c2_key), C2)
)

global_subj <- global_data %>%
  group_by(subject, condition) %>%
  summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE)),
    .groups = "drop")

nodal_subj <- nodal_data %>%
  group_by(subject, condition) %>%
  summarise(across(where(is.numeric) & !any_of("module"),
    \(x) mean(x, na.rm = TRUE)),
    .groups = "drop")

# Excluir métricas redundantes o no-hipótesis del pool FDR.
# Globales:
#   mean_degree      = transformación lineal de density;
#   n_communities + min_module_size + median_module_size + n_singletons =
#   diagnósticos de la partición consensuada, no objeto
#   de inferencia;
#   Q_range / Q_sd / cons_iterations = diagnósticos de convergencia Louvain.
# Nodales (medias post-summarise sobre los 232 ROIs):
#   strength          = mean(strength)   mean_strength (global) por

```

```

#           construcción (l. 667 vs 775; mismo vector);
# degree      = mean(degree)      density × (n-1) en el grafo
#           umbralizado (linealidad equivalente a la de
#           mean_degree global con density);
# local_clustering = mean(transitivity weighted) clustering (global)
#           salvo isolates="zero" vs na.rm=TRUE - diferencia
#           numérica despreciable al 20 % de densidad;
# within_module_z = centrado por construcción → media 0
#           independientemente de condición.
# Pool resultante = 8 globales + 4 nodales (eigenvector, betweenness,
#           closeness, participation) + 1 avanzada (degree_entropy)
#           = 13 tests. La aritmética del FDR coincide con la
#           declaración del callout §6.1.1.
numeric_metrics_global <- setdiff(
  names(select(global_subj, where(is.numeric))),
  c("subject", "threshold_used", "mean_degree", "n_communities",
    "min_module_size", "median_module_size", "n_singletons",
    "Q_range", "Q_sd", "cons_iterations"))

numeric_metrics_nodal <- setdiff(
  names(select(nodal_subj, where(is.numeric))),
  c("subject", "module", "within_module_z",
    "strength", "degree", "local_clustering"))

results_global <- map_dfr(numeric_metrics_global,
  ~analizar_metrica(global_subj, .x, C1, C2))

# Marco estadístico S0 (2026-05-23): la corrección por atenuación basada en
# ICC test-retest se ha retirado del pipeline. `d_z` se reporta como tamaño
# de efecto primario (test-coupled con pwr.t.test en §7.1) y `d_av` se
# reporta en paralelo para comparabilidad con la literatura (Lakens 2013).
# La robustez se cierra en §7 con L00 + bootstrap + permutación intra-sujeto
# + convergencia inter-pipeline. La columna legacy `d_report` (alias de
# cohen_d) se retiró en la ronda 1 de revisión por ser duplicado puro.

results_nodal <- map_dfr(numeric_metrics_nodal,
  ~analizar_metrica(nodal_subj, .x, C1, C2))

# Métricas avanzadas (degree_entropy → pool FDR topología). La modularidad
# con signo Q* fue retirada de §3 en la ronda 1 de revisión (H2): la
# implementación reutilizaba la partición Louvain de §2.1 sobre |w|
# umbralizado, por lo que el Q* reportado era restringido a esa partición

```

```

# y no el Q* maximizado sobre signed que Rubinov & Sporns (2011) introducen.
# El pool topología combinado queda con 8 globales + 4 nodales no-redundantes
# + degree_entropy = 13 tests si degree_entropy está disponible; 12 si falta.
# Las tres nodales redundantes con globales (strength, degree, local_clustering)
# se retiraron del pool en la ronda 1 de revisión de §6.1.
results_advanced <- tibble()
if (!is.null(am$degree_entropy_results)) {
  results_advanced <- analizar_metrica(am$degree_entropy_results,
                                       "degree_entropy", C1, C2)
}

# Diagnóstico de normalidad (H11 ronda 1: ahora cubre las tres tablas del
# pool topología). Informativo - no selector de test primario; el Wilcoxon
# se reporta siempre en paralelo (Lumley 2002; Rochon 2012). Con N=6 el
# Shapiro-Wilk tiene potencia muy baja (S3 ronda 1): la ausencia de p<0.05
# no garantiza normalidad, sólo «no evidencia contra», y el cierre
# cuantitativo se hace en §7.3 (QQ plots).
normality_check <- bind_rows(
  results_global %>% mutate(plano = "global"),
  results_nodal %>% mutate(plano = "nodal"),
  results_advanced %>% mutate(plano = "avanzada")
) %>%
  filter(!is.na(shapiro_p) & shapiro_p < 0.05) %>%
  mutate(sig_t = !is.na(p_value) & p_value < 0.05,
         sig_w = !is.na(p_wilcox) & p_wilcox < 0.05,
         concordan = sig_t == sig_w)
if (nrow(normality_check) > 0) {
  cat(sprintf("\n[%s] NORMALIDAD - Shapiro-Wilk p < 0.05 (pool topología, las 3 tablas):\n",
             cd$id))
  for (i in seq_len(nrow(normality_check))) {
    nc <- normality_check[i, ]
    status <- if (nc$concordan) "OK - concuerdan"
               else "DISCREPANCIA - considerar Wilcoxon como primario"
    cat(sprintf(" · [%-8s] %-22s Shapiro p=%.3f | t p=%.4f | W p=%.4f | %s\n",
               nc$plano, nc$metric, nc$shapiro_p, nc$p_value, nc$p_wilcox, status))
  }
  if (all(normality_check$concordan))
    cat(" -> Todos concuerdan: no-normalidad no afecta conclusiones.\n")
}

# H5 ronda 1: `order_tbl` se imprime para transparencia pero no se incluye
# en la lista devuelta - ningún chunk de §6.2-§7 lo consume y mantenerlo

```

```

# como payload generaba ruido.
list(
  C1          = C1,
  C2          = C2,
  RESULTS_DS  = RESULTS_DS,
  out_dir     = out_dir,
  plot_dir    = plot_dir,
  out_adv     = out_adv,
  cond_colors = cond_colors,
  global_subj = global_subj,
  nodal_subj  = nodal_subj,
  numeric_metrics_global = numeric_metrics_global,
  numeric_metrics_nodal  = numeric_metrics_nodal,
  results_global  = results_global,
  results_nodal   = results_nodal,
  results_advanced = results_advanced,
  graph_dir       = gd$graph_dir,
  common_subjects = cd$common_subjects,
  pipeline_name   = cd$pipeline[1]
)
}

STATS_13_1 <- setNames(
  lapply(ACTIVE_DATASETS, function(ds)
    make_stats_13_1(CONNECTOMES[[ds]], GRAPHS[[ds]], ADVANCED_METRICS[[ds]]),
  ACTIVE_DATASETS)

```

6.1.1 6.1.1 FDR within-family: justificación y aplicación

! Nota metodológica — Estrategia de corrección por comparaciones múltiples (FDR)

Criterio: Se aplica BH-FDR **within-family** (por familia de tests) como criterio primario de significación en todas las secciones de §2.

Justificación:

1. **Definición de familia:** Cada sección del pipeline testa una pregunta conceptualmente distinta (métricas globales/nodales, POS/NEG, between-network, seed-based). Según Benjamini & Hochberg (1995), la familia de hipótesis debe reflejar un conjunto de tests que responden a una misma pregunta inferencial. Agrupar todos los tests del pipeline (~300) en un solo pool BH-FDR sería excesivamente conservador y no refleja la estructura jerárquica del análisis.

2. **Estándar en neuroimagen:** La corrección within-family es la práctica habitual en estudios de conectividad funcional (Zalesky et al., 2010; Fornito et al., 2016, *Fundamentals of Brain Network Analysis*).
3. **Potencia estadística:** Con N pequeño (N=6 en ds006072, N=15 en ds003059), la potencia ya es limitada. Un pool global penalizaría desproporcionadamente las familias pequeñas (ej. las 13 métricas topología corregidas junto con 200 tests seed-based).

Familias definidas:

Sección	Familia	N tests	Pool FDR
§2.1 + §3.2	Métricas globales + nodales no-redundantes + avanzadas (topología)	13	Conjunto (misma pregunta: PSIL MTP en grafos). 8 globales + 4 nodales (eigenvector, betweenness, closeness, participation) + 1 avanzada (degree_entropy, Viol 2017). Se retiraron del pool las nodales strength , degree y local_clustering por ser redundantes con mean_strength , density × (n-1) y clustering respectivamente (ronda 1 de revisión §6.1). La modularity_signed Q* se retiró de §3 en la ronda 1 (Q* restringido a la partición Louvain de §2.1, no maximizado sobre signed). Independiente (BH-FDR conjunta, dominio dinámico). Agregado synch_entropy_assoc fuera del pool (a priori direccional).
§3.1	Entropía sincronía intra-red por red Yeo-7 (Carhart-Harris 2014 #1)	7	

§6.2	Convergencia inter-pipeline (pool topología por pipeline)	13 por pipeline	Within-pipeline (BH sobre los 13 tests de cada pipeline) — preserva la semántica del FDR de §6.1 (no cross-pipeline). El criterio S0 mira sólo <code>sign(d_z)</code> (6/7 ds006072, 3/3 ds003059); la FDR within-pipeline se reporta en paralelo como <code>n_sig_FDR</code> para transparencia, no entra en el criterio S0. Output: flag <code>converge_S0</code> en <code>pipeline_convergence_summary.csv</code>; tabla larga métrica × pipeline en <code>pipeline_convergence_long.csv</code>; heatmap en <code>pipeline_convergence_heatmap.png</code>.
§6.3	POS/NEG global	3	Independiente
§6.3	POS/NEG por red × métrica	7 por métrica	(conectividad signed) Por métrica (7 redes)
§6.4	Between-network (inter-red 8×8)	28	28 pares de redes
§6.5	Seed-based Tian (5 semillas subcortex × 232 ROIs)	232 por semilla	Within-seed (5 familias independientes BH-FDR). Análisis exploratorio terminal, no participa de la regla de credibilidad S0.
Referencia: Benjamini & Hochberg (1995); [Inferencia Estadística: U8, pág 14-17].			

```

# FDR conjunto within-family: globales + nodales + degree_entropy = "topología
# de grafo". Pool dinámico: 8 globales + 4 nodales no-redundantes + 1 avanzada
# (degree_entropy) = 13 tests si degree_entropy está disponible; 12 si falta.
# La modularity_signed Q* se retiró de §3 en la ronda 1 (H2). Las nodales
# redundantes con globales (strength, degree, local_clustering) se retiraron
# del pool en la ronda 1 de §6.1. La cuenta exacta del pool aplicado se
# imprime al final por dataset (cf. `message()` con length(all_p_91)).
for (ds in ACTIVE_DATASETS) {
  st <- STATS_13_1[[ds]]
  C1 <- st$C1; C2 <- st$C2

  all_p_91 <- c(st$results_global$p_value,
               st$results_nodal$p_value,
               st$results_advanced$p_value)
  all_fdr_91 <- p.adjust(all_p_91, method = "fdr")

  n_global <- nrow(st$results_global)
  n_nodal <- nrow(st$results_nodal)
  n_advanced <- nrow(st$results_advanced)

  st$results_global$p_FDR <- all_fdr_91[seq_len(n_global)]
  st$results_global <- st$results_global %>%
    arrange(p_FDR) %>%
    mutate(significant = !is.na(p_FDR) & p_FDR < 0.05)

  cat(sprintf("\n=== [%s] Resultados globales ===\n", ds))
  # Marco S0 (2026-05-23): d_z como effect size primario; d_av en paralelo
  # para comparabilidad (Lakens 2013). La robustez se cierra en §7 (L00 +
  # bootstrap + permutación + convergencia inter-pipeline).
  col_c1 <- paste0("mean_", C1); col_c2 <- paste0("mean_", C2)
  st$results_global %>%
    select(any_of(c("metric", col_c1, col_c2, "diff", "d_z", "d_av",
                    "p_value", "p_wilcox", "p_FDR")))) %>%
    arrange(p_FDR) %>% print(n = Inf)

  st$results_nodal$p_FDR <- all_fdr_91[n_global + seq_len(n_nodal)]
  st$results_nodal <- st$results_nodal %>%
    arrange(p_FDR) %>%
    mutate(significant = !is.na(p_FDR) & p_FDR < 0.05)

  st$results_advanced$p_FDR <- all_fdr_91[n_global + n_nodal + seq_len(n_advanced)]
  st$results_advanced <- st$results_advanced %>%

```

```

    arrange(p_FDR) %>%
    mutate(significant = !is.na(p_FDR) & p_FDR < 0.05)

write.csv(st$results_global,
          file.path(st$out_dir, "global_metrics_stats.csv"), row.names = FALSE)
write.csv(st$results_nodal,
          file.path(st$out_dir, "nodal_metrics_stats.csv"), row.names = FALSE)
write.csv(st$results_advanced,
          file.path(st$out_adv, "advanced_tests.csv"), row.names = FALSE)
message(sprintf("[%s] FDR §6.1: %d tests (global=%d + nodal=%d + avanzadas=%d)",
                ds, length(all_p_91), n_global, n_nodal, n_advanced))

# H10 ronda 1: la `table_TFM_style.csv` se retiró por ser un subset
# huérfano (sin consumidor downstream) de `global_metrics_stats.csv`.
# Para reconstruirla, basta con leer global_metrics_stats.csv y
# seleccionar las columnas {metric, mean_*, diff, d_z, d_av, p_value,
# p_wilcox, p_FDR, significant}.

STATS_13_1[[ds]] <- st
}

```

6.1.2 6.1.2 Visualización de resultados: boxplots, rainclouds, forest-style dot plot

```

for (ds in ACTIVE_DATASETS) {
  st      <- STATS_13_1[[ds]]
  C1      <- st$C1; C2 <- st$C2
  ds_tag  <- sprintf("%s vs %s", C1, C2)
  cond_colors <- st$cond_colors
  am      <- ADVANCED_METRICS[[ds]]

  # H7 ronda 1: `results_nodal` agrupa por MÉTRICA (post-summarise por
  # sujeto), no por ROI - 4 filas tras la depuración del pool en ronda 1
  # (eigenvector, betweenness, closeness, participation). Por eso el
  # ranking y el top son por métrica, no por ROI.
  metric_ranking <- st$results_nodal %>% filter(!is.na(d_z)) %>%
    arrange(desc(abs(d_z)))
  top_metrics <- head(metric_ranking, 10)

  # Boxplots globales
  for (m in st$numeric_metrics_global) {

```

```

p <- ggplot(st$global_subj, aes(x=condition, y=.data[[m]], fill=condition)) +
  geom_boxplot(width=0.5, outlier.shape=NA) +
  geom_jitter(width=0.12, size=2.5, alpha=0.8) +
  scale_fill_manual(values = cond_colors) +
  theme_minimal(base_size=13) +
  theme(legend.position="none",
        plot.title=element_text(size=10)) +
  labs(title=sprintf("[%s · Global] Boxplot | %s | %s", ds, m, ds_tag),
        x="Condición", y=m)
ggsave(file.path(st$plot_dir, paste0("boxplot_", m, ".png")),
        p, width=7, height=5, dpi=300)
}

# Rainclouds globales (las 8 métricas del pool) + composite 2x4
# ordenado por p_FDR ascendente, con bordes que codifican el nivel de
# evidencia (negro: p_t<0.05 cruzan nominal - máxima saliencia; gris
# oscuro: cluster borderline p_FDR<0.15; sin borde para n.s.).
# Sustituye al bloque previo que sólo dibujaba las tres primeras
# métricas, para que el informe R3 pueda mostrar una única figura con
# la lectura completa del bloque global ordenada por evidencia.
#
# `geom_half_violin(trim=FALSE, scale="width")`: imprescindible para que
# métricas con varianza inter-sujeto muy baja (típicamente `density`
# bajo PSIL, donde los 6 sujetos saturan el techo del threshold
# proporcional 20%) no degeneren a un violin invisible; con `scale="width"`
# cada distribución ocupa el ancho disponible aunque su KDE sea muy
# puntiaguda, y `trim=FALSE` extiende las colas hasta el extremo del
# soporte teórico.
rg_panel <- st$results_global %>%
  filter(metric %in% st$numeric_metrics_global, !is.na(p_FDR)) %>%
  mutate(
    relevance = case_when(
      !is.na(p_value) & p_value < 0.05 ~ "sig",
      !is.na(p_FDR) & p_FDR < 0.15 ~ "border",
      TRUE ~ "ns"
    ),
    border_col = case_when(
      relevance == "sig" ~ "#000000",
      relevance == "border" ~ "#7F7F7F",
      TRUE ~ NA_character_
    ),
    border_lwd = case_when(

```

```

      relevance == "sig"      ~ 1.6,
      relevance == "border" ~ 1.2,
      TRUE                  ~ 0
    ),
    title_col = case_when(
      relevance == "sig"      ~ "#000000",
      relevance == "border" ~ "#404040",
      TRUE                  ~ "grey45"
    ),
    title_face = ifelse(relevance %in% c("sig","border"), "bold", "plain")
  ) %>%
  arrange(p_FDR, p_value)

make_global_raincloud <- function(m, p_t, p_FDR, dz,
                                  border_col, border_lwd,
                                  title_col, title_face) {
  df_rc <- st$global_subj %>% transmute(subject, condition, value=.data[[m]])
  panel_title <- sprintf("%s (d_z = %+2f)\np_t = %.3f · p_FDR = %.3f",
                        m, dz, p_t, p_FDR)
  bg_rect <- if (is.na(border_col)) {
    element_blank()
  } else {
    element_rect(color = border_col, fill = NA, linewidth = border_lwd)
  }
  ggplot(df_rc, aes(x=condition, y=value, fill=condition)) +
    gghalves::geom_half_violin(side="l", alpha=0.5, trim=FALSE,
                              scale="width") +
    geom_boxplot(width=0.16, outlier.shape=NA, fatten=2, alpha=0.85) +
    geom_line(aes(group=subject), color="grey55", linewidth=0.4, alpha=0.7) +
    geom_jitter(width=0.08, size=2.4, alpha=0.85,
                color = "grey15", stroke = 0.3) +
    scale_fill_manual(values = cond_colors) +
    theme_minimal(base_size=11) +
    theme(legend.position="none",
          plot.title=element_text(size=9, color=title_col, face=title_face,
                                   hjust=0.5),
          plot.background=bg_rect,
          plot.margin=margin(6, 6, 6, 6)) +
    labs(title=panel_title, y=m, x=NULL)
}

raincloud_panels <- pmap(

```

```

list(rg_panel$metric, rg_panel$p_value, rg_panel$p_FDR, rg_panel$d_z,
     rg_panel$border_col, rg_panel$border_lwd,
     rg_panel$title_col, rg_panel$title_face),
make_global_raincloud
)
names(raincloud_panels) <- rg_panel$metric

# Mantener los PNGs individuales (consumidos por sensitivity logs y
# como anclajes externos), pero ahora para las 8 métricas.
for (i in seq_along(raincloud_panels)) {
  ggsave(file.path(st$plot_dir,
                  paste0("raincloud_", names(raincloud_panels)[i], ".png")),
        raincloud_panels[[i]], width=7, height=5, dpi=300)
}

# Composite 3x2 con las 6 métricas más informativas del pool topología,
# ordenadas por |d_z| descendente para mostrar mayor saliencia arriba.
# mean_strength y modularity quedan fuera del composite (siguen disponibles
# como PNGs individuales en st$plot_dir) para permitir paneles más grandes
# en la memoria sin sobrecargar la lectura.
composite_order <- c("assortativity", "char_path_length",
                    "clustering",    "global_efficiency",
                    "density",       "small_worldness")
composite_panels <- raincloud_panels[composite_order]
composite_raincloud <- patchwork::wrap_plots(composite_panels,
                                             ncol = 2, nrow = 3) +

  patchwork::plot_annotation(
    title    = sprintf("[%s · Global] Rainclouds del pool topología (6 métricas focales)",
                      ds, ds_tag),
    subtitle = "Borde negro: p_t < 0.05 (cruzan nominal). Borde gris: p_FDR < 0.15 (clust",
    theme    = theme(plot.title    = element_text(size=12, face="bold"),
                     plot.subtitle = element_text(size=10))
  )
ggsave(file.path(st$plot_dir, "raincloud_global_composite.png"),
      composite_raincloud, width=9, height=13.5, dpi=300)

# Forest-style dot plot (nadales por métrica)
p_effect <- st$results_nodal %>%
  filter(!is.na(d_z)) %>%
  mutate(
    metric    = fct_reorder(metric, d_z),
    sig_label = case_when(

```



```

    !is.na(p_FDR) & p_FDR < 0.001 ~ "FDR<0.001",
    !is.na(p_FDR) & p_FDR < 0.01 ~ "FDR<0.01",
    !is.na(p_FDR) & p_FDR < 0.05 ~ "FDR<0.05",
    TRUE ~ "n.s."
  ),
  sig_label = factor(sig_label, levels = c("FDR<0.001", "FDR<0.01", "FDR<0.05", "n.s."))
) %>%
ggplot(aes(x = d_z, y = metric, color = sig_label)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey50") +
  geom_segment(aes(xend = 0, yend = metric), color = "grey80", linewidth = 0.4) +
  geom_point(aes(size = sig_label)) +
  geom_text_repel(aes(label = ifelse(sig_label != "n.s.",
                                     sprintf("dz=%.2f", d_z), "")),
                 size = 3, nudge_x = 0.02) +
  scale_color_manual(
    values = c("FDR<0.001"="#C0392B", "FDR<0.01"="#E67E22", "FDR<0.05"="#2980B9", "n.s."="grey"),
    name = "Significación") +
  scale_size_manual(values = c("FDR<0.001"=4, "FDR<0.01"=3.5, "FDR<0.05"=3, "n.s."=2),
                    guide = "none") +
  theme_minimal(base_size = 13) +
  labs(title = sprintf("[%s · Nodal] Efecto vs significación | %s", ds, ds_tag),
       x = expression(d[z]~de~Cohen), y = NULL,
       caption = "Punto = d_z; color = nivel FDR; métricas ordenadas por d_z")
ggsave(file.path(st$plot_dir, "effect_size_significance.png"),
       p_effect, width = 8, height = 6, dpi = 300)

p_metric_map <- st$results_nodal %>%
  filter(!is.na(d_z)) %>%
  mutate(metric=fct_reorder(metric, d_z)) %>%
  ggplot(aes(x=1, y=metric, fill=d_z)) +
  geom_tile() +
  scale_fill_gradient2(low="blue", mid="white", high="red", midpoint=0, name="d_z") +
  theme_void(base_size=9) + theme(axis.text.y=element_text(size=7)) +
  labs(title=sprintf("[%s · Nodal] Tamaño del efecto | %s", ds, ds_tag))
ggsave(file.path(st$plot_dir, "effect_size_map.png"),
       p_metric_map, width=5, height=10, dpi=300)

p_bar_top_metrics <- ggplot(top_metrics, aes(x=fct_reorder(metric, abs(d_z)), y=abs(d_z)))
  geom_col(fill="steelblue") + coord_flip() +
  theme_minimal(base_size=10) +
  labs(title=sprintf("[%s · Nodal] Top métricas por |d_z| | %s", ds, ds_tag),
       x="", y="|d_z|")

```

```

paper_fig <- (p_effect | p_bar_top_metrics) / p_metric_map +
  plot_layout(heights=c(2,1)) +
  plot_annotation(title=sprintf("[%s] Resumen estadístico | %s", ds, ds_tag))
ggsave(file.path(st$plot_dir,"paper_figure.png"),
  paper_fig, width=12, height=10, dpi=300)

# Métricas avanzadas
plot_dir_adv_ds <- file.path(st$RESULTS_DS, "figures/stats/advanced")
dir.create(plot_dir_adv_ds, showWarnings = FALSE, recursive = TRUE)

raincloud_adv <- function(df, ylab, title, subtitle) {
  ggplot(df, aes(x = condition, y = value, fill = condition)) +
    ggthemes::geom_half_violin(side = "l", alpha = 0.45, trim = FALSE) +
    geom_boxplot(width = 0.14, outlier.shape = NA, alpha = 0.85) +
    geom_jitter(width = 0.07, size = 2.5, alpha = 0.85) +
    geom_line(aes(group = subject), color = "grey55", linewidth = 0.4, alpha = 0.6) +
    scale_fill_manual(values = cond_colors) +
    theme_minimal(base_size = 12) + theme(legend.position = "none") +
    labs(title = title, subtitle = subtitle, x = "Condición", y = ylab)
}

# Tras §3 r1 (retirada de Q* signed y SampEn) y §6.1 r1 sólo queda una
# métrica avanzada visualizable (degree_entropy). El panel combinado de
# métricas avanzadas se eliminó en §6.1 r1 (H9) por dead-code: con un
# único panel coincidía con el raincloud individual ya escrito.
if (!is.null(am$degree_entropy_results)) {
  de_long <- am$degree_entropy_results %>%
    rename(value = degree_entropy) %>% filter(!is.na(value))
  if (nrow(de_long) > 0) {
    p_deg_entropy <- raincloud_adv(de_long, "S de la distribución de grados (bits)",
      sprintf("[%s · Global] Shannon entropy de grados", ds),
      "Viol et al. 2017 - entropic brain hypothesis")
    ggsave(file.path(plot_dir_adv_ds, "adv_raincloud_degree_entropy.png"),
      p_deg_entropy, width = 6, height = 5, dpi = 300)
  }
}
}

```

6.1.3 6.1.3 Análisis de influencia Leave-One-Out (LOO) — métricas globales y nodales

```
#=====
# § 6.1.3 - Análisis de influencia LOO - métricas globales y nodales
#=====

for (ds in ACTIVE_DATASETS) {
  st <- STATS_13_1[[ds]]
  C1 <- st$C1; C2 <- st$C2
  gd <- GRAPHS[[ds]]
  am <- ADVANCED_METRICS[[ds]]
  cd <- CONNECTOMES[[ds]]

  out_robust_dir <- file.path(st$RESULTS_DS, "stats/statistical_results/robust")
  dir.create(out_robust_dir, showWarnings = FALSE, recursive = TRUE)

  # --- Métricas globales ---
  global_long <- bind_rows(
    map_dfr(st$common_subjects, function(s) {
      f_c1 <- file.path(gd$graph_dir, "global", cd$conditions[1],
                        paste0(s, "_global_metrics.csv"))
      f_c2 <- file.path(gd$graph_dir, "global", cd$conditions[2],
                        paste0(s, "_global_metrics.csv"))
      if (!file.exists(f_c1) || !file.exists(f_c2)) return(NULL)
      bind_rows(
        read.csv(f_c1) %>% mutate(subject = s, condition = C1),
        read.csv(f_c2) %>% mutate(subject = s, condition = C2)
      )
    })
  )

  # LOO alineado al pool FDR (H6 ronda 1): se itera sobre todas las globales
  # del pool - `st$numeric_metrics_global` se construyó en `make_stats_13_1`
  # con las mismas exclusiones que el FDR conjunto.
  global_metrics_loo <- intersect(st$numeric_metrics_global, names(global_long))

  loo_global <- map(global_metrics_loo, function(m) {
    loo_influence_analysis(global_long, m, m, C1, C2)
  })
  names(loo_global) <- global_metrics_loo

  robust_global <- map_dfr(loo_global, ~.x$robust)
```

```

infl_global <- map_dfr(loo_global, ~.x$influence)
write.csv(robust_global,
          file.path(out_robust_dir, "global_metrics_robust.csv"), row.names = FALSE)
write.csv(infl_global,
          file.path(out_robust_dir, "global_metrics_influence.csv"), row.names = FALSE)

cat(sprintf("\n=== [%s] L00 - métricas globales ===\n", ds))
walk(names(loo_global), function(nm) {
  res <- loo_global[[nm]]
  cat(sprintf("%-22s | influyentes: %d | acuerdo 3 tests: %s\n",
              nm, res$n_influential,
              ifelse(res$agreement, "SI", "NO - revisar")))
})

# --- Métricas nodales (media por sujeto) ---
nodal_long <- bind_rows(
  map_dfr(st$common_subjects, function(s) {
    f_c1 <- file.path(gd$graph_dir, "nodal", cd$conditions[1],
                      paste0(s, "_nodal_metrics.csv"))
    f_c2 <- file.path(gd$graph_dir, "nodal", cd$conditions[2],
                      paste0(s, "_nodal_metrics.csv"))
    if (!file.exists(f_c1) || !file.exists(f_c2)) return(NULL)
    bind_rows(
      read.csv(f_c1) %>% mutate(subject = s, condition = C1),
      read.csv(f_c2) %>% mutate(subject = s, condition = C2)
    )
  })
)

# L00 nodal alineado al pool FDR (H6 ronda 1): se promedia cada métrica
# nodal no-redundante por sujeto×condición (cf. `numeric_metrics_nodal`
# construido en `make_stats_13_1` con las exclusiones de strength, degree
# y local_clustering por redundancia con globales). El nombre del L00 se
# prefija con `nodal_` para que no colisione con el de la métrica global
# cuando existan ambos en el output.
nodal_metrics_loo <- intersect(st$numeric_metrics_nodal, names(nodal_long))
nodal_mean_long <- nodal_long %>%
  group_by(subject, condition) %>%
  summarise(across(all_of(nodal_metrics_loo), \(x) mean(x, na.rm = TRUE)),
            .groups = "drop")

loo_nodal <- map(nodal_metrics_loo, function(m) {

```

```

    loo_influence_analysis(nodal_mean_long, m, paste0("nodal_", m), C1, C2)
  })
  names(loo_nodal) <- paste0("nodal_", nodal_metrics_loo)

  robust_nodal <- map_dfr(loo_nodal, ~.x$robust)
  infl_nodal   <- map_dfr(loo_nodal, ~.x$influence)
  write.csv(robust_nodal,
            file.path(out_robust_dir, "nodal_metrics_robust.csv"), row.names = FALSE)
  write.csv(infl_nodal,
            file.path(out_robust_dir, "nodal_metrics_influence.csv"), row.names = FALSE)

  cat(sprintf("\n=== [%s] L00 - métricas nodales (media por sujeto) ===\n", ds))
  walk(names(loo_nodal), function(nm) {
    res <- loo_nodal[[nm]]
    cat(sprintf("%-30s | influyentes: %d | acuerdo 3 tests: %s\n",
              nm, res$n_influential,
              ifelse(res$agreement, "SI", "NO - revisar"))))
  })

  # --- Métricas avanzadas: degree_entropy ---
  # (la modularity_signed Q* fue retirada de §3 en la ronda 1 de revisión)
  loo_advanced <- list()
  if (!is.null(am$degree_entropy_results)) {
    loo_advanced$adv_degree_entropy <- loo_influence_analysis(
      am$degree_entropy_results, "degree_entropy", "adv_degree_entropy", C1, C2)
  }
  loo_advanced <- loo_advanced[!vapply(loo_advanced, is.null, logical(1))]

  robust_adv <- map_dfr(loo_advanced, ~.x$robust)
  infl_adv   <- map_dfr(loo_advanced, ~.x$influence)
  write.csv(robust_adv,
            file.path(out_robust_dir, "advanced_robust.csv"), row.names = FALSE)
  write.csv(infl_adv,
            file.path(out_robust_dir, "advanced_influence.csv"), row.names = FALSE)

  cat(sprintf("\n=== [%s] L00 - métricas avanzadas ===\n", ds))
  walk(names(loo_advanced), function(nm) {
    res <- loo_advanced[[nm]]
    cat(sprintf("%-25s | influyentes: %d | acuerdo 3 tests: %s\n",
              nm, res$n_influential,
              ifelse(res$agreement, "SI", "NO - revisar"))))
    if (res$n_influential > 0) {

```

```

    cat("  Sujetos influyentes:\n")
    print(res$influence %>%
          filter(influential) %>%
          select(any_of(c("subject", C1, C2, "diff",
                          "loo_influence", "diff_zscore"))))
  }
})

message(sprintf("[%s] OK §6.1 pipeline estadístico completado", ds))
}

```

6.2 6.2 Convergencia inter-pipeline del contraste — operacionalización del criterio S0

Esta sub-sección **operacionaliza** el criterio de convergencia inter-pipeline definido en el callout-important S0 de §6.1 (cf. callout §6.1 para la definición canónica de **converge_S0** y **S0_THRESHOLD_BY_DS**). Hasta ahora ese criterio vivía sólo como promesa textual: §4 calcula correlaciones r entre los **valores** de las métricas en pipelines distintos (control técnico: ¿los pipelines producen valores parecidos?), pero **no** ejecuta el contraste PSIL vs MTP dentro de cada pipeline (control inferencial: ¿el efecto se sostiene al cambiar el preprocesamiento?). §6.2 cierra esa brecha replicando los tests pareados de §6.1 sobre el pool topología en cada pipeline de control y produciendo, por métrica, un *flag* de convergencia que se incorpora al juicio de credibilidad.

Diseño operativo. El pool de §6.2 es el mismo de §6.1 — las 13 métricas del callout §6.1.1 (8 globales + 4 nodales agregadas por sujeto + **degree_entropy**). El pipeline GSR (0C+36P+GSR en ds006072, AsIs+GSR en ds003059) **cuenta como uno más del pool** — la discordancia GSR no-GSR es por sí misma información de credibilidad (Murphy y Fox, 2017) y el **summary** añade columnas **n_dir_noGSR** para discutirla sin duplicar la lógica de inferencia. El criterio S0 no exige significación estadística por pipeline (con N=6 sería excesivamente restrictivo), pero el **summary** reporta **n_sig_raw** y **n_sig_FDR** en paralelo para transparencia.

Reutilización vs recálculo. El pipeline primario reutiliza el pool ya construido en §6.1 / §3 — **STATS_13_1[[ds]]\$global_subj**, **\$nodal_subj** y **ADVANCED_METRICS[[ds]]\$degree_entropy_results** — sin recalcular. Para cada pipeline de **control** se cargan las matrices con **load_subject_matrices()** (§1.1) y se recalculan las métricas con las mismas funciones que §2.1 (**mat_to_igraph_full**, **mat_to_igraph**, **calc_global_metrics**, **calc_nodal_metrics**) y la **shannon_entropy_degree** de §3. El umbral proporcional es siempre el primario (**PRIMARY_THRESHOLD = 0.20**), de modo que la única fuente de varianza entre pipelines es el preprocesamiento.

Output. Por dataset se escriben **results/<ds>/stats/statistical_results/pipeline_convergence_long.** (tabla larga métrica × pipeline con **d_z**, **d_av**, **p_value**, **p_wilcox**, **p_FDR**, **sig_raw**, **sig_FDR**, **sign_dz**, **dir_match**), **pipeline_convergence_summary.csv** (una fila por métrica con **n_dir**,

`n_dir_noGSR`, `n_sig_raw`, `n_sig_FDR`, `n_total`, `dz_primary`, `converge_S0`) y un heatmap métrica $\times$ pipeline coloreado por d_z con marcador de significación. La FDR es **within-pipeline** (BH sobre los 13 tests de cada pipeline) — preserva la semántica del FDR de §6.1; una FDR cross-pipeline confundiría sensibilidad metodológica con significación.

Acoplamiento productor $\rightarrow$ consumidor. §6.2 consume `CONNECTOMES`, `GRAPHS`, `STATS_13_1`, `ADVANCED_METRICS` (§1–§3, §6.1) y `CONTROL_PIPELINES_BY_DS` (§4). Produce CSVs + figura por dataset; no escribe estado en memoria consumido aguas abajo (no hace falta shim). §7 lo lee sólo en prosa.

Coste. Carga + grafos + métricas para 6 pipelines de control en $ds006072 \times 6$ sujetos $\times$ 2 condiciones + 2 pipelines en $ds003059 \times 15$ sujetos $\times$ 2 condiciones. Orden de magnitud del coste actual de §4, ampliado del pool de 4 métricas básicas a las 13 del pool topología completo.

```
#####
# § 6.2 - Convergencia inter-pipeline del contraste - criterio S0
#####
# Replica los tests pareados del pool topología (13 métricas) en cada pipeline
# de denoising y produce el flag `converge_S0` por métrica. Definición canónica
# del criterio (`n_dir >= S0_THRESHOLD_BY_DS`) y la tabla de umbrales: callout-
# important §6.1.
#
# Pool = 8 globales (mean_strength, global_efficiency, char_path_length,
#           density, clustering, modularity, assortativity, small_worldness)
#       + 4 nodales (eigenvector, betweenness, closeness, participation;
#           agregadas por sujeto $\times$ condición = mean sobre 232 ROIs)
#       + 1 avanzada (degree_entropy de Viol 2017, sobre grafo umbralizado
#           al PRIMARY_THRESHOLD = 20 %)
#       = 13 tests por pipeline.
#
# Primario: reusa STATS_13_1[[ds]]$global_subj / $nodal_subj y
#           ADVANCED_METRICS[[ds]]$degree_entropy_results (ya calculados).
# Controles: recalcula matrices  $\rightarrow$  grafos  $\rightarrow$  métricas con las MISMAS funciones
#           que §2.1 (mat_to_igraph_full, mat_to_igraph, calc_global_metrics,
#           calc_nodal_metrics) + shannon_entropy_degree de §3.
#
# GSR: cuenta como un pipeline más del pool. El summary añade columna
# n_dir_noGSR para discutir Murphy & Fox (2017) sin duplicar la lógica.
#
# Outputs (por dataset):
#   results/<ds>/stats/statistical_results/pipeline_convergence_long.csv
#   results/<ds>/stats/statistical_results/pipeline_convergence_summary.csv
#   results/<ds>/figures/stats/pipeline_convergence_heatmap.png
```

```

#
# FDR within-pipeline (BH sobre los 13 tests de cada pipeline) - alineado a
# la fila "$6.2" del callout $6.1.1.
#=====

library(tidyverse)
library(igraph)

# Estado en memoria heredado de las secciones previas (cd, gd, am, ctrl, etc.).
stopifnot(
  "$6.2 requiere STATS_13_1 (ejecutar $6.1 antes)" = exists("STATS_13_1"),
  "$6.2 requiere ADVANCED_METRICS (ejecutar $3 antes)" = exists("ADVANCED_METRICS"),
  "$6.2 requiere CONTROL_PIPELINES_BY_DS (ejecutar $4 antes)" = exists("CONTROL_PIPELINES_BY_DS"),
  "$6.2 requiere CONNECTOMES (ejecutar $1.1 antes)" = exists("CONNECTOMES"),
  "$6.2 requiere ACTIVE_DATASETS (definido en $1.1)" = exists("ACTIVE_DATASETS"),
  "$6.2 requiere PRIMARY_THRESHOLD (definido en $2.1)" = exists("PRIMARY_THRESHOLD")
)

# Funciones puras heredadas de $1.1, $2.1, $3 y $6.1 - error temprano si la
# sesión no ha cargado los chunks previos en orden (mensaje legible vs el
# "could not find function" críptico de R).
stopifnot(
  "$6.2 requiere load_subject_matrices() de $1.1" = exists("load_subject_matrices"),
  "$6.2 requiere mat_to_igraph_full() de $2.1" = exists("mat_to_igraph_full"),
  "$6.2 requiere mat_to_igraph() de $2.1" = exists("mat_to_igraph"),
  "$6.2 requiere calc_global_metrics() de $2.1" = exists("calc_global_metrics"),
  "$6.2 requiere calc_nodal_metrics() de $2.1" = exists("calc_nodal_metrics"),
  "$6.2 requiere shannon_entropy_degree() de $3" = exists("shannon_entropy_degree"),
  "$6.2 requiere analizar_metrica() de $6.1" = exists("analizar_metrica")
)

POOL_GLOBAL <- c("mean_strength", "global_efficiency", "char_path_length",
  "density", "clustering", "modularity", "assortativity",
  "small_worldness")
POOL_NODAL <- c("eigenvector", "betweenness", "closeness", "participation")
POOL_ADVANCED <- "degree_entropy"
POOL_ALL <- c(POOL_GLOBAL, POOL_NODAL, POOL_ADVANCED)

# Check defensivo: si en una ronda futura `calc_global_metrics` o
# `calc_nodal_metrics` ganan/pierden una métrica, $6.1 ajustará su pool
# automáticamente (vía `numeric_metrics_global/_nodal`) pero $6.2 seguiría
# con sus constantes hardcoded. Este `stopifnot` falla ruidosamente si los

```



```

# dos pools divergen, en lugar de propagar resultados parciales aguas abajo.
for (.ds_check in ACTIVE_DATASETS) {
  .st <- STATS_13_1[.ds_check]
  stopifnot(
    "POOL_GLOBAL desincronizado con numeric_metrics_global (§6.1)" =
      setequal(POOL_GLOBAL, .st$numeric_metrics_global),
    "POOL_NODAL desincronizado con numeric_metrics_nodal (§6.1)" =
      setequal(POOL_NODAL, .st$numeric_metrics_nodal)
  )
}
rm(.ds_check, .st)

# Umbral S0 por dataset - cf. callout-important §6.1.
SO_THRESHOLD_BY_DS <- list(ds006072 = 6L, ds003059 = 3L)

# Recalcula el pool (13 métricas) para un pipeline desde sus matrices
compute_pool_pipeline <- function(s1, s2, atlas_labels, C1, C2, subjs) {
  combos <- tidyr::crossing(subject = subjs, condition = c(C1, C2))
  purrr::pmap_dfr(combos, function(subject, condition) {
    mat <- if (condition == C1) s1[[subject]] else s2[[subject]]
    if (is.null(mat)) return(NULL)
    G_full <- mat_to_igraph_full(mat, atlas_labels)
    G_thr <- mat_to_igraph(mat, atlas_labels)
    gm <- calc_global_metrics(G_full, G_thr, mat)
    nm <- calc_nodal_metrics(G_full, G_thr,
                           louvain_memb = attr(gm, "louvain_membership"))
    deg_S <- shannon_entropy_degree(igraph::degree(G_thr))
    tibble(
      subject = subject, condition = condition,
      !!!as.list(gm[1, POOL_GLOBAL]),
      !!!setNames(lapply(POOL_NODAL, \(c) mean(nm[[c]]), na.rm = TRUE)),
        POOL_NODAL),
      degree_entropy = deg_S
    )
  })
}

# Pool del PRIMARIO ya construido en §6.1 / §3
get_primary_pool <- function(ds) {
  st <- STATS_13_1[[ds]]
  am <- ADVANCED_METRICS[[ds]]
  g <- st$global_subj %>% select(subject, condition, any_of(POOL_GLOBAL))
}

```

```

n <- st$nodal_subj %>% select(subject, condition, any_of(POOL_NODAL))
d <- if (!is.null(am$degree_entropy_results))
  am$degree_entropy_results %>% select(subject, condition, degree_entropy)
else
  tibble(subject = character(), condition = character(),
          degree_entropy = numeric())
reduce(list(g, n, d), full_join, by = c("subject", "condition"))
}

for (ds in ACTIVE_DATASETS) {
  st <- STATS_13_1[[ds]]
  cd <- CONNECTOMES[[ds]]
  ctrl <- CONTROL_PIPELINES_BY_DS[[ds]]
  if (is.null(ctrl)) {
    warning(sprintf("$6.2 (%s): CONTROL_PIPELINES_BY_DS sin entrada - saltando.", ds))
    next
  }
  C1 <- st$C1; C2 <- st$C2
  c1_key <- cd$conditions[1]; c2_key <- cd$conditions[2]
  primary <- st$pipeline_name
  out_dir <- st$out_dir
  plot_dir <- st$plot_dir
  s0_thr <- S0_THRESHOLD_BY_DS[[ds]] %||% (nrow(ctrl) + 1L)

  cat(sprintf("\n##### $6.2 Convergencia inter-pipeline - %s (primario %s, criterio
              ds, primary, s0_thr))

  # 1) Pool del primario (preconstruido).
  pool_primary <- get_primary_pool(ds) %>%
    mutate(pipeline = primary, has_gsr = FALSE)

  # 2) Pool por cada pipeline de control - recálculo desde las matrices.
  pool_controls <- map(seq_len(nrow(ctrl)), function(i) {
    name <- ctrl$name[i]; conn_dir <- ctrl$conn_dir[i]; has_gsr <- ctrl$has_gsr[i]
    d1 <- file.path(conn_dir, c1_key); d2 <- file.path(conn_dir, c2_key)
    if (!dir.exists(d1) || !dir.exists(d2)) {
      warning(sprintf("$6.2 (%s) | %s: directorios %s/%s no encontrados - saltando.",
                      ds, name, basename(d1), basename(d2)))
    }
    return(NULL)
  })
  s1 <- load_subject_matrices(d1, sprintf("_mean_%s\\.csv$", c1_key))
  s2 <- load_subject_matrices(d2, sprintf("_mean_%s\\.csv$", c2_key))

```

```

subjs <- intersect(intersect(names(s1), names(s2)), cd$common_subjects)
if (length(subjs) == 0L) {
  warning(sprintf("$6.2 (%s) | %s: sin sujetos comunes con %s - saltando.",
    ds, name, primary))
  return(NULL)
}
message(sprintf(" %s: recalculando pool topología (n=%d sujetos comunes)...",
  name, length(subjs)))
compute_pool_pipeline(s1, s2, cd$atlas_labels, C1, C2, subjs) %>%
  mutate(pipeline = name, has_gsr = has_gsr)
})
pool_controls <- pool_controls[!vapply(pool_controls, is.null, logical(1))]
pool_long <- bind_rows(pool_primary, bind_rows(pool_controls))

if (n_distinct(pool_long$pipeline) < s0_thr) {
  warning(sprintf("$6.2 (%s): sólo %d pipelines cargados (umbral S0 = %d). El criterio num
    ds, n_distinct(pool_long$pipeline), s0_thr, ds))
}

# 3) Tests pareados por (pipeline × métrica) - reusa analizar_metrica ($6.1).
results_long <- map_dfr(unique(pool_long$pipeline), function(pipe) {
  df_pipe <- pool_long %>% filter(pipeline == pipe)
  has_gsr_pipe <- unique(df_pipe$has_gsr)
  map_dfr(P00L_ALL, function(m) {
    if (!m %in% names(df_pipe)) return(NULL)
    analizar_metrica(df_pipe, m, C1, C2) %>%
      mutate(pipeline = pipe, has_gsr = has_gsr_pipe, .before = 1)
  })
})

# 4) FDR within-pipeline + flags de significación + signo del efecto.
results_long <- results_long %>%
  group_by(pipeline) %>%
  mutate(p_FDR = p.adjust(p_value, method = "fdr")) %>%
  ungroup() %>%
  mutate(
    sig_raw = !is.na(p_value) & p_value < 0.05,
    sig_FDR = !is.na(p_FDR) & p_FDR < 0.05,
    sign_dz = sign(d_z)
  )

# 5) Dirección del primario por métrica → flag dir_match.

```

```

# Edge case intencional: si el primario tiene d_z = NA o sign = 0 en alguna
# métrica (p. ej. `degree_entropy` cuando `am$degree_entropy_results` es
# NULL), `dir_match` queda FALSE para todos los controles → `n_dir = 0` y
# `n_dir_noGSR = 0` → `converge_S0 = FALSE` aunque los controles converjan
# entre sí. La métrica «falla S0 por falta de referencia primaria», no por
# divergencia real; la pista para reconocer este caso en el summary es
# `dz_primary` = NA o `sign_primary` = 0 (filas que se ordenan al fondo
# por el `arrange(desc(abs(dz_primary)))` con NA tratado como -Inf).
dir_primary <- results_long %>%
  filter(pipeline == primary) %>%
  select(metric, sign_primary = sign_dz, dz_primary = d_z)
results_long <- left_join(results_long, dir_primary, by = "metric") %>%
  mutate(dir_match = !is.na(sign_dz) & !is.na(sign_primary) &
    sign_dz != 0 & sign_dz == sign_primary)

write.csv(results_long,
  file.path(out_dir, "pipeline_convergence_long.csv"),
  row.names = FALSE)

# 6) Summary por métrica: n_dir (incluye primario), n_dir_noGSR, n_sig_*,
# converge_S0. `n_total` (entero) se incluye como columna paramétrica para
# que el CSV permita calcular `n_dir / n_total` sin abrir el chunk.
n_total <- n_distinct(results_long$pipeline)
summary_tbl <- results_long %>%
  group_by(metric) %>%
  summarise(
    n_dir = sum(dir_match, na.rm = TRUE),
    n_dir_noGSR = sum(dir_match & !has_gsr, na.rm = TRUE),
    n_sig_raw = sum(sig_raw, na.rm = TRUE),
    n_sig_FDR = sum(sig_FDR, na.rm = TRUE),
    sign_primary = unique(sign_primary),
    dz_primary = unique(dz_primary),
    .groups = "drop"
  ) %>%
  mutate(
    n_total = n_total,
    converge_S0 = n_dir >= s0_thr
  ) %>%
  arrange(desc(converge_S0), desc(abs(dz_primary)))

write.csv(summary_tbl,
  file.path(out_dir, "pipeline_convergence_summary.csv"),

```

```

    row.names = FALSE)

# 7) Informe en consola.
cat(sprintf("\n == Convergencia S0 (%s, criterio %d/%d) ===\n", ds, s0_thr, n_total))
cat(sprintf(" %-22s %6s %12s %10s %10s %s\n",
            "Métrica", "n_dir", "dz_primario", "n_sig_raw", "n_sig_FDR", "Converge"))
cat(" ", strrep("-", 80), "\n", sep = "")
for (i in seq_len(nrow(summary_tbl))) {
  row_i <- summary_tbl[i, ]
  mark <- if (isTRUE(row_i$converge_S0)) " " else " "
  cat(sprintf(" %-22s %6d %12.3f %10d %10d %s\n",
              row_i$metric, row_i$n_dir, row_i$dz_primary,
              row_i$n_sig_raw, row_i$n_sig_FDR, mark))
}
n_converge <- sum(summary_tbl$converge_S0)
cat(sprintf("\n → %d/%d métricas convergen (S0: sign(d_z) consistente en %d/%d pipelines)
            n_converge, nrow(summary_tbl), s0_thr, n_total))

# 8) Figura: heatmap métrica × pipeline coloreado por d_z, marcador */** según p.
pipe_levels <- c(primary,
                  ctrl$name[!ctrl$has_gsr],
                  ctrl$name[ ctrl$has_gsr])
pipe_levels <- intersect(pipe_levels, unique(results_long$pipeline))
results_plot <- results_long %>%
  mutate(
    pipeline = factor(pipeline, levels = pipe_levels),
    metric    = factor(metric, levels = POOL_ALL),
    sig_lab   = dplyr::case_when(
      !is.na(p_FDR) & p_FDR < 0.05 ~ "***",
      !is.na(p_value) & p_value < 0.05 ~ "*",
      TRUE ~ ""
    )
  )
# Separadores verticales: la sólida marca el límite primario controles;
# la discontinua (sólo si hay al menos un pipeline GSR) marca el límite
# no-GSR GSR dentro del bloque de controles.
n_ctrl_noGSR <- sum(!ctrl$has_gsr)
has_any_gsr <- any(ctrl$has_gsr)
p_heat <- ggplot(results_plot,
                 aes(x = pipeline, y = fct_rev(metric), fill = d_z)) +
  geom_tile(color = "white", linewidth = 0.4) +
  geom_text(aes(label = sig_lab), size = 4, color = "black") +

```

```

    geom_vline(xintercept = 1.5, color = "grey25", linewidth = 0.7)
  if (has_any_gsr && n_ctrl_noGSR > 0L) {
    p_heat <- p_heat +
      geom_vline(xintercept = 1.5 + n_ctrl_noGSR,
                  color = "grey40", linewidth = 0.5, linetype = "dashed")
  }
  p_heat <- p_heat +
    scale_fill_gradient2(low = "#3498DB", mid = "white", high = "#E74C3C",
                          midpoint = 0, name = expression(d[z]),
                          na.value = "grey88") +
    labs(
      title = sprintf("[%s] Convergencia inter-pipeline | %s vs %s",
                       ds, C1, C2),
      subtitle = sprintf("Criterio S0 (dirección): %d/%d pipelines con sign(d_z)==sign del p
                          s0_thr, n_total, n_converge, nrow(summary_tbl)),
      caption = "Color = d_z. * p_raw<0.05 | ** p_FDR<0.05 (within-pipeline, BH). Vertical
      x = NULL, y = NULL
    ) +
    theme_minimal(base_size = 11) +
    theme(
      axis.text.x = element_text(angle = 35, hjust = 1),
      panel.grid = element_blank(),
      plot.title = element_text(face = "bold")
    )
  ggsave(file.path(plot_dir, "pipeline_convergence_heatmap.png"),
          p_heat,
          width = max(7, 1.05 * n_total + 2),
          height = max(5, 0.42 * length(POOL_ALL) + 2.2),
          dpi = 300)

  message(sprintf("[%s] OK §6.2 - convergencia inter-pipeline (CSVs + heatmap).", ds))
}

```

6.3 6.3 Conectividad positiva, negativa y anticorrelaciones (POS/NEG)

Esta sección descompone la matriz de conectividad funcional en sus componentes **positivo (POS)** y **negativo (NEG)** y los analiza por separado, porque cada uno refleja **mecanismos funcionales distintos** y no es válido tratarlos como una variable signed continua una vez que se ha aplicado regresión de confounds (Fox et al., 2005, *PNAS* 102:9673; Chai et al., 2012, *NeuroImage* 59:1420; Schwarz & McGonigle, 2010, *NeuroImage*).

Significado funcional de POS y NEG. Las correlaciones positivas reflejan **sincronía**

neuronal directa entre regiones (mecanismos excitatorios o de eferencia común; Fox et al., 2009, *J Neurophysiol* 101:3270). Las correlaciones negativas (anticorrelaciones), cuando son robustas, reflejan **antagonismo funcional** entre redes con roles complementarios — clásicamente entre la *Default Mode Network* (DMN) y la *task-positive network* (TPN, especialmente DAN y Cont) (Fox et al., 2005). La existencia y plausibilidad biológica de las anticorrelaciones ha sido objeto de debate intenso por su sensibilidad al *Global Signal Regression* (GSR): Murphy et al. (2009, *NeuroImage* 44:893) demostraron que GSR **matemáticamente fuerza** una distribución centrada en cero y por tanto introduce anticorrelaciones espurias; en contraposición, Fox et al. (2009) y Chai et al. (2012) mostraron que las anticorrelaciones DMN-TPN persisten **sin GSR** mediante CompCor o regresión fisiológica, sustentando un origen neural genuino al menos parcialmente. **Murphy & Fox (2017, *NeuroImage* 154:169)** consolidaron esta literatura en 8 declaraciones de consenso, de las cuales tres son críticas para este análisis:

1. *No existe un único pre-procesamiento “correcto”*: distintas estrategias revelan facetas complementarias del mismo sistema.
2. *La presencia de anticorrelaciones tras GSR puede ser artefactual*; pero su **persistencia sin GSR** apoya plausibilidad biológica.
3. *Reportar el análisis con y sin GSR* es la práctica recomendada para validar findings sobre conectividad signed.

Aquí explotamos el marco de triangulación de PsiloPrep §5.2 / PsiloPrep §3.13: el pipeline primario **TEDANA+aCompCor** no aplica GSR (regresa 24P motion + 5 PC WM + 5 PC CSF sobre la salida ME-ICA, preservando la varianza global de origen neural; Behzadi 2007, Murphy & Fox 2017), mientras que el control de sensibilidad **OC+36P+GSR** sí lo aplica. Si una diferencia PSIL—MTP sobrevive en ambos, no es producto del centrado matemático.

Hipótesis psicodélicas a priori:

1. **POS↑ ↓segregación inter-red** (Müller et al. 2018, *NeuroImage* 178:114; Lord et al. 2019, *NeuroImage* 199:127): PSIL aumenta la conectividad positiva *entre* redes funcionalmente segregadas, reduciendo la modularidad efectiva.
2. **NEG↑ (menos negativa, más positiva) entre DMN y DAN/TPN** (Carhart-Harris 2014, *Front Hum Neurosci* 8:20 §“INCREASED NETWORK ENTROPY”; Carhart-Harris et al. 2016, *PNAS* 113:4853): la desdiferenciación funcional bajo PSIL reduce la anticorrelación canónica DMN TPN. **Aclaración para este dataset:** sin GSR (pipeline primario TEDANA+aCompCor), la anticorrelación canónica DMN TPN es cuantitativamente débil o positiva incluso bajo placebo (Murphy & Fox 2017 §“Anticorrelations and GSR”; Chai et al. 2012). La predicción se traduce aquí, operacionalmente, a «DMN DAN es más positiva bajo PSIL que bajo MTP» — el sub-análisis (d) la testa one-sided en esa dirección.

Diseño analítico. Cuatro sub-análisis sobre la decomposición POS/NEG:

Sub-análisis	Unidad	N tests	Hipótesis	Pool FDR
(a) POS/NEG/RATIO global	matriz completa	2 (POS, NEG) + 1 descriptivo (RATIO)	bilateral	within {POS, NEG}; RATIO fuera del pool
(b) POS/NEG por red intra	8 redes × {POS, NEG, RATIO}	24	bilateral	dentro de cada métrica
(c) Asimetría inter-red 8×8	36 pares (8 intra + 28 inter) × {full, pos, neg}	108	bilateral	dentro de cada variante
(d) DMN DAN específico	1 par	1	PSIL > MTP (one-sided)	independiente (a priori direccional)

Atlas extendido. Para acomodar las **8 redes** (Yeo-7 cortical + pseudo-red Subcortex con los 32 ROIs Tian S2), se reusa la asignación `get_network_label` de §1.2: las etiquetas que empiezan por "7Networks_" mantienen su red Yeo-7; las restantes (Tian) se agrupan bajo "Subcortex". Esto permite preguntas como *¿la conectividad DMN Subcortex (incluyendo HIP, AMY, NAc) cambia bajo PSIL?* — relevante porque Carhart-Harris 2014 §"MEDIAL TEMPORAL LOBES AND PRIMARY CONSCIOUSNESS" sitúa al hipocampo como driver de la transición a estados primarios bajo psilocibina.

i Validación cruzada con §4 (control de pipelines)

§4 ya cuantificó la sensibilidad de las métricas topológicas (`mean_strength`, `global_efficiency`, `clustering`, `modularity`) al pipeline de denoising. Aquí extendemos el control específicamente a POS/NEG comparando TEDANA+aCompCor (no-GSR, primario) vs OC+36P+GSR (control GSR). La predicción Murphy & Fox 2017 es que NEG global **debe ser cuantitativamente menor** (más negativo) bajo OC+36P+GSR por el centrado matemático del GSR; lo crítico no es la magnitud absoluta sino que la **diferencia PSIL—MTP** sobreviva en ambos pipelines.

```
#####
# §6.3 - Conectividad positiva y negativa (POS/NEG) - dual dataset
#
# Cuatro sub-análisis (a)-(d) sobre la descomposición POS/NEG de la matriz
# de FC + control GSR Murphy & Fox 2017 + LOO POS/NEG. Cf. tabla
# «Diseño analítico» en la prosa.
#
# Marco S0: tests pareados intra-sujeto vía `analizar_metrica` (§6.1).
```



```

# El método "parallel (t + Wilcoxon)" reporta los dos p-values en paralelo
# para que el lector elija según el supuesto de normalidad - `shapiro_p`
# se conserva en cada CSV como DIAGNÓSTICO DESCRIPTIVO (no gobierna la
# selección del test: bajo el marco S0 ambos se reportan y se evalúa la
# convergencia por triangulación). FDR within-family por sub-análisis.
# RATIO se reporta descriptivamente con p sin corregir (H6-A r1) - no
# entra en el pool BH de (a).
#=====
library(tidyverse)
library(effsize)

for (ds in ACTIVE_DATASETS) {
  cd      <- CONNECTOMES[[ds]]
  c1_key  <- cd$conditions[1]
  c2_key  <- cd$conditions[2]
  C1      <- toupper(c1_key)
  C2      <- toupper(c2_key)
  RESULTS_DS      <- file.path(PROJECT_ROOT, "results", ds)
  CONN_DIR_PRIMARY <- cd$conn_dir    # ruta real en disco; cd$pipeline es etiqueta legible (p
  conn_root      <- file.path(PROJECT_ROOT, "derivatives", ds, "conn")
  gsr_pipe_name  <- grep("GSR",
                        list.dirs(conn_root, recursive = FALSE, full.names = FALSE),
                        value = TRUE, ignore.case = TRUE)[1]
  CONN_DIR_GSR <- if (!is.na(gsr_pipe_name)) file.path(conn_root, gsr_pipe_name) else ""
  out_dir      <- file.path(RESULTS_DS, "stats/statistical_results/POS_NEG")
  dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)

  # Atlas
  ATLAS_LABELS_CSV <- file.path(RESULTS_DS, "stats",
                                paste0("atlas_labels_", ds, ".csv"))
  atlas_labels <- read.csv(ATLAS_LABELS_CSV, stringsAsFactors = FALSE)$label[-1]
  stopifnot(length(atlas_labels) == cd$n_rois)

  get_network_label <- function(label) {
    if (startsWith(label, "7Networks_")) strsplit(label, "_")[[1]][3] else "Subcortex"
  }
  node_network <- factor(vapply(atlas_labels, get_network_label, character(1)),
                        levels = c("Vis", "SomMot", "DorsAttn", "SalVentAttn",
                                   "Limbic", "Cont", "Default", "Subcortex"))
  NET_LEVELS <- levels(node_network)
  cat(sprintf("\n[%s] Nodos por red:\n", ds)); print(table(node_network))
}

```

```

# ---- Funciones utilitarias -----
compute_pos_neg <- function(mat) {
  upper <- mat[upper.tri(mat)]; pos <- upper[upper > 0]; neg <- upper[upper < 0]
  list(POS = if (length(pos) > 0) mean(pos) else NA_real_,
       NEG = if (length(neg) > 0) mean(neg) else NA_real_,
       RATIO = if (length(neg) > 0 && length(pos) > 0)
               mean(pos) / abs(mean(neg)) else NA_real_)
}

compute_by_network <- function(mat, node_net) {
  map_dfr(levels(node_net), function(net) {
    idx <- which(node_net == net)
    if (length(idx) < 2) return(NULL)
    m <- compute_pos_neg(mat[idx, idx])
    tibble(network = net, POS = m$POS, NEG = m$NEG, RATIO = m$RATIO)
  })
}

compute_internetnetwork <- function(mat, node_net) {
  nets <- levels(node_net); rows <- list()
  for (a in nets) for (b in nets) {
    i <- which(node_net == a); j <- which(node_net == b)
    if (length(i) == 0 || length(j) == 0) {
      rows[[length(rows)+1L]] <- tibble(net1=a, net2=b,
                                         mean_full=NA_real_, mean_pos=NA_real_, mean_neg=NA_real_, n_edges=0L)
      next
    }
    vals <- if (a == b) { sub <- mat[i,i]; sub[upper.tri(sub)] } else as.vector(mat[i,j])
    rows[[length(rows)+1L]] <- tibble(net1=a, net2=b,
                                     mean_full = mean(vals, na.rm=TRUE),
                                     mean_pos = if (any(vals>0, na.rm=TRUE)) mean(vals[vals>0], na.rm=TRUE) else NA_real_,
                                     mean_neg = if (any(vals<0, na.rm=TRUE)) mean(vals[vals<0], na.rm=TRUE) else NA_real_,
                                     n_edges = length(vals))
  }
  bind_rows(rows)
}

# `read_mean_mat` localiza el CSV y delega en `read_matrix` (§1.1) para
# respetar los checks contractuales del productor (simetría tol 1e-8,
# sin NaN/Inf, tipo numérico). Coherente con H9-A (ronda 1 §6.3): los
# CSVs vienen de la extracción Python y deben pasar los mismos chequeos
# que el carga de §1.1.

```

```

read_mean_mat <- function(condition_dir, subj, cond) {
  pat <- paste0("_mean_", cond, "\\\\.csv$")
  f <- list.files(file.path(condition_dir, subj), pattern = pat, full.names = TRUE)
  if (length(f) == 0) return(NULL)
  mat <- read_matrix(f[1])
  diag(mat) <- 0
  mat
}

discover_subjects <- function(conn_dir, c1, c2) {
  d1 <- file.path(conn_dir, c1); d2 <- file.path(conn_dir, c2)
  if (!dir.exists(d1) || !dir.exists(d2)) return(character())
  intersect(
    setdiff(list.dirs(d1, recursive=FALSE, full.names=FALSE), "group"),
    setdiff(list.dirs(d2, recursive=FALSE, full.names=FALSE), "group"))
}

#=====
# (a) POS/NEG global
#=====
subjects <- discover_subjects(CONN_DIR_PRIMARY, c1_key, c2_key)
message(sprintf("[%s] Sujetos comunes (pipeline primario): %d", ds, length(subjects)))
c1_dir_primary <- file.path(CONN_DIR_PRIMARY, c1_key)
c2_dir_primary <- file.path(CONN_DIR_PRIMARY, c2_key)

pos_neg_global <- map_dfr(subjects, function(subj) {
  m_p <- read_mean_mat(c1_dir_primary, subj, c1_key)
  m_m <- read_mean_mat(c2_dir_primary, subj, c2_key)
  if (is.null(m_p) || is.null(m_m)) return(NULL)
  pp <- compute_pos_neg(m_p); mm <- compute_pos_neg(m_m)
  tibble(subject = rep(subj, 2), condition = c(C1, C2),
          POS = c(pp$POS, mm$POS), NEG = c(pp$NEG, mm$NEG),
          RATIO = c(pp$RATIO, mm$RATIO))
})
write.csv(pos_neg_global, file.path(out_dir, "POS_NEG_global.csv"), row.names = FALSE)

#=====
# (b) POS/NEG por red intra
#=====
pos_neg_network <- map_dfr(subjects, function(subj) {
  m_p <- read_mean_mat(c1_dir_primary, subj, c1_key)
  m_m <- read_mean_mat(c2_dir_primary, subj, c2_key)

```

```

    if (is.null(m_p) || is.null(m_m)) return(NULL)
    bind_rows(
      compute_by_network(m_p, node_network) %>% mutate(subject = subj, condition = C1),
      compute_by_network(m_m, node_network) %>% mutate(subject = subj, condition = C2))
  })
write.csv(pos_neg_network, file.path(out_dir, "POS_NEG_network.csv"), row.names = FALSE)

#=====
# (c) Asimetria POS/NEG inter-red 8x8
#=====
inter_net_data <- map_dfr(subjects, function(subj) {
  m_p <- read_mean_mat(c1_dir_primary, subj, c1_key)
  m_m <- read_mean_mat(c2_dir_primary, subj, c2_key)
  if (is.null(m_p) || is.null(m_m)) return(NULL)
  bind_rows(
    compute_internetnetwork(m_p, node_network) %>% mutate(subject = subj, condition = C1),
    compute_internetnetwork(m_m, node_network) %>% mutate(subject = subj, condition = C2))
})
write.csv(inter_net_data, file.path(out_dir, "POS_NEG_internetnetwork_long.csv"), row.names =

inter_net_paired <- inter_net_data %>%
  rowwise() %>%
  mutate(pair_key = paste(sort(c(net1, net2)), collapse = "\u2194")) %>%
  ungroup() %>%
  group_by(pair_key, subject, condition) %>%
  summarise(mean_full = mean(mean_full, na.rm = TRUE),
            mean_pos  = mean(mean_pos,  na.rm = TRUE),
            mean_neg  = mean(mean_neg,  na.rm = TRUE), .groups = "drop")

inter_pairs_unique <- unique(inter_net_paired$pair_key)
cat(sprintf("\n[%s] Pares unicos inter/intra-red: %d\n", ds, length(inter_pairs_unique)))

inter_net_results <- map_dfr(c("mean_full", "mean_pos", "mean_neg"), function(met) {
  map_dfr(inter_pairs_unique, function(pk) {
    df <- inter_net_paired %>% filter(pair_key == pk)
    res <- analizar_metrica(df, met, C1, C2)
    res$pair <- pk; res$variant <- met; res
  })
}) %>%
  group_by(variant) %>%
  mutate(p_FDR = p.adjust(p_value, "fdr")) %>%
  ungroup()

```

```

write.csv(inter_net_results,
          file.path(out_dir, "POS_NEG_internetnetwork_tests.csv"), row.names = FALSE)

cat(sprintf("\n=== [%s] Inter-red top 5 |d_z| (mean_full) ===\n", ds))
print(as.data.frame(inter_net_results %>%
  filter(variant == "mean_full", !is.na(d_z)) %>%
  arrange(desc(abs(d_z))) %>% select(pair, d_z, d_av, p_value, p_FDR) %>%
  slice_head(n = 5)), row.names = FALSE)

#=====
# (d) DMN <-> DAN one-sided
#
# N1 (ronda 1 §6.3): bajo el pipeline primario sin GSR (TEDANA+aCompCor)
# la anticorrelación canónica DMN DAN es DÉBIL o positiva incluso bajo
# placebo en este dataset (consistente con Chai 2012 + Murphy & Fox 2017
# - sin centrado matemático, las anticorrelaciones biológicas son
# cuantitativamente pocas). La predicción a priori 2 ("anticorrelación
# se reduce bajo PSIL") se operacionaliza aquí como "DMN DAN es más
# POSITIVA bajo PSIL que bajo MTP" - el test one-sided greater preserva
# la direccionalidad teórica de Carhart-Harris 2014 §"Entropic brain"
# aunque el signo absoluto de la correlación sea positivo.
#=====
dmn_dan_pairs <- inter_net_paired %>%
  filter(pair_key == "Default\u2194DorsAttn") %>%
  select(subject, condition, r_DMN_DAN = mean_full)
write.csv(dmn_dan_pairs, file.path(out_dir, "DMN_DAN_per_subject.csv"), row.names = FALSE)

dmn_dan_wide <- dmn_dan_pairs %>%
  pivot_wider(names_from = condition, values_from = r_DMN_DAN) %>%
  filter(!is.na(.data[[C1]]), !is.na(.data[[C2]])) %>%
  arrange(subject)

if (nrow(dmn_dan_wide) >= 3) {
  t_dd_1s <- t.test(dmn_dan_wide[[C1]], dmn_dan_wide[[C2]],
                    paired = TRUE, alternative = "greater")
  t_dd_2s <- t.test(dmn_dan_wide[[C1]], dmn_dan_wide[[C2]], paired = TRUE)
  w_dd <- wilcox.test(dmn_dan_wide[[C1]], dmn_dan_wide[[C2]],
                     paired = TRUE, alternative = "greater", exact = FALSE)
  d_dd <- mean(dmn_dan_wide[[C1]] - dmn_dan_wide[[C2]], na.rm = TRUE) /
    sd(dmn_dan_wide[[C1]] - dmn_dan_wide[[C2]], na.rm = TRUE)

  cat(sprintf("\n=== [%s] DMN \u2194 DAN (one-sided) ===\n", ds))

```

```

cat(sprintf("  N=%d  Media %s=%.4f  Media %s=%.4f  dz=%.3f\n",
            nrow(dmn_dan_wide), C1, mean(dmn_dan_wide[[C1]], na.rm=TRUE),
            C2, mean(dmn_dan_wide[[C2]], na.rm=TRUE), d_dd))
cat(sprintf("  t 1s: t(%.0f)=%.3f p=%.4f | t 2s: p=%.4f | W 1s: V=%.0f p=%.4f\n",
            t_dd_1s$parameter, t_dd_1s$statistic, t_dd_1s$p.value,
            t_dd_2s$p.value, w_dd$statistic, w_dd$p.value))

# `d_z` aquí es cálculo manual (mean( $\Delta$ )/sd( $\Delta$ )) independiente de
# `analizar_metrica`; nombrado coherente con el contrato r2 de §6.1
# (Lakens 2013 eq. 8; antes `cohen_dz` por inercia, renombrado en
# ronda 1 de §6.3, H7-A).
write.csv(tibble(metric = "DMN_DAN", n_pairs = nrow(dmn_dan_wide),
                !!paste0("mean_", C1) := mean(dmn_dan_wide[[C1]], na.rm=TRUE),
                !!paste0("mean_", C2) := mean(dmn_dan_wide[[C2]], na.rm=TRUE),
                d_z = d_dd, p_onesided = t_dd_1s$p.value,
                p_twosided = t_dd_2s$p.value, p_wilcoxon = w_dd$p.value,
                hypothesis = sprintf("%s > %s (entropic brain)", C1, C2)),
          file.path(out_dir, "DMN_DAN_test.csv"), row.names = FALSE)
} else {
  cat(sprintf("\n[%s] [DMN-DAN] Sin pares suficientes.\n", ds))
}

# El sub-análisis (e) Mueller 2013 - variabilidad inter-sujeto por arista -
# se retiró en la ronda 1 de §6.3 (H10-C). Razón: Wilcoxon pareado sobre
# 26 796 aristas infla el N efectivo a ~26 796 y produce significancia
# trivial (p ~ 10 ) para shifts ínfimos (~4.6 % en mediana) que no se
# corresponden con la magnitud de efecto que Mueller 2013 reporta
# (~20-30 % en ratio de varianza inter-sujeto). El test no respeta la
# unidad pareada del diseño (sujeto, no arista). Si se quisiera
# reintroducir, el camino correcto sería ratio + bootstrap por sujeto.

#=====
# Tests globales pareados + por red
#
# H6-A (ronda 1): el pool FDR de sub-análisis (a) se limita a {POS, NEG}
# como tests independientes. RATIO se calcula por sujeto como
# mean(pos_i)/|mean(neg_i)| sobre el upper.tri del propio sujeto (no es
# función de las medias de grupo de POS y NEG), pero sigue compartiendo
# numerador y denominador con ellas - incluirlo en el pool BH
# inflaría conservadoramente los q sin aportar un test independiente.
# RATIO se reporta como métrica DESCRIPTIVA con su p sin corregir;
# `p_FDR` queda en NA para RATIO.

```

```

#=====
res_global_pn <- map_dfr(c("POS","NEG","RATIO"),
                        ~analizar_metrica(pos_neg_global, .x, C1, C2))
is_pool_a <- res_global_pn$metric %in% c("POS","NEG")
res_global_pn$p_FDR <- NA_real_
res_global_pn$p_FDR[is_pool_a] <- p.adjust(res_global_pn$p_value[is_pool_a], "fdr")
write.csv(res_global_pn, file.path(out_dir, "POS_NEG_global_stats.csv"), row.names = FALSE)

# El bloque LMM `lmer(value ~ condition + network + (1|subject))` se retiró
# en la ronda 1 de §6.3 (H1-A). Razón: el random intercept único por
# sujeto no captura la dependencia intra-sujeto entre las 8 medidas-por-red,
# tratándolas como parcialmente independientes y dividiendo el SE por
# ~sqrt(8). Producía p ~ 10 para POS contra p = 0.075 del paired test
# honesto sobre los mismos datos - falsa evidencia por pseudoreplicación
# (Barr et al. 2013). Los sub-análisis (a) paired global + (b) paired
# por-red cubren las dos preguntas inferenciales del bloque sin recurrir
# a un LMM. Si se quisiera reintroducir, la estructura correcta sería
# `(condition | subject) + (1 | subject:network)`.

pn_long_by_network <- pos_neg_network %>%
  pivot_longer(c(POS, NEG, RATIO), names_to = "metric", values_to = "value") %>%
  mutate(subject = factor(subject),
         network = factor(network, levels = NET_LEVELS))

res_by_net <- map_dfr(c("POS","NEG","RATIO"), function(m) {
  df_m <- pn_long_by_network %>% filter(metric == m)
  map_dfr(NET_LEVELS, function(net) {
    df_net <- df_m %>% filter(as.character(network) == net)
    if (n_distinct(df_net$subject) < 3)
      return(tibble(metric=m, network=net, p_value=NA_real_,
                    p_wilcox=NA_real_, d_z=NA_real_, d_av=NA_real_,
                    method="insufficient_n"))
    res <- analizar_metrica(df_net, "value", C1, C2)
    tibble(metric=m, network=net, p_value=res$p_value, p_wilcox=res$p_wilcox,
          d_z=res$d_z, d_av=res$d_av, method=res$method)
  })
}) %>%
  group_by(metric) %>% mutate(p_FDR = p.adjust(p_value, "fdr")) %>% ungroup()
write.csv(res_by_net, file.path(out_dir, "POS_NEG_network_pareado.csv"), row.names = FALSE)

#=====
# Control GSR (Murphy & Fox 2017) - test del CONTRASTE PSIL-MTP en pipeline

```

```

# primario (no-GSR) vs control GSR. La pregunta no es si las medias absolutas
# de POS/NEG difieren (es trivialmente sí: GSR comprime POS y empuja NEG
# hacia valores más negativos por el centrado matemático, Murphy 2009).
# La pregunta es si la DIFERENCIA PSIL-MTP sobrevive en ambos pipelines
# con la misma dirección (Murphy & Fox 2017 §"Recommendations").
#=====
if (nzchar(CONN_DIR_GSR) && dir.exists(CONN_DIR_GSR)) {
  subjects_gsr <- discover_subjects(CONN_DIR_GSR, c1_key, c2_key)
  message(sprintf("\n[%s] Sujetos GSR: %d", ds, length(subjects_gsr)))
  if (length(subjects_gsr) >= 3) {
    c1_dir_gsr <- file.path(CONN_DIR_GSR, c1_key)
    c2_dir_gsr <- file.path(CONN_DIR_GSR, c2_key)

    # (a) global bajo GSR
    pos_neg_gsr <- map_dfr(subjects_gsr, function(subj) {
      m_p <- read_mean_mat(c1_dir_gsr, subj, c1_key)
      m_m <- read_mean_mat(c2_dir_gsr, subj, c2_key)
      if (is.null(m_p) || is.null(m_m)) return(NULL)
      pp <- compute_pos_neg(m_p); mm <- compute_pos_neg(m_m)
      tibble(subject=rep(subj,2), condition=c(C1,C2),
              POS=c(pp$POS,mm$POS), NEG=c(pp$NEG,mm$NEG), RATIO=c(pp$RATIO,mm$RATIO))
    })
    write.csv(pos_neg_gsr,
              file.path(out_dir, paste0("POS_NEG_global_", basename(CONN_DIR_GSR), ".csv")),
              row.names = FALSE)
    comparison <- inner_join(
      pos_neg_global %>% rename(POS_PRIM=POS, NEG_PRIM=NEG, RATIO_PRIM=RATIO),
      pos_neg_gsr %>% rename(POS_GSR=POS, NEG_GSR=NEG, RATIO_GSR=RATIO),
      by = c("subject","condition"))
    write.csv(comparison, file.path(out_dir, "POS_NEG_primary_vs_GSR.csv"), row.names = FALSE)

    # Test del contraste PSIL-MTP bajo GSR para (a) POS y NEG (RATIO fuera
    # del pool FDR, cf. H6-A). Tabla de comparación: estadísticos primario
    # vs GSR + chequeo de dirección.
    res_global_pn_gsr <- map_dfr(c("POS","NEG"),
                                ~analizar_metrica(pos_neg_gsr, .x, C1, C2)) %>%
      mutate(p_FDR_gsr = p.adjust(p_value, "fdr"))
    gsr_compare_global <- tibble(
      metric      = res_global_pn$metric,
      d_z_prim    = res_global_pn$d_z,
      p_prim      = res_global_pn$p_value,
      d_z_gsr     = res_global_pn_gsr$d_z[match(res_global_pn$metric, res_global_pn_gsr$metric)]
    )
  }
}

```



```

    p_gsr      = res_global_pn_gsr$p_value[match(res_global_pn$metric, res_global_pn_gsr$
) %>%
  mutate(sign_match = !is.na(d_z_prim) & !is.na(d_z_gsr) &
          sign(d_z_prim) == sign(d_z_gsr))
write.csv(gsr_compare_global,
          file.path(out_dir, "POS_NEG_global_primary_vs_GSR_tests.csv"),
          row.names = FALSE)

# (b) por-red bajo GSR: mismo cálculo que pos_neg_network pero sobre
# el stack GSR. Test pareado por red+métrica con FDR within-metric
# idéntico a la rama primaria, y comparación de signo d_z primario vs GSR.
pos_neg_network_gsr <- map_dfr(subjects_gsr, function(subj) {
  m_p <- read_mean_mat(c1_dir_gsr, subj, c1_key)
  m_m <- read_mean_mat(c2_dir_gsr, subj, c2_key)
  if (is.null(m_p) || is.null(m_m)) return(NULL)
  bind_rows(
    compute_by_network(m_p, node_network) %>% mutate(subject = subj, condition = C1),
    compute_by_network(m_m, node_network) %>% mutate(subject = subj, condition = C2))
})
pn_long_by_network_gsr <- pos_neg_network_gsr %>%
  pivot_longer(c(POS, NEG, RATIO), names_to = "metric", values_to = "value") %>%
  mutate(subject = factor(subject),
         network = factor(network, levels = NET_LEVELS))
res_by_net_gsr <- map_dfr(c("POS", "NEG", "RATIO"), function(m) {
  df_m <- pn_long_by_network_gsr %>% filter(metric == m)
  map_dfr(NET_LEVELS, function(net) {
    df_net <- df_m %>% filter(as.character(network) == net)
    if (n_distinct(df_net$subject) < 3)
      return(tibble(metric=m, network=net, p_value=NA_real_,
                    p_wilcox=NA_real_, d_z=NA_real_, d_av=NA_real_,
                    method="insufficient_n"))
    res <- analizar_metrica(df_net, "value", C1, C2)
    tibble(metric=m, network=net, p_value=res$p_value, p_wilcox=res$p_wilcox,
           d_z=res$d_z, d_av=res$d_av, method=res$method)
  })
}) %>%
  group_by(metric) %>% mutate(p_FDR = p.adjust(p_value, "fdr")) %>% ungroup()
gsr_compare_network <- res_by_net %>%
  select(metric, network, d_z_prim = d_z, p_prim = p_value, p_FDR_prim = p_FDR) %>%
  left_join(res_by_net_gsr %>%
            select(metric, network, d_z_gsr = d_z, p_gsr = p_value, p_FDR_gsr = p_FDR)
            by = c("metric", "network")) %>%

```

```

      mutate(sign_match = !is.na(d_z_prim) & !is.na(d_z_gsr) &
              sign(d_z_prim) == sign(d_z_gsr))
write.csv(gsr_compare_network,
          file.path(out_dir, "POS_NEG_network_primary_vs_GSR_tests.csv"),
          row.names = FALSE)

# Resumen ejecutivo en consola
cat(sprintf("\n=== [%s] Control GSR (Murphy & Fox 2017) ===\n", ds))
cat(sprintf("  Pipeline GSR: %s | N=%d sujetos\n",
            basename(CONN_DIR_GSR), length(subjects_gsr)))
cat("\n (a) Contraste global PSIL-MTP (signo y p sin FDR):\n")
for (i in seq_len(nrow(gsr_compare_global))) {
  r <- gsr_compare_global[i,]
  cat(sprintf("    %-6s | d_z prim=%+.3f p=%.4f | d_z GSR=%+.3f p=%.4f | sign_match=%s\n",
              r$metric, r$d_z_prim, r$p_prim, r$d_z_gsr, r$p_gsr,
              ifelse(r$sign_match, "SI", "NO")))
}
cat("\n (b) Contraste por-red: convergencia de signo primario vs GSR (24 cells):\n")
conv <- gsr_compare_network %>%
  summarise(n_total      = n(),
            n_evaluable  = sum(!is.na(d_z_prim) & !is.na(d_z_gsr)),
            n_match      = sum(sign_match, na.rm = TRUE))
cat(sprintf("    evaluables: %d/%d | signo coincide: %d/%d (%.0f%%)\n",
            conv$n_evaluable, conv$n_total, conv$n_match, conv$n_evaluable,
            100 * conv$n_match / max(conv$n_evaluable, 1L)))
} else {
  cat(sprintf("\n[%s] GSR: <3 sujetos.\n", ds))
}
} else {
  cat(sprintf("\n[%s] GSR: pipeline no encontrado.\n", ds))
}

#=====
# LOO - POS/NEG global
#=====
out_robust_dir <- file.path(RESULTS_DS, "stats/statistical_results/robust")
dir.create(out_robust_dir, showWarnings = FALSE, recursive = TRUE)
loo_posneg <- map(c("POS", "NEG", "RATIO"), function(m)
  loo_influence_analysis(pos_neg_global, m, paste0("posneg-", m), C1, C2))
names(loo_posneg) <- c("POS", "NEG", "RATIO")
write.csv(map_dfr(loo_posneg, ~.x$robust),
          file.path(out_robust_dir, "posneg_global_robust.csv"), row.names = FALSE)

```

```

write.csv(map_dfr(loo_posneg, ~.x$influence),
          file.path(out_robust_dir, "posneg_global_influence.csv"), row.names = FALSE)
cat(sprintf("\n=== [%s] LOO POS/NEG ===\n", ds))
walk(names(loo_posneg), function(nm) {
  res <- loo_posneg[[nm]]
  cat(sprintf("%-8s | influyentes: %d | acuerdo: %s\n",
              nm, res$n_influential, ifelse(res$agreement, "SI", "NO - revisar")))
})
message(sprintf("[%s] OK §6.3 POS/NEG completado", ds))
}

```

6.3.1 6.3.1 Visualizaciones POS/NEG

```

=====
# §6.3.1 - Visualizaciones POS/NEG - dual dataset
=====
library(tidyverse)
library(gghalves)

for (ds in ACTIVE_DATASETS) {
  cd      <- CONNECTOMES[[ds]]
  C1      <- toupper(cd$conditions[1])
  C2      <- toupper(cd$conditions[2])
  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_dir   <- file.path(RESULTS_DS, "stats/statistical_results/POS_NEG")
  plot_dir  <- file.path(RESULTS_DS, "figures/stats")
  dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)

  pos_neg_global    <- read.csv(file.path(out_dir, "POS_NEG_global.csv"))
  pos_neg_network    <- read.csv(file.path(out_dir, "POS_NEG_network.csv"))
  res_by_net         <- read.csv(file.path(out_dir, "POS_NEG_network_pareado.csv"))
  inter_net_results <- tryCatch(
    read.csv(file.path(out_dir, "POS_NEG_internetnetwork_tests.csv")), error = function(e) NULL,
    dmn_dan_pairs    <- tryCatch(
      read.csv(file.path(out_dir, "DMN_DAN_per_subject.csv")),          error = function(e) NULL,

  # Paleta de condiciones: magenta #C2185B (PSIL) + teal oscuro #00695C
  # (MTP) - rojo/azul reservados
  # para POS/NEG en §5 y §6.3.
  col_cond    <- setNames(c("#C2185B", "#00695C"), c(C1, C2))

```

```

NET_LEVELS <- c("Vis", "SomMot", "DorsAttn", "SalVentAttn", "Limbic",
               "Cont", "Default", "Subcortex")
lbl <- sprintf("%s vs %s", C1, C2)

#=====
# 1. Rainclouds globales POS / NEG / RATIO + composite 1x3
#
# Se mantienen los PNGs individuales para retrocompatibilidad y como
# anclajes externos, y se añade el composite `POSNEG_raincloud_global.png`
# con los tres rainclouds en una fila por el orden canónico POS · NEG ·
# RATIO, consumido por R5 como figura central del sub-análisis (a).
# `geom_half_violin(trim=FALSE, scale="width")`: análogo a §6.1.2, evita
# que paneles con baja varianza inter-sujeto degeneren a un violin
# invisible (típicamente el RATIO bajo PSIL, con sub-P4 y sub-P6 cerca
# de 17 y sub-P3 a 3).
#=====
make_global_posneg_raincloud <- function(m) {
  df_rc <- pos_neg_global %>%
    select(subject, condition, value = all_of(m)) %>%
    filter(!is.na(value))
  ggplot(df_rc, aes(x = condition, y = value, fill = condition)) +
    gghalves::geom_half_violin(side = "l", alpha = 0.45, trim = FALSE,
                              scale = "width") +
    geom_boxplot(width = 0.15, outlier.shape = NA, alpha = 0.8, fatten = 2) +
    geom_jitter(width = 0.07, size = 2.5, alpha = 0.85,
                color = "grey15", stroke = 0.3) +
    geom_line(aes(group = subject), color = "grey60", linewidth = 0.4,
              alpha = 0.6) +
    scale_fill_manual(values = col_cond) +
    theme_minimal(base_size = 13) + theme(legend.position = "none") +
    labs(title = paste0("[GRUPO · Global] Conectividad ", m),
         x = "Condición", y = paste0("Conectividad media (", m, ")"))
}

posneg_global_panels <- setNames(lapply(c("POS", "NEG", "RATIO"),
                                         make_global_posneg_raincloud),
                                c("POS", "NEG", "RATIO"))

for (m in names(posneg_global_panels)) {
  ggsave(file.path(plot_dir, paste0("POSNEG_raincloud_", m, ".png")),
          posneg_global_panels[[m]], width = 6, height = 5, dpi = 300)
}

```

```

composite_posneg_global <- patchwork::wrap_plots(posneg_global_panels,
                                                ncol = 3, nrow = 1) +
  patchwork::plot_annotation(
    title = sprintf("[%s · Global] Rainclouds POS · NEG · RATIO | %s",
                    ds, lbl),
    subtitle = "Sub-análisis (a) de §6.3 - conectividad signada agregada por sujeto y cond.",
    theme = theme(plot.title = element_text(size = 13, face = "bold"),
                  plot.subtitle = element_text(size = 10))
  )
ggsave(file.path(plot_dir, "POSNEG_raincloud_global.png"),
        composite_posneg_global, width = 16, height = 5, dpi = 300)

#=====
# 2. Barplot balance POS/NEG por red
#
# r3 quirúrgica §6.3.1 (2026-05-24): POS y NEG se agregan por separado
# (bind_rows de dos summarise) en lugar de un filter conjunto + pivot.
# El filter previo `!is.na(POS), !is.na(NEG)` descartaba la fila entera
# cuando una sola de las dos era NA - y para SomMot, DorsAttn, SalVentAttn
# bajo PSIL los 6 sujetos tienen NEG = NA (no hay anticorrelaciones
# intra-red bajo psicodélico sin GSR, consistente con Müller 2018 +
# Murphy & Fox 2017). Aquel filter ocultaba la condición PSIL entera en
# esas tres redes - ahora la barra POS aparece y la ausencia de barra
# NEG comunica visualmente la ausencia de anticorrelaciones (que es el
# findings real, no un dato faltante).
#=====
net_summary <- bind_rows(
  pos_neg_network %>%
    filter(!is.na(POS)) %>%
    group_by(network, condition) %>%
    summarise(mean = mean(POS, na.rm = TRUE),
              se = sd(POS, na.rm = TRUE) / sqrt(n()),
              sign = "POS", .groups = "drop"),
  pos_neg_network %>%
    filter(!is.na(NEG)) %>%
    group_by(network, condition) %>%
    summarise(mean = mean(NEG, na.rm = TRUE),
              se = sd(NEG, na.rm = TRUE) / sqrt(n()),
              sign = "NEG", .groups = "drop")
) %>%
  mutate(network = factor(network, levels = NET_LEVELS),
         sign = factor(sign, levels = c("POS", "NEG")))

```

```

p_bar <- ggplot(net_summary, aes(x = network, y = mean, fill = condition, alpha = sign)) +
  geom_col(position = position_dodge(0.8), width = 0.7) +
  geom_errorbar(aes(ymin = mean - se, ymax = mean + se),
                position = position_dodge(0.8), width = 0.25, linewidth = 0.5) +
  scale_fill_manual(values = col_cond) +
  scale_alpha_manual(values = c("POS" = 1, "NEG" = 0.5),
                     labels = c("POS" = "Positiva", "NEG" = "Negativa")) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey40") +
  theme_minimal(base_size = 12) +
  theme(axis.text.x = element_text(angle = 35, hjust = 1)) +
  labs(title = paste0("[GRUPO · Redes] Balance POS/NEG por red | ", lbl),
       x = "Red funcional", y = "Conectividad media", fill = "Condición", alpha = "Signo")
ggsave(file.path(plot_dir, "POSNEG_balance_por_red.png"),
        p_bar, width = 9, height = 5, dpi = 300)

#=====
# 3. Heatmap -log10(FDR) por red x métrica
#=====
p_heat <- res_by_net %>%
  filter(!is.na(p_FDR)) %>%
  mutate(log_p = -log10(pmax(p_FDR, 1e-10)),
         network = factor(network, levels = NET_LEVELS),
         metric = factor(metric, levels = c("POS", "NEG", "RATIO"))) %>%
  ggplot(aes(x = metric, y = network, fill = log_p)) +
  geom_tile(color = "white", linewidth = 0.4) +
  geom_text(aes(label = ifelse(p_FDR < 0.05, "*", "")),
            size = 5, vjust = 0.8, color = "black") +
  scale_fill_gradient(low = "white", high = "#C0392B", name = "-log (FDR)") +
  theme_minimal(base_size = 12) + theme(axis.text.x = element_text(face = "bold")) +
  labs(title = paste0("[GRUPO · Redes] Significación | ", lbl),
       subtitle = "* FDR < 0.05", x = "Métrica", y = "Red funcional")
ggsave(file.path(plot_dir, "POSNEG_heatmap_significacion.png"),
        p_heat, width = 6, height = 5, dpi = 300)

#=====
# 4. Dotplot -log10(FDR)
#=====
p_dot <- res_by_net %>%
  filter(!is.na(p_FDR)) %>%
  mutate(log_p = -log10(pmax(p_FDR, 1e-10)),
         sig = p_FDR < 0.05,
         red_label = paste0(network, " (", metric, ")")) %>%

```

```

    arrange(desc(log_p)) %>%
    mutate(red_label = fct_inorder(red_label)) %>%
    ggplot(aes(x = log_p, y = red_label, color = metric, shape = sig)) +
    geom_vline(xintercept = -log10(0.05), linetype = "dashed",
              color = "grey50", linewidth = 0.5) +
    geom_point(size = 3) +
    scale_color_manual(values = c("POS" = "#E74C3C", "NEG" = "#2980B9", "RATIO" = "#8E44AD")) +
    scale_shape_manual(values = c("TRUE" = 16, "FALSE" = 1),
                      labels = c("TRUE" = "FDR < 0.05", "FALSE" = "n.s.)) +
    theme_minimal(base_size = 11) + theme(axis.text.y = element_text(size = 8)) +
    labs(title = paste0("[GRUPO · Redes] Ranking significación | ", lbl),
         x = "-log (FDR)", y = "Red × métrica",
         color = "Métrica", shape = "Significación")
ggsave(file.path(plot_dir, "POSNEG_dotplot_significacion.png"),
        p_dot, width = 7, height = 7, dpi = 300)

#=====
# 4-bis. Composite heatmap + dotplot de significación en una fila
#
# Combina el heatmap -log (FDR) red × métrica y el dotplot del ranking
# en una sola imagen `POSNEG_heatmap_dotplot.png`, consumida por R5
# como figura única del sub-análisis (b) cribado FDR. El balance
# POS/NEG por red (`POSNEG_balance_por_red.png`) se mantiene como
# figura separada del mismo sub-análisis.
#=====
composite_posneg_fdr <- (p_heat | p_dot) +
  patchwork::plot_layout(widths = c(1, 1.2)) +
  patchwork::plot_annotation(
    title = sprintf("[%s · Redes] Significación FDR red × métrica | %s",
                    ds, lbl),
    subtitle = "Heatmap -log (FDR) (izquierda) + ranking por celda (derecha) - sub-análisis",
    theme = theme(plot.title = element_text(size = 13, face = "bold"),
                  plot.subtitle = element_text(size = 10))
  )
ggsave(file.path(plot_dir, "POSNEG_heatmap_dotplot.png"),
        composite_posneg_fdr, width = 14, height = 7, dpi = 300)

#=====
# 5. Heatmap inter-red 8x8 - d_z (test-coupled, §6.1 r2 contract)
#=====
if (!is.null(inter_net_results)) {
  inter_d <- inter_net_results %>%

```

```

    filter(variant == "mean_full") %>%
    separate(pair, into = c("net_a", "net_b"), sep = " ", remove = FALSE) %>%
    mutate(net_a = factor(net_a, levels = NET_LEVELS),
           net_b = factor(net_b, levels = NET_LEVELS))
inter_d_sym <- bind_rows(
  inter_d,
  inter_d %>% rename(net_a = net_b, net_b = net_a) %>% filter(net_a != net_b))
p_inter <- ggplot(inter_d_sym, aes(x = net_a, y = net_b, fill = d_z)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%s%s",
                                ifelse(is.na(d_z), "", sprintf("%.2f", d_z)),
                                ifelse(!is.na(p_FDR) & p_FDR < 0.05, "*",
                                ifelse(!is.na(p_value) & p_value < 0.05, ".", ""))),
              size = 3) +
  scale_fill_gradient2(low = "#2980B9", mid = "white", high = "#C0392B",
                       midpoint = 0,
                       name = sprintf("d_z\n(%s-%s)", C1, C2),
                       na.value = "grey90") +
  theme_minimal(base_size = 11) +
  theme(axis.text.x = element_text(angle = 35, hjust = 1, face = "bold"),
        axis.text.y = element_text(face = "bold")) +
  labs(title = paste0("[GRUPO · Inter-red] d_z 8x8 | ",
                      C1, " - ", C2, " (mean_full)",
                      subtitle = "* p_FDR<0.05 ; · p<0.05 (sin corregir)", x = "", y = "")
  ggsave(file.path(plot_dir, "POSNEG_inter_red_heatmap.png"),
         p_inter, width = 8, height = 7, dpi = 300)
} else {
  message(sprintf("[%s][fig 5] inter_net_results no disponible.", ds))
}

#=====
# 6. Scatter DMN <-> DAN
#=====
if (!is.null(dmn_dan_pairs) && nrow(dmn_dan_pairs) > 0) {
  dmn_dan_wide <- dmn_dan_pairs %>%
    pivot_wider(names_from = condition, values_from = r_DMN_DAN) %>%
    filter(!is.na(.data[[C1]]), !is.na(.data[[C2]]))
  rng <- range(c(dmn_dan_wide[[C1]], dmn_dan_wide[[C2]]), na.rm = TRUE)
  p_dd <- ggplot(dmn_dan_wide, aes(x = .data[[C2]], y = .data[[C1]])) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey60") +
    geom_hline(yintercept = 0, linetype = "dotted", color = "grey75") +
    geom_vline(xintercept = 0, linetype = "dotted", color = "grey75") +

```



```

geom_point(size = 4, color = "#E74C3C", alpha = 0.85) +
ggrepel::geom_text_repel(aes(label = subject), size = 3) +
coord_equal(xlim = rng, ylim = rng) +
theme_minimal(base_size = 12) +
labs(title = paste0("[GRUPO] DMN DAN - test direccional ", C1, " > ", C2),
      subtitle = paste0("Hipótesis: anticorrelación DMN-TPN se reduce bajo ", C1,
                        "\nLínea diagonal = identidad"),
      x = paste0("r(DMN, DAN) bajo ", C2),
      y = paste0("r(DMN, DAN) bajo ", C1))
ggsave(file.path(plot_dir, "POSNEG_DMN_DAN_scatter.png"),
       p_dd, width = 7, height = 6, dpi = 300)
} else {
  message(sprintf("[%s] [fig 6] DMN-DAN no disponible.", ds))
}

# Fig 7 (densidad varianza por arista Mueller 2013) eliminada en ronda 1
# de §6.3 (H10-C) junto con el sub-análisis (e) edge variability -
# Wilcoxon sobre 26 796 aristas pareadas inflaba la significancia para
# shifts triviales en magnitud.

message(sprintf("[%s] OK Visualizaciones POS/NEG completadas", ds))
}

```

6.4 6.4 Conectividad inter-red 8×8 (between-network) — Yeo-7 + Subcortex Tian

Esta sección testa formalmente los **28 pares puramente inter-red** (Yeo-7 cortical + Subcortex Tian) en el contraste PSIL vs MTP, añadiendo un **Bayes Factor pareado** para distinguir un nulo informativo de la simple falta de potencia. El test pareado sobre `mean_full` por par ya está hecho en §6.3(c) (sobre los 36 pares = 28 inter + 8 intra, con FDR within-variant); §6.4 reusa esos resultados, los filtra a los 28 puramente inter, recalcula FDR-BH dentro de esa familia y añade tres piezas que §6.3 no produce: la cuantificación bayesiana, los heatmaps 8×8 con anotación de significancia (* / .) y el LOO selectivo por par sugestivo.

Marco teórico. Bajo psicodélicos serotoninérgicos (LSD, psilocibina, DMT) la firma neural más replicada es un **incremento de la integración global** acompañado de una **reducción de la segregación de redes** (Carhart-Harris et al., 2016, *PNAS* 113:4853; Tagliazucchi et al., 2016, *Curr Biol* 26:1043; Müller et al., 2018, *NeuroImage Clin* 18:694; Lord et al., 2019, *NeuroImage* 199:127; Daws et al., 2022, *Nat Med* 28:844). Esto se observa cuantitativamente como un **aumento del cross-talk inter-red** y una **disminución de la cohesión intra-red**, especialmente para la *Default Mode Network* (DMN) y las redes transmodales (frontoparietal control, salience). Roseman et al. (2014, *Front Hum Neurosci* 8:204) reportaron por primera vez este patrón inter-red con psilocibina (con MDMA como comparador activo).

Marco unificado: REBUS y la jerarquía cortical aplanada. Carhart-Harris & Friston (2019, *Pharmacol Rev* 71:316) propusieron el modelo *RElaxed Beliefs Under Psychedelics* (REBUS), basado en *predictive coding* jerárquico, según el cual los psicodélicos relajan la precisión de las creencias previas codificadas en cortex transmodal, aumentando la sensibilidad a inputs sensoriales unimodales. **Girn et al. (2022, *NeuroImage* 256:119220)** testaron directamente esta predicción mediante *gradient mapping* (Margulies et al., 2016) en los datasets ds003059 (LSD) y Carhart-Harris 2012 (psilocibina) y mostraron una **contracción significativa del gradiente principal unimodal-transmodal** bajo ambos psicodélicos, con incrementos de gradiente en redes unimodales (Vis, SomMot) y descensos en redes transmodales (DMN, Cont). Para psilocibina específicamente reportaron decrementos significativos en la red Default y aumentos en la red Somatomotora (FDR-corrected sobre 6 redes Yeo). La interpretación operacional para nuestro análisis 8×8 es: **se predicen incrementos de conectividad PSIL—MTP en pares inter-red transmodal unimodal y entre redes transmodales, con disminución de la auto-cohesión intra-DMN** (la cual se cuantifica en §6.3(b) por separado).

Predicciones a priori (PSIL > MTP, dirigidas). La dirección de cada predicción está fijada por el marco REBUS y por los hallazgos empíricos de Girn et al. (2022) **antes** de mirar los datos; ningún one-sided se elige *post-hoc* a partir del signo observado. El chunk reporta dos familias en paralelo: los 28 pares con tests two-sided + FDR-BH within-family 28 (sin priorizar dirección), y un sub-análisis dedicado one-sided sobre los 5 pares a priori restantes con su propio FDR-BH within-family 5 (Default DorsAttn no entra en este sub-análisis porque §6.3(d) ya lo testa one-sided con cita a Carhart-Harris 2014).

1. **Default DorsAttn** ↑ — anticorrelación canónica reducida (testado one-sided en §6.3(d)).
2. **Default SomMot** y **Default Vis** ↑ — cross-talk transmodal-unimodal (Girn 2022).
3. **Default Subcortex (Tian)** ↑ — desinhibición DMN-MTL bajo psicodélicos (Carhart-Harris 2014, *Front Hum Neurosci* §“MEDIAL TEMPORAL LOBES AND PRIMARY CONSCIOUSNESS”; el hipocampo-amígdala forman la mayor parte de Tian S2).
4. **Cont Vis** y **Cont SomMot** ↑ — paralelo de (2) sobre la red frontoparietal control.

i Interpretación de un resultado nulo con N=6

La literatura recoge que los **efectos inter-red corticales bajo psicodélicos son consistentemente débiles** y poco solapantes entre estudios cuando se examinan pares específicos de redes (Müller et al., 2018; Carhart-Harris et al., 2016; Roseman et al., 2014). Müller et al. lo formalizan así: “*Although decreased within-network and increased between-network FC is often listed as a consistent finding with serotonergic psychedelics, the effects with respect to specific networks and network pairs has limited overlap across drugs and datasets*”. Con N 6, ningún par sobrevivirá FDR (28 tests) salvo para efectos $d > 1.5$. Por ello el chunk añade **Bayes Factor pareado** (Rouder et al. 2009, *Psychon Bull Rev* 16:225; Dienes 2014, *Front Psychol* 5:781) calculado con `BayesFactor::ttestBF` bajo prior Cauchy de localización 0 y escala `rscale = 0.707` ($= \sqrt{2}/2$, prior “medium” de

Rouder 2009; la elección por defecto del paquete) para distinguir “ausencia de evidencia” ($BF_{01} \approx 1$) de “evidencia de ausencia” ($BF_{01} > 3$). El resultado más probable, dado el tamaño muestral, es **inconcluso para la mayoría de pares**; los efectos individuales se reportarán como exploratorios y el patrón global (signo de los 28 Δ) se contrasta con la predicción de $\uparrow$ integración global mediante un test de signos binomial y un Wilcoxon signed-rank sobre el vector de deltas.

i Acoplamiento §6.3(c) $\rightarrow$ §6.4

Aspecto	§6.3(c)	§6.4
Pares testados	36 (8 intra + 28 inter)	28 (solo inter, vista filtrada de §6.3(c))
Variantes	mean_full / mean_pos / mean_neg	mean_full
Cómputo del test	analizar_metrica (origen)	reusa los outputs de §6.3(c) — no recalcula
Pool FDR	within-variant ($3 \times 36 = 108$ tests; BH dentro de cada variante)	within-family 28 inter (BH); más una family one-sided de 5 pares REBUS aparte
Effect size	d_z, d_av	d_z, d_av (heredados; H7 de §6.3)
Bayes Factor	no	sí (28 pares two-sided + 5 pares one-sided REBUS)
LOO	global de §6.1.3	selectivo: sólo en pares con $p_FDR < 0.10$
Visualización	barplot POS/NEG por red + raincloud + heatmap signed	pheatmap PSIL/MTP/ Δ con * (FDR<0.05) / . (FDR<0.10); §6.4.1 añade valores numéricos completos + dotplot Δ coloreado por nivel BF
Foco	descomposición POS/NEG con todas las variantes	redistribución neta inter-red + cuantificación bayesiana + auditoría agregada del signo

```

=====
# §6.4 - Conectividad inter-red 8x8 (between-network) - dual dataset
#-----
# H2-A (cierre r1): reusa POS_NEG_internetnetwork_tests.csv y _long.csv de
#   §6.3(c) en lugar de recalcular t-tests + Wilcoxon que ya están en disco.

```

```

# Aporta sobre §6.3(c): FDR within-family 28 (vs within-variant 36 de
# §6.3), Bayes Factor (rouder_2009, prior Cauchy rscale=0.707), L00
# selectivo por par sugestivo, heatmaps 8x8 con asteriscos y test de
# signos global sobre los 28  $\Delta$ .
# H1-A (cierre r1): el BF se calcula con group_modify (paralelo a §6.3 sec 2);
# la implementación r0 usaba rowwise + filter(net1 == .data$net1,...) dentro
# de un mutate, lo que hacía que ttestBF recibiera los 28 pares juntos en
# cada llamada (BF10 idéntico = 2.57e+22 para todos los pares).
# H4-A (cierre r1): sub-análisis REBUS one-sided sobre 5 pares a priori
# (Default SomMot, Default Vis, Default Subcortex, Cont Vis, Cont SomMot;
# Default DorsAttn se testa en §6.3(d)) con FDR-BH within-family 5 y BF
# one-sided (nullInterval = c(0, Inf)). La dirección está fijada por Girn
# 2022 / REBUS antes de ver datos, no se elige post-hoc.
# H5-A (cierre r1): contrato d_z/d_av heredado de §6.3 (no se recalcula
# statistic t-stat).
# H6-A (cierre r1): shapiro_p es diagnóstico descriptivo, NO selector de
# test; t-test y Wilcoxon se reportan siempre en paralelo (Lakens 2013;
# Rochon 2012).
# H8-B (cierre r1): `library(rstatix)` retirada (no se usaba); BayesFactor y
# pheatmap se mantienen inline porque sólo aquí se cargan.
# N2-A (cierre r1): test de signos binomial + Wilcoxon signed-rank sobre los
# 28  $\Delta$  para reportar la consistencia direccional agregada (la firma REBUS
# "↑integración global" es un patrón colectivo, no un par individual).
#=====
library(tidyverse)
library(pheatmap)
library(BayesFactor)

for (ds in ACTIVE_DATASETS) {
  cd      <- CONNECTOMES[[ds]]
  c1_key  <- cd$conditions[1]
  c2_key  <- cd$conditions[2]
  C1      <- toupper(c1_key)
  C2      <- toupper(c2_key)
  RESULTS_DS    <- file.path(PROJECT_ROOT, "results", ds)
  out_dir       <- file.path(RESULTS_DS, "stats/statistical_results/advanced")
  plot_dir      <- file.path(RESULTS_DS, "figures/stats/advanced")
  out_robust_dir <- file.path(RESULTS_DS, "stats/statistical_results/robust")
  out_posneg_dir <- file.path(RESULTS_DS, "stats/statistical_results/POS_NEG")
  dir.create(out_dir,          showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_dir,        showWarnings = FALSE, recursive = TRUE)
  dir.create(out_robust_dir,   showWarnings = FALSE, recursive = TRUE)

```

```

NET_LEVELS <- c("Vis", "SomMot", "DorsAttn", "SalVentAttn", "Limbic",
               "Cont", "Default", "Subcortex")

# ---- 1. Reusar los tests pareados que §6.3(c) ya hizo sobre mean_full ----
pn_tests_file <- file.path(out_posneg_dir, "POS_NEG_internetnetwork_tests.csv")
pn_long_file <- file.path(out_posneg_dir, "POS_NEG_internetnetwork_long.csv")
stopifnot(
  "$6.4 requiere ejecutar §6.3(c) antes (POS_NEG_internetnetwork_tests.csv ausente)" =
    file.exists(pn_tests_file),
  "$6.4 requiere ejecutar §6.3(c) antes (POS_NEG_internetnetwork_long.csv ausente)" =
    file.exists(pn_long_file))
pn_tests <- read.csv(pn_tests_file, stringsAsFactors = FALSE)
pn_long <- read.csv(pn_long_file, stringsAsFactors = FALSE)

bn_tests <- pn_tests %>%
  filter(variant == "mean_full") %>%
  tidyr::separate(pair, into = c("net1", "net2"),
                 sep = " ", remove = FALSE, fill = "right") %>%
  filter(net1 != net2) %>% # quita los 8 intra
  select(-variant, -p_FDR, -metric) %>%
  rename(delta = diff) %>%
  mutate(net1 = factor(net1, levels = NET_LEVELS),
         net2 = factor(net2, levels = NET_LEVELS),
         p_FDR = p.adjust(p_value, method = "fdr"), # re-FDR sobre 28
         significant = !is.na(p_FDR) & p_FDR < 0.05,
         pair = paste0(net1, " ", net2)) %>%
  arrange(p_FDR)
write.csv(bn_tests, file.path(out_dir, "between_network_tests.csv"),
         row.names = FALSE)

cat(sprintf("\n[%s] Pares inter-red reusados de §6.3(c): %d (esperados 28)\n",
          ds, nrow(bn_tests)))

# ---- 2. Resumen ejecutivo ----
cat(sprintf("\n=== [%s] Between-network 28 pares (two-sided) ===\n", ds))
cat(sprintf("  Sig. FDR < 0.05: %d/28 | Sugestivos FDR < 0.10: %d/28\n",
          sum(bn_tests$significant, na.rm = TRUE),
          sum(bn_tests$p_FDR < 0.10, na.rm = TRUE)))
print(as.data.frame(bn_tests %>% filter(!is.na(p_value)) %>%
  select(pair, delta, d_z, d_av, p_value, p_FDR, significant) %>%
  arrange(p_value)), row.names = FALSE)

```

```

# ---- 3. Datos por sujeto x condicion x par (long) para BF, LOO y heatmaps ----
between_net <- pn_long %>%
  rowwise() %>%
  mutate(pair_key = paste(sort(c(as.character(net1), as.character(net2))),
                           collapse = " ")) %>%
  ungroup() %>%
  group_by(pair_key, subject, condition) %>%
  summarise(connectivity = mean(mean_full, na.rm = TRUE), .groups = "drop") %>%
  tidyr::separate(pair_key, into = c("net1", "net2"),
                  sep = " ", remove = TRUE) %>%
  filter(net1 != net2) %>%
  mutate(net1 = factor(net1, levels = NET_LEVELS),
         net2 = factor(net2, levels = NET_LEVELS))
write.csv(between_net, file.path(out_dir, "between_network_connectivity.csv"),
         row.names = FALSE)

# ---- 4. LOO para pares sugestivos ----
pares_sugestivos <- bn_tests %>% filter(p_FDR < 0.10)
if (nrow(pares_sugestivos) > 0) {
  loo_bn_list <- pmap(
    list(as.character(pares_sugestivos$net1),
         as.character(pares_sugestivos$net2),
         pares_sugestivos$pair),
    function(n1, n2, par_name) {
      df_par <- between_net %>%
        filter(as.character(net1) == n1, as.character(net2) == n2) %>%
        select(subject, condition, connectivity)
      df_par2 <- df_par %>% rename(!!par_name := connectivity)
      loo_influence_analysis(df_par2, par_name, par_name, C1, C2)
    })
  names(loo_bn_list) <- pares_sugestivos$pair
  write.csv(map_dfr(loo_bn_list, ~if (!is.null(.x)) .x$robust),
            file.path(out_robust_dir, "between_network_robust.csv"), row.names = FALSE)
  write.csv(map_dfr(loo_bn_list, ~if (!is.null(.x)) .x$influence),
            file.path(out_robust_dir, "between_network_influence.csv"), row.names = FALSE)
  cat(sprintf("\n=== [%s] LOO pares sugestivos ===\n", ds))
  walk(names(loo_bn_list), function(nm) {
    res <- loo_bn_list[[nm]]
    if (is.null(res)) return()
    cat(sprintf("  %-35s | influyentes: %d | acuerdo: %s\n",
                nm, res$n_influential, ifelse(res$agreement, "SI", "NO")))
  })
}

```

```

} else {
  cat(sprintf("\n[%s] Ningun par FDR < 0.10 - L00 no aplicable.\n", ds))
}

# ---- 5. Bayes Factor pareado two-sided (28 pares) ----
bn_bayes_calc <- between_net %>%
  group_by(net1, net2) %>%
  group_modify(function(df, keys) {
    df_par <- df %>%
      pivot_wider(names_from = condition, values_from = connectivity) %>%
      filter(!is.na(.data[[C1]]), !is.na(.data[[C2]])) %>%
      arrange(subject)
    bf10 <- if (nrow(df_par) < 3) NA_real_ else tryCatch(
      extractBF(ttestBF(x = df_par[[C1]], y = df_par[[C2]],
        paired = TRUE))$bf,
      error = \(e) NA_real_)
    tibble(BF10 = bf10)
  }) %>% ungroup()

bn_bayes <- bn_tests %>%
  left_join(bn_bayes_calc, by = c("net1", "net2")) %>%
  mutate(
    BF01 = 1 / BF10,
    bf_evidence = case_when(
      is.na(BF10) ~ "indeterminado",
      BF10 > 10 ~ paste0("fuerte H1 (", C1, " != ", C2, ")"),
      BF10 > 3 ~ "moderada H1",
      BF10 > 1 ~ "anecdótica H1",
      BF10 > 1/3 ~ "anecdótica H0",
      BF10 > 1/10 ~ "moderada H0 (no efecto)",
      TRUE ~ "fuerte H0"))

write.csv(bn_bayes %>% select(pair, delta, d_z, d_av, p_value, p_FDR,
                             BF10, BF01, bf_evidence),
          file.path(out_dir, "between_network_BayesFactor.csv"), row.names = FALSE)

cat(sprintf("\n=== [%s] Bayes Factor between-network two-sided (28 pares) ===\n", ds))
cat(sprintf(" BF10 > 3: efecto | 1/3..3: INCONCLUSO | BF01 > 3: nulo\n"))
print(as.data.frame(bn_bayes %>%
  select(pair, delta, p_FDR, BF10, BF01, bf_evidence) %>% arrange(desc(BF10))),
      row.names = FALSE)
n_h1_str <- sum(bn_bayes$BF10 > 10, na.rm = TRUE)

```



```

n_h1_mod <- sum(bn_bayes$BF10 > 3 & bn_bayes$BF10 <= 10, na.rm = TRUE)
n_inconc <- sum(bn_bayes$BF10 >= 1/3 & bn_bayes$BF10 <= 3, na.rm = TRUE)
n_h0_mod <- sum(bn_bayes$BF01 > 3 & bn_bayes$BF01 <= 10, na.rm = TRUE)
n_h0_str <- sum(bn_bayes$BF01 > 10, na.rm = TRUE)
cat(sprintf(" %d fuerte H1, %d mod H1, %d inconclusos, %d mod H0, %d fuerte H0\n",
            n_h1_str, n_h1_mod, n_inconc, n_h0_mod, n_h0_str))

# ---- 6. Sub-análisis REBUS one-sided (5 pares a priori PSIL > MTP) ----
# Default DorsAttn se excluye porque §6.3(d) ya lo testa one-sided con
# cita a Carhart-Harris 2014 (entropic brain). La dirección PSIL > MTP de
# los 5 pares restantes está fijada por Girn 2022 / REBUS antes de mirar
# datos: cross-talk transmodal-unimodal (Default SomMot, Default Vis),
# desinhibición DMN-MTL (Default Subcortex) y paralelo Cont unimodal
# (Cont Vis, Cont SomMot). FDR-BH within-family 5; BF one-sided con
# nullInterval = c(0, Inf) (Morey & Rouder 2018 BayesFactor §"Directional
# hypotheses"; cf. Rouder 2009 eq. 17).
rebus_pairs <- tibble::tribble(
  ~n1,      ~n2,
  "Default", "SomMot",
  "Default", "Vis",
  "Default", "Subcortex",
  "Cont",    "Vis",
  "Cont",    "SomMot"
) %>% rowwise() %>%
  mutate(pair_key = paste(sort(c(n1, n2)), collapse = " ")) %>% ungroup()

rebus_tests <- pmap_dfr(
  list(rebus_pairs$n1, rebus_pairs$n2, rebus_pairs$pair_key),
  function(n1, n2, par_name) {
    df_par <- between_net %>%
      filter((as.character(net1) == n1 & as.character(net2) == n2) |
             (as.character(net1) == n2 & as.character(net2) == n1)) %>%
      pivot_wider(names_from = condition, values_from = connectivity) %>%
      filter(!is.na(.data[[C1]]), !is.na(.data[[C2]])) %>%
      arrange(subject)
    n <- nrow(df_par)
    if (n < 3) return(tibble(pair = par_name, n = n, delta = NA_real_,
                              d_z = NA_real_, t = NA_real_, p_one = NA_real_,
                              BF10_one = NA_real_))
    diffs <- df_par[[C1]] - df_par[[C2]]
    t1s <- t.test(df_par[[C1]], df_par[[C2]], paired = TRUE,
                  alternative = "greater")
  }
)

```



```

bf_one <- tryCatch(
  extractBF(ttestBF(x = df_par[[C1]], y = df_par[[C2]],
    paired = TRUE, nullInterval = c(0, Inf)))$bf[1],
  error = \(e) NA_real_)
tibble(pair      = par_name,
  n            = n,
  delta        = mean(diffs, na.rm = TRUE),
  d_z          = mean(diffs, na.rm = TRUE) / sd(diffs, na.rm = TRUE),
  t            = as.numeric(t1s$statistic),
  p_one        = t1s$p.value,
  BF10_one     = bf_one)
}) %>%
mutate(p_FDR_one      = p.adjust(p_one, method = "fdr"),
  significant_one     = !is.na(p_FDR_one) & p_FDR_one < 0.05)
write.csv(rebus_tests,
  file.path(out_dir, "between_network_REBUS_one_sided.csv"),
  row.names = FALSE)

cat(sprintf("\n=== [%s] REBUS one-sided (5 pares a priori PSIL > MTP) ===\n", ds))
cat(sprintf("  Sig. FDR_one < 0.05: %d/5 | BF10_one > 3: %d/5\n",
  sum(rebus_tests$significant_one, na.rm = TRUE),
  sum(rebus_tests$BF10_one > 3, na.rm = TRUE)))
print(as.data.frame(rebus_tests %>%
  select(pair, n, delta, d_z, t, p_one, p_FDR_one, BF10_one, significant_one)),
  row.names = FALSE)

# ---- 7. Test de signos sobre los 28 Δ (firma REBUS agregada) ----
# Si las 28 medias inter-red comparten dirección PSIL > MTP, eso es
# evidencia mucho más fuerte contra la nula global "PSIL = MTP" que la
# supervivencia FDR de cualquier par individual. Test de signos binomial
# (sólo signo de Δ) + Wilcoxon signed-rank one-sample (signo + magnitud
# rank-based) sobre el vector de 28 deltas.
n_pos <- sum(bn_tests$delta > 0, na.rm = TRUE)
n_neg <- sum(bn_tests$delta < 0, na.rm = TRUE)
n_tot <- n_pos + n_neg
sign_test <- if (n_tot > 0) binom.test(n_pos, n_tot, p = 0.5) else NULL
wilcox_global <- tryCatch(wilcox.test(bn_tests$delta, mu = 0, exact = FALSE),
  error = \(e) NULL)
sign_summary <- tibble::tibble(
  n_pos      = n_pos,
  n_neg      = n_neg,
  n_total    = n_tot,

```

```

    binomial_p = if (!is.null(sign_test))      sign_test$p.value      else NA_real_,
    wilcox_p    = if (!is.null(wilcox_global)) wilcox_global$p.value else NA_real_,
    mean_delta  = mean(bn_tests$delta, na.rm = TRUE),
    median_delta = median(bn_tests$delta, na.rm = TRUE))
write.csv(sign_summary,
          file.path(out_dir, "between_network_sign_test.csv"),
          row.names = FALSE)
cat(sprintf("\n=== [%s] Firma agregada sobre los 28  $\Delta$  ===\n", ds))
cat(sprintf("   $\Delta > 0$ : %d/%d | Binomial p = %.3g | Wilcoxon signed-rank p = %.3g\n",
          n_pos, n_tot, sign_summary$binomial_p, sign_summary$wilcox_p))

# ---- 8. Matrices 8x8 para heatmaps ----
make_8x8 <- function(df_cond, value_col = "connectivity") {
  mat <- matrix(NA_real_, nrow = length(NET_LEVELS), ncol = length(NET_LEVELS),
                dimnames = list(NET_LEVELS, NET_LEVELS))
  for (i in seq_len(nrow(df_cond))) {
    n1 <- as.character(df_cond$net1[i])
    n2 <- as.character(df_cond$net2[i])
    v <- df_cond[[value_col]][i]
    if (!(n1 %in% NET_LEVELS) || !(n2 %in% NET_LEVELS)) next
    mat[n1, n2] <- v; mat[n2, n1] <- v
  }
  diag(mat) <- NA; mat
}

bn_group <- between_net %>%
  group_by(net1, net2, condition) %>%
  summarise(connectivity = mean(connectivity, na.rm = TRUE), .groups = "drop")

mat_c1 <- make_8x8(bn_group %>% filter(condition == C1))
mat_c2 <- make_8x8(bn_group %>% filter(condition == C2))
nets_ok <- intersect(rownames(mat_c1), rownames(mat_c2))
mat_c1 <- mat_c1[nets_ok, nets_ok]
mat_c2 <- mat_c2[nets_ok, nets_ok]
mat_delta <- mat_c1 - mat_c2

sig_mat <- matrix("", nrow = length(nets_ok), ncol = length(nets_ok),
                  dimnames = list(nets_ok, nets_ok))
for (i in seq_len(nrow(bn_tests))) {
  n1 <- as.character(bn_tests$net1[i])
  n2 <- as.character(bn_tests$net2[i])
  if (!(n1 %in% nets_ok) || !(n2 %in% nets_ok)) next

```

```

    label <- ifelse(!is.na(bn_tests$significant[i]) && bn_tests$significant[i], "*",
                    ifelse(!is.na(bn_tests$p_FDR[i]) && bn_tests$p_FDR[i] < 0.10, "†", ""))
    sig_mat[n1, n2] <- label; sig_mat[n2, n1] <- label
  }

# Etiqueta combinada del Delta: "0.29 *" / "0.18 ." / "0.10" en celdas inter-red,
# cadena vacia en la diagonal (que ya es NA y se pinta con na_col).
cell_label_mat <- matrix("", nrow = length(nets_ok), ncol = length(nets_ok),
                          dimnames = list(nets_ok, nets_ok))
for (i in seq_along(nets_ok)) {
  for (j in seq_along(nets_ok)) {
    if (i == j) next
    v <- mat_delta[i, j]
    if (is.na(v)) next
    s <- sig_mat[i, j]
    cell_label_mat[i, j] <- if (nzchar(s)) sprintf("%.2f %s", v, s) else sprintf("%.2f", v)
  }
}

# ---- 9. Figuras (pheatmap 8x8) ----
# Dimensiones unificadas 900x700 entre las tres figuras para que la version
# combinada (Delta + C1 + C2) tenga sub-paneles de tamaño homogéneo. La
# diagonal NA se suprime del display_numbers para que se vea solo el na_col.
display_c1 <- ifelse(is.na(mat_c1), "", sprintf("%.2f", mat_c1))
display_c2 <- ifelse(is.na(mat_c2), "", sprintf("%.2f", mat_c2))
# Escala unificada para C1 y C2: rango común de los datos finitos -> mismo
# valor numérico se mapea al mismo color en los dos paneles, lo que hace
# honesta la comparación visual PSIL vs MTP. La diagonal NA sigue
# pintándose con na_col.
c12_range <- range(c(mat_c1, mat_c2), na.rm = TRUE)
c12_breaks <- seq(c12_range[1], c12_range[2], length.out = 101)
c12_color <- colorRampPalette(c("white", "#C0392B"))(100)

# Tamaños de fuente escalados al panel: PSIL y MTP a tamaño moderado (van
# como paneles inferiores a media anchura del composite); Delta proporcionalmente
# mayor (~1.6x-1.7x) porque ocupa anchura completa y doble altura.
png(file.path(plot_dir, "bn_heatmap_C1.png"), width=900, height=700, res=130)
ph_c1 <- pheatmap(mat_c1, cluster_rows=FALSE, cluster_cols=FALSE,
                  color=c12_color, breaks=c12_breaks,
                  display_numbers=display_c1, fontsize_number=12,
                  fontsize=14, fontsize_row=12, fontsize_col=12,
                  na_col="grey90", border_color="white",

```

```

    main=paste0("[Inter-red 8x8] Conectividad media | ", C1))
dev.off()

png(file.path(plot_dir, "bn_heatmap_C2.png"), width=900, height=700, res=130)
ph_c2 <- pheatmap(mat_c2, cluster_rows=FALSE, cluster_cols=FALSE,
  color=c12_color, breaks=c12_breaks,
  display_numbers=display_c2, fontsize_number=12,
  fontsize=14, fontsize_row=12, fontsize_col=12,
  na_col="grey90", border_color="white",
  main=paste0("[Inter-red 8x8] Conectividad media | ", C2))
dev.off()

png(file.path(plot_dir, "bn_heatmap_Delta.png"), width=900, height=700, res=130)
ph_delta <- pheatmap(mat_delta, cluster_rows=FALSE, cluster_cols=FALSE,
  color=colorRampPalette(c("#2980B9", "white", "#C0392B"))(100),
  display_numbers=cell_label_mat, fontsize_number=20,
  fontsize=24, fontsize_row=18, fontsize_col=18,
  na_col="grey90", border_color="white",
  main=paste0("[Inter-red 8x8] Delta ", C1, " - ", C2,
    "\n* FDR<0.05 | † FDR<0.10 | diag NA"))
dev.off()

# Figura combinada en dos filas: bloque superior con Delta a anchura
# completa como foco del contraste (2x lineal vs los paneles inferiores,
# conservando aspect ratio 900:700); bloque inferior con PSIL | MTP lado
# a lado a su tamaño nominal. PNG de 1800x2100 (vs 2700x700 del layout
# 1x3 anterior) - heights = c(2, 1) en grid.arrange materializa el 2x
# lineal de Delta.
library(gridExtra)
png(file.path(plot_dir, "bn_heatmap_combined.png"), width=1800, height=2100, res=130)
grid.arrange(
  ph_delta$gtable,
  arrangeGrob(ph_c1$gtable, ph_c2$gtable, ncol = 2),
  ncol = 1, heights = c(2, 1))
dev.off()

bn_tests_plot <- bn_tests %>%
  filter(!is.na(delta)) %>%
  mutate(sig_level = ifelse(significant, "FDR < 0.05", "n.s."),
    sig_level = factor(sig_level, levels = c("FDR < 0.05", "n.s.")),
    pair = fct_reorder(pair, abs(delta)))

```

```

p_dot_bn <- ggplot(bn_tests_plot, aes(x=delta, y=pair, color=delta>0, shape=sig_level)) +
  geom_vline(xintercept=0, linetype="dashed", color="grey50") +
  geom_point(size=3) +
  scale_color_manual(values=c("TRUE"="#C0392B", "FALSE"="#2980B9"), guide="none") +
  scale_shape_manual(values=c("FDR < 0.05"=16, "n.s."=1), name="Significacion") +
  theme_minimal(base_size=11) + theme(axis.text.y=element_text(size=9)) +
  labs(title = paste0("[Inter-red 8x8] Delta conectividad | ", C1, " - ", C2),
       subtitle = paste0("Rojo = ", C1, " > ", C2, " | Azul = ", C2, " > ", C1, " | Fill = ",
                        x = paste0("Delta conectividad media (", C1, " - ", C2, ")"),
                        y = "Par de redes")
ggsave(file.path(plot_dir, "bn_dotplot_delta.png"),
       p_dot_bn, width=7, height=7, dpi=300)

write.csv(data.frame(mat_delta), file.path(out_dir, "between_network_delta_matrix.csv"))

message(sprintf("[%s] OK Between-network 8x8 completado", ds))
message(sprintf(" Sig. FDR < 0.05: %d/28", sum(bn_tests$significant, na.rm=TRUE)))
}

```

6.4.1 6.4.1 Visualizaciones inter-red 8x8 (con barras de error y BayesFactor)

Esta sub-sección produce las visualizaciones de soporte de §6.4. El reparto es deliberado: §6.4 genera los heatmaps con anotación de significancia (* para $FDR < 0.05$, . para $FDR < 0.10$) y el dotplot ordenado por `|delta|`, que son las figuras pensadas para la memoria del TFM; §6.4.1 genera las versiones complementarias con anotación numérica completa de cada celda (para inspección detallada en los anexos) y el dotplot con barras $\pm SE$ inter-sujeto coloreado por nivel del Bayes Factor (para discutir qué pares quedan “inconclusos” vs los que tienen evidencia bayesiana de efecto o de nulo).

```

#=====
# §6.4.1 - Visualizaciones inter-red 8x8 (SE + BayesFactor) - dual dataset
#=====
library(tidyverse)
library(pheatmap)

for (ds in ACTIVE_DATASETS) {
  cd <- CONNECTOMES[[ds]]
  C1 <- toupper(cd$conditions[1])
  C2 <- toupper(cd$conditions[2])
  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_adv <- file.path(RESULTS_DS, "stats/statistical_results/advanced")
}

```

```

plot_dir <- file.path(RESULTS_DS, "figures/stats/advanced")
dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)

between_net <- read.csv(file.path(out_adv, "between_network_connectivity.csv"))
bn_bayes_csv <- file.path(out_adv, "between_network_BayesFactor.csv")
bn_bayes <- if (file.exists(bn_bayes_csv)) read.csv(bn_bayes_csv) else NULL

NET_LEVELS <- c("Vis", "SomMot", "DorsAttn", "SalVentAttn", "Limbic",
               "Cont", "Default", "Subcortex")

#=====
# 1. Heatmaps C1 / C2 / Delta con todos los valores anotados
#=====
make_net_matrix <- function(df_cond) {
  nets_present <- unique(c(df_cond$net1, df_cond$net2))
  nets <- intersect(NET_LEVELS, nets_present)
  mat <- matrix(0, nrow = length(nets), ncol = length(nets),
               dimnames = list(nets, nets))
  for (i in seq_len(nrow(df_cond))) {
    n1 <- df_cond$net1[i]; n2 <- df_cond$net2[i]
    if (!n1 %in% nets || !n2 %in% nets) next
    mat[n1, n2] <- df_cond$connectivity[i]
    mat[n2, n1] <- df_cond$connectivity[i]
  }
  mat
}

bn_group <- between_net %>%
  group_by(net1, net2, condition) %>%
  summarise(connectivity = mean(connectivity, na.rm = TRUE), .groups = "drop")

mat_c1 <- make_net_matrix(bn_group %>% filter(condition == C1))
mat_c2 <- make_net_matrix(bn_group %>% filter(condition == C2))

nets_common <- intersect(rownames(mat_c1), rownames(mat_c2))
mat_c1 <- mat_c1[nets_common, nets_common]
mat_c2 <- mat_c2[nets_common, nets_common]
mat_delta <- mat_c1 - mat_c2

# Nota: el heatmap del Delta vive en §6.4 (`bn_heatmap_Delta.png`) e incluye
# ya los valores numericos por celda y los marcadores FDR; aqui se mantienen
# solo las versiones por condicion (C1, C2) como anexo de inspeccion.

```

```

for (info in list(
  list(mat = mat_c1, file = "adv_between_net_C1.png",
        title = paste0("[GRUPO Inter-red 8x8] Conectividad | ", C1)),
  list(mat = mat_c2, file = "adv_between_net_C2.png",
        title = paste0("[GRUPO Inter-red 8x8] Conectividad | ", C2))
)) {
  col_ramp <- if (grepl("Delta", info$title))
    colorRampPalette(c("#2980B9", "white", "#C0392B"))(100)
  else
    colorRampPalette(c("white", "#C0392B"))(100)
  png(file.path(plot_dir, info$file), width = 720, height = 620, res = 130)
  pheatmap(info$mat,
            cluster_rows = FALSE, cluster_cols = FALSE, color = col_ramp,
            display_numbers = TRUE, number_format = "%.2f", fontsize_number = 8,
            fontsize_row = 10, fontsize_col = 10, border_color = "white",
            main = info$title)
  dev.off()
}

#=====
# 2. Dotplot Delta +- SE, coloreado por bf_evidence
#=====

bn_delta <- between_net %>%
  pivot_wider(names_from = condition, values_from = connectivity,
              id_cols = c(subject, net1, net2)) %>%
  filter(!is.na(.data[[C1]]), !is.na(.data[[C2]])) %>%
  mutate(delta = .data[[C1]] - .data[[C2]]) %>%
  group_by(net1, net2) %>%
  summarise(mean_delta = mean(delta, na.rm = TRUE),
            se_delta = sd(delta, na.rm = TRUE) / sqrt(n()),
            .groups = "drop") %>%
  mutate(pair = paste0(net1, " ", net2),
         pair = fct_reorder(pair, abs(mean_delta)))

if (!is.null(bn_bayes)) {
  bn_delta <- bn_delta %>%
    left_join(bn_bayes %>% select(pair, bf_evidence, BF10, BF01), by = "pair") %>%
    mutate(
      bf_simple = case_when(
        is.na(bf_evidence) ~ "n/a",
        grepl("fuerte H1", bf_evidence) ~ "fuerte H1",
        grepl("moderada H1", bf_evidence) ~ "moderada H1",

```

```

    grepl("anecdótica H1", bf_evidence) ~ "anecdótica H1",
    grepl("anecdótica H0", bf_evidence) ~ "anecdótica H0",
    grepl("moderada H0", bf_evidence) ~ "moderada H0",
    grepl("fuerte H0", bf_evidence) ~ "fuerte H0",
    TRUE ~ bf_evidence),
  bf_simple = factor(bf_simple,
    levels = c("fuerte H1", "moderada H1", "anecdótica H1",
      "anecdótica H0", "moderada H0", "fuerte H0", "n/a"))
} else {
  bn_delta$bf_simple <- factor("n/a")
}

bf_palette <- c("fuerte H1"="#922B21", "moderada H1"="#C0392B", "anecdótica H1"="#E8A399",
  "anecdótica H0"="#A6BCD4", "moderada H0"="#2980B9", "fuerte H0"="#1A5276",
  "n/a"="grey60")

p_dot_bn <- ggplot(bn_delta, aes(x = mean_delta, y = pair, color = bf_simple)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey50") +
  geom_errorbar(aes(xmin = mean_delta - se_delta, xmax = mean_delta + se_delta),
    width = 0.35, linewidth = 0.5, orientation = "y") +
  geom_point(size = 3) +
  scale_color_manual(values = bf_palette, drop = FALSE, name = "Bayes Factor") +
  theme_minimal(base_size = 11) + theme(axis.text.y = element_text(size = 9)) +
  labs(
    title = paste0("[GRUPO Inter-red 8x8] Delta conectividad | ", C1, " - ", C2),
    subtitle = "Color = BF evidence. Barras = +-SE inter-sujeto.",
    x = paste0("Delta conectividad media (", C1, " - ", C2, ")"),
    y = "Par de redes")
ggsave(file.path(plot_dir, "adv_between_net_dotplot_delta.png"),
  p_dot_bn, width = 9, height = 7, dpi = 300)

message(sprintf("[%s] OK Visualizaciones inter-red 8x8 completadas", ds))
}

```

6.5 6.5 Conectividad basada en semillas (seed-based) — subcortex Tian para REBUS-1

Análisis **hypothesis-driven** restringido a las **5 semillas subcorticales del atlas Tian S2** (hipocampo, amígdala, tálamo, núcleo accumbens, putamen). El scope se redujo desde una propuesta inicial de 12 semillas (7 corticales + 5 subcorticales) tras constatar que el bloque cortical duplica con peor potencia lo que §6.4 (inter-red 8×8) ya cuantifica con 28

pares a nivel de red. Las 5 Tian son, en cambio, el **único punto del pipeline** que separa el subcortex por estructura — §6.4 colapsa Subcortex en una sola red, perdiendo la distinción HIP/AMY/THA/NAc/PUT que la predicción REBUS-1 necesita.

Para cada semilla se computa el vector de conectividad funcional con los 232 ROIs y se testa pareado PSIL vs MTP por ROI con corrección BH-FDR within-seed (5 familias independientes $\times$ 232 tests = 1160 por construcción; 1136 efectivos tras anular las 24 celdas seed-to-self que la propia semilla ocuparía como destino — el vector excluye el bloque seed $\times$ seed para evitar el sesgo de mezclar la diagonal-cero con FC reales en una media multi-ROI).

Rationale por semilla (con cita bibliográfica):

Semilla Tian	ROIs capturadas	Hipótesis a priori bajo PSIL	Referencia
Hipocampo	aHIP + pHIP bilateral (4 ROIs)	$\uparrow$ FC con DMN cortex (mPFC, PCC); decoupling MTL-cortex; $\uparrow$ varianza intra-MTL	Carhart-Harris 2014 §“Medial temporal lobes and primary consciousness”; Kaelen et al. 2015, <i>Psychopharmacology</i> 232:3607
Amígdala	lAMY + mAMY bilateral (4 ROIs)	Cambios FC con DMN; predicción direccional débil	Kaelen et al. 2015, <i>Psychopharmacology</i> 232:3607; Roseman et al. 2018, <i>Neuropharmacology</i> 142:263
Tálamo	THA- {DP,VP,VA,DA} bilateral (8 ROIs)	$\uparrow$ thalamo-cortical FC vía 5-HT _{2A} presináptica $\rightarrow$ integración cortex transmodal	Preller et al. 2018, <i>eLife</i> 7:e35082; Müller et al. 2017, <i>Acta Psychiatr Scand</i> 136:648
Núcleo accumbens	NAc-{shell,core} bilateral (4 ROIs)	$\uparrow$ FC global; alteración hub centrality reward	Müller et al. 2018, <i>NeuroImage Clin</i> 18:694
Putamen	aPUT + pPUT bilateral (4 ROIs)	$\downarrow$ CBF y $\downarrow$ FC reportados con psilocibina	Carhart-Harris et al. 2012, <i>PNAS</i> 109:2138

Las 24 ROIs Tian capturadas son las 5 estructuras a priori bajo REBUS-1; las 8 restantes del atlas (Globo Pálido aGP+pGP, Caudado aCAU+pCAU bilateral) no se testan como semillas porque no hay predicción direccional para ellas en la literatura psicodélica revisada.

Marco interpretativo. Todas las semillas Tian operan como **origen** del análisis y el cortex transmodal (Default, Cont, SalVentAttn) como **destino preferente** bajo la predicción 1 de la **REBUS / entropic brain hypothesis** (Carhart-Harris & Friston 2019; Carhart-Harris 2014):

Desinhibición top-down de cortex transmodal sobre estructuras subcorticales ↑ FC desde las 5 semillas Tian hacia DMN/Cont/SalVentAttn bajo PSIL, con efecto especialmente marcado para HIP→DMN (Carhart-Harris 2014 §MTL) y THA→cortex transmodal (Preller 2018, Müller 2017).

El **heatmap semilla × red destino** (`seed_heatmap_by_network.png`) materializa esta predicción de forma directa: cada celda es el **d_z** medio del contraste PSIL–MTP de la fila-semilla con todas las ROIs de la red-destino. La columna Default/Cont/SalVentAttn de cada semilla Tian es la lectura primaria REBUS-1 del análisis. El resto de visualizaciones (top 10 ROIs por semilla, dotplot de mediana effect size, raincloud de FC media) son lecturas exploratorias complementarias.

La segunda predicción REBUS (aplanamiento del gradiente unimodal-transmodal, Girn 2022) no se operacionaliza en §6.5 porque requería semillas corticales V1/dlPFC — esa predicción queda cubierta por §6.4 en la familia inter-red 8×8 (par Cont Vis del bloque REBUS one-sided).

Resolución de las 5 semillas Tian en el atlas combinado

Cada semilla se resuelve por *substring literal* (`grep1(..., fixed = TRUE)`) sobre las etiquetas del atlas combinado Schaefer 200 + Tian S2. Las etiquetas Tian tienen el formato `Subcortex_<prefijo>-{rh, lh}` (p. ej. `Subcortex_aHIP-rh`, `Subcortex_THA-DP-lh`); los patrones declarados — `c("aHIP", "pHIP")`, `c("lAMY", "mAMY")`, `"THA-"`, `c("Nac-shell", "Nac-core")`, `c("aPUT", "pPUT")` — capturan exactamente las 4/4/8/4/4 ROIs anatómicamente esperadas. El chunk imprime las cuentas resueltas y las contrasta con el conteo esperado: si alguna desviase, lo marca con `!!` en la consola para que la discrepancia sea visible antes de interpretar los resultados (la semilla se procesa de todos modos con el subconjunto detectado — preferimos visibilidad explícita a fallar silenciosamente).

Una versión previa de este chunk usaba `\b` (word boundary) sobre las mismas etiquetas y se comportaba como bug silencioso: `\b` no genera límite entre dos letras consecutivas ni entre `_` y letra, por lo que matches como `\bHIP` sobre `Subcortex_aHIP-rh` devuelven 0 — las 5 semillas Tian se omitían sin que se produjera ningún output. Los patrones literales `fixed = TRUE` eliminan esta clase de error.

Potencia con N=5–6 — análisis exploratorio

Con N=5–6 sujetos pareados (ds006072) y ~228 tests efectivos por semilla bajo FDR-BH (232 ROIs menos las 4–8 auto-referenciales que el bloque `seed×seed` anula), la potencia para detectar diferencias FC `seed×ROI` individuales es muy limitada incluso tras la

reducción a 5 semillas (1136 tests efectivos en lugar de los 2784 originales). Los resultados se reportan como **exploratorios y descriptivos**: el heatmap seed $\times$ red destino (5 $\times$ 8) es la lectura primaria REBUS-1, agregada y robusta; el ranking por $|d_z|$ identifica ROIs candidatas para replicación cuando se integre el subconjunto 2 (N 6) o se amplíe con los participantes de replicación P1R-P5R (cf. README ds006072). La sección **sí participa de la regla de credibilidad S0** desde la ronda 2 vía §6.5.1, que replica el análisis sobre los siete pipelines de control y reporta `n_dir`, `n_dir_noGSR` y `sign_match_GSR` por cada celda seed $\times$ red. Lo que el N = 6 no puede dar — supervivencia FDR per se y CI bootstrap excluyente del cero a nivel ROI — sigue pendiente de muestra mayor.

```
#####
# §6.5 - Seed-based connectivity (Tian-only) - dual dataset
#####
# Análisis hypothesis-driven sobre las 5 semillas subcorticales Tian S2:
# HIP, AMY, THA, NAc, PUT. Reducido desde 12 semillas tras constatar que las
# 7 corticales duplicaban §6.4 (inter-red 8 $\times$ 8) con peor potencia. Las 5 Tian
# son el único punto del pipeline que separa el subcortex por estructura para
# operacionalizar la predicción REBUS-1 (Carhart-Harris & Friston 2019,
# Carhart-Harris 2014): desinhibición top-down de cortex transmodal sobre
# MTL/estriado.
#
# Por cada semilla:
#   1. Resolver índices en el atlas por substring literal (fixed=TRUE).
#   2. Computar vector FC con los 232 ROIs (mean de filas de la semilla,
#       con NA en el bloque seed $\times$ seed para evitar sesgo seed-to-self).
#   3. Test pareado C1 vs C2 por ROI:  $d_z + d_{av}$  manuales (no effsize), t y
#       Wilcoxon en paralelo, ambos con FDR-BH within-seed.
#   4. Heatmap seed  $\times$  red destino = lectura primaria REBUS-1.
#
# Referencias (en BIBLIOGRAPHY.md):
#   Carhart-Harris 2014 §MTL, Carhart-Harris & Friston 2019 REBUS,
#   Kaelen 2015 (música/MTL), Roseman 2018 (amígdala/MDD), Preller 2018
#   (thalamic 5-HT2A), Müller 2017 (thalamo-cortical core driver),
#   Müller 2018 (hub LSD), Carhart-Harris 2012 PNAS (CBF/FC PSIL).
# Effect size:  $d_z$ ,  $d_{av}$  manuales (Lakens 2013; Borenstein 2009) - coherente
# con §6.1/§6.3/§6.4.
#####

library(tidyverse)
library(pheatmap)
library(ggghalves)
```

```

for (ds in ACTIVE_DATASETS) {
  cd <- CONNECTOMES[[ds]]
  c1_key <- cd$conditions[1]; c2_key <- cd$conditions[2]
  C1 <- toupper(c1_key); C2 <- toupper(c2_key)
  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  # Nota: las matrices se sirven en memoria desde cd$mean_subject ($1.1), por
  # lo que no se reconstruye CONN_DIR_PRIMARY aquí - el patrón obsoleto de
  # rederivar `file.path(..., cd$pipeline)` heredaba el bug R0 destapado en
  # §6.3 (la etiqueta legible `TEDANA+aCompCor` no es el dir real
  # `TEDANA_aCompCor`). cd$conn_dir queda disponible en cd si alguna lectura
  # futura lo necesita.

  out_seed <- file.path(RESULTS_DS, "stats/statistical_results/seed_based")
  plot_seed <- file.path(RESULTS_DS, "figures/stats/seed_based")
  dir.create(out_seed, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_seed, showWarnings = FALSE, recursive = TRUE)

  # H6-A: reusar atlas_labels ya cargado y validado por §1.1 en memoria.
  atlas_labels <- cd$atlas_labels
  stopifnot(length(atlas_labels) == cd$n_rois)

  get_network_label <- function(label) {
    if (startsWith(label, "7Networks_")) strsplit(label, "_")[[1]][3]
    else "Subcortex"
  }
  node_network <- vapply(atlas_labels, get_network_label, character(1))
  atlas_clean <- gsub("^7Networks_", "", atlas_labels)

  # --- 5 semillas Tian a priori (REBUS-1) -----
  # H1-A: patrones literales con fixed=TRUE. Las 24 ROIs Tian capturadas son
  # las 5 estructuras a priori (HIP=4, AMY=4, THA=8, NAc=4, PUT=4); las 8
  # restantes (Globo Pálido aGP+pGP, Caudado aCAU+pCAU bilateral) no entran
  # porque no hay predicción direccional para ellas en la literatura
  # psicodélica revisada. Etiquetas reales (Subcortex_aHIP-rh, Subcortex_THA-
  # DP-lh, etc.); ver callout-note "Resolución de las 5 semillas Tian".
  seeds <- list(
    HIP = list(patterns = c("aHIP", "pHIP"),
               label    = "Hipocampo (Tian: aHIP+pHIP bilateral)",
               ref      = "Carhart-Harris 2014 §MTL; Kaelen 2015",
               n_expected = 4L),
    AMY = list(patterns = c("lAMY", "mAMY"),
               label    = "Amígdala (Tian: lAMY+mAMY bilateral)",

```

```

        ref      = "Kaelen 2015; Roseman 2018",
        n_expected = 4L),
THA = list(patterns = "THA-",
        label     = "Talamo (Tian: DP+VP+VA+DA bilateral)",
        ref       = "Preller 2018 eLife; Muller 2017",
        n_expected = 8L),
NAc = list(patterns = c("NAc-shell", "NAc-core"),
        label     = "Nucleo Accumbens (Tian: shell+core bilateral)",
        ref       = "Muller 2018 NeuroImage Clin",
        n_expected = 4L),
PUT = list(patterns = c("aPUT", "pPUT"),
        label     = "Putamen (Tian: aPUT+pPUT bilateral)",
        ref       = "Carhart-Harris 2012 PNAS",
        n_expected = 4L)
)

resolve_seed <- function(patterns, labels) {
  which(Reduce(`|`, lapply(patterns, function(p)
    grepl(p, labels, fixed = TRUE))))
}

seed_indices <- imap(seeds, function(s, nm) resolve_seed(s$patterns, atlas_labels))
seed_indices <- seed_indices[sapply(seed_indices, length) > 0]

cat(sprintf("\n=== [%s] Semillas Tian resueltas en atlas ===\n", ds))
for (nm in names(seed_indices)) {
  n_obs <- length(seed_indices[[nm]])
  n_exp <- seeds[[nm]]$n_expected
  ok    <- if (n_obs == n_exp) "OK" else sprintf("!! (esperadas %d)", n_exp)
  cat(sprintf("  %-4s  n=%2d ROIs %-18s | %s\n",
    nm, n_obs, ok, seeds[[nm]]$label))
}
seeds_skipped <- setdiff(names(seeds), names(seed_indices))
if (length(seeds_skipped) > 0)
  cat(sprintf("  Omitidas (0 ROIs): %s\n", paste(seeds_skipped, collapse=" ")))

# --- Matrices por sujeto (todas en memoria desde $1.1) -----
# H5-A: el primario sirve directo desde cd$mean_subject; ningún fallback a
# disco ($1.1 valida en carga con stopifnot dim+simetría+ rango).
subjects <- cd$common_subjects
cat(sprintf("Sujetos comunes [%s]: %d\n", ds, length(subjects)))

```

```

# H8-A: poner NA en el bloque seed*seed antes del colMean evita el sesgo
# seed-to-self cuando |seed_idx| > 1 (todas las Tian: 4-8 ROIs). Las
# columnas que pertenecen a la propia semilla quedan como NaN y se filtran
# más adelante; el ranking top10 no devuelve "HIP top match = HIP".
compute_seed_vec <- function(mat, seed_idx) {
  if (is.null(mat) || length(seed_idx) == 0) return(rep(NA_real_, nrow(mat)))
  mat_clean <- mat
  mat_clean[seed_idx, seed_idx] <- NA_real_
  colMeans(mat_clean[seed_idx, , drop = FALSE], na.rm = TRUE)
}

seed_data <- map_dfr(subjects, function(subj) {
  mat_c1 <- cd$mean_subject[[c1_key]][[subj]]
  mat_c2 <- cd$mean_subject[[c2_key]][[subj]]
  stopifnot(!is.null(mat_c1), !is.null(mat_c2),
            nrow(mat_c1) == length(atlas_labels),
            nrow(mat_c2) == length(atlas_labels))

  imap_dfr(seed_indices, function(sidx, sname) {
    vec_c1 <- compute_seed_vec(mat_c1, idx)
    vec_c2 <- compute_seed_vec(mat_c2, idx)
    tibble(
      subject = subj,
      seed = sname,
      roi = seq_along(atlas_labels),
      roi_label = atlas_clean,
      network = node_network,
      fc_c1 = vec_c1,
      fc_c2 = vec_c2,
      fc_delta = vec_c1 - vec_c2
    )
  })
})

write.csv(seed_data, file.path(out_seed, "seed_connectivity_all.csv"),
          row.names = FALSE)

message(sprintf("[%s] Tests pareados seed x ROI...", ds))
# H4-A: d_z y d_av calculados manualmente (Lakens 2013 eq. 4 y 8; Borenstein
# 2009 cap. 4). effsize::cohen.d(paired=TRUE) devuelve d_rm - no se usa.
# H7-B: p_wilcox FDR-corregida en paralelo a p_value y reportada como
# sensibilidad robusta.

```

```

seed_stats <- seed_data %>%
  group_by(seed, roi, roi_label, network) %>%
  summarise(
    n = sum(!is.na(fc_c1) & !is.na(fc_c2)),
    !!paste0("mean_", C1) := mean(fc_c1, na.rm = TRUE),
    !!paste0("mean_", C2) := mean(fc_c2, na.rm = TRUE),
    mean_delta = mean(fc_delta, na.rm = TRUE),
    d_z = tryCatch({
      ok <- !is.na(fc_c1) & !is.na(fc_c2)
      diffs <- fc_c1[ok] - fc_c2[ok]
      if (length(diffs) < 3 || sd(diffs, na.rm = TRUE) == 0) NA_real_
      else mean(diffs, na.rm = TRUE) / sd(diffs, na.rm = TRUE)
    }, error = \ (e) NA_real_),
    d_av = tryCatch({
      ok <- !is.na(fc_c1) & !is.na(fc_c2)
      if (sum(ok) < 3) NA_real_
      else {
        sd_avg <- mean(c(sd(fc_c1[ok], na.rm = TRUE),
                          sd(fc_c2[ok], na.rm = TRUE)), na.rm = TRUE)
        if (is.na(sd_avg) || sd_avg == 0) NA_real_
        else mean(fc_c1[ok] - fc_c2[ok], na.rm = TRUE) / sd_avg
      }
    }, error = \ (e) NA_real_),
    p_value = tryCatch({
      ok <- !is.na(fc_c1) & !is.na(fc_c2)
      if (sum(ok) < 3) NA_real_
      else t.test(fc_c1[ok], fc_c2[ok], paired = TRUE)$p.value
    }, error = \ (e) NA_real_),
    p_wilcox = tryCatch({
      ok <- !is.na(fc_c1) & !is.na(fc_c2)
      if (sum(ok) < 3) NA_real_
      else wilcox.test(fc_c1[ok], fc_c2[ok],
                       paired = TRUE, exact = FALSE)$p.value
    }, error = \ (e) NA_real_),
    .groups = "drop"
  )

seed_stats <- seed_stats %>%
  group_by(seed) %>%
  mutate(p_FDR = p.adjust(p_value, "fdr"),
         p_wilcox_FDR = p.adjust(p_wilcox, "fdr"),
         significant = !is.na(p_FDR) & p_FDR < 0.05,

```

```

        sig_wilcox = !is.na(p_wilcox_FDR) & p_wilcox_FDR < 0.05) %>%
ungroup()

cat(sprintf("\n=== [%s] Seed-based connectivity - resumen ===\n", ds))
seed_summary <- seed_stats %>%
  filter(!is.na(p_value)) %>%
  group_by(seed) %>%
  summarise(
    n_tests      = n(),
    n_sig        = sum(significant, na.rm = TRUE),
    n_sig_wilcox = sum(sig_wilcox, na.rm = TRUE),
    max_d_z      = max(abs(d_z), na.rm = TRUE),
    median_d_z   = median(d_z, na.rm = TRUE),
    .groups      = "drop"
  ) %>%
  left_join(
    tibble(seed = names(seeds),
           label = sapply(seeds, function(x) x$label),
           ref   = sapply(seeds, function(x) x$ref)),
    by = "seed")
print(as.data.frame(seed_summary), row.names = FALSE)

write.csv(seed_stats, file.path(out_seed, "seed_stats_per_ROI.csv"), row.names = FALSE)
write.csv(seed_summary, file.path(out_seed, "seed_summary.csv"), row.names = FALSE)

top_seed <- seed_stats %>%
  filter(!is.na(d_z)) %>%
  group_by(seed) %>%
  slice_max(abs(d_z), n = 10) %>%
  ungroup()
write.csv(top_seed, file.path(out_seed, "seed_top10_ROI.csv"), row.names = FALSE)

#--- Visualizaciones seed-based ---
# Paleta de condiciones: magenta #C2185B (PSIL) + teal oscuro #00695C
# (MTP) - rojo/azul reservados
# para POS/NEG en §5 y §6.3.
col_cond <- setNames(c("#C2185B", "#00695C"), c(C1, C2))
lbl      <- sprintf("%s vs %s", C1, C2)

# 1. Barplot d_z top 10 ROIs por semilla
# H11-B: caption explícito sobre rol exploratorio del top10 con N pequeño.
# imap (no iwalk) para retener los plots y poder componer el composite

```



```

# PUT/HIP/THA que consume R7.
seed_top_plots <- imap(seed_indices, function(sidx, sname) {
  seed_label <- seeds[[sname]]$label
  df_top <- top_seed %>% filter(seed == sname) %>%
    mutate(roi_label = fct_reorder(roi_label, d_z),
           sig = !is.na(p_FDR) & p_FDR < 0.05)

  if (nrow(df_top) == 0) return(NULL)

  p <- ggplot(df_top, aes(x = roi_label, y = d_z, fill = d_z > 0)) +
    geom_col(width = 0.65) +
    geom_text(aes(label = ifelse(sig, "*", ""),
                      vjust = ifelse(d_z >= 0, -0.3, 1.2)), size = 5) +
    geom_hline(yintercept = 0, linewidth = 0.4) +
    scale_fill_manual(values = c("TRUE" = "#E74C3C", "FALSE" = "#3498DB"),
                      guide = "none") +
    coord_flip() +
    theme_minimal(base_size = 11) +
    theme(
      plot.title = element_text(size = 10),
      plot.caption = element_text(hjust = 0, size = 8)
    ) +
    labs(
      title = paste0("[Semilla: ", seed_label, "] Top 10 ROIs | ", lbl),
      subtitle = paste0("* FDR < 0.05 * rojo = ", C1, " > ", C2,
                        " * azul = ", C2, " > ", C1),
      caption = paste0("Ref: ", seeds[[sname]]$ref, "\n",
                        "Exploratorio: con N pequeño, el ordenamiento por |d_z| ",
                        "sobre-representa ROIs con sd(diferencias) baja;\n",
                        "los valores altos no son estimaciones puntuales generalizables.")
      x = "ROI", y = expression(italic(d[z]))
    )

  ggsave(file.path(plot_seed, paste0("seed_", sname, "_top_ROIs.png")),
          p, width = 8, height = 5.8, dpi = 300)
  p
})

# 1b. Composite 1x3 PUT / HIP / THA - figura central del bloque seed-based
# de R7. El orden replica la secuencia narrativa del informe: PUT (caso
# dominante, reversión direccional del a priori), HIP (predicción REBUS-1
# clásica, supervivencia FDR sub-umbral), THA (única supervivencia FDR

```

```

# no-PUT, patrón distribuido). El composite se introduce una sola vez en
# el cuerpo de R7 y las subsecciones de HIP/THA refieren a sus paneles.
composite_seeds <- c("PUT", "HIP", "THA")
if (all(!sapply(seed_top_plots[composite_seeds], is.null))) {
  panel_labels <- c(PUT = "PUT (putamen)",
                    HIP = "HIP (hipocampo)",
                    THA = "THA (talamo)")
  panels_top <- map2(seed_top_plots[composite_seeds],
                    panel_labels[composite_seeds],
                    function(p, ttl) {
      p + labs(title = ttl, subtitle = NULL, caption = NULL) +
      theme(
        plot.title = element_text(size = 22,
                                   face = "bold",
                                   hjust = 0.5),
        axis.text = element_text(size = 16),
        axis.title = element_text(size = 17),
        plot.subtitle = element_blank(),
        plot.caption = element_blank()
      )
    })
  composite_top <- patchwork::wrap_plots(panels_top, nrow = 1) +
  patchwork::plot_annotation(
    title = paste0("[Seed-based Tian] Top 10 ROIs por |d_z| | PUT / HIP / THA |",
    subtitle = paste0("* FDR-BH within-seed < 0.05 * rojo = ", C1, " > ", C2,
                      " * azul = ", C2, " > ", C1),
    theme = theme(
      plot.title = element_text(size = 26, face = "bold"),
      plot.subtitle = element_text(size = 18)
    )
  )
  ggsave(file.path(plot_seed, "seed_top_ROIs_composite.png"),
          composite_top, width = 22, height = 9, dpi = 300, limitsize = FALSE)
}

# 2. Heatmap d_z medio: semilla x red (8 redes Yeo-7 + Subcortex)
# Lectura primaria REBUS-1: columnas Default / Cont / SalVentAttn
NET_LEVELS <- c("Vis", "SomMot", "DorsAttn", "SalVentAttn", "Limbic",
               "Cont", "Default", "Subcortex")
heat_net <- seed_stats %>%
  filter(!is.na(d_z)) %>%
  mutate(network = factor(network, levels = NET_LEVELS)) %>%

```

```

group_by(seed, network) %>%
  summarise(mean_d_z = mean(d_z, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = network, values_from = mean_d_z) %>%
  column_to_rownames("seed")

nets_present <- intersect(NET_LEVELS, colnames(heat_net))
heat_net <- heat_net[, nets_present, drop = FALSE]

if (nrow(heat_net) > 0 && ncol(heat_net) > 0) {
  png(file.path(plot_seed, "seed_heatmap_by_network.png"),
      width = 1000, height = 600, res = 130)
  pheatmap(heat_net,
            cluster_rows = FALSE, cluster_cols = FALSE,
            color = colorRampPalette(c("#3498DB", "white", "#E74C3C"))(100),
            display_numbers = TRUE, number_format = "%.2f",
            fontsize_number = 8,
            border_color = "white", fontsize_row = 10, fontsize_col = 9,
            main = paste0("[Seed-based Tian] d_z medio semilla x red destino | ", lbl,
                          "\nLectura REBUS-1: columnas Default / Cont / SalVentAttn (rojo =
                          C1, " > ", C2, ")"))
  dev.off()
}

# 3. Raincloud de conectividad media por semilla (C1 vs C2)
seed_avg <- seed_data %>%
  group_by(subject, seed) %>%
  summarise(
    !!paste0("FC_", C1) := mean(fc_c1, na.rm = TRUE),
    !!paste0("FC_", C2) := mean(fc_c2, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  pivot_longer(starts_with("FC_"),
               names_to = "condition", values_to = "FC",
               names_prefix = "FC_")

p_rain_seed <- ggplot(seed_avg, aes(x = condition, y = FC, fill = condition)) +
  geom_half_violin(side = "l", alpha = 0.45, trim = FALSE) +
  geom_boxplot(width = 0.13, outlier.shape = NA) +
  geom_jitter(width = 0.06, size = 2, alpha = 0.8) +
  geom_line(aes(group = subject), color = "grey55",
            linewidth = 0.35, alpha = 0.6) +
  scale_fill_manual(values = col_cond) +

```

```

facet_wrap(~seed, scales = "free_y", ncol = 3) +
theme_minimal(base_size = 11) +
theme(legend.position = "none",
      strip.text = element_text(face = "bold")) +
labs(
  title = paste0("[Seed-based Tian] Conectividad media por semilla | ", lbl),
  subtitle = "5 semillas subcorticales Tian S2 (HIP, AMY, THA, NAc, PUT)",
  x = "Condicion", y = "Conectividad media (FC z)"
)

n_seeds_resolved <- length(seed_indices)
ggsave(file.path(plot_seed, "seed_raincloud_all_seeds.png"),
  p_rain_seed,
  width = 12,
  height = 3 * ceiling(n_seeds_resolved / 3),
  dpi = 300, limitsize = FALSE)

# 4. Dotplot ranking de tamaño de efecto por semilla
seed_overall <- seed_stats %>%
  filter(!is.na(d_z)) %>%
  group_by(seed) %>%
  summarise(
    median_d_z = median(d_z, na.rm = TRUE),
    iqr_lo = quantile(d_z, 0.25, na.rm = TRUE),
    iqr_hi = quantile(d_z, 0.75, na.rm = TRUE),
    n_sig = sum(p_FDR < 0.05, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  left_join(tibble(seed = names(seeds),
                  label = sapply(seeds, function(x) x$label)),
    by = "seed") %>%
  mutate(label = fct_reorder(label, median_d_z))

p_overall <- ggplot(seed_overall,
  aes(x = median_d_z, y = label, color = median_d_z > 0)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey50") +
  geom_errorbar(aes(xmin = iqr_lo, xmax = iqr_hi),
    width = 0.3, linewidth = 0.5, orientation = "y") +
  geom_point(aes(size = n_sig)) +
  scale_color_manual(values = c("TRUE" = "#E74C3C", "FALSE" = "#3498DB"),
    guide = "none") +
  scale_size_continuous(name = "ROIs sig.\n(FDR<0.05)",

```

```

        breaks = c(0, 5, 10, 20, 50)) +
theme_minimal(base_size = 11) +
labs(
  title      = paste0("[Seed-based Tian] Resumen efecto por semilla | ",
                      C1, " - ", C2),
  subtitle   = paste0("Punto = mediana d_z (", C1, " - ", C2,
                      ") * barras = IQR * tamaño = ROIs sig. FDR<0.05"),
  x = expression(paste("Mediana ", italic(d[z]))), y = ""
)

ggsave(file.path(plot_seed, "seed_overall_effect_by_seed.png"),
        p_overall, width = 8, height = 5, dpi = 300)

# H12-A: mensaje final sin la separación cortical/subcortical (todo Tian).
message(sprintf("[%s] OK Seed-based Tian completada", ds))
message(sprintf("  Semillas resueltas: %d/%d (HIP/AMY/THA/NAc/PUT)",
                length(seed_indices), length(seeds)))
message("  Resultados: ", out_seed)
message("  Figuras:    ", plot_seed)
}

```

6.6 6.5.1 Convergencia inter-pipeline del seed-based (criterio S0)

El cierre original de §6.5 dejaba el seed-based **fuera** de la regla de credibilidad S0 (cf. callout-warning del chunk: «sus outputs son terminales en el flujo del pipeline»). R7 levantó esa limitación: con la firma cabecera del PUT (114 supervivientes FDR; $\|d_z\|_{\max} = 6.80$) y el caveat de Murphy & Fox (2017) sobre la dependencia de las medias positivas globales al régimen no-GSR, conviene re-evaluar el patrón seed-based en los siete pipelines de control (los mismos que §6.2 usa para topología) y aplicar el criterio S0 que R4 formalizó.

El procedimiento es paralelo al de §6.2 pero adaptado a la geometría seed $\times$ ROI: para cada uno de los siete pipelines se reconstruye el vector de FC desde cada semilla Tian hacia los 232 ROIs (mismas reglas de §6.5: máscara seed $\times$ seed, contraste pareado intra-sujeto), se obtiene d_z por (semilla, ROI), y se agrega a d_z medio por (semilla, red destino) — exactamente la lectura primaria REBUS-1 del heatmap de §6.5. Sobre las 40 celdas (5 semillas $\times$ 8 redes) se aplica $n_{\text{dir}} = 6/7$ para `converge_S0` y se reporta $n_{\text{dir_noGSR}}$ (6/6) y `sign_match_GSR` para visibilizar la dirección del pipeline GSR contra el primario.

```

#=====
# §6.5.1 - Seed-based S0 convergence (7 pipelines)
#=====
# Insumo: derivatives/ds006072/conn/<pipe>/{psil,mtp}/<subj>/<subj>_mean_<cond>.csv

```

```

# para los 7 pipelines × 6 sujetos × 2 condiciones = 84 matrices 232x232.
# Productos:
#   seed_pipeline_per_ROI.csv           (5 × 232 × 7 = 8120 filas: d_z, p_FDR)
#   seed_pipeline_summary.csv           (5 × 7 = 35 filas: n_sig_FDR por celda)
#   seed_network_pipeline_long.csv      (5 × 8 × 7 = 280 filas: mean d_z)
#   seed_pipeline_convergence_summary.csv (5 × 8 = 40 celdas con S0 flags)
#   seed_pipeline_convergence_per_ROI.csv (1160 celdas con S0 a nivel ROI)
#   seed_PUT_top10_S0.csv               (10 ROIs PUT cabecera con flags S0)
#   seed_pipeline_convergence_heatmap.png
#
# S0_THRESH (ds006072) = 6L; n_dir cuenta sobre los 7 pipelines, n_dir_noGSR
# sobre los 6 no-GSR. El primario es TEDANA_aCompCor y el GSR-pipe es
# OC_36P_GSR (los nombres son los de los directorios bajo conn/, no las
# etiquetas legibles que $4 usa en CONTROL_PIPELINES_BY_DS).
#=====
library(pheatmap)

for (ds in ACTIVE_DATASETS) {
  if (ds != "ds006072") {
    cat(sprintf("[%s] §6.5.1 omitida (los 7 pipelines de control son ds006072-only).\n", ds))
    next
  }
  cd <- CONNECTOMES[[ds]]
  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_seed <- file.path(RESULTS_DS, "stats/statistical_results/seed_based")
  plot_seed <- file.path(RESULTS_DS, "figures/stats/seed_based")
  dir.create(out_seed, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_seed, showWarnings = FALSE, recursive = TRUE)

  CONN_ROOT <- file.path(PROJECT_ROOT, "derivatives", ds, "conn")
  PIPELINES <- c("TEDANA_aCompCor", "TEDANA", "TEDANA_24P",
                 "OC_siegel", "OC_24P", "OC_aCompCor", "OC_36P_GSR")
  PRIMARY <- "TEDANA_aCompCor"
  GSR_PIPE <- "OC_36P_GSR"
  S0_THRESH <- 6L

  atlas_labels <- cd$atlas_labels
  get_network_label <- function(label) {
    if (startsWith(label, "7Networks_")) strsplit(label, "_")[[1]][3]
    else "Subcortex"
  }
  node_network <- vapply(atlas_labels, get_network_label, character(1))

```

```

atlas_clean <- gsub("^7Networks_", "", atlas_labels)

# Mismas 5 semillas Tian que §6.5
seeds <- list(
  HIP = list(patterns = c("aHIP", "pHIP"),          n_expected = 4L),
  AMY = list(patterns = c("lAMY", "mAMY"),          n_expected = 4L),
  THA = list(patterns = "THA-",                    n_expected = 8L),
  NAc = list(patterns = c("NAc-shell", "NAc-core"), n_expected = 4L),
  PUT = list(patterns = c("aPUT", "pPUT"),          n_expected = 4L)
)
resolve_seed <- function(patterns, labels)
  which(Reduce(`|`, lapply(patterns, function(p) grepl(p, labels, fixed = TRUE))))
seed_indices <- imap(seeds, function(s, nm) resolve_seed(s$patterns, atlas_labels))
for (nm in names(seed_indices))
  stopifnot(length(seed_indices[[nm]]) == seeds[[nm]]$n_expected)

read_mean_pipe <- function(pipe, cond, subj) {
  f <- file.path(CONN_ROOT, pipe, cond, subj,
    sprintf("%s_mean_%s.csv", subj, cond))
  if (!file.exists(f)) return(NULL)
  mat <- as.matrix(read.csv(f, row.names = 1, check.names = FALSE))
  storage.mode(mat) <- "numeric"; mat
}
compute_seed_vec_pipe <- function(mat, seed_idx) {
  if (is.null(mat)) return(NULL)
  mat[seed_idx, seed_idx] <- NA_real_
  colMeans(mat[seed_idx, , drop = FALSE], na.rm = TRUE)
}

c1_key <- cd$conditions[1]; c2_key <- cd$conditions[2]
subjects <- cd$common_subjects
cat(sprintf("\n=== §6.5.1 [%s] seed-based S0 (7 pipelines × %d sujetos) ===\n",
  ds, length(subjects)))

bin_size <- length(PIPELINES) * length(subjects) * length(seeds)
all_long <- vector("list", bin_size); k <- 0L
for (pipe in PIPELINES) {
  cat(sprintf(" [%s] ...", pipe))
  for (subj in subjects) {
    m1 <- read_mean_pipe(pipe, c1_key, subj)
    m2 <- read_mean_pipe(pipe, c2_key, subj)
    if (is.null(m1) || is.null(m2)) next
  }
}

```

```

    for (sname in names(seed_indices)) {
      sidx <- seed_indices[[sname]]
      k <- k + 1L
      all_long[[k]] <- tibble(
        pipeline = pipe, subject = subj, seed = sname,
        roi = seq_along(atlas_labels), roi_label = atlas_clean,
        network = node_network,
        fc_c1 = compute_seed_vec_pipe(m1, sidx),
        fc_c2 = compute_seed_vec_pipe(m2, sidx)
      )
    }
  }
  cat(" OK\n")
}
all_long <- all_long[seq_len(k)]
long_df <- bind_rows(all_long)

# Tests por (pipeline, seed, ROI) - d_z manual + p_t + FDR within (pipe, seed)
seed_stats_pipe <- long_df %>%
  group_by(pipeline, seed, roi, roi_label, network) %>%
  summarise(
    n = sum(!is.na(fc_c1) & !is.na(fc_c2)),
    mean_delta = mean(fc_c1 - fc_c2, na.rm = TRUE),
    d_z = {
      ok <- !is.na(fc_c1) & !is.na(fc_c2)
      d <- fc_c1[ok] - fc_c2[ok]
      if (length(d) < 3 || sd(d) == 0) NA_real_ else mean(d) / sd(d)
    },
    p_value = {
      ok <- !is.na(fc_c1) & !is.na(fc_c2)
      if (sum(ok) < 3) NA_real_
      else tryCatch(t.test(fc_c1[ok], fc_c2[ok], paired = TRUE)$p.value,
        error = function(e) NA_real_)
    },
    .groups = "drop"
  ) %>%
  group_by(pipeline, seed) %>%
  mutate(p_FDR = p.adjust(p_value, "fdr"),
    sig_FDR = !is.na(p_FDR) & p_FDR < 0.05) %>%
  ungroup()

write.csv(seed_stats_pipe,

```



```

        file.path(out_seed, "seed_pipeline_per_ROI.csv"),
        row.names = FALSE)

seed_pipe_summary <- seed_stats_pipe %>%
  filter(!is.na(d_z)) %>%
  group_by(pipeline, seed) %>%
  summarise(n_tests = n(),
            n_sig_FDR = sum(sig_FDR, na.rm = TRUE),
            max_dz     = max(abs(d_z), na.rm = TRUE),
            median_dz  = median(d_z, na.rm = TRUE),
            min_pFDR   = min(p_FDR, na.rm = TRUE),
            .groups = "drop")
write.csv(seed_pipe_summary,
          file.path(out_seed, "seed_pipeline_summary.csv"),
          row.names = FALSE)

# Agregado seed × red destino × pipeline (lectura primaria REBUS-1)
NET_LEVELS <- c("Vis","SomMot","DorsAttn","SalVentAttn","Limbic","Cont",
               "Default","Subcortex")
seed_net_long <- seed_stats_pipe %>%
  filter(!is.na(d_z)) %>%
  mutate(network = factor(network, levels = NET_LEVELS)) %>%
  group_by(pipeline, seed, network) %>%
  summarise(mean_dz = mean(d_z, na.rm = TRUE), n_rois = n(),
            .groups = "drop")
write.csv(seed_net_long,
          file.path(out_seed, "seed_network_pipeline_long.csv"),
          row.names = FALSE)

# Criterio S0 (n_dir >= 6/7), separando GSR
s0_net <- seed_net_long %>%
  group_by(seed, network) %>%
  summarise(
    dz_primary      = mean_dz[pipeline == PRIMARY],
    sign_primary    = sign(dz_primary),
    n_dir           = sum(sign(mean_dz) == sign_primary, na.rm = TRUE),
    n_dir_noGSR     = sum(sign(mean_dz[pipeline != GSR_PIPE]) == sign_primary,
                          na.rm = TRUE),
    n_total         = n(),
    converge_S0     = n_dir >= S0_THRESH,
    sign_match_GSR  = sign(mean_dz[pipeline == GSR_PIPE]) == sign_primary,
    .groups         = "drop")

```

```

)
write.csv(s0_net,
          file.path(out_seed, "seed_pipeline_convergence_summary.csv"),
          row.names = FALSE)

cat(sprintf(" S0 (n_dir>=6/7) en seed x red: %d / 40 celdas\n",
            sum(s0_net$converge_S0)))
cat(sprintf(" noGSR (6/6): %d / 40 ; GSR sign_match: %d / 40\n",
            sum(s0_net$n_dir_noGSR == 6),
            sum(s0_net$sign_match_GSR, na.rm = TRUE)))

# Convergencia a nivel de ROI individual (1136 celdas efectivas)
roi_conv <- seed_stats_pipe %>%
  filter(!is.na(d_z)) %>%
  group_by(seed, roi, roi_label, network) %>%
  summarise(
    dz_primary      = d_z[pipeline == PRIMARY],
    sign_primary    = sign(dz_primary),
    n_dir           = sum(sign(d_z) == sign_primary, na.rm = TRUE),
    n_dir_noGSR     = sum(sign(d_z[pipeline != GSR_PIPE]) == sign_primary,
                          na.rm = TRUE),
    converge_S0     = n_dir >= S0_THRESH,
    sign_match_GSR  = sign(d_z[pipeline == GSR_PIPE]) == sign_primary,
    .groups         = "drop"
  )
write.csv(roi_conv,
          file.path(out_seed, "seed_pipeline_convergence_per_ROI.csv"),
          row.names = FALSE)

# Top 10 PUT del primario con flags S0
top10_put <- seed_stats_pipe %>%
  filter(pipeline == PRIMARY, seed == "PUT", !is.na(d_z)) %>%
  arrange(desc(abs(d_z))) %>% slice_head(n = 10) %>%
  select(roi, roi_label, network, d_z_primary = d_z, p_FDR_primary = p_FDR)
top10_put_S0 <- top10_put %>%
  left_join(roi_conv %>% filter(seed == "PUT") %>%
            select(roi, n_dir, n_dir_noGSR, converge_S0, sign_match_GSR),
            by = "roi")
write.csv(top10_put_S0,
          file.path(out_seed, "seed_PUT_top10_S0.csv"),
          row.names = FALSE)

```

```

# Heatmap convergencia seed × red destino
hm_dz <- s0_net %>%
  mutate(network = factor(network, levels = NET_LEVELS),
         seed     = factor(seed, levels = c("HIP", "AMY", "THA", "NAc", "PUT"))) %>%
  select(seed, network, dz_primary) %>%
  pivot_wider(names_from = network, values_from = dz_primary) %>%
  column_to_rownames("seed") %>% as.matrix()
hm_annot <- s0_net %>%
  mutate(network = factor(network, levels = NET_LEVELS),
         seed     = factor(seed, levels = c("HIP", "AMY", "THA", "NAc", "PUT")),
         annot    = ifelse(converge_S0,
                          paste0(n_dir, "*"), as.character(n_dir))) %>%
  select(seed, network, annot) %>%
  pivot_wider(names_from = network, values_from = annot) %>%
  column_to_rownames("seed") %>% as.matrix()

png(file.path(plot_seed, "seed_pipeline_convergence_heatmap.png"),
     width = 1200, height = 600, res = 130)
pheatmap(hm_dz, cluster_rows = FALSE, cluster_cols = FALSE,
         color = colorRampPalette(c("#3498DB", "white", "#E74C3C"))(100),
         display_numbers = hm_annot,
         fontsize_number = 9, number_color = "black",
         border_color = "white",
         main = paste0("[Seed-based S0] convergencia inter-pipeline (7 pipelines)\n",
                       "Numero = n_dir | * = converge S0 (>=6/7) | color = d_z primario"))
dev.off()

message(sprintf("[%s] OK $6.5.1 - S0 seed-based completada", ds))
}

```

6.7 6.6 Correlación cerebro-experiencia (MEQ30, EBI)

Esta sub-sección complementa el §2.1 con la pregunta nuclear de la tradición psicodélica: **¿la magnitud de los cambios topológicos cerebrales bajo PSIL covaría con la intensidad de la experiencia subjetiva?** Conectar el sustrato neural con el reporte fenomenológico es el sello de los estudios pivote de psilocibina (Griffiths et al., 2006, *Psychopharmacology* 187:268; MacLean et al., 2011, *J Psychopharmacol* 25:1453; Carhart-Harris & Friston, 2019, *Pharmacol Rev* 71:316).

Instrumentos subjetivos (disponibles en BehavioralAssessments/Psilocybin-NonIdentifiableInfor_DATA_*.csv de ds006072):

- **MEQ30 — Revised Mystical Experience Questionnaire** (Barrett, Johnson & Griffiths, 2015, *J Psychopharmacol* 29:1182). 30 ítems puntuados 0-5, agrupados en cuatro sub-escalas: *Mystical* (15 ítems: unidad interna/externa, calidad noética, sacralidad), *Positive Mood* (6), *Transcendence of Time/Space* (6), *Ineffability* (3). Validado como predictor de cambios persistentes en personalidad y bienestar tras psilocibina (MacLean et al., 2011).
- **EBI — Emotional Breakthrough Inventory** (Roseman et al., 2019, *J Psychopharmacol* 33:1076). 6 ítems puntuados 0-100 que cuantifican la “ruptura emocional” (catarsis, resolución de conflictos no resueltos). Predictor de outcomes clínicos en estudios de psilocibina-asistida.

Diseño analítico. Para cada sujeto se calcula la diferencia intra-sujeto $\Delta\text{brain} = \text{PSIL} - \text{MTP}$ en dos métricas representativas:

- Δ global efficiency (familia integración funcional; §2.1);
- Δ synch entropy_{assoc} (familia entropía dinámica; §3.1, Carhart-Harris et al. 2014).

La modularidad con signo Q^* fue retirada de §3 en la ronda 1 de revisión (la implementación reutilizaba la partición Louvain de §2.1 — Q^* restringido a esa partición, no maximizado sobre signed), por lo que `delta_Q_signed` ya no figura como variable cerebral. La SampEn (Wang 2014, Lebedev 2016) nunca se llegó a implementar; el alias engañoso que apuntaba a la synch entropy se eliminó.

Se correlaciona (Spearman, robusta a no-normalidad y outliers) cada sub-escala MEQ + EBI **on-PSIL** con cada Δbrain . Como **control sham** se reporta la diferencia MEQ_PSIL — MEQ_MTP: bajo MTP las puntuaciones deben ser 0 si el cuestionario es específico al estado psicodélico (validación interna del placebo activo).

Predicciones a priori (entropic brain hypothesis):

1. **MEQ_total** $\uparrow$ **$\Delta\text{synch_entropy_assoc}$** $\uparrow$ — experiencia mística intensa mayor desorden de la sincronía intra-red en redes de asociación (DMN, Cont, SalVentAttn), réplica directa de Carhart-Harris (2014, Fig. 6).
2. **MEQ_total** $\uparrow$ **$\Delta\text{global_efficiency}$** $\uparrow$ — mayor integración global (Müller et al. 2018, *NeuroImage*; Lord et al. 2019, *NeuroImage*).

Inferencia. Spearman con bootstrap percentil (R=10 000) para CI al 95 % y p de permutación de etiquetas (R=10 000) en paralelo. Con N 6 la asintótica de `cor.test` es poco fiable; el bootstrap aporta CI más conservador, pero igualmente sensible a la potencia limitada. Corrección BH-FDR within-family (6 predictores $\times$ 2 Δbrain = 12 tests). La ronda 2 (2026-05-26) añade un **bloque LOO sobre las correlaciones** (contrato r3 de §6.1.3 adaptado a Spearman: regla $|z| > 2$ de Tukey sobre el vector `loo_influence_i = |_full - _{-i}|`); reporta `correlations_loo_robust.csv` y `correlations_loo_influence.csv` por celda predictor $\times$ Δbrain para certificar que la firma direccional no depende de un sujeto único.

⚠ Potencia con N=5-6 — análisis exploratorio

Una correlación Spearman con N=5 tiene 4 g.l. efectivos: para detectar $\rho=0.7$ con potencia 0.8 a $\alpha=0.05$ se requieren N 12. Las correlaciones de esta sub-sección se reportan como **exploratorias y descriptivas**, no confirmatorias. Los efectos deben replicarse al integrar el subconjunto 2 (N 6) o, idealmente, ampliando con datos de los participantes de replicación P1R–P5R (N=4 adicionales según README de ds006072).

i Mapping ítem→subescala MEQ30 — verificado contra Barrett 2015

El mapping de los 30 ítems `meq_*` a las 4 sub-escalas (Mystical 15 / Positive Mood 6 / Transcendence Time-Space 6 / Ineffability 3) está fijado por la validación factorial confirmatoria de Barrett, Johnson & Griffiths (2015) Fig. 1 (CFI=0.906, factor loadings 0.61–0.97). El cruce con las columnas REDCap del CSV de Siegel 2024 se hace por correspondencia semántica entre el texto del ítem (Fig. 1) y el sufijo de cada columna (e.g. `meq_lust` = “loss of usual sense of **time**” → Time-Space; `meq_innerworld` = “oneness in **inner world**” → Mystical). El stopifnot inline confirma cardinalidad 30 sin solape. **MEQ_total** (suma de los 30 ítems, convención Barrett 2015 p. 5) es **inequívoco** y el predictor primario del reporte; las 4 sub-escalas son complementarias y se reportan en el heatmap.

```
#####
# FASE 6 - Análisis estadístico (ds006072): PSIL vs MTP
# 6.6 - Correlación cerebro × experiencia (MEQ30, EBI × Δbrain)
#####
# Variables procedentes de CONNECTOMES[[ds]], STATS_13_1[[ds]],
# ADVANCED_METRICS[[ds]]. Solo ejecuta para ds006072 (ds003059 no tiene
# archivos MEQ30/EBI).
#
# Refs:
#   - Barrett, Johnson & Griffiths 2015, J Psychopharmacol 29:1182 - MEQ30
#   - Griffiths et al. 2006, Psychopharmacology - MEQ + psilocybin
#   - MacLean et al. 2011, J Psychopharmacol - MEQ openness
#   - Roseman et al. 2019, J Psychopharmacol - EBI
#   - Carhart-Harris 2014, Front Hum Neurosci - DMN/ego/alpha
#   - Müller et al. 2018, Lord et al. 2019, NeuroImage - global integration
#
# Escalas y scoring (ronda 1, verificado contra PDFs de validación):
#   - MEQ items: 6-point Likert 0-5 (Barrett 2015 p. 5: 0=none, 5=extreme).
#     MEQ_total = suma de los 30 ítems (Barrett 2015 convention, lineage
#     Johns Hopkins; Siegel 2024 sigue esta convención) [0, 150].
#     Sub-escalas: Mystical [0,75], Positive_Mood [0,30], Time_Space [0,30],
```

```

# Ineffability [0,15].
# - EBI items: VAS 0-100 con incrementos de 1 (Roseman 2019 p. 3).
# EBI_total = media de los 6 ítems (Roseman 2019 p. 5: "the average of
# the six items was used as the total EBI score") [0, 100].
# Roseman valida con la media, no la suma - escala canónica del campo.
#
# Reproducibilidad: bootstrap CI (R=10 000) y permutación p_perm (R=10 000)
# se siembran con GLOBAL_SEED (definido en header de §2.1). Guard defensivo
# del header del chunk cubre ejecución aislada.
#=====

library(boot)
library(ggrepel)
library(patchwork)

if (!exists("GLOBAL_SEED")) GLOBAL_SEED <- 2025L # guard ejecución aislada

# Path del CSV conductual raw (vive en el dataset OpenNeuro, no en
# derivatives/). Si tu copia local del dataset está en otra unidad, edita
# esta línea.
BEHAV_CSV_DS006072 <- "F:/JIVillL/openneuro/ds006072/BehavioralAssessments/Psilocybin-NonIder

for (ds in ACTIVE_DATASETS) {
  if (ds != "ds006072") {
    cat(sprintf("[%s] §6.6 omitida: no hay datos MEQ30/EBI para este dataset.\n", ds))
    next
  }
  cd <- CONNECTOMES[[ds]]
  s1 <- STATS_13_1[[ds]]
  am <- ADVANCED_METRICS[[ds]]

  common_subjects <- cd$common_subjects
  global_subj <- s1$global_subj
  # H1 ronda 1 (§3): el alias engañoso `sampen_results` <- am$synch_summary`
  # se eliminó; lo que realmente había era la synch entropy intra-red de
  # §3.1.
  # H2 ronda 1 (§3): la modularity_signed Q* se retiró de §3 - sin
  # `delta_Q_signed`.
  synch_results <- am$synch_summary # per-subject synch entropy; NULL si sin timeseries

  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_dir <- file.path(RESULTS_DS, "stats/statistical_results")

```

```

plot_dir  <- file.path(RESULTS_DS, "figures/stats")

if (!file.exists(BEHAV_CSV_DS006072)) {

  warning(" CSV conductual no encontrado en ", BEHAV_CSV_DS006072,
    " - $6.6 omitida hasta que el archivo esté disponible.")

} else {

  out_dir_be  <- file.path(out_dir, "brain_experience")
  plot_dir_be <- file.path(plot_dir, "brain_experience")
  dir.create(out_dir_be, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_dir_be, showWarnings = FALSE, recursive = TRUE)

  # -----
  # 1. Lectura del CSV conductual + emparejamiento dose  drug
  # -----
  behav <- read.csv(BEHAV_CSV_DS006072, stringsAsFactors = FALSE,
    na.strings = c("", "NA", "n/a", " "),
    check.names = FALSE)
  # La columna 2 (sin nombre) contiene el patient_id "P1".. "P7"
  names(behav)[2] <- "patient_id"
  behav <- behav %>%
    mutate(subject = paste0("sub-", patient_id))

  # Eventos de dosis: dose_1_arm_X y dose_2_arm_X (X varía por arm REDCap)
  dose_rows <- behav %>%
    filter(grepl("^dose_[12]_arm_", redcap_event_name)) %>%
    mutate(dose_num = as.integer(sub("^dose_(\\d+)_.*", "\\1",
      redcap_event_name)))

  # Mapear dose → drug usando participants.tsv (cargado en $6.1 vía
  # PARTICIPANTS_TSV_DS006072; recargar si la variable `participants` no
  # está en el entorno actual - p.ej. ejecución aislada de $6.6).
  if (!exists("participants")) {
    participants <- read.delim(PARTICIPANTS_TSV_DS006072,
      stringsAsFactors = FALSE)
  }
  dose_rows <- dose_rows %>%
    left_join(participants %>%
      transmute(subject = participant_id, drug1, drug2),
      by = "subject") %>%

```

```

mutate(condition = if_else(dose_num == 1L, drug1, drug2)) %>%
filter(condition %in% c("PSIL", "MTP"))

# -----
# 2. MEQ30 ítems + sub-escalas (Barrett, Johnson & Griffiths 2015 Fig. 1)
# -----
# El mapping ítem→sub-escala está fijado por la validación factorial
# confirmatoria de Barrett 2015 Fig. 1 (CFI=0.906, factor loadings
# 0.61-0.97). Cruce columna REDCap → texto del ítem por correspondencia
# semántica del sufijo (e.g. meq_lust = "Loss of Usual Sense of Time" →
# Time-Space; meq_innerworld = "Oneness in inner world" → Mystical).
# El stopifnot confirma cardinalidad 30 sin solape.
MEQ_ITEMS <- c(
  "meq_lust",          "meq_amaze",          "meq_cantdescribe",
  "meq_gaininsightknow", "meq_infinity",          "meq_unity",
  "meq_luss",          "meq_tender",          "meq_ultreal",
  "meq_cantdojustice", "meq_luswyw",          "meq_peace",
  "meq_outsidetime",   "meq_freefromself",    "meq_spiritheight",
  "meq_pure",          "meq_ecstasy",          "meq_allone",
  "meq_nospacebounds", "meq_innerworld",      "meq_reverence",
  "meq_timeless",      "meq_convultreal",      "meq_holy",
  "meq_awarelife",     "meq_fusion",           "meq_awe",
  "meq_unityultreal",  "meq_cantcomm",         "meq_joy"
)
stopifnot("Esperan 30 items MEQ" = length(MEQ_ITEMS) == 30L)

# H1 ronda 1: mapping verificado contra Barrett 2015 Fig. 1. Versión
# anterior tenía un swap quasi-simétrico de 4 ítems entre Mystical y
# Time_Space (meq_luswyw + meq_nospacebounds estaban en Mystical pero
# son "loss usual sense of where you were" + "no space boundaries" =
# Time_Space; meq_innerworld + meq_pure estaban en Time_Space pero son
# "oneness inner world" + "pure being/pure awareness" = Mystical).
# El stopifnot inferior pasaba con ambos mappings porque conserva
# cardinalidad - el bug era invisible al check defensivo.
MEQ_SUBSCALES <- list(
  Mystical      = c("meq_freefromself", "meq_pure", "meq_innerworld",
                    "meq_fusion", "meq_unityultreal", "meq_infinity",
                    "meq_unity", "meq_allone", "meq_awarelife",
                    "meq_gaininsightknow", "meq_ultreal",
                    "meq_convultreal", "meq_spiritheight",
                    "meq_reverence", "meq_holy"),
  Positive_Mood = c("meq_amaze", "meq_tender", "meq_peace",

```



```

      "meq_ecstasy", "meq_awe", "meq_joy"),
Time_Space    = c("meq_lust", "meq_luss", "meq_luswyw",
      "meq_outsidetime", "meq_nospacebounds",
      "meq_timeless"),
Ineffability  = c("meq_cantdescribe", "meq_cantdojustice",
      "meq_cantcomm")
)
all_in_subscales <- unlist(MEQ_SUBSCALES, use.names = FALSE)
stopifnot("Subscales MEQ deben cubrir 30 items únicos" =
  length(all_in_subscales) == 30L &&
  setequal(all_in_subscales, MEQ_ITEMS))

EBI_ITEMS <- c("emobrkinv_edf", "emobrkinv_respct", "emobrkinv_expcem",
  "emobrkinv_emobrkr", "emobrkinv_closure",
  "emobrkinv_release")

# Comprobar columnas presentes
miss_meq <- setdiff(MEQ_ITEMS, names(dose_rows))
miss_ebi <- setdiff(EBI_ITEMS, names(dose_rows))
if (length(miss_meq) > 0)
  warning(" MEQ items ausentes en CSV: ", paste(miss_meq, collapse = ", "))
if (length(miss_ebi) > 0)
  warning(" EBI items ausentes en CSV: ", paste(miss_ebi, collapse = ", "))
MEQ_PRESENT <- intersect(MEQ_ITEMS, names(dose_rows))
EBI_PRESENT <- intersect(EBI_ITEMS, names(dose_rows))

# Asegurar tipo numérico (la lectura por defecto puede dejarlos como char
# si hay celdas vacías codificadas como " ")
dose_rows <- dose_rows %>%
  mutate(across(all_of(c(MEQ_PRESENT, EBI_PRESENT)), as.numeric))

# H11 ronda 1: verificación empírica de NAs en items MEQ/EBI. El uso de
# `na.rm = TRUE` en rowSums/rowMeans abajo es justificable bajo MTP
# (ausencia de experiencia → 0) pero cuestionable bajo PSIL (NA puede
# significar ítem no respondido ausencia de experiencia). Con N=6
# cualquier NA bajo PSIL desproporciona. Si esta verificación reporta
# NAs, hay que revisar el CSV antes de fiarse de los scores.
paired_rows <- dose_rows %>%
  filter(condition %in% c("PSIL", "MTP"))
meq_mat_all <- as.matrix(paired_rows[, MEQ_PRESENT, drop = FALSE])
ebi_mat_all <- as.matrix(paired_rows[, EBI_PRESENT, drop = FALSE])
meq_mat_psil <- as.matrix(paired_rows[paired_rows$condition == "PSIL",

```

```

                                MEQ_PRESENT, drop = FALSE])
ebi_mat_psil <- as.matrix(paired_rows[paired_rows$condition == "PSIL",
                                EBI_PRESENT, drop = FALSE])
na_check <- data.frame(
  n_rows      = nrow(paired_rows),
  n_rows_psil = sum(paired_rows$condition == "PSIL"),
  meq_na_total = sum(is.na(meq_mat_all)),
  ebi_na_total = sum(is.na(ebi_mat_all)),
  meq_na_psil  = sum(is.na(meq_mat_psil)),
  ebi_na_psil  = sum(is.na(ebi_mat_psil))
)
cat("\n[6.6 NA-check] Items MEQ/EBI con NA por condición:\n")
print(na_check, row.names = FALSE)
if (na_check$meq_na_psil > 0 || na_check$ebi_na_psil > 0) {
  warning(" NAs en items MEQ/EBI bajo PSIL - revisar CSV; con N=6 el ",
    "`na.rm = TRUE` puede sesgar scores. Considerar exclusión ",
    "de sujetos afectados.")
}
rm(paired_rows, meq_mat_all, ebi_mat_all, meq_mat_psil, ebi_mat_psil)

# Scoring por sub-escala. Dos convenciones distintas según el instrumento:
# - MEQ: SUMA de los ítems (Barrett 2015 p. 5, lineage Johns Hopkins).
#   MEQ_total [0, 150], MEQ_Mystical [0, 75], etc.
# - EBI: MEDIA de los ítems (Roseman 2019 p. 5, validation paper).
#   EBI_total [0, 100] (escala VAS original).
# `na.rm = TRUE`: ítems sin respuesta cuentan como 0 - defendible bajo
# MTP (ausencia de experiencia); ver H11 check arriba para PSIL.
behav_scores <- dose_rows %>%
  select(subject, condition, all_of(MEQ_PRESENT), all_of(EBI_PRESENT)) %>%
  mutate(MEQ_total = rowSums(across(all_of(MEQ_PRESENT)), na.rm = TRUE))
for (sc in names(MEQ_SUBSCALES)) {
  cols <- intersect(MEQ_SUBSCALES[[sc]], names(behav_scores))
  behav_scores[[paste0("MEQ_", sc)]] <- rowSums(
    behav_scores[, cols, drop = FALSE], na.rm = TRUE)
}
# H3 ronda 1: EBI_total = media (no suma) según Roseman 2019 p. 5
# ("the average of the six items was used as the total EBI score").
# Spearman es invariante a la escala pero los valores publicados deben
# estar en [0, 100] para comparar con la literatura.
behav_scores$EBI_total <- rowMeans(
  behav_scores[, EBI_PRESENT, drop = FALSE], na.rm = TRUE)

```

```

# Quedarnos solo con los sujetos que también tienen datos cerebrales
behav_scores <- behav_scores %>%
  filter(subject %in% common_subjects) %>%
  select(subject, condition, MEQ_total, MEQ_Mystical, MEQ_Positive_Mood,
         MEQ_Time_Space, MEQ_Ineffability, EBI_total)

cat("\n[§6.6] Behavioral scores cargados:\n")
print(as.data.frame(behav_scores), row.names = FALSE)

write.csv(behav_scores,
         file.path(out_dir_be, "MEQ_EBI_subject_scores.csv"),
         row.names = FALSE)

# -----
# 3. Δ cerebro por sujeto (PSIL - MTP)
# -----
# H8 ronda 1: la variable cerebral se llama explícitamente
# `delta_synch_entropy_assoc` (no `delta_synch_entropy` a secas) para
# distinguirla de un futuro `_sensory` (control negativo §3.1) si se
# añade. Origen: `synch_entropy_assoc` z-scored (§3.1 H4-C).
delta_geff <- global_subj %>%
  select(subject, condition, global_efficiency) %>%
  pivot_wider(names_from = condition, values_from = global_efficiency) %>%
  filter(!is.na(PSIL), !is.na(MTP)) %>%
  transmute(subject, delta_global_efficiency = PSIL - MTP)

delta_synch_entropy_assoc <- if (!is.null(synch_results)) {
  synch_results %>%
    select(subject, condition, synch_entropy_assoc) %>%
    pivot_wider(names_from = condition, values_from = synch_entropy_assoc) %>%
    filter(!is.na(PSIL), !is.na(MTP)) %>%
    transmute(subject, delta_synch_entropy_assoc = PSIL - MTP)
} else {
  tibble(subject = character(0), delta_synch_entropy_assoc = numeric(0))
}

brain_deltas <- delta_geff %>%
  full_join(delta_synch_entropy_assoc, by = "subject")

write.csv(brain_deltas,
         file.path(out_dir_be, "brain_delta_subject.csv"),
         row.names = FALSE)

```

```

# -----
# 4. Pareo predictor (MEQ/EBI on-PSIL) × respuesta (Abrain)
# -----
# Predictor primario: MEQ/EBI durante la sesión PSIL (intensidad de la
# experiencia). MEQ/EBI durante MTP se reporta como sham check (debe 0).
behav_psil <- behav_scores %>%
  filter(condition == "PSIL") %>%
  select(-condition) %>%
  rename_with(~paste0(.x, "_PSIL"), -subject)
behav_mtp <- behav_scores %>%
  filter(condition == "MTP") %>%
  select(-condition) %>%
  rename_with(~paste0(.x, "_MTP"), -subject)

cor_data <- behav_psil %>%
  full_join(behav_mtp, by = "subject") %>%
  inner_join(brain_deltas, by = "subject")

cat(sprintf("\nN sujetos con MEQ/EBI + Abbrain pareados: %d\n",
  nrow(cor_data)))

# -----
# 5. Spearman + bootstrap percentil CI + p por permutación (R = 10 000)
# -----
# H2 ronda 1: `set.seed(GLOBAL_SEED)` antes de cada bootstrap y de cada
# permutación - CI y p_perm reproducibles entre ejecuciones. La semilla
# global se define en el header de §2.1 (GLOBAL_SEED <- 2025L); el
# guard del header del chunk cubre ejecución aislada.
# N1 ronda 1: añadida `p_perm` por permutación de etiquetas como
# diagnóstico paralelo a la `p` asintótica de cor.test (poco fiable
# con N=5-6). La FDR-BH se aplica sobre la `p` asintótica para
# mantener la familia comparable con la literatura; `p_perm` se reporta
# sin corregir como diagnóstico.
spearman_boot <- function(x, y, R = 10000L) {
  ok <- complete.cases(x, y)
  x <- x[ok]; y <- y[ok]
  if (length(x) < 4) {
    return(tibble(rho = NA_real_, p = NA_real_, p_perm = NA_real_,
                  lo = NA_real_, hi = NA_real_, n = length(x)))
  }
  # Si la varianza de cualquier vector es 0, Spearman no está definido
  if (sd(x) == 0 || sd(y) == 0) {

```

```

    return(tibble(rho = NA_real_, p = NA_real_, p_perm = NA_real_,
                  lo = NA_real_, hi = NA_real_, n = length(x)))
  }
  fit <- suppressWarnings(
    cor.test(x, y, method = "spearman", exact = FALSE))
  set.seed(GLOBAL_SEED)
  bs <- tryCatch({
    boot::boot(data.frame(x, y), R = R, statistic = function(d, i) {
      tryCatch(
        if (sd(d$x[i]) == 0 || sd(d$y[i]) == 0) NA_real_
        else cor(d$x[i], d$y[i], method = "spearman"),
        error = function(e) NA_real_)
      })
  }, error = function(e) NULL)
  ci <- if (!is.null(bs))
    tryCatch(boot::boot.ci(bs, type = "perc")$percent[4:5],
              error = function(e) c(NA_real_, NA_real_))
  else c(NA_real_, NA_real_)
  # p_perm: permutar la etiqueta de y (independencia bajo H0), recalculando
  # y reportar fracción de |_perm| / |_obs| (two-sided).
  set.seed(GLOBAL_SEED)
  rho_obs <- unname(fit$estimate)
  perm_rho <- tryCatch(
    replicate(R, suppressWarnings(
      cor(x, sample(y), method = "spearman"))),
    error = function(e) rep(NA_real_, R))
  p_perm <- mean(abs(perm_rho) >= abs(rho_obs), na.rm = TRUE)
  tibble(rho = rho_obs, p = fit$p.value, p_perm = p_perm,
         lo = ci[1], hi = ci[2], n = length(x))
}

predictors <- c("MEQ_total_PSIL", "MEQ_Mystical_PSIL",
               "MEQ_Positive_Mood_PSIL", "MEQ_Time_Space_PSIL",
               "MEQ_Ineffability_PSIL", "EBI_total_PSIL")
# H8 ronda 1: nombre completo `delta_synch_entropy_assoc`
brain_metrics <- c("delta_global_efficiency", "delta_synch_entropy_assoc")
brain_metrics <- intersect(brain_metrics, names(cor_data))

cor_grid <- expand_grid(predictor = predictors, brain = brain_metrics) %>%
  rowwise() %>%
  mutate(res = list(spearman_boot(cor_data[[predictor]],
                                cor_data[[brain]]))) %>%

```

```

ungroup() %>%
unnest(res) %>%
mutate(p_FDR = p.adjust(p, method = "fdr"))

cat("\n=== Spearman: predictor (MEQ/EBI on-PSIL) × Δbrain (PSIL - MTP) ===\n")
cat("Bootstrap percentil CI 95 % (R=10 000); BH-FDR sobre p asintótica;\n")
cat("p_perm = permutación de etiquetas (R=10 000), diagnóstica sin corregir.\n\n")
print(as.data.frame(
  cor_grid %>%
    arrange(p_FDR) %>%
    mutate(across(c(rho, lo, hi), ~round(.x, 3)),
      p = signif(p, 3),
      p_perm = signif(p_perm, 3),
      p_FDR = signif(p_FDR, 3))),
  row.names = FALSE)

write.csv(cor_grid,
  file.path(out_dir_be, "correlations_MEQ_EBI_brain.csv"),
  row.names = FALSE)

# -----
# 5b. LOO sobre las correlaciones Spearman
# -----
# Adaptación a del contrato r3 de §6.1.3 (regla |z|>2 de Tukey 1977).
# Para una correlación no hay vector `diff = C1 - C2`; la influencia
# natural de cada sujeto es la variación absoluta de al eliminarlo:
# loo_influence_i = |_full - _{-i}|
# Se estandariza sobre los N valores y se marca influyente con |z|>2.
# Acuerdo: el signo de se conserva entre todos / sin influyentes y la
# magnitud no cambia en más de 0.3 (umbral arbitrario coherente con la
# escala [-1,+1] de ; se mantiene como descriptor, no como gatekeeper).
loo_correlation_analysis <- function(x, y, subject,
  predictor_name, brain_name) {
  ok <- complete.cases(x, y)
  x <- x[ok]; y <- y[ok]; subject <- subject[ok]
  n <- length(x)
  if (n < 4 || sd(x) == 0 || sd(y) == 0) return(NULL)

  rho_full <- cor(x, y, method = "spearman")
  rho_loo <- vapply(seq_len(n), function(i) {
    xi <- x[-i]; yi <- y[-i]
    if (sd(xi) == 0 || sd(yi) == 0) NA_real_

```

```

    else suppressWarnings(cor(xi, yi, method = "spearman"))
  }, numeric(1))

loo_influence <- abs(rho_full - rho_loo)
loo_zscore    <- as.numeric(scale(loo_influence))
Z_THRESH     <- 2
influential   <- !is.na(loo_zscore) & abs(loo_zscore) > Z_THRESH

if (any(influential, na.rm = TRUE)) {
  keep <- !influential
  if (sum(keep) >= 4 && sd(x[keep]) > 0 && sd(y[keep]) > 0) {
    rho_no_infl <- suppressWarnings(
      cor(x[keep], y[keep], method = "spearman"))
  } else {
    rho_no_infl <- NA_real_
  }
} else {
  rho_no_infl <- rho_full
}

agreement <- if (is.na(rho_no_infl)) NA
             else (sign(rho_full) == sign(rho_no_infl)) &&
                (abs(abs(rho_full) - abs(rho_no_infl)) < 0.3)

infl_tbl <- tibble(
  predictor    = predictor_name,
  brain        = brain_name,
  subject      = subject,
  x            = x,
  y            = y,
  rho_loo      = rho_loo,
  loo_influence = loo_influence,
  loo_zscore    = loo_zscore,
  influential   = influential
)
robust_tbl <- tibble(
  predictor    = predictor_name,
  brain        = brain_name,
  n            = n,
  rho_full     = rho_full,
  rho_no_infl  = rho_no_infl,
  n_influential = sum(influential, na.rm = TRUE),

```

```

      agreement      = agreement,
      zscore_thresh = Z_THRESH
    )
    list(robust = robust_tbl, influence = infl_tbl)
  }

loo_correlations <- map(seq_len(nrow(cor_grid)), function(k) {
  row <- cor_grid[k, ]
  loo_correlation_analysis(
    x      = cor_data[[row$predictor]],
    y      = cor_data[[row$brain]],
    subject = cor_data$subject,
    predictor_name = row$predictor,
    brain_name   = row$brain
  )
})
loo_correlations <- loo_correlations[
  !vapply(loo_correlations, is.null, logical(1))]
robust_corr <- map_dfr(loo_correlations, ~.x$robust)
infl_corr   <- map_dfr(loo_correlations, ~.x$influence)

write.csv(robust_corr,
  file.path(out_dir_be, "correlations_loo_robust.csv"),
  row.names = FALSE)
write.csv(infl_corr,
  file.path(out_dir_be, "correlations_loo_influence.csv"),
  row.names = FALSE)

cat("\n=== L00 sobre correlaciones Spearman (|z|>2 sobre loo_influence) ===\n")
print(as.data.frame(
  robust_corr %>%
    arrange(desc(abs(rho_full))) %>%
    mutate(across(c(rho_full, rho_no_infl), ~round(.x, 3))),
  row.names = FALSE)

# Sham check: MTP debe puntuar 0 en MEQ y EBI. H9 ronda 1: ampliado a
# los 6 predictores (3 sub-escalas MEQ adicionales) para que la validación
# interna cubra toda la familia que entra al FDR.
sham_check <- behav_scores %>%
  group_by(condition) %>%
  summarise(across(c(MEQ_total, MEQ_Mystical, MEQ_Positive_Mood,
    MEQ_Time_Space, MEQ_Ineffability, EBI_total),

```



```

        list(mean = mean, sd = sd),
        .names = "{.col}_{.fn}"),
        .groups = "drop")
cat("\n[Sham check] MEQ/EBI por condición (MTP debería 0):\n")
print(as.data.frame(sham_check), row.names = FALSE)

write.csv(sham_check,
          file.path(out_dir_be, "sham_check_MEQ_EBI.csv"),
          row.names = FALSE)

# -----
# 6. Visualizaciones
# -----
# H6 ronda 1: eliminado `geom_smooth(method="lm")` - la línea de Pearson
# contradice visualmente el Spearman reportado en el annotate, y con
# N=5-6 puntos los smoothers introducen más ruido visual que señal. Los
# puntos + label numérico hablan por sí solos.
scatter_be <- function(df, x_var, y_var) {
  cinfo <- cor_grid %>% filter(predictor == x_var, brain == y_var)
  rho_lab <- if (nrow(cinfo) > 0 && !is.na(cinfo$rho))
    sprintf("=%.2f [%.2f, %.2f]\np=%.3f, p_perm=%.3f, n=%d",
            cinfo$rho, cinfo$lo, cinfo$hi, cinfo$p, cinfo$p_perm, cinfo$n)
  else " = NA"

  # Posición de la etiqueta : por defecto en la esquina superior izquierda;
  # para delta_synch_entropy_assoc se mueve a la superior derecha porque
  # sub-P3 cae cerca de la esquina superior izquierda y la etiqueta se
  # solapaba con el punto.
  if (y_var == "delta_synch_entropy_assoc") {
    label_x <- Inf; label_hjust <- 1.05
  } else {
    label_x <- -Inf; label_hjust <- -0.05
  }

  p <- ggplot(df, aes(x = .data[[x_var]], y = .data[[y_var]])) +
    geom_point(size = 3, alpha = 0.85, color = "#E74C3C", na.rm = TRUE) +
    ggrepel::geom_text_repel(aes(label = subject), size = 3,
                            color = "grey30", na.rm = TRUE) +
    annotate("text", x = label_x, y = Inf, label = rho_lab,
             hjust = label_hjust, vjust = 1.5, size = 3.0, color = "grey30") +
    geom_hline(yintercept = 0, linetype = "dashed", color = "grey60") +
    theme_minimal(base_size = 10) +

```

```

      labs(x = sub("_PSIL$", "", x_var), y = y_var)
    p
  }

panel_plots <- list()
for (px in predictors) {
  for (br in brain_metrics) {
    panel_plots[[paste(px, br, sep = "__")]] <-
      scatter_be(cor_data, px, br)
  }
}

fig_panel <- patchwork::wrap_plots(panel_plots,
                                   ncol = length(brain_metrics)) +
  plot_annotation(
    title = "[GRUPO] Correlación cerebro × experiencia subjetiva",
    subtitle = "Spearman con bootstrap CI 95 % y p_perm (R=10 000)",
    caption = "Filas: MEQ_total, Mystical, Positive Mood, Time/Space, Ineffability, EBI_t",
    theme = theme(plot.title = element_text(size = 12, face = "bold"),
                  plot.subtitle = element_text(size = 9),
                  plot.caption = element_text(size = 8))
  )

ggsave(file.path(plot_dir_be, "panel_MEQ_EBI_x_delta_brain.png"),
        fig_panel,
        width = 4 * length(brain_metrics),
        height = 3 * length(predictors),
        dpi = 300, limitsize = FALSE)

# Heatmap resumen de . H10 ronda 1: las celdas con =NA quedan vacías
# (en lugar de imprimir "0.00" sobre fondo gris claro como hacía antes).
hm_df <- cor_grid %>%
  mutate(predictor_short = sub("_PSIL$", "", predictor),
         sig = case_when(
           !is.na(p_FDR) & p_FDR < 0.05 ~ "*",
           !is.na(p) & p < 0.05 ~ ".",
           TRUE ~ ""))
p_hm <- ggplot(hm_df, aes(x = brain, y = predictor_short, fill = rho)) +
  geom_tile(color = "white") +
  geom_text(aes(label = ifelse(is.na(rho), "",
                              sprintf("%.2f%s", rho, sig))),
            size = 3.5, na.rm = TRUE) +

```

```

scale_fill_gradient2(low = "#2980B9", mid = "white", high = "#C0392B",
                     midpoint = 0, limits = c(-1, 1), name = " ",
                     na.value = "grey90") +
theme_minimal(base_size = 11) +
theme(axis.text.x = element_text(angle = 25, hjust = 1)) +
labs(title = "[GRUPO] Heatmap - MEQ/EBI × Δbrain",
     subtitle = "* p_FDR<0.05 ; · p<0.05 (sin corregir); n=5-6",
     x = "", y = "")
ggsave(file.path(plot_dir_be, "heatmap_correlations.png"),
       p_hm, width = 7, height = 5, dpi = 300)

cat(sprintf("\n[§6.6] Outputs:\n Stats:  %s\n Figures: %s\n",
          out_dir_be, plot_dir_be))

} # cierre del else file.exists(BEHAV_CSV_DS006072)
} # end for ds

```

7 7 Análisis de sensibilidad y robustez

Las cuatro sub-secciones que siguen evalúan la robustez del pool topología cerrado en §6.1 — **13 tests** (8 globales + 4 nodales no-redundantes + `degree_entropy`), el mismo pool declarado en el callout-important §6.1.1. Cada sub-sección toma un eje distinto (detectabilidad a priori, permutación pareada, supuestos de normalidad, IC bootstrap del efecto) y los cuatro se construyen sobre dos helpers comunes (`pool_13_long` y `get_paired_vectors`) que se declaran a continuación.

```

#=====
# §7 - Helpers comunes
#=====
# Cierre r2 §7 (H1-A unificación pool, H11-A degree_entropy incluida):
#   pool_13_long(ds, st, am)
#     devuelve un tibble largo (subject, condition, metric, value, plano)
#     con las 13 métricas del pool §6.1.1: 8 globales + 4 nodales
#     no-redundantes + 1 avanzada (degree_entropy). `plano` {"global",
#     "nodal", "advanced"} marca la procedencia de cada fila para facetado
#     posterior y trazabilidad.
#   get_paired_vectors(long, metric_name, c1, c2)
#     devuelve list(g1, g2, n, subs) con los dos vectores pareados (mismo
#     orden de sujeto) para una métrica concreta; NULL si N<3. Centraliza
#     el patrón intersect/arrange/pull que usan §7.2, §7.3 y §7.4.
#

```

```
# Cierre r2 §7 (H2-A reproducibilidad): GLOBAL_SEED guard defensivo para
# ejecución aislada. La cascada H2-D §6.2 ya definió GLOBAL_SEED=2025L en
# el header de §2.1; este guard cubre el caso de re-ejecutar §7 sin haber
# cargado §2.1 primero.
```

```
#=====
```

```
library(tidyverse)
if (!exists("GLOBAL_SEED")) GLOBAL_SEED <- 2025L

pool_13_long <- function(ds, st, am) {
  long_global <- st$global_subj %>%
    select(subject, condition, all_of(st$numeric_metrics_global)) %>%
    pivot_longer(-c(subject, condition),
                 names_to = "metric", values_to = "value") %>%
    mutate(plano = "global")

  long_nodal <- st$nodal_subj %>%
    select(subject, condition, all_of(st$numeric_metrics_nodal)) %>%
    pivot_longer(-c(subject, condition),
                 names_to = "metric", values_to = "value") %>%
    mutate(plano = "nodal")

  long_advanced <- if (!is.null(am$degree_entropy_results)) {
    am$degree_entropy_results %>%
      transmute(subject, condition,
                metric = "degree_entropy",
                value = degree_entropy,
                plano = "advanced")
  } else {
    tibble(subject = character(), condition = character(),
           metric = character(), value = numeric(), plano = character())
  }

  bind_rows(long_global, long_nodal, long_advanced)
}

get_paired_vectors <- function(long, metric_name, c1, c2) {
  d <- long %>% filter(metric == metric_name)
  subs <- intersect(
    d %>% filter(condition == c1) %>% pull(subject),
    d %>% filter(condition == c2) %>% pull(subject)
  )
}
```

```

if (length(subs) < 3) return(NULL)
g1 <- d %>% filter(condition == c1, subject %in% subs) %>%
  arrange(subject) %>% pull(value)
g2 <- d %>% filter(condition == c2, subject %in% subs) %>%
  arrange(subject) %>% pull(value)
list(g1 = g1, g2 = g2, n = length(subs), subs = sort(subs))
}

```

7.1 7.1 Detectabilidad del diseño (sensitivity analysis, no post-hoc power)

Con $N=5-6$ sujetos pareados, la mayoría de los tests de §6.1 estarán *infrapotenciados* para detectar efectos pequeños o medianos. Esta sub-sección **no calcula potencia post-hoc** (uso del d observado para evaluar la potencia del test que lo detectó), una práctica matemáticamente inválida y desaconsejada (Hoenig & Heisey, 2001, *Am Stat* 55:19; Lenth, 2007, *Am Stat* 61:38: “*Observed power is a one-to-one function of the p-value*”). En su lugar plantea una **pregunta a priori sobre el diseño**: ¿qué tamaño de efecto d_z puede detectar este crossover con N pareados al 80 % de potencia y $\alpha=0.05$? La respuesta es independiente de los datos.

Marco metodológico:

- $d_z = \text{mean}(\Delta)/\text{sd}(\Delta)$ es el effect size pareado correcto para `pwr.t.test(type="paired")` (Lakens, 2013, *Front Psychol* 4:863). d_{av} se reporta separadamente para comparabilidad con la literatura.
- $d_{\min,80}$ = el menor d_z tal que $1 - \beta \geq 0.80$ con N pareados, $\alpha=0.05$ (Cohen, 1988). Para $N=5$ se obtiene $d_{\min,80} \approx 1.6$, para $N=6 \approx 1.4$, para $N=10 \approx 1.0$.
- La **corrección por atenuación de fiabilidad** basada en ICC test-retest se ha retirado del pipeline (decisión S0, 2026-05-23): con $N=5-6$ e intra-sesión el ICC no es estimable de forma útil ni mide la fiabilidad inter-sesión relevante. La robustez se cierra en §7.2–§7.4 con permutación intra-sujeto y bootstrap del IC del efecto, sin pasar por una fórmula de corrección que asume varianza de medida bien estimada.

Clasificación: cada métrica del pool §6.1.1 (13 tests) se etiqueta con cuatro casos cruzando p_FDR (sig/n.s.) $\times$ $|dz|$ ($< d_{\min,80}$):

Caso	Lectura
sig +	dz
sig +	dz
n.s. +	dz
n.s. +	dz

```

#=====
# §7.1 - Detectabilidad del diseño (análisis de sensibilidad a priori)
#=====
# Pregunta a priori: ¿qué tamaño de efecto puede detectar el diseño?
# Refs: Hoenig & Heisey 2001; Lenth 2007; Cohen 1988; Lakens 2013.
# Marco S0 (2026-05-23): sin corrección por atenuación basada en ICC; la
# robustez se cierra en §7.2 (permutación), §7.3 (supuestos) y §7.4
# (bootstrap del IC del d).
# Cierre r2 §7 (H1-A unificación pool): la clasificación cubre las 13
# métricas del pool §6.1.1 - 8 globales + 4 nodales + 1 avanzada
# (degree_entropy). El reparto se conserva en la columna `plano` para
# facetado y trazabilidad. H4-A renombra `cohen_d` → `d_z` en `power_curve`
# para alinear con el contrato §6.1 r2.
#=====

library(tidyverse)
library(pwr)

for (ds in ACTIVE_DATASETS) {
  cd <- CONNECTOMES[[ds]]
  s1 <- STATS_13_1[[ds]]
  C1 <- s1$C1
  C2 <- s1$C2

  PIPELINE_NAME <- cd$pipeline
  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_dir <- file.path(RESULTS_DS, "stats/statistical_results/robust")
  plot_dir <- file.path(RESULTS_DS, "figures/stats/robust")
  dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)

  results_global <- s1$results_global
  results_nodal <- s1$results_nodal
  results_advanced <- s1$results_advanced
  # p_FDR is added by s6-1-1-fdr-tables; guard for standalone execution
  if (!"p_FDR" %in% names(results_global))
    results_global$p_FDR <- p.adjust(results_global$p_value, "fdr")
  if (!"p_FDR" %in% names(results_nodal))
    results_nodal$p_FDR <- p.adjust(results_nodal$p_value, "fdr")
  if (nrow(results_advanced) > 0 && !"p_FDR" %in% names(results_advanced))
    results_advanced$p_FDR <- p.adjust(results_advanced$p_value, "fdr")
  n_subj <- length(cd$common_subjects)

```

```

if (n_subj < 3) {
  cat(sprintf("[%s] §7.1 omitida: N < 3 pareados.\n", ds)); next
}
message(sprintf("[%s · %s] n sujetos pareados para análisis de sensibilidad: %d",
  ds, PIPELINE_NAME, n_subj))

# --- Función auxiliar: potencia para un d y n dados ---
compute_power <- function(d, n, sig = 0.05) {
  if (is.na(d) || is.na(n) || n < 3) return(NA_real_)
  tryCatch(
    pwr.t.test(n = n, d = abs(d), sig.level = sig, type = "paired",
      alternative = "two.sided")$power,
    error = \(e) NA_real_
  )
}

# --- Función auxiliar: n necesario para 80% potencia dado un d ---
compute_n_required <- function(d, power_target = 0.80, sig = 0.05) {
  if (is.na(d) || abs(d) < 0.01) return(NA_real_)
  tryCatch(
    ceiling(pwr.t.test(d = abs(d), power = power_target, sig.level = sig,
      type = "paired", alternative = "two.sided")$n),
    error = \(e) NA_real_
  )
}

# ---- 1. Curva de sensibilidad: ¿qué d mínimo detecta el diseño? ----
# H4-A: la columna de d en `power_curve` se llama `d_z` (no `cohen_d`)
# para alinear con el contrato §6.1 r2 (Lakens 2013): la curva es de
# potencia de pwr.t.test(paired) que toma d_z como entrada.
d_seq <- seq(0.1, 2.0, by = 0.05)
power_curve <- tibble(
  d_z = d_seq,
  power = map_dbl(d_seq, compute_power, n = n_subj)
)
d_min_80 <- power_curve %>% filter(power >= 0.80) %>% slice(1) %>% pull(d_z)

cat(sprintf("\n=== [%s · %s] Análisis de sensibilidad | N = %d pareados | sig = 0.05 ===\n",
  ds, PIPELINE_NAME, n_subj))
cat(sprintf("d mínimo detectable con 80%% potencia: %.2f\n", d_min_80))
cat("Interpretación: efectos con |d| < ", round(d_min_80, 2),
  " podrían ser reales pero indetectables con esta muestra.\n")

```

```

# Tabla de referencia: potencia para efectos estándar
ref_effects <- tibble(
  categoria = c("Pequeño", "Mediano", "Grande", "Muy grande"),
  d_ref     = c(0.2, 0.5, 0.8, 1.2),
  potencia  = map_dbl(d_ref, compute_power, n = n_subj),
  n_requerido = map_dbl(d_ref, compute_n_required)
)
cat("\nPotencia del diseño para efectos estándar (Cohen, 1988):\n")
print(as.data.frame(ref_effects), row.names = FALSE)

# ---- 2. Clasificación de métricas por detectabilidad ----
# NO es potencia post-hoc. Es una clasificación de si el efecto observado
# cae dentro o fuera del rango de sensibilidad del diseño.
# Marco S0 (2026-05-23): sin rama de corrección por atenuación ICC.

classify_sensitivity <- function(df, label) {
  # Effect sizes tras §6.1 r1 (H2-C): la tabla `results_global`/`nodal`
  # ahora trae `d_z` y `d_av` como columnas separadas (antes había
  # `cohen_d` que era duplicado de `dz`). Aquí se usa d_z (test-coupled
  # con pwr.t.test); d_av se reporta en paralelo para comparabilidad.
  df %>%
    filter(!is.na(d_z)) %>%
    mutate(
      abs_dz      = abs(d_z),
      detectable  = abs_dz >= d_min_80,
      n_required  = map_dbl(d_z, compute_n_required),
      interpretacion = case_when(
        p_FDR < 0.05 & detectable ~
          "Significativo + detectable (resultado fiable)",
        p_FDR < 0.05 & !detectable ~
          "Significativo pero |d| < umbral (posible sobreestimación)",
        p_FDR >= 0.05 & !detectable ~
          "No significativo + infradetectable (no descarta efecto real)",
        p_FDR >= 0.05 & detectable ~
          "No significativo pero detectable (evidencia contra el efecto)",
        TRUE ~ "Sin clasificar"
      ),
      tipo = label
    ) %>%
    select(tipo, metric, d_z, d_av, abs_dz, p_FDR, detectable,
           n_required, interpretacion) %>%
    arrange(desc(abs_dz))

```



```

}

# H1-A: las tres tablas del pool §6.1.1 (globales, nodales, avanzada)
# se pasan por la misma clasificación. `sensitivity_all` cubre las 13
# métricas; el plano queda registrado en `tipo` para facetado.
sensitivity_global <- classify_sensitivity(results_global, "global")
sensitivity_nodal <- classify_sensitivity(results_nodal, "nodal")
sensitivity_advanced <- if (nrow(results_advanced) > 0)
  classify_sensitivity(results_advanced, "advanced")
else tibble()
sensitivity_all <- bind_rows(sensitivity_global, sensitivity_nodal,
                             sensitivity_advanced)

write.csv(sensitivity_all, file.path(out_dir, "sensitivity_analysis.csv"),
          row.names = FALSE)

cat(sprintf("\n=== [%s · %s] Clasificación de métricas por detectabilidad ===\n",
          ds, PIPELINE_NAME))
cat(sprintf("Umbral de detección nominal: |dz| %.2f (80%% potencia, N=%d)\n\n",
          d_min_80, n_subj))

cat("--- Métricas globales ---\n")
print(sensitivity_global %>%
      select(metric, d_z, d_av, p_FDR, detectable,
             n_required, interpretacion) %>%
      as.data.frame(), row.names = FALSE)

cat("\n--- Métricas nodales ---\n")
print(sensitivity_nodal %>%
      select(metric, d_z, d_av, p_FDR, detectable, n_required, interpretacion) %>%
      as.data.frame(), row.names = FALSE)

if (nrow(sensitivity_advanced) > 0) {
  cat("\n--- Métricas avanzadas (degree_entropy) ---\n")
  print(sensitivity_advanced %>%
        select(metric, d_z, d_av, p_FDR, detectable,
               n_required, interpretacion) %>%
        as.data.frame(), row.names = FALSE)
}

# Resumen
n_infra <- sum(!sensitivity_all$detectable & sensitivity_all$p_FDR >= 0.05,

```

```

        na.rm = TRUE)
if (n_infra > 0) {
  cat(sprintf("\nLimitación: %d métricas no significativas tienen |d| < %.2f.\n",
            n_infra, d_min_80))
  cat("No se puede descartar que exista un efecto real de tamaño pequeño-mediano.\n")
  cat("Para futuros estudios, se necesitarían:\n")
  cat(sprintf("  - n = %d para detectar d = 0.5 (mediano)\n",
            compute_n_required(0.5)))
  cat(sprintf("  - n = %d para detectar d = 0.3 (pequeño-mediano)\n",
            compute_n_required(0.3)))
}

# ---- 3. Figuras ----
# H9-B: PIPELINE_NAME se incluye en los títulos para trazabilidad del
# dataset×pipeline al que pertenece cada figura.

# Curva de sensibilidad (válida: NO usa datos observados)
p_curve <- ggplot(power_curve, aes(x = d_z, y = power)) +
  geom_line(color = "#2E86AB", linewidth = 1) +
  geom_hline(yintercept = 0.80, linetype = "dashed", color = "grey40") +
  geom_vline(xintercept = d_min_80, linetype = "dotted", color = "#E74C3C") +
  annotate("text", x = d_min_80 + 0.05, y = 0.15,
    label = paste0("dz = ", round(d_min_80, 2)), hjust = 0, size = 4) +
  annotate("text", x = 1.8, y = 0.82, label = "80%", size = 3.5, color = "grey40") +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  theme_minimal(base_size = 11) +
  theme(plot.title = element_text(size = 11, face = "bold"),
    plot.subtitle = element_text(size = 9)) +
  labs(
    title = sprintf("[%s · Sensibilidad] N = %d pareados | = 0.05",
      PIPELINE_NAME, n_subj),
    subtitle = paste0("d mínimo detectable con 80% potencia: ",
      round(d_min_80, 2)),
    x = "d_z (Lakens 2013; pwr.t.test paired)",
    y = "Potencia (1 - )"
  )

ggsave(file.path(plot_dir, "power_curve.png"), p_curve,
  width = 7, height = 5, dpi = 300)

# Dotplot: |d observado| vs umbral de detección
# H8-A: cubre el pool 13 facetado por plano (global/nodal/advanced); el

```

```

# reorder se hace en la columna `metric_ord` para preservar la jerarquía
# |dz| dentro de cada facet (Tidy approach: factor en cada panel).
p_detect <- sensitivity_all %>%
  arrange(tipo, abs_dz) %>%
  mutate(metric_ord = factor(metric, levels = unique(metric))) %>%
  ggplot(aes(x = abs_dz, y = metric_ord, color = detectable)) +
  geom_vline(xintercept = d_min_80, linetype = "dashed", color = "grey40") +
  geom_point(size = 3) +
  scale_color_manual(
    values = c("TRUE" = "#28B463", "FALSE" = "#E74C3C"),
    labels = c("TRUE" = paste0("|dz| > ", round(d_min_80, 2), " (detectable)"),
               "FALSE" = paste0("|dz| < ", round(d_min_80, 2), " (infradetectable)"))
  ) +
  facet_grid(tipo ~ ., scales = "free_y", space = "free_y") +
  theme_minimal(base_size = 12) +
  labs(
    title = sprintf("[%s · %s · Sensibilidad] |dz| vs umbral | N = %d",
                     ds, PIPELINE_NAME, n_subj),
    subtitle = sprintf("Pool §6.1.1 (13 métricas) · efectos a la derecha de dz = %.2f son
                        round(d_min_80, 2)),
    x = "|dz| observado",
    y = "", color = ""
  )

ggsave(file.path(plot_dir, "sensitivity_by_metric.png"), p_detect,
        width = 7.5, height = 6.5, dpi = 300)

message(sprintf(
  "OK [%s · %s] Análisis de sensibilidad | N=%d | d_min (80%%) = %.2f",
  ds, PIPELINE_NAME, n_subj, d_min_80))
message(sprintf(" Detectables: %d/%d | Infradetectables: %d/%d",
  sum(sensitivity_all$detectable), nrow(sensitivity_all),
  sum(!sensitivity_all$detectable), nrow(sensitivity_all)))
} # end for ds

```

7.2 Tests de permutación pareados (sign-flip exacto)

Con N pequeño los tests paramétricos asumen normalidad de las diferencias y son sensibles a outliers; los tests de permutación son **libres de supuestos distribucionales** (Edgington & Onghena, 2007, *Randomization Tests*; Pesarin & Salmaso, 2010). Para diseño pareado within-subject la estrategia canónica es la **permutación por inversión de signo**: bajo H el

sign de la diferencia $\Delta_i = \text{PSIL}_i - \text{MTP}_i$ es intercambiable. Hay 2^N asignaciones de signos posibles, lo que limita el espacio nulo cuando N es pequeña.

⚠ Espacio de permutación finito con N pequeña

Con $N = 5$ sujetos pareados, el espacio exacto de permutaciones tiene $2^5 = 32$ outcomes posibles; con $N = 6$, $2^6 = 64$; con $N = 15$, $2^{15} = 32\,768$. **El p-valor mínimo alcanzable es por tanto $1/(2^N + 1)$ tras la corrección de Phipson & Smyth (2010, *Stat Appl Genet Mol Biol* 9:39) — $\approx 0,030$ con $N=5$, $\approx 0,015$ con $N=6$, $\approx 3 \cdot 10^{-5}$ con $N=15$.** Para ds006072 ($N=6$) la corrección BH-FDR sobre 13 tests puede no devolver ningún $p_{\text{FDR}} < 0,05$ aun con efectos reales alineados, por la granularidad del p-valor mínimo. El chunk **enumera el espacio completo de 2^N outcomes** siempre que $N \leq 16$ (cubre los dos datasets: $N = 6$ ds006072, $N = 15$ ds003059); para $N > 16$ usa Monte Carlo $R=10\,000$.

El criterio de éxito de esta sub-sección es la **concordancia direccional con §6.1** (paramétrico): si t-test pareado y permutación coinciden en **sign(Δ)** y **orden de magnitud de p**, el resultado paramétrico es robusto al supuesto de normalidad. La columna **agreement** requiere ambas condiciones; las discrepancias se reportan como flags para reanálisis. El pool testado son las **13 métricas del callout §6.1.1** (8 globales + 4 nodales no-redundantes + **degree_entropy**), no las 17 columnas brutas de **global_subj** — el helper **pool_13_long** del chunk §7.0 garantiza el alineamiento exacto con la familia FDR de §6.1.

```
#=====
# FASE 7.2 - Análisis de sensibilidad y robustez (dual-dataset)
# 7.2 - Tests de permutación pareados (sign-flip)
#=====
# Refs: Edgington & Onghena 2007; Pesarin & Salmaso 2010; Phipson & Smyth 2010
# (p mínimo con espacio de permutación finito).
#
# Cierre r2 §7 (H7-A): n_max_exact = 16L. Con N=15 (ds003059) el espacio
# exacto  $2^{15} = 32\,768$  outcomes se enumera en < 1 s. Antes el umbral 8L
# disparaba MC R=10 000 para ds003059; ahora se reserva MC sólo para N>16
# hipotéticos. Pico de memoria con N=15: ~256 KB por métrica × 13 = 3 MB.
# (N2 ronda 1: coste computacional anotado para futura referencia.)
#
# Cierre r2 §7 (H2-A): semilla = GLOBAL_SEED (2025L) como el resto del
# pipeline (cascada H2-D §6.2). El guard del chunk §7.0 cubre ejecución
# aislada.
#
# Cierre r2 §7 (H1-A + H11-A): el pool testado son las 13 métricas del
# callout §6.1.1 (no las 17 columnas crudas de global_subj). Las
# diferencias pareadas se obtienen vía `get_paired_vectors` sobre el
```

```

# tibble largo de `pool_13_long`. La familia BH-FDR de §7.2 tiene tamaño
# 13 - mismo tamaño que la familia paramétrica de §6.1; las dos `p_FDR`
# se interpretan como BH dentro de familias de igual tamaño con métodos
# distintos (paramétrico vs permutacional), que es la triangulación que
# §7.2 declara. (N1 ronda 1: documentado para el lector.)
#
# Cierre r2 §7 (H3-A): la concordancia se descompone en `sign_match`
# (signo(obs_diff) == signo(d_z)) y `p_order_match` (mismo orden de
# magnitud entre p_param y p_perm). `agreement = "acuerdo"` requiere
# ambas condiciones - el código ahora honra la prosa.
#=====

library(tidyverse)
stopifnot(exists("pool_13_long"), exists("get_paired_vectors"),
          exists("GLOBAL_SEED"))

for (ds in ACTIVE_DATASETS) {
  cd <- CONNECTOMES[[ds]]
  s1 <- STATS_13_1[[ds]]
  am <- ADVANCED_METRICS[[ds]]
  C1 <- s1$C1
  C2 <- s1$C2
  PIPELINE_NAME <- cd$pipeline

  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_dir <- file.path(RESULTS_DS, "stats/statistical_results/robust")
  plot_dir <- file.path(RESULTS_DS, "figures/stats/robust")
  dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)

  long_13 <- pool_13_long(ds, s1, am)
  metric_planos <- long_13 %>% distinct(metric, plano)

  # --- Permutación pareada (exacta o Monte Carlo según N) ---
  perm_paired_test <- function(long, metric_name, c1, c2,
                                n_perm = 10000L, seed = GLOBAL_SEED,
                                n_max_exact = 16L) {
    set.seed(seed)
    v <- get_paired_vectors(long, metric_name, c1, c2)
    if (is.null(v))
      return(tibble(metric = metric_name, n_subj = NA_integer_,
                    obs_diff = NA_real_, p_perm = NA_real_,

```

```

mode = "insufficient_n", n_perm_effective = NA_integer_))

diffs    <- v$g1 - v$g2
N        <- length(diffs)
obs_diff <- mean(diffs, na.rm = TRUE)

if (N <= n_max_exact) {
  sign_grid <- as.matrix(do.call(expand.grid,
                                replicate(N, c(-1L, 1L), simplify = FALSE)))
  null_dist <- as.numeric(sign_grid %*% diffs / N)
  mode_used <- sprintf("exact (2~%d=%d perms)", N, length(null_dist))
  n_eff     <- length(null_dist)
} else {
  null_dist <- replicate(n_perm, {
    signs <- sample(c(-1L, 1L), N, replace = TRUE)
    mean(signs * diffs, na.rm = TRUE)
  })
  mode_used <- sprintf("MC R=%d", n_perm)
  n_eff     <- n_perm
}

# Phipson & Smyth (2010): p = (b + 1) / (M + 1), b = #{|null| |obs|}.
b <- sum(abs(null_dist) >= abs(obs_diff), na.rm = TRUE)
p_perm <- (b + 1) / (n_eff + 1)

tibble(metric = metric_name, n_subj = N, obs_diff = obs_diff,
        p_perm = p_perm, mode = mode_used, n_perm_effective = n_eff)
}

# --- Ejecutar sobre las 13 métricas del pool §6.1.1 ---
message(sprintf("[%s · %s] Ejecutando tests de permutación pareados (pool 13)...",
                ds, PIPELINE_NAME))
results_perm <- map2_dfr(metric_planos$metric, metric_planos$plano,
                        function(m, p) {
                          perm_paired_test(long_13, m, C1, C2) %>%
                            mutate(plano = p)
                        }) %>%
  mutate(p_perm_FDR = p.adjust(p_perm, "fdr"))

# --- Comparar con resultados paramétricos de §6.1 ---
# Tras §6.1 r1 (H2-C), las tablas results_global/_nodal/_advanced traen
# `d_z` como effect size primario y `d_av` en paralelo. La comparación

```

```

# paramétrico permutación usa d_z porque es la variante test-coupled.
results_param <- bind_rows(
  s1$results_global %>% mutate(plano = "global"),
  s1$results_nodal %>% mutate(plano = "nodal"),
  s1$results_advanced %>% mutate(plano = "advanced")
)
if (!"p_FDR" %in% names(results_param))
  results_param$p_FDR <- p.adjust(results_param$p_value, "fdr")
results_param <- results_param %>%
  select(metric, plano, p_value, p_FDR, d_z, method)

# H3-A: concordancia direccional descompuesta en signo + magnitud de p.
# `sign_match` requiere que el signo de obs_diff (permutación) coincida
# con el signo del d_z paramétrico; `p_order_match` requiere que ambos
# p_FDR estén en el mismo orden de magnitud ( $|\Delta \log_{10}| < 1$ ). El
# `agreement = "acuerdo"` se da sólo cuando ambas condiciones se
# cumplen - alinea el código con la prosa de §7.2.
comparison <- results_perm %>%
  left_join(results_param, by = c("metric", "plano")) %>%
  mutate(
    sign_match = !is.na(d_z) & d_z != 0 &
      sign(obs_diff) == sign(d_z),
    p_order_match = !is.na(p_perm_FDR) & !is.na(p_FDR) &
      abs(log10(p_perm_FDR + 1e-10) - log10(p_FDR + 1e-10)) < 1,
    agreement = case_when(
      is.na(p_perm_FDR) | is.na(p_FDR) ~ "indeterminado",
      sign_match & p_order_match ~ "acuerdo",
      TRUE ~ "DISCREPANCIA"
    )
  ) %>%
  select(metric, plano, n_subj, mode, d_z, method,
    obs_diff, p_param = p_FDR, p_perm = p_perm_FDR,
    sign_match, p_order_match, agreement) %>%
  arrange(p_perm)

write.csv(comparison,
  file.path(out_dir, "permutation_vs_parametric.csv"),
  row.names = FALSE)

cat(sprintf("\n=== [%s · %s] Permutación vs Paramétrico (%s vs %s) ===\n",
  ds, PIPELINE_NAME, C1, C2))
n_eff_max <- max(results_perm$n_perm_effective, na.rm = TRUE)

```

```

cat(sprintf("Modo: %s | p-min alcanzable = %s\n",
           unique(na.omit(comparison$mode))[1],
           if (is.finite(n_eff_max)) sprintf("%.4g", 1 / (n_eff_max + 1)) else "N/A"))
print(as.data.frame(comparison), row.names = FALSE)

# --- Visualización: scatter p-param vs p-perm ---
p_compare_data <- comparison %>% filter(!is.na(p_param), !is.na(p_perm))
if (nrow(p_compare_data) > 0) {
  p_compare <- p_compare_data %>%
    ggplot(aes(x = -log10(p_param + 1e-10), y = -log10(p_perm + 1e-10),
               color = agreement, shape = plano, label = metric)) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey50") +
    geom_point(size = 3) +
    ggrepel::geom_text_repel(size = 3, max.overlaps = 13) +
    scale_color_manual(values = c("acuerdo" = "#28B463",
                                   "DISCREPANCIA" = "#E74C3C",
                                   "indeterminado" = "grey60")) +
    theme_minimal(base_size = 11) +
    theme(plot.title = element_text(size = 11, face = "bold"),
          plot.subtitle = element_text(size = 9)) +
    labs(
      title = sprintf("[%s] Paramétrico vs Permutación | %s vs %s",
                      PIPELINE_NAME, C1, C2),
      subtitle = "Pool §6.1.1 (13 métricas) · diagonal = acuerdo perfecto",
      x = "-log (p FDR paramétrico)",
      y = "-log (p FDR permutación)",
      color = "Concordancia",
      shape = "Plano"
    )
  ggsave(file.path(plot_dir, "permutation_vs_parametric.png"),
         p_compare, width = 7.5, height = 6, dpi = 300)
} else {
  message(sprintf("[%s · %s] §7.2 scatter omitido: sin p_perm no-NA",
                  ds, PIPELINE_NAME))
}

disc <- comparison %>% filter(agreement == "DISCREPANCIA")
n_total <- nrow(comparison)
n_agree <- sum(comparison$agreement == "acuerdo", na.rm = TRUE)
if (nrow(disc) > 0) {
  message(sprintf(" [%s · %s] %d/%d métricas con discrepancia entre métodos:",
                  ds, PIPELINE_NAME, nrow(disc), n_total))
}

```



```

        print(disc %>% select(metric, plano, d_z, obs_diff, p_param, p_perm,
                             sign_match, p_order_match))
    } else {
        message(sprintf("OK [%s · %s] Acuerdo entre paramétrico y permutación: %d/%d",
                        ds, PIPELINE_NAME, n_agree, n_total))
    }

    message(sprintf("OK [%s · %s] Permutation testing completado", ds, PIPELINE_NAME))
} # end for ds

```

7.3 7.3 Verificación de supuestos — QQ-plots y Shapiro-Wilk heredado

El t-test pareado asume normalidad de las **diferencias intra-sujeto** $\Delta_i = \text{PSIL}_i - \text{MTP}_i$. La práctica clásica de Shapiro-Wilk como **selector** entre paramétrico y no paramétrico (two-stage testing) es metodológicamente problemática: con N pequeño Shapiro-Wilk tiene baja potencia (~30 % con N=10 para detectar desviaciones moderadas; Razali & Wah, 2011, *J Stat Model Anal* 2:21), por lo que la “no detección” no garantiza normalidad — y la elección condicional infla el error de Tipo I del procedimiento combinado (Lumley et al., 2002, *Annu Rev Public Health* 23:151; Rochon et al., 2012, *BMC Med Res Methodol* 12:81). En §6.1 se reporta **shapiro_p** exclusivamente **como diagnóstico**, sin selección condicional, y se ejecutan siempre tanto t-test como Wilcoxon en paralelo.

Esta sub-sección **no recalcula** el Shapiro — hereda la columna **shapiro_p** ya presente en `STATS_13_1[[ds]]$results_global/_nodal/_advanced` para las 13 métricas del pool §6.1.1 — y añade lo que sí es propio: **QQ-plots** sobre las diferencias pareadas para inspección visual. Con N=6 (ds006072) el análisis cuantitativo de Shapiro tiene poca potencia, pero la inspección visual permite detectar outliers o asimetrías marcadas que condicionarían la interpretación; con N=15 (ds003059) Shapiro gana algo de potencia y los QQ-plots son más informativos. Si la conclusión cualitativa cambia entre t-test y Wilcoxon (cf. §6.1 print de “DIAGNÓSTICO DE NORMALIDAD”), Wilcoxon se reporta como primario para esa métrica.

```

#=====
# FASE 7.3 - Análisis de sensibilidad y robustez (dual-dataset)
# 7.3 - Verificación de supuestos (QQ-plots; Shapiro-Wilk heredado de §6.1)
#=====
# Refs: Razali & Wah 2011; Lumley et al. 2002; Rochon et al. 2012.
#
# Cierre r2 §7 (H6-B): el cálculo numérico de Shapiro se elimina aquí
# como duplicado. `STATS_13_1[[ds]]$results_*` ya contiene `shapiro_p`
# para las 13 métricas del pool §6.1.1; §7.3 sólo añade la inspección
# visual (QQ-plots) y la columna `note` con la justificación de baja
# potencia. El CSV de archivo se renombra `QQ_diagnostico.csv` para

```

```

# reflejar el cambio de alcance (no duplica más a Shapiro de §6.1).
#=====
library(tidyverse)
stopifnot(exists("pool_13_long"), exists("get_paired_vectors"))

for (ds in ACTIVE_DATASETS) {
  cd <- CONNECTOMES[[ds]]
  s1 <- STATS_13_1[[ds]]
  am <- ADVANCED_METRICS[[ds]]
  C1 <- s1$C1
  C2 <- s1$C2
  PIPELINE_NAME <- cd$pipeline

  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_app <- file.path(RESULTS_DS, "stats/statistical_results/robust")
  plot_app <- file.path(RESULTS_DS, "figures/stats/robust")
  dir.create(out_app, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_app, showWarnings = FALSE, recursive = TRUE)

  long_13 <- pool_13_long(ds, s1, am)
  metric_planos <- long_13 %>% distinct(metric, plano)

  # 1. Heredar Shapiro-Wilk de §6.1 y anotar (cobertura: 13 métricas).
  shapiro_heredado <- bind_rows(
    s1$results_global %>% mutate(plano = "global"),
    s1$results_nodal %>% mutate(plano = "nodal"),
    s1$results_advanced %>% mutate(plano = "advanced")
  ) %>%
  select(metric, plano, shapiro_p) %>%
  semi_join(metric_planos, by = c("metric", "plano")) %>%
  mutate(
    normal = !is.na(shapiro_p) & shapiro_p > 0.05,
    note = case_when(
      is.na(shapiro_p) ~ "no calculable (N<3 o varianza nula)",
      shapiro_p > 0.05 ~ "No evidencia contra normalidad (baja potencia con N pequeño, Razón de verosimilitud)",
      TRUE ~ "Desviación detectada - preferir Wilcoxon como primario"
    )
  ) %>%
  arrange(plano, metric)

  cat(sprintf("\n--- [%s · %s] Shapiro-Wilk (heredado de §6.1, pool 13) ---\n",
    ds, PIPELINE_NAME))
}

```

```

print(as.data.frame(shapiro_hereditado), row.names = FALSE)

write.csv(shapiro_hereditado,
          file.path(out_app, "QQ_diagnostico.csv"),
          row.names = FALSE)

# 2. QQ-plots de las diferencias (las 13 métricas del pool).
metricas <- metric_planos$metric
n_metrics <- length(metricas)
n_cols <- 4L
n_rows <- ceiling(n_metrics / n_cols)

png(file.path(plot_app, "QQ_plots_diferencias.png"),
     width = 1500, height = 350 * n_rows, res = 150)
par(mfrow = c(n_rows, n_cols), mar = c(4, 4, 2, 1),
     oma = c(0, 0, 3, 0))
for (m in metricas) {
  v <- get_paired_vectors(long_13, m, C1, C2)
  if (!is.null(v) && length(v$g1) >= 3) {
    diffs <- v$g1 - v$g2
    qqnorm(diffs, main = paste0("QQ Δ(", C1, "-", C2, "): ", m),
           pch = 19, col = "#E74C3C", cex.main = 0.95)
    qqline(diffs, col = "grey40", lwd = 1.5)
  } else {
    plot.new(); title(main = paste(m, "(insuf. data)"))
  }
}
mtext(sprintf("[%s · %s] QQ-plots de las diferencias pareadas (pool §6.1.1, 13 métricas)",
              ds, PIPELINE_NAME), outer = TRUE, cex = 1.0, font = 2)
par(mfrow = c(1, 1))
dev.off()

message(sprintf("[%s · %s] §7.3 guardado: QQ_diagnostico.csv + QQ_plots_diferencias.png",
               ds, PIPELINE_NAME))
} # end for ds

```

7.4 Bootstrap del intervalo de confianza del tamaño de efecto

Con $N=5-6$ sujetos, los intervalos de confianza paramétricos para d_z asumen normalidad asintótica que no se cumple. El **bootstrap no paramétrico** (Efron, 1979, *Ann Stat* 7:1; Efron & Tibshirani, 1993, *An Introduction to the Bootstrap*) reemplaza esa hipótesis por re-muestreo

con reposición de los N pares observados $(g_1^{(i)}, g_2^{(i)}) = (\text{PSIL}_i, \text{MTP}_i)$, calculando en cada réplica $b = 1, \dots, R$ los dos effect sizes en paralelo: $d_z^{(b)} = \bar{\Delta}^{(b)} / \text{sd}(\Delta^{(b)})$ y $d_{av}^{(b)} = \bar{\Delta}^{(b)} / \sqrt{s_1^{(b)}, s_2^{(b)}}$ (cierre §6.1 r2: ambas variantes en paralelo, Lakens 2013; Borenstein 2009).

Variante BCa (bias-corrected and accelerated; Efron, 1987, *JASA* 82:171; Kelley, 2007, *Educ Psychol Meas* 67:738): corrige sesgo y asimetría de la distribución bootstrap mediante dos parámetros de calibración. Es la elección recomendada para tamaños de efecto, especialmente con N pequeño y distribuciones asimétricas. Se aplica por separado a d_z y d_{av} con la misma muestra bootstrap conjunta (`index = 1, 2` de `boot.ci`).

⚠ Limitación intrínseca del bootstrap con $N=5-6$

El bootstrap re-sampla de la **distribución empírica** observada, que con $N=5$ sólo contiene 5 pares únicos. Las réplicas bootstrap son **combinatorias finitas** (con reposición: $5^5 = 3125$ posibles bootstrap samples, pero muchos contienen repeticiones que infrarepresentan la verdadera distribución). El IC bootstrap tendrá **cobertura reducida** respecto al nominal 95 % cuando $N < 10$ (Davison & Hinkley, 1997, *Bootstrap Methods*, Ch. 5). El IC se reporta aquí como **descriptor del rango plausible de d_z y d_{av}** , no como inferencia confirmatoria; un IC que excluye 0 con $N=5$ debe replicarse en muestras mayores antes de declararse robusto. Con $N=15$ (ds003059) la cobertura es notablemente mejor pero sigue siendo aproximada.

```
#=====
# FASE 7.4 - Análisis de sensibilidad y robustez (dual-dataset)
# 7.4 - Bootstrap pareado del IC de d_z y d_av
#=====
# Refs: Efron 1979, 1987; Efron & Tibshirani 1993; Kelley 2007;
#       Davison & Hinkley 1997 (limitación con N pequeño).
#
# Cierre r2 §7 (H5-A): el bootstrap toma los pares (g1, g2) en lugar
# de las diferencias precomputadas, lo que permite estimar IC para d_z y
# d_av en la misma muestra bootstrap conjunta (boot.ci index=1,2). El
# CSV se renombra `bootstrap_CI_dz.csv` (antes `bootstrap_CI_cohen_d.csv`,
# legacy pre-r2) y añade columnas `d_av`, `ci_lo_av`, `ci_hi_av`.
#
# Cierre r2 §7 (H10-A): n_boot = 10000L (antes 5000L). Convención BCa
# estándar (Efron 1987; Kelley 2007); coste adicional negligible
# (~10-20 s por dataset × pool 13).
#
# Cierre r2 §7 (H1-A + H11-A): pool §6.1.1 (13 métricas) vía
# `pool_13_long` y `get_paired_vectors` del chunk §7.0. Cubre globales,
# nodales y degree_entropy en una única iteración.
#
```

```

# Cierre r2 §7 (H2-A): semilla = GLOBAL_SEED (2025L).
#=====
library(boot)
library(tidyverse)
stopifnot(exists("pool_13_long"), exists("get_paired_vectors"),
          exists("GLOBAL_SEED"))

for (ds in ACTIVE_DATASETS) {
  cd <- CONNECTOMES[[ds]]
  s1 <- STATS_13_1[[ds]]
  am <- ADVANCED_METRICS[[ds]]
  C1 <- s1$C1
  C2 <- s1$C2
  PIPELINE_NAME <- cd$pipeline

  RESULTS_DS <- file.path(PROJECT_ROOT, "results", ds)
  out_app <- file.path(RESULTS_DS, "stats/statistical_results/robust")
  plot_app <- file.path(RESULTS_DS, "figures/stats/robust")
  dir.create(out_app, showWarnings = FALSE, recursive = TRUE)
  dir.create(plot_app, showWarnings = FALSE, recursive = TRUE)

  long_13 <- pool_13_long(ds, s1, am)
  metric_planos <- long_13 %>% distinct(metric, plano)

  # Bootstrap conjunto de (d_z, d_av) sobre los pares (g1, g2). El statistic
  # devuelve un vector de dos componentes; boot.ci se invoca con index=1
  # (d_z) e index=2 (d_av). Si sd(g1)==0 o sd(g2)==0, d_av es NA pero
  # d_z sigue siendo computable mientras sd(diff)>0.
  bootstrap_d <- function(long, metric_name, c1, c2,
                          n_boot = 10000L, seed = GLOBAL_SEED) {
    set.seed(seed)
    v <- get_paired_vectors(long, metric_name, c1, c2)
    if (is.null(v))
      return(tibble(metric = metric_name, n = NA_integer_,
                    d_z = NA_real_, ci_lo_dz = NA_real_, ci_hi_dz = NA_real_,
                    d_av = NA_real_, ci_lo_av = NA_real_, ci_hi_av = NA_real_,
                    ci_method = "n<3", n_boot = NA_integer_))

    diffs_obs <- v$g1 - v$g2
    if (length(unique(diffs_obs)) < 2)
      return(tibble(metric = metric_name, n = v$n,
                    d_z = NA_real_, ci_lo_dz = NA_real_, ci_hi_dz = NA_real_,

```

```

      d_av = NA_real_, ci_lo_av = NA_real_, ci_hi_av = NA_real_,
      ci_method = "constant", n_boot = NA_integer_))

df_pair <- data.frame(g1 = v$g1, g2 = v$g2)
boot_fn <- function(d, indices) {
  s <- d[indices, , drop = FALSE]
  dif <- s$g1 - s$g2
  if (nrow(s) < 2 || sd(dif) == 0) return(c(NA_real_, NA_real_))
  dz <- mean(dif) / sd(dif)
  d_av <- if (sd(s$g1) == 0 || sd(s$g2) == 0) NA_real_
    else mean(dif) / mean(c(sd(s$g1), sd(s$g2)))
  c(dz, d_av)
}
boot_result <- boot(df_pair, statistic = boot_fn, R = n_boot)

ci_method <- "BCa"
pull_ci <- function(idx) {
  tryCatch(
    boot.ci(boot_result, type = "bca", conf = 0.95, index = idx)$bca[4:5],
    error = function(e) {
      ci_method <- "percentile (BCa fallback)"
      tryCatch(boot.ci(boot_result, type = "perc", conf = 0.95,
        index = idx)$percent[4:5],
        error = function(e2) c(NA_real_, NA_real_))
    }
  )
}
ci_dz <- pull_ci(1)
ci_av <- pull_ci(2)

d_z_obs <- mean(diffs_obs) / sd(diffs_obs)
d_av_obs <- if (sd(v$g1) == 0 || sd(v$g2) == 0) NA_real_
  else mean(diffs_obs) / mean(c(sd(v$g1), sd(v$g2)))

tibble(metric = metric_name, n = v$n,
  d_z = d_z_obs,
  ci_lo_dz = ci_dz[1], ci_hi_dz = ci_dz[2],
  d_av = d_av_obs,
  ci_lo_av = ci_av[1], ci_hi_av = ci_av[2],
  ci_method = ci_method, n_boot = n_boot)
}

```

```

message(sprintf("[%s · %s] Ejecutando bootstrap BCa (pool 13)...",
               ds, PIPELINE_NAME))
boot_results <- map2_dfr(metric_planos$metric, metric_planos$plano,
                        function(m, p) {
                          bootstrap_d(long_13, m, C1, C2) %>%
                            mutate(plano = p)
                        }) %>%
  select(metric, plano, everything())

cat(sprintf("\n--- [%s · %s] Bootstrap IC d_z y d_av (pool 13) ---\n",
           ds, PIPELINE_NAME))
print(as.data.frame(boot_results), row.names = FALSE)
write.csv(boot_results,
          file.path(out_app, "bootstrap_CI_dz.csv"),
          row.names = FALSE)

# Forest plot: una sola figura con dos paneles (d_z y d_av) en facets.
# Construcción directa de las dos filas long-format por métrica para
# evitar pivot_longer con nombres irregulares (d_z vs ci_lo_dz).
boot_long_dz <- boot_results %>%
  filter(!is.na(d_z)) %>%
  transmute(metric, plano, effect = "d_z",
            d = d_z, ci_lo = ci_lo_dz, ci_hi = ci_hi_dz)
boot_long_av <- boot_results %>%
  filter(!is.na(d_av)) %>%
  transmute(metric, plano, effect = "d_av",
            d = d_av, ci_lo = ci_lo_av, ci_hi = ci_hi_av)
boot_long <- bind_rows(boot_long_dz, boot_long_av) %>%
  mutate(effect = factor(effect, levels = c("d_z", "d_av")))

if (nrow(boot_long) > 0) {
  p_boot <- ggplot(boot_long,
                  aes(x = d, y = fct_reorder(metric, abs(d), .fun = max),
                      color = plano)) +
    geom_vline(xintercept = 0, linetype = "dashed", color = "grey50") +
    geom_errorbar(aes(xmin = ci_lo, xmax = ci_hi),
                  width = 0.3, orientation = "y", linewidth = 0.5) +
    geom_point(size = 3) +
    facet_wrap(~ effect, nrow = 1, scales = "free_x") +
    scale_color_brewer(palette = "Set1") +
    theme_minimal(base_size = 11) +
    theme(plot.title = element_text(size = 11, face = "bold"),

```

```

        plot.subtitle = element_text(size = 9)) +
labs(
  title    = sprintf("[%s · Bootstrap IC] Forest plot - d_z y d_av con IC 95%% (BCa)"
                      PIPELINE_NAME),
  subtitle = sprintf("Pool (13 métricas) · R = %d bootstraps · %s - %s",
                      unique(na.omit(boot_results$n_boot))[1], C1, C2),
  caption  = "BCa con fallback a percentil si BCa falla; IC con N<10 es descriptivo (D
  x = sprintf("Effect size (%s - %s)", C1, C2),
  y = "", color = "Plano"
)
ggsave(file.path(plot_app, "bootstrap_forest_plot.png"),
        p_boot, width = 9, height = 6, dpi = 300)
}

message(sprintf("[%s · %s] Bootstrap guardado: bootstrap_CI_dz.csv / bootstrap_forest_plot
              ds, PIPELINE_NAME))
} # end for ds

```